

# Dokumentacja: React

---

## Spis treści

---

- 1. Wprowadzenie
  - 1.1. Czym jest React
  - 1.2. Czym jest Single Page Application (SPA)
  - 1.3. Deklaratywność vs imperatywność
  - 1.4. Virtual DOM — jak React aktualizuje stronę
  - 1.5. Jak korzystać z tego poradnika
- 2. Środowisko pracy
  - 2.1. Node.js, npm i npx
  - 2.2. Instalacja Node.js
  - 2.3. Sprawdzanie wersji
  - 2.4. Czym jest Create React App
  - 2.5. Tworzenie nowego projektu
  - 2.6. Uruchamianie projektu
  - 2.7. Struktura katalogów
  - 2.8. Czyszczenie projektu startowego
  - 2.9. Skrypty npm
  - 2.10. Instalacja dodatkowych bibliotek
- 3. Podstawy JavaScript potrzebne w React
  - 3.1. Zmienne — const, let, var
  - 3.2. Typy danych
  - 3.3. Operatory arytmetyczne
  - 3.4. Operatory porównania
  - 3.5. Operatory logiczne
  - 3.6. Template stringi (szablony napisów)
  - 3.7. Instrukcja warunkowa if / else if / else
  - 3.8. Operator trójargumentowy (ternary)
  - 3.9. Funkcje — deklaracja i wyrażenie
  - 3.10. Funkcje strzałkowe (arrow functions)
  - 3.11. Tablice — tworzenie i podstawowe metody
  - 3.12. Metody tablic kluczowe w React — map, filter, find, reduce
  - 3.13. Obiekty
  - 3.14. Destrukturyzacja tablic i obiektów
  - 3.15. Operator spread (...)
  - 3.16. Import i export modułów
  - 3.17. Konwersje typów
  - 3.18. Metody napisów
  - 3.19. Truthy i falsy
  - 3.20. Konsola przeglądarki — console.log()
  - 3.21. Instrukcja warunkowa switch
  - 3.22. Pętle (for, while, do...while) i iteracja
  - 3.23. Asynchroniczność (Promises, async/await, try/catch)
  - 3.24. Dodatkowe metody tablic — forEach, some, every, slice, splice, concat
  - 3.25. Obiekt Math — losowanie, zaokrąglanie, min/max
  - 3.26. Obiekt Date — data i czas
  - 3.27. setTimeout i setInterval — opóźnienia i interwały
  - 3.28. Operator ?? (nullish coalescing) i ?. (optional chaining)

- 3.29. Obsługa błędów — try / catch / finally
- 3.30. Wyrażenia regularne (RegExp) — podstawy
- 3.31. Odczyt plików lokalnych (File API, FileReader)
- 4. JSX — składnia widoku
  - 4.1. Czym jest JSX
  - 4.2. Wstawianie wartości JavaScript w JSX
  - 4.3. Atrybuty HTML vs JSX
  - 4.4. Zasada jednego elementu nadrzędnego
  - 4.5. Fragmenty — puste znaczniki
  - 4.6. Komentarze w JSX
  - 4.7. Atrybuty boolean
  - 4.8. Co można wstawiać w klamrach — podsumowanie
  - 4.9. Tagi samozamykające z HTML w JSX (Zasada zamknięcia)
  - 4.10. Multimedia ze źródłem (Audio, Soundplayery i Wideo)
  - 4.11. Elementy osadzone: Iframe (Mapy, Embedy z YouTube)
- 5. Komponenty
  - 5.1. Czym jest komponent
  - 5.2. Pierwszy komponent funkcyjny
  - 5.3. Komponent statyczny — bez stanu
  - 5.4. Kompozycja — komponenty w komponentach
  - 5.5. Podział na pliki — osobne komponenty
  - 5.6. Props — przekazywanie danych do komponentu
  - 5.7. Props — destrukuryzacja
  - 5.8. Props — wartości domyślne
  - 5.9. Children — zawartość między znacznikami
  - 5.10. Kiedy dzielić komponent na mniejsze
- 6. Stylowanie
  - 6.1. CSS w projekcie React (CRA)
  - 6.2. className zamiast class
  - 6.3. Style inline w JSX
  - 6.4. Dynamiczne klasy CSS
  - 6.5. Dynamiczne style inline
  - 6.6. Organizacja plików CSS
- 7. Zdarzenia (Events)
  - 7.1. onClick — obsługa kliknięcia
  - 7.2. onChange — zmiana wartości pola
  - 7.3. onSubmit — wysłanie formularza
  - 7.4. onBlur — utrata fokusa
  - 7.5. Przekazywanie argumentów do handlera
  - 7.6. Obiekt zdarzenia (event)
  - 7.7. Najczęstsze zdarzenia — tabela
- 8. Stan komponentu — useState
  - 8.1. Po co jest stan
  - 8.2. Składnia useState
  - 8.3. Stan liczbowy — licznik
  - 8.4. Stan tekstowy
  - 8.5. Stan boolean — przełącznik
  - 8.6. Aktualizacja na podstawie poprzedniego stanu
  - 8.7. Reset stanu
  - 8.8. Stan nie aktualizuje się natychmiast

- 8.9. Lazy initial state
- 8.10. Zmienna lokalna vs stan — różnica
- 9. Formularze kontrolowane
  - 9.1. Czym jest formularz kontrolowany
  - 9.2. Input text
  - 9.3. Input number
  - 9.4. Input password
  - 9.5. Select — lista rozwijana
  - 9.6. Textarea
  - 9.7. Checkbox
  - 9.8. Checkbox jako switch (Bootstrap)
  - 9.9. Radio — wybór jednej opcji
  - 9.10. Range — suwak
  - 9.11. Formularz jako jeden obiekt stanu
  - 9.12. Walidacja formularza
  - 9.13. Reset formularza
- 10. Renderowanie warunkowe
  - 10.1. if przed return
  - 10.2. Operator trójargumentowy w JSX
  - 10.3. Operator && — warunkowe wyświetlanie
  - 10.4. Komunikaty błędów walidacji
  - 10.5. Obsługa pustej listy
- 11. Tablice i renderowanie list
  - 11.1. Renderowanie tablicy przez map()
  - 11.2. Atrybut key — dlaczego jest wymagany
  - 11.3. Lista numerowana
  - 11.4. Dodawanie elementu do tablicy stanu
  - 11.5. Usuwanie elementu z tablicy stanu
  - 11.6. Aktualizacja jednego elementu w tablicy
  - 11.7. Sortowanie tablicy w stanie
- 12. Obiekty w stanie
  - 12.1. Model danych — tablica obiektów
  - 12.2. Kopiowanie obiektu — spread
  - 12.3. Formularz jako obiekt stanu
  - 12.4. Dane z pliku przepisane do kodu
- 13. Bootstrap w React — skrót
  - 13.1. Rola Bootstrapa w aplikacji React
  - 13.2. Instalacja i import stylów
  - 13.3. Klasy Bootstrapa w JSX
  - 13.4. Komponenty zależne od stanu Reacta
  - 13.5. React-Bootstrap czy zwykłe klasy
  - 13.6. Mini przykład: formularz i karta
- 14. Obrazy i zasoby statyczne
  - 14.1. Obrazy z folderu public
  - 14.2. Obrazy z folderu src — import
  - 14.3. Obraz zależny od stanu
  - 14.4. Obrazy w kolekcjach (tablicach obiektów)
  - 14.5. Atrybut alt — dostępność
- 15. Przepływ danych — props w górę i w dół
  - 15.1. Dane płyną z góry na dół (top-down)

- 15.2. Callback — dziecko zgłasza zdarzenie rodzicowi
- 15.3. Lifting state up — podnoszenie stanu
- 15.4. Pełny przykład wieloplikowy z przepływem danych
- 16. useEffect i efekty uboczne
  - 16.1. Po co jest useEffect
  - 16.2. useEffect przy starcie aplikacji
  - 16.3. Tablica zależności
  - 16.4. Cleanup — sprzątanie efektu
  - 16.5. localStorage — zapis i odczyt danych
  - 16.6. Typowe pułapki useEffect
- 17. useRef — referencje do elementów DOM
  - 17.1. Czym jest useRef
  - 17.2. Ustawianie fokusa na polu
  - 17.3. useRef vs useState
- 18. Dane lokalne, JSON i fetch
  - 18.1. Tablice danych w kodzie
  - 18.2. Import pliku JSON
  - 18.3. Fetch z folderu public
  - 18.4. Parsowanie danych tekstowych
  - 18.5. Czym jest API
  - 18.6. Endpoint, metoda HTTP i status odpowiedzi
  - 18.7. Pobieranie danych z zewnętrznego API
  - 18.8. Loading, błąd i pusta lista
  - 18.9. Wysyłanie danych metodą POST
  - 18.10. Parametry w adresie URL
  - 18.11. Dobre praktyki przy pracy z API
- 19. Logika aplikacji poza JSX
  - 19.1. Funkcje pomocnicze
  - 19.2. Osobne moduły z logiką
  - 19.3. Oddzielenie UI od obliczeń
- 20. Organizacja projektu
  - 20.1. Nazewnictwo plików i komponentów
  - 20.2. Folder components
  - 20.3. Folder data
  - 20.4. Folder utils
  - 20.5. Przykładowa struktura projektu
- 21. Debugowanie
  - 21.1. Konsola przeglądarki
  - 21.2. React DevTools
  - 21.3. Typowe błędy składni
  - 21.4. Typowe błędy stanu
  - 21.5. Typowe błędy formularzy
- 22. Najczęstsze pułapki i jak ich unikać
  - 22.1. Brak key w pętli map()
  - 22.2. Mutowanie stanu zamiast tworzenia kopii
  - 22.3. Odczyt stanu zaraz po ustawieniu
  - 22.4. Brak event.preventDefault() w formularzu
  - 22.5. Zapomnienie o import useState
  - 22.6. Wywołanie funkcji zamiast przekazania referencji
- 23. Build i publikacja projektu



- 23.1. npm run build
- 23.2. Co zawiera folder build
- 23.3. Typowe problemy przy buildzie
- 24. Dobre praktyki UI i dostępność
  - 24.1. Typ przycisku — button vs submit
  - 24.2. Label i htmlFor
  - 24.3. Semantyczny układ strony
- 25. Routing i Nawigacja w SPA (react-router-dom)
  - 25.1. Czym jest Client-Side Routing?
  - 25.2. BrowserRouter, Routes i Route
  - 25.3. Linkowanie pomiędzy podstronami używając [<Link>](#)
  - 25.4. Nawigacja z poziomu kodu (useNavigate)
  - 25.5. Parametry w ścieżkach (useParams)
- 26. Wzorce praktyczne
  - 26.1. Formularz rejestracji
  - 26.2. Zapisy na kurs
  - 26.3. Formularz filmu
  - 26.4. Formularz zamówienia pizzy
  - 26.5. Formularz wyceny ubezpieczenia OC pojazdu
  - 26.6. Formularz rezerwacji wizyty lekarskiej
  - 26.7. Generator i podgląd CV (Live CV Builder)
  - 26.8. Formularz ankiety z oceną gwiazdkową
  - 26.9. Kalkulator wyceny szafy na wymiar
  - 26.10. Kalkulator BMI
  - 26.11. Przelicznik walut
  - 26.12. Kalkulator spalania paliwa i kosztów podróży
  - 26.13. Kalkulator rat kredytu (symulator)
  - 26.14. Kalkulator zapotrzebowania kalorycznego (BMR i TDEE)
  - 26.15. Kalkulator wieku psa (ludzkie lata)
  - 26.16. Kalkulator czasu pracy i wynagrodzenia
  - 26.17. Konwerter systemów liczbowych
  - 26.18. Generator hasła
  - 26.19. Kości do gry z blokowaniem
  - 26.20. Gra w zgadywanie liczb (Za dużo / Za mało)
  - 26.21. Kamień, Papier, Nożyce
  - 26.22. Rzut monetą ze statystykami i historią
  - 26.23. Galeria zdjęć z kategoriami
  - 26.24. Lista zadań (Todo App) — wieloplikowy
  - 26.25. Widok kart z filtrami i wyszukiwaniem
  - 26.26. Algorytmy — sumowanie, zliczanie, filtrowanie
  - 26.27. Galeria zdjęć z lightboxem i ulubionymi
  - 26.28. Książka adresowa z wyszukiwarką i tagami
  - 26.29. Biblioteczka książek ze statusem przeczytania
  - 26.30. Wyszukiwarka przepisów kulinarnych po składnikach
  - 26.31. Dzienniczek ocen z obliczaniem średniej ważonej
  - 26.32. Lista zakupów z podziałem na działy
  - 26.33. Mixer kolorów RGB
  - 26.34. Licznik z historią operacji
  - 26.35. Prosta Playlista Audio (Odtwarzacz ze stanem)
  - 26.36. Akordeon FAQ z widocznością (Sekcje Rozwijane)

- 26.37. CSS Gradient Generator
- 26.38. Licznik słów, znaków i czasu czytania
- 26.39. Minutnik Kuchenny (Odliczanie)
- 26.40. Kreator i podgląd menu restauracji (Karta dań)
- 26.41. Interaktywny Quiz wiedzy (5 pytań)
- 26.42. Tablica Kanban (Zadania w kolumnach)
- 26.43. System rezerwacji miejsc w kinie (Siatka miejsc)
- 26.44. Akordeon FAQ z wyszukiwarką pytań
- 26.45. Wyszukiwarka użytkowników z API
- 27. Najczęstsze błędy (Wyjątki i błędy konsolowe)
  - 27.1. SyntaxError: Unexpected token
  - 27.2. TypeError: Cannot read properties of undefined
  - 27.3. ReferenceError: X is not defined
  - 27.4. Warning: Each child in a list should have a unique "key" prop
  - 27.5. Error: Rendered fewer hooks than expected
  - 27.6. Error: Invalid hook call
  - 27.7. Error: Too many re-renders
  - 27.8. Error: Objects are not valid as a React child
  - 27.9. Warning: A component is changing an uncontrolled input to be controlled
  - 27.10. Module not found: Can't resolve
  - 27.11. TypeError: Failed to fetch
  - 27.12. CORS error: Access-Control-Allow-Origin
  - 27.13. TypeError: X is not a function
  - 27.14. Error: Element type is invalid
  - 27.15. Warning: React does not recognize the prop on a DOM element
  - 27.16. TypeError: Assignment to constant variable
  - 27.17. Unhandled Promise Rejection
  - 27.18. SyntaxError: Cannot use import statement outside a module
  - 27.19. JSON.parse error: Unexpected token
  - 27.20. Error: Minified React error

## 1. Wprowadzenie

React najłatwiej zrozumieć jako sposób opisywania ekranu na podstawie danych. Nie myślisz wtedy: „znajdź element i zmień mu tekst”, tylko: „dla tych danych pokaż taki widok”. Taka zmiana myślenia jest ważniejsza niż sama składnia JSX.

```
function StatusPołączenia({ online }) {  
  return (  
    <p className={online ? "text-success" : "text-danger"}>  
      {online ? "Połączono" : "Brak połączenia"}  
    </p>  
  );  
}
```

W przykładzie widok nie jest ręcznie aktualizowany. Wystarczy zmienić wartość `online`, a React obliczy, jak powinien wyglądać aktualny fragment interfejsu.

### 1.1. Czym jest React

React warto rozumieć jako warstwę, która zamienia dane na interfejs. Komponent nie powinien sam wyszukiwać elementu w DOM i zmieniać mu tekstu; powinien dostać dane i zwrócić JSX opisujący aktualny widok.

```
function UserStatus({ name, active }) {  
  return <p>{name}: {active ? "aktywny" : "nieaktywny"}</p>;  
}
```

React to biblioteka JavaScript stworzona przez zespół Facebooka (Meta) w 2013 roku. Służy do budowania interfejsów użytkownika (UI). Nie jest pełnym frameworkiem — odpowiada wyłącznie za warstwę widoku. Oznacza to, że React nie narzuca sposobu obsługi routingu, zapytań do serwera ani zarządzania bazą danych. W podstawowych projektach te rzeczy nie są zazwyczaj potrzebne.

Najważniejsza zasada Reacta brzmi: **widok jest funkcją danych**. Jeżeli zmienią się dane (stan) w komponencie, React automatycznie odświeży odpowiedni fragment strony. Programista nie musi ręcznie wyszukiwać elementów DOM i zmieniać ich tekstu — React robi to sam.

React opiera się na **komponentach**. Komponent to funkcja JavaScript, która zwraca fragment widoku (napisany w składni JSX, która wygląda jak HTML). Cała aplikacja jest drzewem komponentów — od jednego głównego (`App`) aż po najmniejsze przyciski i etykiety.

## 1.2. Czym jest Single Page Application (SPA)

W SPA pierwszy dokument HTML jest ładowany raz, a dalsze zmiany widoku wykonuje JavaScript. To oznacza, że stan aplikacji może pozostać w pamięci podczas przechodzenia między ekranami, o ile nie odświeżysz całej strony.

W praktyce zwykły link `<a href="/profil">` może przeładować aplikację, a link z routera Reacta zmieni widok bez resetowania stanu.

SPA, czyli Single Page Application (aplikacja jednostronicowa), to aplikacja webowa, która działa na jednej stronie HTML. Przeglądarka łączy plik `index.html` oraz pliki JavaScript i CSS. Od tego momentu widok zmienia się bez pełnego przeładowania strony — wszystko odbywa się dynamicznie po stronie klienta (przeglądarki).

Proste aplikacje Reactowe są właśnie małymi SPA. W typowych, mniejszych projektach zwykle:

- nie ma backendu (serwera)
- nie ma bazy danych
- nie ma routingu (wielu podstron)
- dane są wpisane w kodzie lub skopiowane z pliku `dane.txt`

## 1.3. Deklaratywność vs imperatywność

Deklaratywność najlepiej widać wtedy, gdy ten sam widok ma kilka stanów. Zamiast ręcznie dopisywać i usuwać klasy w DOM, opisujesz warunek w JSX.

```
function SaveInfo({ saved }) {  
  return <p className={saved ? "text-success" : "text-warning"}>  
    {saved ? "Zapisano zmiany" : "Masz niezapisane zmiany"}  
  </p>;  
}
```

Przesiadka z czystego JavaScriptu na Reacta wymaga zmiany sposobu myślenia.

**Podejście imperatywne (czysty JavaScript / DOM):** Mówisz krok po kroku, *jak* coś zrobić. „Znajdź przycisk, dodaj do niego zdarzenie, znajdź akapit, zmień jego tekst na »Cześć«”. Musisz ręcznie pilnować każdego elementu na stronie.

```
// Podejście imperatywne – czysty JavaScript
const btn = document.querySelector("#btn");
const output = document.querySelector("#output");

btn.addEventListener("click", function () {
  output.textContent = "Kliknięto przycisk!";
});
```

**Podejście deklaratywne (React):** Mówisz, *co* chcesz osiągnąć. „Chcę, aby ten akapit zawsze wyświetlał to, co jest w zmiennej `komunikat`”. React sam zajmie się aktualizacją DOM, gdy zmienna ulegnie zmianie.

```
// Podejście deklaratywne – React
// Plik: src/App.js
import { useState } from "react";

function App() {
  const [komunikat, setKomunikat] = useState("Czekam na kliknięcie...");

  return (
    <div>
      <button onClick={() => setKomunikat("Kliknięto przycisk!")}>
        Kliknij
      </button>
      <p>{komunikat}</p>
    </div>
  );
}

export default App;
```

To jak zamawianie pizzy: w podejściu imperatywnym wchodzisz do kuchni i instruujesz kucharza, ile mąki i wody ma dodać. W podejściu deklaratywnym (React) składasz zamówienie: „Chcę margheritę”. Restauracja (React) sama dba o to, byś ją otrzymał.

#### 1.4. Virtual DOM — jak React aktualizuje stronę

Virtual DOM nie oznacza, że React zawsze jest automatycznie najszybszy. Oznacza, że React porównuje opis poprzedniego i nowego widoku, a następnie aktualizuje realny DOM tylko tam, gdzie wynik renderowania faktycznie się zmienił.

Dlatego tak ważne są stabilne `key` w listach i niemutowalne aktualizacje stanu. Dzięki nim React potrafi poprawnie rozpoznać, który element jest ten sam, a który został dodany, usunięty albo zmieniony.

Zrozumienie tego mechanizmu pomaga pisać lepszy kod:

1. **Komponenty** to Twoje „klocki” — funkcje zwracające widok.
2. Gdy zmienia się **stan** (`useState`), React tworzy w pamięci nową kopię widoku (tzw. **Virtual DOM**).
3. React **porównuje** tę kopię z tym, co aktualnie widzi użytkownik na ekranie.
4. React znajduje **tylko te różnice** (np. zmienił się tekst w jednym polu) i tylko je przesyła do prawdziwego DOM przeglądarki.

Dzięki temu aplikacje są szybkie, a programista nie musi martwić się o ręczne manipulowanie elementami HTML.

#### 1.5. Jak korzystać z tego poradnika

Ten temat warto zamknąć małym komponentem testowym. Zmień dane wejściowe i sprawdź, czy widok aktualizuje się bez ręcznego dotykania DOM.

| OBSZAR     | CO TRZEBA UMIEĆ                                                                  | PO CO                        |
|------------|----------------------------------------------------------------------------------|------------------------------|
| Środowisko | <code>node</code> , <code>npm</code> , <code>npx</code> , <code>npm start</code> | uruchomienie projektu        |
| JavaScript | zmienne, funkcje, tablice, obiekty                                               | pisanie logiki               |
| JSX        | <code>className</code> , <code>{zmienna}</code> , fragmenty                      | pisanie widoku               |
| Komponenty | funkcje zwracające JSX                                                           | dzielenie UI                 |
| Props      | przekazywanie danych do komponentu                                               | ponowne użycie               |
| Stan       | <code>useState</code>                                                            | reakcja na akcje użytkownika |
| Zdarzenia  | <code>onClick</code> , <code>onChange</code> , <code>onSubmit</code>             | obsługa interakcji           |
| Formularze | kontrolowane pola                                                                | odczyt danych                |
| Listy      | <code>map</code> , <code>filter</code> , <code>key</code>                        | dynamiczne dane              |
| Bootstrap  | klasy CSS                                                                        | szybki wygląd                |
| Zasoby     | obrazy z <code>public</code> i <code>src</code>                                  | karty, miniatury, ikony      |

## 2. Środowisko pracy

Środowisko pracy w React składa się z kilku warstw: Node.js uruchamia narzędzia, npm pobiera paczki, bundler przetwarza JSX i CSS, a przeglądarka pokazuje wynik. Gdy coś nie działa, warto ustalić, na której warstwie pojawia się problem: instalacja, uruchomienie serwera, kompilacja, czy dopiero działanie aplikacji w przeglądarce.

| PROBLEM                     | GDZIE SZUKAĆ PRZYCZYNY     | TYPOWA REAKCJA                       |
|-----------------------------|----------------------------|--------------------------------------|
| komenda nie istnieje        | Node.js / npm / terminal   | sprawdź wersje i katalog projektu    |
| aplikacja się nie kompiluje | importy, JSX, zależności   | przeczytaj pierwszy błąd w terminalu |
| strona jest pusta           | konsola przeglądarki       | sprawdź błąd JavaScript              |
| styl się nie ładuje         | import CSS / ścieżka pliku | sprawdź nazwę pliku i import         |

### 2.1. Node.js, npm i npx

`node` uruchamia JavaScript poza przeglądarką, `npm` zarządza paczkami, a `npx` potrafi uruchomić narzędzie bez ręcznej instalacji globalnej. W projekcie React większość problemów środowiskowych zaczyna się od złej wersji Node albo od uruchomienia komendy poza katalogiem projektu.

```
node -v
npm -v
pwd # lub cd na Windows: sprawdź aktualny folder
```

Zanim zaczniesz pracę z Reactem, musisz zrozumieć trzy narzędzia:

| NARZĘDZIE | CO TO JEST                               | DO CZEGO SŁUŻY                                                            |
|-----------|------------------------------------------|---------------------------------------------------------------------------|
| Node.js   | Środowisko uruchomieniowe JavaScript     | Uruchamia JavaScript poza przeglądarką, obsługuje narzędzia deweloperskie |
| npm       | Menedżer pakietów (Node Package Manager) | Instaluje biblioteki (np. React, Bootstrap), zarządza zależnościami       |
| npx       | Narzędzie do uruchamiania pakietów       | Uruchamia paczkę bez globalnej instalacji (np. Create React App)          |

Dodatkowo ważne pojęcia:

| POJĘCIE                        | OPIS                                                             |
|--------------------------------|------------------------------------------------------------------|
| <code>package.json</code>      | Plik opisujący projekt — nazwa, wersja, zależności, skrypty      |
| <code>node_modules</code>      | Folder z pobranymi bibliotekami — nie edytujemy go ręcznie       |
| <code>package-lock.json</code> | Dokładne wersje zainstalowanych pakietów — nie edytujemy ręcznie |

## 2.2. Instalacja Node.js

W części „Instalacja Node.js” zapisz konkretną komendę, oczekiwany efekt i miejsce wykonania. Jeśli coś nie działa, porównaj te trzy rzeczy: czy jesteś w katalogu z `package.json`, czy zależności są zainstalowane i czy terminal pokazuje błąd narzędzia czy błąd kodu.

Node.js pobieramy ze strony <https://nodejs.org>. Wybieramy wersję **LTS** (Long Term Support), która jest stabilna i sprawdzona. Instalator automatycznie instaluje także `npm` i `npx`.

**Windows / macOS:** Pobierz instalator ze strony, uruchom go i postępuj zgodnie z instrukcjami.

**Sprawdzenie po instalacji:** Po zainstalowaniu otwórz terminal (wiersz poleceń) i wpisz:

```
node -v
# Wynik np.: v20.11.0

npm -v
# Wynik np.: 10.2.4

npx --version
# Wynik np.: 10.2.4
```

Jeżeli polecenia zwracają numery wersji, instalacja się powiodła.

## 2.3. Sprawdzanie wersji

Sprawdzanie wersji ma sens wtedy, gdy zapiszesz wynik i porównasz go z wymaganiami projektu. Jeśli projekt był tworzony na innej wersji Node, po aktualizacji zależności mogą pojawić się błędy, których nie widać w samym kodzie Reacta. Warto sprawdzić też `npm outdated`, gdy projekt długo nie był ruszany.

Przed tworzeniem nowego projektu warto upewnić się, że narzędzia są zainstalowane:

```
# Sprawdzenie wersji node.js
node -v

# Sprawdzenie wersji npm
npm -v

# Sprawdzenie wersji npx
npx --version
```

Minimalne wymagania dla Create React App:

- Node.js w wersji 14 lub nowszej (zalecana 18+)
- npm w wersji 6 lub nowszej

## 2.4. Czym jest Create React App

Create React App jest starszym sposobem tworzenia projektu. Warto umieć rozpoznać jego strukturę, bo dużo istniejących projektów nadal go używa. W nowszych projektach często spotkasz Vite, ale podstawowe komponenty Reacta, propsy, stan i efekty działają tak samo.

| NARZĘDZIE | TYPOWY START             | FOLDER PRODUKCYJNY |
|-----------|--------------------------|--------------------|
| CRA       | <code>npm start</code>   | <code>build</code> |
| Vite      | <code>npm run dev</code> | <code>dist</code>  |

Create React App (skrót: CRA) to narzędzie, które tworzy gotowy projekt Reactowy z pełną konfiguracją. Nie trzeba ręcznie konfigurować bundlera (Webpack), transpilera (Babel) ani serwera deweloperskiego — CRA robi to za nas.

CRA tworzy projekt z:

- Reactem i ReactDOM
- Babeliem (kompiluje nowoczesny JavaScript i JSX do kodu zrozumiałego dla przeglądarek)
- Webpackiem (łączy pliki w jeden bundle)
- Serwerem deweloperskim z automatycznym odświeżaniem
- Środowiskiem do testów (`jest`)
- Gotowymi skryptami npm (`start`, `build`, `test`)

**Uwaga:** Create React App jest obecnie narzędziem, które nie jest już aktywnie rozwijane przez zespół Reacta. Dla nowych projektów profesjonalnych rekomendowane są narzędzia jak Vite. Jednak CRA nadal doskonale sprawdza się do nauki i podstawowych środowisk projektowych, dlatego w tej dokumentacji używamy go świadomie.

## 2.5. Tworzenie nowego projektu

Po utworzeniu projektu zawsze wykonaj trzy kontrole: czy powstał `package.json`, czy zainstalowały się zależności oraz czy serwer deweloperski pokazuje adres lokalny. Jeśli brakuje `node_modules`, uruchom `npm install` w folderze projektu.

```
cd moja-aplikacja
npm install
npm start
```

Projekt tworzysz w folderze, w którym chcesz mieć katalog aplikacji. Nazwa projektu powinna być:

- pisana małymi literami
- bez spacji
- bez polskich znaków
- słowa oddzielone myślnikami

```
# Tworzenie nowego projektu o nazwie "moja-aplikacja"
npx create-react-app moja-aplikacja

# Wejście do folderu projektu
cd moja-aplikacja

# Uruchomienie serwera deweloperskiego
npm start
```

Po uruchomieniu `npm start` przeglądarka powinna automatycznie otworzyć adres `http://localhost:3000` z domyślną stroną startową Reacta.

Jeżeli port 3000 jest zajęty, terminal zapyta, czy użyć innego portu. Potwierdź klawiszem `Y`.

## 2.6. Uruchamianie projektu

W części „Uruchamianie projektu” zapisz konkretną komendę, oczekiwany efekt i miejsce wykonania. Jeśli coś nie działa, porównaj te trzy rzeczy: czy jesteś w katalogu z `package.json`, czy zależności są zainstalowane i czy terminal pokazuje błąd narzędzia czy błąd kodu.

Po utworzeniu projektu najczęściej używasz dwóch poleceń:

```
# Uruchomienie serwera deweloperskiego – używasz codziennie przy pisaniu kodu
npm start

# Zatrzymanie serwera – ctrl + C w terminalu
```

Serwer deweloperski:

- Obserwuje zmiany w plikach źródłowych
- Po zapisaniu pliku automatycznie odświeża stronę w przeglądarce
- Wyświetla błędy kompilacji w terminalu i w przeglądarce
- Działa pod adresem `http://localhost:3000`

2.7. Struktura katalogów

Struktura katalogów pokazuje, które pliki są częścią aplikacji, a które konfiguracją narzędzi. `src` zawiera kod przetwarzany przez bundler, `public` pliki kopiowane bezpośrednio, a `package.json` opisuje zależności i komendy. Przy błędach importu najpierw sprawdzaj, czy plik jest w odpowiednim miejscu.

Po utworzeniu projektu przez CRA otrzymujesz następującą strukturę:

```
moja-aplikacja/
├─ node_modules/      # Pobrane biblioteki (nie edytujemy)
├─ public/             # Pliki statyczne
│   ├─ index.html     # Główny plik HTML aplikacji
│   ├─ favicon.ico    # Ikona zakładki przeglądarki
│   └─ manifest.json  # Manifest aplikacji webowej
├─ src/               # Kod źródłowy aplikacji
│   ├─ App.js         # Główny komponent aplikacji
│   ├─ App.css        # Style głównego komponentu
│   ├─ App.test.js    # Testy głównego komponentu
│   ├─ index.js       # Punkt wejścia – montuje React do HTML
│   ├─ index.css      # Style globalne
│   ├─ logo.svg       # Logo Reacta (do usunięcia)
│   └─ reportWebVitals.js # Metryki wydajności (do usunięcia)
├─ package.json       # Konfiguracja projektu i zależności
├─ package-lock.json  # Zablokowane wersje pakietów
└─ README.md         # Opis projektu
```

Najważniejsze pliki:

| PLIK                           | ROLA                                                                                               |
|--------------------------------|----------------------------------------------------------------------------------------------------|
| <code>public/index.html</code> | Główny plik HTML — zawiera <code>&lt;div id="root"&gt;</code> , do którego React montuje aplikację |
| <code>src/index.js</code>      | Punkt wejścia — montuje komponent <code>App</code> do elementu <code>#root</code> w HTML           |
| <code>src/App.js</code>        | Główny komponent aplikacji — tutaj zaczynasz pracę                                                 |
| <code>src/App.css</code>       | Style dla komponentu App                                                                           |
| <code>src/index.css</code>     | Style globalne aplikacji                                                                           |
| <code>package.json</code>      | Lista zależności i skrypty npm                                                                     |

Jak to działa — przepływ od HTML do komponentu:



```
←!— public/index.html – uproszczony widok →  
  
<!DOCTYPE html>  
<html lang="pl">  
  <head>  
    <meta charset="utf-8" />  
    <title>Moja Aplikacja React</title>  
  </head>  
  <body>  
    ←!— React montuje całą aplikację do tego elementu →  
    <div id="root"></div>  
  </body>  
</html>
```

```
// src/index.js – punkt wejścia aplikacji  
import React from "react";  
import ReactDOM from "react-dom/client";  
import "./index.css";  
import App from "./App";  
  
// React znajduje element o id="root" w HTML  
// i montuje do niego komponent App  
const root = ReactDOM.createRoot(document.getElementById("root"));  
root.render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
);
```

```
// src/App.js – główny komponent  
import "./App.css";  
  
function App() {  
  return (  
    <div>  
      <h1>Witaj w React!</h1>  
    </div>  
  );  
}  
  
export default App;
```

`React.StrictMode` to komponent, który w trybie deweloperskim pomaga wykryć potencjalne problemy. Renderuje komponenty dwukrotnie, aby sprawdzić, czy nie mają efektów ubocznych. W trybie produkcyjnym (`npm run build`) StrictMode jest ignorowany.

## 2.8. Czyszczenie projektu startowego

Czyszczenie projektu startowego nie polega tylko na usunięciu logo. Po wyczyszczeniu warto zostawić minimalny `App`, jeden plik CSS i działający import. Dzięki temu każdy kolejny błąd będzie pochodził z Twojego kodu, a nie z nieużywanych plików szablonu.

Po utworzeniu projektu warto usunąć niepotrzebne pliki startowe i zacząć od czystego szablonu. Oto minimalny zestaw plików po wyczyszczeniu:

```
// src/App.js – wyczyszczony komponent główny
import './App.css';

function App() {
  return (
    <main>
      <h1>Moja aplikacja React</h1>
      <p>Projekt działa poprawnie.</p>
    </main>
  );
}

export default App;
```

```
/* src/App.css – wyczyszczony plik stylów */
/* Na razie pusty – dodamy style później */
```

```
// src/index.js – bez zmian, zostaje jak jest
import React from "react";
import ReactDOM from "react-dom/client";
import './index.css';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Pliki, które można usunąć:

- `src/logo.svg`
- `src/App.test.js`
- `src/reportWebVitals.js`
- `src/setupTests.js`

## 2.9. Skrypty npm

W części „Skrypty npm” zapisz konkretną komendę, oczekiwany efekt i miejsce wykonania. Jeśli coś nie działa, porównaj te trzy rzeczy: czy jesteś w katalogu z `package.json`, czy zależności są zainstalowane i czy terminal pokazuje błąd narzędzia czy błąd kodu.

Skrypty są zdefiniowane w pliku `package.json`. Dzięki nim nie trzeba pamiętać pełnych poleceń:

```
{
  "scripts": {
    "start": "react-scripts start",
```

```

    "build": "react-scripts build",
    "test": "react-scripts test",
    "eject": "react-scripts eject"
  }
}

```

| POLECENIE                      | KIEDY UŻYWAĆ                         | EFEKT                                                  |
|--------------------------------|--------------------------------------|--------------------------------------------------------|
| <code>npm start</code>         | Podczas pisania kodu                 | Uruchamia serwer deweloperski                          |
| <code>npm run build</code>     | Przed publikacją / oddaniem projektu | Tworzy zoptymalizowany folder <code>build</code>       |
| <code>npm test</code>          | Przy testowaniu                      | Uruchamia testy                                        |
| <code>npm install nazwa</code> | Gdy dodajesz bibliotekę              | Pobiera pakiet i aktualizuje <code>package.json</code> |

Polecenia `eject` prawie nigdy nie używaj. Wyciąga ukrytą konfigurację Webpacka i Babela, co komplikuje projekt.

## 2.10. Instalacja dodatkowych bibliotek

W części „Instalacja dodatkowych bibliotek” zapisz konkretną komendę, oczekiwany efekt i miejsce wykonania. Jeśli coś nie działa, porównaj te trzy rzeczy: czy jesteś w katalogu z `package.json`, czy zależności są zainstalowane i czy terminal pokazuje błąd narzędzia czy błąd kodu.

Aby zainstalować bibliotekę (np. Bootstrap), używasz polecenia `npm install`:

```

# Instalacja bootstrapa
npm install bootstrap

# Instalacja wielu bibliotek naraz
npm install bootstrap react-icons

```

Po instalacji biblioteka pojawia się w `node_modules` i w sekcji `dependencies` w `package.json`. Od tego momentu możesz ją importować w swoich plikach.

## 3. Podstawy JavaScript potrzebne w React

JavaScript w React nie jest dodatkiem, tylko podstawą. JSX pozwala pisać znaczniki podobne do HTML, ale każda decyzja widoku opiera się na zwykłych wartościach JavaScript: tablicach, obiektach, funkcjach, warunkach i metodach takich jak `map()` albo `filter()`.

```

const produkty = [
  { id: 1, nazwa: "Monitor", cena: 799, aktywny: true },
  { id: 2, nazwa: "Mysz", cena: 99, aktywny: false },
];

const widoczneProdukty = produkty.filter((produkt) => produkt.aktywny);
const suma = widoczneProdukty.reduce((acc, produkt) => acc + produkt.cena, 0);

```

Takie przygotowanie danych najlepiej robić przed `return`, a w JSX zostawić samo renderowanie.

React jest biblioteką JavaScript, więc znajomość podstaw tego języka jest niezbędna. Ten rozdział prezentuje elementy JavaScriptu, które pojawiają się najczęściej w kodzie Reactowym.

### 3.1. Zmienne — const, let, var

W React domyślnie wybieraj `const`, nawet dla tablic i obiektów. Nie oznacza to, że obiekt jest zamrożony; oznacza tylko, że zmienna nie dostanie nowej referencji przez znak `=`. Dla stanu i tak stworzysz nowe kopie przez `setState`.

```
const products = [];  
const user = { name: "Anna" };  
let index = 0; // tylko tam, gdzie wartość faktycznie się zmienia
```

Zmienna to "pudełko", w którym przechowujesz dane (tekst, liczby, tablice), by móc ich później użyć w programie. W nowoczesnym JavaScriptcie zmienne deklarujemy za pomocą dwóch głównych słów kluczowych: `const` i `let`. Słowo `var` to przeżytek, którego w React już się nie używa.

```
// 1. const – STAŁA REFERENCJA (Domyślny wybór w React!)  
// Używamy go, gdy wiemy, że wartość przypisana do zmiennej nie ulegnie zmianie  
// poprzez przypisanie znakiem równości '='.  
const nazwaAplikacji = "Mój React App";  
const maxUzytkownikow = 100;  
// nazwaAplikacji = "Inna aplikacja"; // BŁĄD! Nie można przypisać ponownie do const  
  
// 2. let – ZMIENNA (Używaj tylko tam, gdzie to konieczne)  
// Używamy go, gdy wartość z założenia będzie się zmieniać (np. w pętlach).  
let licznik = 0;  
licznik = licznik + 1; // OK! Zwiększamy wartość o 1  
licznik = 10; // OK! Nadpisujemy nową wartością  
  
// 3. var – STARY STYL (Unikaj)  
// Działa podobnie do let, ale ma przestarzały sposób działania tzw. zasięgu (scope),  
// co prowadzi do błędów. W React udajemy, że 'var' nie istnieje.  
var staryStyl = "nie używaj w React";
```

**Zakres widoczności (Scope):** Zmienne `let` i `const` "żyją" tylko w obrębie bloku kodu `{ ... }`, w którym zostały utworzone.

```
if (true) {  
  const tajneHaslo = "12345";  
  console.log(tajneHaslo); // OK, działa  
}  
// console.log(tajneHaslo); // BŁĄD! tajneHaslo nie istnieje poza klamrami if-a
```

| SŁOWO KLUCZOWE     | CZY MOŻNA PRZYPISAĆ PONOWNIE | ZASIĘG                   | TYPOWE UŻYCIE                           |
|--------------------|------------------------------|--------------------------|-----------------------------------------|
| <code>const</code> | Nie                          | Blokowy <code>{ }</code> | Tablice, obiekty, funkcje, stałe teksty |
| <code>let</code>   | Tak                          | Blokowy <code>{ }</code> | Licznik w pętli, zmienna pomocnicza     |
| <code>var</code>   | Tak                          | Funkcyjny                | Stary kod — niezalecane                 |

**Ważne:** `const` nie oznacza, że wartość jest niemodyfikowalna. Oznacza, że nie można **przypisać ponownie** zmiennej. Jeżeli `const` wskazuje na tablicę lub obiekt, można modyfikować ich zawartość:

```
const lista = [1, 2, 3];  
lista.push(4); // OK – modyfikujemy zawartość tablicy
```

```
// lista = [5, 6, 7];    // błąd – nie można przypisać ponownie

const osoba = { imie: "Jan" };
osoba.imie = "Anna";    // OK – modyfikujemy pole obiektu
// osoba = {};          // błąd – nie można przypisać ponownie
```

### 3.2. Typy danych

W formularzach Reacta typ danych bywa mylący: nawet `input type="number"` zwraca tekst. Jeśli liczba ma być użyta w obliczeniach, zamień ją jawnie przez `Number()` albo przechowuj pusty string jako stan pola.

```
const valueFromInput = "42";
const amount = Number(valueFromInput);
```

JavaScript ma kilka podstawowych typów danych:

| TYP                    | OPIS                                      | PRZYKŁADY                                                          |
|------------------------|-------------------------------------------|--------------------------------------------------------------------|
| <code>string</code>    | Tekst (napis)                             | <code>"Ala"</code> , <code>'React'</code> , <code>`szablon`</code> |
| <code>number</code>    | Liczba (całkowita lub zmiennoprzecinkowa) | <code>5</code> , <code>3.14</code> , <code>-10</code>              |
| <code>boolean</code>   | Wartość logiczna                          | <code>true</code> , <code>false</code>                             |
| <code>undefined</code> | Brak przypisanej wartości                 | <code>let x;</code> → <code>x</code> jest <code>undefined</code>   |
| <code>null</code>      | Celowy brak wartości                      | <code>let wynik = null;</code>                                     |
| <code>object</code>    | Obiekt (kolekcja par klucz-wartość)       | <code>{ imie: "Jan", wiek: 25 }</code>                             |
| <code>array</code>     | Tablica (technicznie obiekt)              | <code>[1, 2, 3]</code> , <code>["a", "b"]</code>                   |

```
// Przykłady każdego typu
const tytul = "Lista kursów";    // string
const liczba = 42;                // number
const cena = 19.99;              // number (zmiennoprzecinkowa)
const aktywny = true;            // boolean
let wybrany = undefined;          // undefined
let wynik = null;                // null
const kurs = { nazwa: "React" };  // object
const oceny = [5, 4, 3, 5, 4];    // array
```

Sprawdzanie typu:

```
console.log(typeof "tekst");    // "string"
console.log(typeof 42);         // "number"
console.log(typeof true);       // "boolean"
console.log(typeof undefined);  // "undefined"
console.log(typeof null);       // "object" (historyczny błąd JS)
console.log(typeof [1, 2]);     // "object" (tablice to obiekty)
console.log(Array.isArray([1])); // true (prawidłowy sposób sprawdzenia tablicy)
```

Typy prymitywne vs referencyjne:

To rozróżnienie jest kluczowe dla zrozumienia, dlaczego w React tworzymy kopie obiektów/tablic zamiast je modyfikować.

```
// PRYMITYWNE (string, number, boolean, null, undefined)
// Kopiowanie tworzy NIEZALEŻNĄ kopię wartości
let a = 5;
let b = a; // b = 5 (kopia wartości)
b = 10;    // a nadal = 5 (zmiana b nie wpływa na a)

// REFERENCYJNE (object, array, function)
// Kopiowanie kopiuje REFERENCJĘ (wskaźnik), nie wartość!
const arr1 = [1, 2, 3];
const arr2 = arr1; // arr2 wskazuje na TĘ SAMĄ tablicę!
arr2.push(4);      // arr1 też ma teraz [1, 2, 3, 4]!

// Dlatego w React robimy KOPIĘ:
const arr3 = [...arr1]; // Nowa, niezależna tablica
```

3.3. Operatory arytmetyczne

Przy operatorze `+` uważaj na mieszanie liczb i tekstów. W React bardzo często wartość pochodzi z formularza, więc bez konwersji możesz dostać konkatenację zamiast dodawania.

```
const a = "2";
const b = "3";
console.log(a + b); // "23"
console.log(Number(a) + Number(b)); // 5
```

| OPERATOR        | NAZWA                       | PRZYKŁAD            | WYNIK                  |
|-----------------|-----------------------------|---------------------|------------------------|
| <code>+</code>  | Dodawanie                   | <code>10 + 5</code> | <code>15</code>        |
| <code>-</code>  | Odejmowanie                 | <code>10 - 5</code> | <code>5</code>         |
| <code>*</code>  | Mnożenie                    | <code>10 * 5</code> | <code>50</code>        |
| <code>/</code>  | Dzielenie                   | <code>10 / 3</code> | <code>3.3333...</code> |
| <code>%</code>  | Reszta z dzielenia (modulo) | <code>10 % 3</code> | <code>1</code>         |
| <code>**</code> | Potęgowanie                 | <code>2 ** 3</code> | <code>8</code>         |
| <code>++</code> | Inkrementacja               | <code>x++</code>    | Zwiększa o 1           |
| <code>--</code> | Dekrementacja               | <code>x--</code>    | Zmniejsza o 1          |

```
const a = 17;
const b = 5;

console.log(a + b); // 22
console.log(a - b); // 12
console.log(a * b); // 85
console.log(a / b); // 3.4
```

```
console.log(a % b); // 2 (reszta z dzielenia 17 przez 5)
console.log(a ** b); // 1419857 (17 do potęgi 5)
```

### Operatory przypisania skróconego:

```
let x = 10;
x += 5; // x = 15 (x = x + 5)
x -= 3; // x = 12
x *= 2; // x = 24
x /= 4; // x = 6
x %= 4; // x = 2
```

### Operator `+` z tekstem — konkatencja (łączenie stringów):

```
// Gdy jeden z operandów jest stringiem, + łączy teksty
"Cześć" + " " + "świecie" // "Cześć świecie"
"Wiek: " + 25 // "Wiek: 25" (liczba zamieniona na string)
5 + "3" // "53" (nie 8! - string wygrywa)
5 + 3 + " zł" // "8 zł" (najpierw 5+3=8, potem 8+" zł")
"Cena: " + 5 + 3 // "Cena: 53" (od lewej: string+5="Cena: 5", potem +"3")

// Dlatego w React preferujemy template stringi:
const wynik = `Cena: ${5 + 3} zł`; // "Cena: 8 zł" - jednoznaczne
```

## 3.4. Operatory porównania

W warunkach renderowania używaj `===` i `!==`. Luźne porównanie potrafi ukryć błąd typu, np. gdy identyfikator z formularza jest tekstem, a identyfikator w danych jest liczbą.

```
const selectedId = Number(event.target.value);
const selected = users.find((user) => user.id === selectedId);
```

| OPERATOR           | ZNACZENIE                           | PRZYKŁAD               | WYNIK              |
|--------------------|-------------------------------------|------------------------|--------------------|
| <code>===</code>   | Równe (ściśle, bez konwersji typów) | <code>5 === 5</code>   | <code>true</code>  |
| <code>!==</code>   | Różne (ściśle)                      | <code>5 !== "5"</code> | <code>true</code>  |
| <code>=</code>     | Równe (z konwersją typów)           | <code>5 = "5"</code>   | <code>true</code>  |
| <code>!=</code>    | Różne (z konwersją typów)           | <code>5 != "5"</code>  | <code>false</code> |
| <code>&lt;</code>  | Mniejsze                            | <code>3 &lt; 5</code>  | <code>true</code>  |
| <code>&gt;</code>  | Większe                             | <code>5 &gt; 3</code>  | <code>true</code>  |
| <code>&lt;=</code> | Mniejsze lub równe                  | <code>5 &lt;= 5</code> | <code>true</code>  |
| <code>&gt;=</code> | Większe lub równe                   | <code>3 &gt;= 5</code> | <code>false</code> |

**Ważna zasada:** W React (i nowoczesnym JavaScriptcie) **zawsze używaj** `===` i `!==` (ściśle porównanie). Operator `=` dokonuje automatycznej konwersji typów, co prowadzi do nieoczywistych wyników:

```
// Ścisłe porównanie (rekomendowane)
console.log(5 === 5);    // true
console.log(5 === "5");  // false (różne typy: number vs string)

// Luźne porównanie (unikaj)
console.log(5 = "5");    // true (JavaScript konwertuje string na number)
console.log(0 = false);  // true (nieintuicyjne)
console.log("" = false); // true (nieintuicyjne)
```

### 3.5. Operatory logiczne

`&&` jest wygodne w JSX, ale pamiętaj, że zwraca wartość, a nie zawsze `true` lub `false`. Szczególnie uważaj na liczbę `0`, bo może zostać wyrenderowana na stronie.

```
{items.length > 0 && <ProductList items={items} />}
```

| OPERATOR                | NAZWA         | PRAWDA GDY...                                     |
|-------------------------|---------------|---------------------------------------------------|
| <code>&amp;&amp;</code> | AND (i)       | Oba warunki są <code>true</code>                  |
| <code>  </code>         | OR (lub)      | Przynajmniej jeden warunek jest <code>true</code> |
| <code>!</code>          | NOT (negacja) | Warunek jest <code>false</code>                   |

```
const wiek = 20;
const maBilet = true;

// AND – oba warunki muszą być prawdziwe
if (wiek ≥ 18 && maBilet) {
  console.log("Możesz wejść");
}

// OR – wystarczy jeden prawdziwy warunek
const x = 5;
if (x < 0 || x > 100) {
  console.log("Poza zakresem");
}

// NOT – odwrócenie wartości logicznej
const zamkniety = false;
if (!zamkniety) {
  console.log("Sklep jest otwarty");
}
```

#### Short-circuit evaluation (skrótowe obliczanie):

Operatory `&&` i `||` nie zawsze zwracają `true` / `false` — zwracają jedną z wartości:

```
// && zwraca PIERWSZĄ wartość falsy LUB ostatnią wartość
"Jan" && "Kowalski" // "Kowalski" (obie truthy → zwraca ostatnią)
"" && "Kowalski"    // "" (pierwsza falsy → zwraca ją)
```



```

null && "cokolwiek" // null

// || zwraca PIERWSZĄ wartość truthy LUB ostatnią wartość
"" || "domyślny" // "domyślny" (pierwsza falsy → szuka dalej)
"Jan" || "domyślny" // "Jan" (pierwsza truthy → zwraca ją)
null || undefined || "ostatni" // "ostatni"

// Praktyczne użycie – wartość domyślna
const imieUsera = pobraneImie || "Anonim";

```

### 3.6. Template stringi (szablony napisów)

Template stringi są bardzo przydatne do klas CSS i komunikatów, ale nie buduj nimi nieczytelnych bloków logiki. Jeśli klasa zależy od wielu warunków, przygotuj zmienną przed `return`.

```

const buttonClass = `btn ${primary ? "btn-primary" : "btn-outline-secondary"}`;
return <button className={buttonClass}>Zapisz</button>;

```

Template stringi to sposób wstawiania zmiennych i wyrażeń do tekstu. Używają **odwrotnych apostrofów** (backtick) ``` zamiast cudzysłowów. Zmienne wstawiamy w `${}`:

```

const imie = "Jan";
const wiek = 25;

// Template string – wstawianie zmiennych
const powitanie = `Cześć, ${imie}! Masz ${wiek} lat.`;
console.log(powitanie); // "Cześć, Jan! Masz 25 lat."

// Wyrażenia w ${}
console.log(`Za 5 lat będziesz mieć ${wiek + 5} lat.`);
// "Za 5 lat będziesz mieć 30 lat."

// Wieloliniowy tekst
const tekst = `Linia pierwsza
Linia druga
Linia trzecia`;

// Wywołanie funkcji i metod w ${}
const imieInput = " jan ";
const msg = `Witaj, ${imieInput.trim().toUpperCase()}!`; // "Witaj, JAN!"

// Warunek (ternary) w template stringu
const status = `Użytkownik jest ${wiek ≥ 18 ? "pełnoletni" : "niepełnoletni"}`;

// Dynamiczne klasy CSS w React (bardzo częste!)
const klasa = `btn btn-${aktywny ? "success" : "danger"} ${duży ? "btn-lg" : ""}`;

```

### 3.7. Instrukcja warunkowa if / else if / else

Klasyczny `if` jest najlepszy, gdy komponent ma zwrócić całkowicie inny widok: loading, błąd albo brak danych. Użyj go przed `return`, a nie bezpośrednio w JSX.

```
if (loading) return <p>Ładowanie...</p>;  
if (error) return <p>Nie udało się pobrać danych.</p>;  
return <UserList users={users} />;
```

Instrukcja `if` pozwala programowi podejmować decyzje. Działa jak "rozwidlenie dróg" – kod pójdzie w jedną stronę, jeśli warunek jest spełniony (`true`), a w drugą, jeśli nie jest (`false`).

```
const wiek = 17;  
  
if (wiek ≥ 18) {  
  // Ten kod wykona się TYLKO, jeśli wiek jest większy lub równy 18  
  console.log("Jesteś pełnoletni, możesz wejść.");  
} else if (wiek ≥ 13) {  
  // Jeśli pierwszy warunek (wiek ≥ 18) zawiodł, sprawdzany jest ten  
  console.log("Jesteś nastolatkiem, dostęp ograniczony.");  
} else {  
  // Jeśli żaden z powyższych warunków nie jest prawdziwy, wykona się 'else'  
  console.log("Jesteś dzieckiem.");  
}
```

**Kluczowa zasada dotycząca `if` w JSX:** W Reakcie funkcja komponentu musi w bloku `return (...)` zwrócić kod JSX (przypominający HTML). **Nie można wstawiać klasycznego bloku `if() { ... }` wewnątrz `return`!** Jeśli chcesz użyć tradycyjnego `if`, musisz to zrobić **przed** słowem kluczowym `return`.

```
// POPRAWNIE: użycie if przed return  
function MojaStrona({ czyZalogowany }) {  
  if (czyZalogowany === true) {  
    return <h1>Witaj w systemie!</h1>;  
  } else {  
    return <h1>Zaloguj się, by zobaczyć treść.</h1>;  
  }  
}
```

### 3.8. Operator trójargumentowy (ternary)

Operator trójargumentowy najlepiej nadaje się do wyboru jednego z dwóch krótkich wariantów. Jeżeli w obu gałęziach masz duże fragmenty JSX, czytelniej będzie wydzielić osobne komponenty.

```
<span>{isAdmin ? "Administrator" : "Użytkownik"}</span>
```

Operator trójargumentowy to skrócona forma `if/else`. Składa się z trzech części: `warunek ? wartość_dlaTrue : wartość_dlaFalse`.

```
const wiek = 20;  
const status = wiek ≥ 18 ? "Pełnoletni" : "Niepełnoletni";  
console.log(status); // "Pełnoletni"
```

Jest niezwykle często używany w JSX do warunkowego wyświetlania:

```
<p>{czyZalogowany ? "Witaj ponownie!" : "Zaloguj się"}</p>
```

### 3.9. Funkcje — deklaracja i wyrażenie

Funkcje deklarowane poza komponentem nie są tworzone ponownie przy każdym renderze. To dobre miejsce dla czystej logiki, np. formatowania ceny albo obliczania sumy.

```
function formatPrice(value) {  
  return `${value.toFixed(2)} zł`;  
}
```

Funkcja to nazwany fragment kodu, który "wykonuje jakąś pracę" i może być wywoływany wielokrotnie. Możesz o niej myśleć jak o maszynie – wrzucasz do niej jakieś składniki (parametry), ona robi coś z nimi w środku, a następnie "wypluwa" wynik (przy pomocy `return`). W React każdy komponent jest właśnie taką funkcją!

W JavaScriptcie funkcje można definiować na dwa sposoby:

```
// 1. Deklaracja funkcji (function declaration)  
function dodaj(a, b) {  
  return a + b;  
}  
  
// 2. Wyrażenie funkcyjne (function expression)  
const pomnoz = function (a, b) {  
  return a * b;  
};  
  
// Wywołanie  
console.log(dodaj(3, 5)); // 8  
console.log(pomnoz(3, 5)); // 15
```

**Parametry domyślne:**

```
function powitaj(imie = "Gościu") {  
  return `Cześć, ${imie}!`;  
}  
  
console.log(powitaj("Anna")); // "Cześć, Anna!"  
console.log(powitaj()); // "Cześć, Gościu!"
```

**Funkcja bez `return` — zwraca `undefined`:**

```
function wyswietl(tekst) {  
  console.log(tekst);  
  // Brak return — funkcja zwraca undefined  
}  
  
const wynik = wyswietl("Cześć"); // wynik = undefined
```

## Wiele wartości — zwracanie obiektu lub tablicy:

```
function obliczStatystyki(liczby) {  
  const suma = liczby.reduce((a, b) => a + b, 0);  
  const srednia = suma / liczby.length;  
  return { suma, srednia }; // Zwracamy obiekt z wieloma wartościami  
}  
  
const { suma, srednia } = obliczStatystyki([10, 20, 30]);
```

### 3.10. Funkcje strzałkowe (arrow functions)

Funkcje strzałkowe często pojawiają się w handlerach. Jeśli musisz przekazać argument, owiń wywołanie w funkcję, żeby nie uruchomić go podczas renderowania.

```
<button onClick={() => removeProduct(product.id)}>Usuń</button>
```

Funkcje strzałkowe to krótsza składnia funkcji, bardzo popularna w React:

```
// Pełna forma funkcji strzałkowej  
const dodaj = (a, b) => {  
  return a + b;  
};  
  
// Skrócona forma — jeśli ciało to jedno wyrażenie, nie trzeba {} i return  
const podwoj = (x) => x * 2;  
  
// Dla jednego parametru nawiasy () są opcjonalne  
const kwadrat = x => x * x;  
  
// Bez parametrów — puste nawiasy obowiązkowe  
const losuj = () => Math.random();
```

## Dlaczego strzałki są ważne w React:

```
// Obsługa zdarzenia — funkcja strzałkowa inline  
<button onClick={() => setLicznik(licznik + 1)}>Dodaj</button>  
  
// Renderowanie listy — strzałka w map  
{kursy.map((kurs) => <li key={kurs}>{kurs}</li>)}  
  
// Filtrowanie — strzałka w filter  
{produkty.filter((p) => p.cena < 100).map((p) => <p key={p.id}>{p.nazwa}</p>)}
```

## Parametr rest ( `...` ) — zbieranie wielu argumentów w tablicę:

```
// Rest parameter zbiera "resztę" argumentów do tablicy  
function suma(...liczby) {  
  return liczby.reduce((acc, n) => acc + n, 0);  
}
```

```

}

suma(1, 2, 3)      // 6
suma(10, 20, 30, 40) // 100

// Połączenie zwykłych parametrów z rest
function log(poziom, ...wiadomosci) {
  wiadomosci.forEach(msg => console.log(`[${poziom}] ${msg}`));
}

log("INFO", "Start", "Połączono", "Gotowe");

```

### 3.11. Tablice — tworzenie i podstawowe metody

Konstrukcję „Tablice — tworzenie i podstawowe metody” najlepiej przećwiczyć na danych, które realnie trafiają do komponentu: tekście z inputa, obiekcie użytkownika albo tablicy produktów. Przygotuj wynik w JavaScriptcie przed JSX, a w widoku zostaw tylko odczyt gotowej wartości.

Tablice (arrays) to uporządkowane kolekcje elementów. W React są fundamentalne — listy, karty, formularze wieloelementowe — wszystko opiera się na tablicach.

```

// Tworzenie tablicy
const kursy = ["HTML", "CSS", "JavaScript", "React"];

// Dostęp do elementów (indeksowanie od 0)
console.log(kursy[0]);    // "HTML"
console.log(kursy[3]);    // "React"
console.log(kursy.length); // 4

// Dodawanie elementu na koniec
kursy.push("Node.js");

// Usuwanie ostatniego elementu
kursy.pop();

// Dodawanie elementu na początek
kursy.unshift("Wstęp do IT");

// Usuwanie pierwszego elementu
kursy.shift();

// Sprawdzenie, czy element istnieje
console.log(kursy.includes("React")); // true
console.log(kursy.indexOf("CSS"));    // 1

// Iteracja przez tablicę
for (let i = 0; i < kursy.length; i++) {
  console.log(`${i + 1}. ${kursy[i]}`);
}

// Iteracja przez for...of
for (const kurs of kursy) {
  console.log(kurs);
}

```

3.12. Metody tablic kluczowe w React — map, filter, find, reduce

Te cztery metody są podstawą większości interfejsów opartych na danych.

| METODA              | ZWRACA                                    | TYPOWE UŻYCIE W REACT                     |
|---------------------|-------------------------------------------|-------------------------------------------|
| <code>map</code>    | nową tablicę                              | renderowanie listy elementów              |
| <code>filter</code> | nową, krótszą tablicę                     | wyszukiwarka, kategorie, aktywne elementy |
| <code>find</code>   | jeden element albo <code>undefined</code> | szczegóły wybranego rekordu               |
| <code>reduce</code> | dowolną wartość                           | suma, grupowanie, statystyki              |

W React niemal wszystkie operacje na listach elementów opierają się na czterech podstawowych metodach wbudowanych w JavaScript. Co kluczowe, w przypadku Reacta zależy nam na "niemutowalności" (immutability), dlatego każda z wymienionych metod **nie modyfikuje oryginalnej tablicy, ale zwraca zupełnie nową**.

Dzięki temu React jest w stanie poprawnie zauważyć zmianę w danych i odświeżyć widok. Unikać należy używania metod klasycznych, jak `push()` czy `splice()`, które ukradkiem zmieniają tablicę bez powiadamiania Reacta.

1. `MAP()` — TRANSFORMACJA ELEMENTÓW I GENEROWANIE LIST HTML

Metoda `map()` pozwala "przejsć" przez każdy element tablicy wejściowej i przekształcić go, jednocześnie tworząc nową tablicę o dokładnie tej samej długości. Jej najczęstszym, kluczowym zadaniem jest generowanie zestawu znaczników (np. `<li>` lub niestandardowych komponentów) na podstawie danych.

```
const liczby = [1, 2, 3, 4, 5];

// Starszy zapis z klasyczną definicją funkcji
const podwojone = liczby.map(function(liczba) {
  return liczba * 2;
}); // Wynik: [2, 4, 6, 8, 10]

// Nowoczesny i preferowany w React zapis strzałkowy:
const potrojone = liczby.map(liczba => liczba * 3);

// WYKORZYSTANIE PRAKTYCZNE: Pobieranie atrybutu (np. nazwy) z każdego obiektu
const produkty = [
  { id: 1, nazwa: "Klawiatura", cena: 200 },
  { id: 2, nazwa: "Myszka", cena: 100 }
];

const zmapowaneNazwy = produkty.map(produkt => produkt.nazwa);
console.log(zmapowaneNazwy); // ["Klawiatura", "Myszka"]
```

Gdy w komponencie React wykorzystujesz funkcję `map()` do generowania znaczników z użyciem kodu JSX/HTML, **masz absolutny obowiązek przypisać do powielanego wrappera atrybut nazwany `key`**. To jedyny sposób, w jaki mechanizmy optymalizacji Reacta potrafią śledzić powtarzalność.

```
function ListaZadan() {
  const listaZadan = ["Zrobić zakupy", "Odebrać paczkę", "Umyć auto"];

  return (
    <ul>
      {listaZadan.map(zadanie => (
        // Zawsze ustawiamy unikalny key! Najlepiej identyfikator (id) lub ciąg znaków, jeśli jest unikalny.

```

```

        <li key={zadanie} className="zadanie">
            {zadanie}
        </li>
    )}
</ul>
);
};

```

## 2. FILTER() — ODSIEWANIE OBIEKTÓW Z KOLEKCJI

Metoda `filter()` wyciąga z elementu wszystkie wpisy, które spełnią Twój dany warunek i umieszcza je w nowo zwracanej powłocie tablicowej. W instrukcji strzałkowej `filter()` określasz logiczny test do sprawdzenia (gdzie wynik prawdziwy `true` zostawia element).

```

const uzytkownicy = [
  { id: 1, imie: "Anna", wiek: 22 },
  { id: 2, imie: "Marek", wiek: 35 },
  { id: 3, imie: "Piotr", wiek: 15 }
];

// Zostawienie pełnoletnich:
const dorosli = uzytkownicy.filter(osoba => osoba.wiek >= 18);
// Zostaną Anna i Marek.

// KLUCZOWY WZORZEC DLA REACT: "Usuwanie" rzeczy używając filter() w Stanie.
// Chcąc np. usunąć usera o id = 2, nie stosujemy splice().
// Pozostawiamy po prostu wszystkich tych, którzy takiego id NIE MAJĄ:
const idDoUsuniecia = 2;
const aktywniUzytkownicy = uzytkownicy.filter(osoba => osoba.id !== idDoUsuniecia);

```

## 3. FIND() — ZWRÓCENIE PIERWSZEGO PASUJĄCEGO WYNIKU

`find()` operuje podobnie jak `filter()`, jednak zamiast sprawdzać do końca i zwracać wyselekcjonowaną tablicę wyników – `find()` zwraca wprost **jeden obiekt**. W momencie, w którym pierwszy obiekt spotka się z wynikiem `true`, cała operacja jest przerywana z celowo odnalezioną pozycją.

```

const powiadomienia = [
  { pId: 101, typ: "Info", tresc: "Witaj" },
  { pId: 105, typ: "Ostrzezenie", tresc: "Błąd serwera" }
];

// Odszukujemy pojedynczy element:
const pierwszyBłąd = powiadomienia.find(element => element.typ === "Ostrzezenie");

console.log(pierwszyBłąd);
// Zawiera tylko sam odzyskany obiekt: { pId: 105, typ: "Ostrzezenie", tresc: "Błąd serwera" }

```

## 4. REDUCE() — ANALIZA CAŁEJ TABLICZY W JEDEN ZWARTY WYNIK

Metoda `reduce()` jest kluczowa tam, gdzie np. z wieloelementowego koszyka w e-commerce musisz wyciągnąć jedną ostateczną cyfrę: "Suma do Zapłaty". Agreguje ona kolejne wiersze przekształcając je za pomocą pętli. Pętla wymusza zdefiniowanie minimum dwóch parametrów. Np. "Akumulatora" (oznaczającego ułamek sumy przechodzący rosnąco dalej z pętli do pętli) oraz reprezentanta "Bieżącej Pozycji" (odczytywanej przez krok w tablicy).

W argumencie numer dwa (dodawanym zawsze po deklaracji skomplikowanej funkcji liczącej) ustalamy stan startowy licznika: zazwyczaj `0`.

```
const koszykKosmetykow = [
  { nazwa: "Krem", cena: 45, ilosc: 2 },
  { nazwa: "Balsam", cena: 15, ilosc: 1 },
  { nazwa: "Szampon", cena: 20, ilosc: 3 }
];

// Sprawny kalkulator:
const sumaCalkowita = koszykKosmetykow.reduce((akumulatorKosztow, aktualnyProdukt) => {

  // Obliczamy cenę cząstkową danej sekcji koszyka wejściowej:
  const cenaJednostkowaRazySztuki = aktualnyProdukt.cena * aktualnyProdukt.ilosc;

  // Podwajamy i zwracamy "narośnięte saldo" przekazując kolejnemu obiegowi nową wagę
  return akumulatorKosztow + cenaJednostkowaRazySztuki;

}, 0); // Kasa przed zakupami z zerowym stanem wyniosła ZERO zł.

console.log(sumaCalkowita); // Ujrzysz całkowitą wyciągniętą pojedynczą liczbę finalną: 165.
```

### 3.13. Obiekty

Obiekty są podstawowym sposobem modelowania danych w aplikacji: użytkownik, produkt, zadanie albo ustawienia formularza. W JSX najczęściej odczytujesz pola przez kropkę, np. `user.name`, ale przy brakujących danych warto używać optional chaining, np. `user.address?.city`.

Obiekty to kolekcje par klucz-wartość. Są podstawą modelowania danych w React:

```
// Tworzenie obiektu
const zdjecie = {
  id: 1,
  nazwa: "kwiat.jpg",
  kategoria: "kwiaty",
  pobrania: 0,
};

// Dostęp do pól – notacja kropkowa
console.log(zdjecie.nazwa); // "kwiat.jpg"
console.log(zdjecie.kategoria); // "kwiaty"

// Dostęp do pól – notacja nawiasowa (przydatna z dynamicznym kluczem)
const pole = "kategoria";
console.log(zdjecie[pole]); // "kwiaty"

// Modyfikacja pola
zdjecie.pobrania = 5;

// Dodanie nowego pola
zdjecie.autor = "Jan Kowalski";
```



```
// Sprawdzenie, czy pole istnieje
console.log("nazwa" in zdjecie); // true
```

## Metody obiektu — Object.keys, Object.values, Object.entries:

```
const osoba = { imie: "Jan", wiek: 25, miasto: "Kraków" };

// Tablica kluczy
Object.keys(osoba) // ["imie", "wiek", "miasto"]

// Tablica wartości
Object.values(osoba) // ["Jan", 25, "Kraków"]

// Tablica par [klucz, wartość]
Object.entries(osoba) // [["imie", "Jan"], ["wiek", 25], ["miasto", "Kraków"]]

// Liczba pól w obiekcie
Object.keys(osoba).length // 3

// Iteracja po obiekcie
Object.entries(osoba).forEach(([klucz, wartosc]) => {
  console.log(`${klucz}: ${wartosc}`);
});

// Sprawdzenie czy obiekt jest pusty
const pusty = {};
Object.keys(pusty).length === 0 // true
```

## Tablica obiektów — najczęstsza struktura danych w React:

```
const filmy = [
  { id: 1, tytuł: "Matrix", rodzaj: "Sci-Fi", rok: 1999 },
  { id: 2, tytuł: "Incepcja", rodzaj: "Sci-Fi", rok: 2010 },
  { id: 3, tytuł: "Titanic", rodzaj: "Dramat", rok: 1997 },
];

// Filtrowanie po rodzaju
const scifi = filmy.filter((f) => f.rodzaj === "Sci-Fi");

// Mapowanie do listy tytułów
const tytuły = filmy.map((f) => f.tytuł);
// ["Matrix", "Incepcja", "Titanic"]
```

### 3.14. Destrukturyzacja tablic i obiektów

Konstrukcję „Destrukturyzacja tablic i obiektów” najlepiej przećwiczyć na danych, które realnie trafiają do komponentu: tekście z inputa, obiekcie użytkownika albo tablicy produktów. Przygotuj wynik w JavaScriptcie przed JSX, a w widoku zostaw tylko odczyt gotowej wartości.

Destrukturyzacja to sposób na „wyciągnięcie” wartości z tablicy lub obiektu do osobnych zmiennych. Jest niezwykle często używana w React.

#### Destrukturyzacja obiektów:

```
const film = { tytuł: "Incepcja", rodzaj: "Sci-Fi", rok: 2010 };

// Zamiast:
// const tytuł = film.tytuł;
// const rodzaj = film.rodzaj;

// Destrukturyzacja:
const { tytuł, rodzaj, rok } = film;
console.log(tytuł); // "Incepcja"
console.log(rodzaj); // "Sci-Fi"
console.log(rok); // 2010
```

### Destrukturyzacja tablic:

```
const kolory = [120, 80, 200];
const [r, g, b] = kolory;
console.log(r); // 120
console.log(g); // 80
console.log(b); // 200
```

### Destrukturyzacja w React — useState:

```
// useState zwraca tablicę [wartość, funkcjaZmieniająca]
// Destrukturyzacja wyciąga oba elementy
const [licznik, setLicznik] = useState(0);
const [imie, setImie] = useState("");
const [aktywny, setAktywny] = useState(false);
```

### Destrukturyzacja props:

```
// Zamiast function Karta(props) { ... props.tytuł ... }
function Karta({ tytuł, opis, cena }) {
  return (
    <div>
      <h3>{tytuł}</h3>
      <p>{opis}</p>
      <p>Cena: {cena} zł</p>
    </div>
  );
}
```

### Alias (zmiana nazwy) przy destrukturyzacji:

```
const osoba = { imie: "Jan", wiek: 25 };
const { imie: firstName, wiek: age } = osoba;
console.log(firstName); // "Jan"
console.log(age); // 25
```

### Wartości domyślne w destruktuzyzacji:

```
const { imie, miasto = "Nieznane" } = { imie: "Jan" };
console.log(miasto); // "Nieznane" (bo nie było w obiekcie)

const [a = 0, b = 0, c = 0] = [10, 20];
console.log(c); // 0 (bo tablica miała tylko 2 elementy)
```

### Zagnieżdżona destruktuzyzacja:

```
const user = {
  imie: "Jan",
  adres: { miasto: "Kraków", kod: "30-001" }
};

const { imie, adres: { miasto, kod } } = user;
console.log(miasto); // "Kraków"
console.log(kod);    // "30-001"
```

### Pomijanie elementów tablicy:

```
const [, drugi, , czwarty] = [10, 20, 30, 40];
console.log(drugi);    // 20
console.log(czwarty); // 40
```

### 3.15. Operator spread (...)

Spread tworzy płytką kopię. To wystarczy przy prostych tablicach i obiektach, ale przy zagnieżdżeniach trzeba skopiować każdy poziom, który zmieniasz.

```
const nextUser = {
  ...user,
  address: { ...user.address, city: "Gdańsk" },
};
```

Operator spread ( `...` ) „rozkłada” tablicę lub obiekt na poszczególne elementy. Jest kluczowy w React do **niemutowalnej aktualizacji stanu**:

### Spread tablicy:

```
const stare = [1, 2, 3];

// Kopia tablicy
const kopia = [...stare];

// Dodanie elementu na koniec (zamiast push)
const nowa = [...stare, 4];
// [1, 2, 3, 4]

// Dodanie elementu na początek
```

```
const nowa2 = [0, ...stare];  
// [0, 1, 2, 3]  
  
// Łączenie tablic  
const a = [1, 2];  
const b = [3, 4];  
const polaczone = [...a, ...b];  
// [1, 2, 3, 4]
```

### Spread obiektu:

```
const stary = { imie: "Jan", wiek: 25 };  
  
// Kopia obiektu  
const kopia = { ...stary };  
  
// Kopia z modyfikacją jednego pola  
const zaktualizowany = { ...stary, wiek: 26 };  
// { imie: "Jan", wiek: 26 }  
  
// Kopia z dodaniem nowego pola  
const rozszerzony = { ...stary, miasto: "Kraków" };  
// { imie: "Jan", wiek: 25, miasto: "Kraków" }
```

### W React — niemutowalna aktualizacja stanu:

```
// Dodanie elementu do tablicy stanu  
setZadania((prev) => [...prev, noweZadanie]);  
  
// Usunięcie elementu z tablicy stanu  
setZadania((prev) => prev.filter((z) => z.id !== idDoUsuniecia));  
  
// Aktualizacja jednego pola obiektu w stanie  
setFormularz((prev) => ({ ...prev, imie: "Anna" }));
```

### Uwaga — spread tworzy PŁYTKĄ kopię (shallow copy):

Spread kopiuje tylko pierwszy poziom. Zagnieżdżone obiekty nadal są współdzielone:

```
const original = { imie: "Jan", adres: { miasto: "Kraków", kod: "30-001" } };  
const kopia = { ...original };  
  
kopia.imie = "Anna"; // OK – nie zmienia oryginału  
kopia.adres.miasto = "Warszawa"; // UWAGA – zmienia też oryginał!  
  
// Głęboka kopia zagnieżdżonego obiektu:  
const gleboka = { ...original, adres: { ...original.adres } };  
gleboka.adres.miasto = "Gdańsk"; // Teraz oryginał jest bezpieczny
```

```
// Alternatywa – structuredClone (nowoczesne przeglądarki)
const pełnaKopia = structuredClone(original);
```

### 3.16. Import i export modułów

Konstrukcję „Import i export modułów” najlepiej przećwiczyć na danych, które realnie trafiają do komponentu: tekście z inputa, obiekcie użytkownika albo tablicy produktów. Przygotuj wynik w JavaScriptcie przed JSX, a w widoku zostaw tylko odczyt gotowej wartości.

Podział kodu na mniejsze pliki (moduły) to fundament pracy z Reactem. Zamiast pisać tysiące linii w jednym pliku `App.js`, wyodrębniamy komponenty, dane i funkcje do osobnych plików, a następnie używamy mechanizmów `export` i `import`, by je ze sobą łączyć.

**1. Export domyślny (Default Export)** Używany najczęściej do eksportowania głównego komponentu z pliku. W jednym pliku może być tylko jeden export domyślny.

```
// Plik: src/components/Header.js
function Header() {
  return <header>Witaj na stronie</header>;
}

// Zazwyczaj na samym dole pliku
export default Header;
```

Importowanie tego pliku:

```
// Plik: src/App.js
// Ważne: Możesz nadać dowolną nazwę podczas importu z funkcji default!
import Header from "./components/Header";
import MojNaglowek from "./components/Header"; // zadziała tak samo!
```

**2. Export nazwany (Named Export)** Używany, gdy z jednego pliku chcemy wyeksportować wiele rzeczy (np. paczka różnych funkcji matematycznych, stałych wartości lub mini-komponentów).

```
// Plik: src/utils/matematyka.js
export function dodaj(a, b) { return a + b; }
export function odejmij(a, b) { return a - b; }
export const WERSJA_API = "1.0.0";
```

Importowanie:

```
// Plik: src/App.js
// Ważne: Przy eksporcie nazwanym musisz użyć dokładnie tych samych nazw wewnątrz nawiasów klamrowych {}.
import { dodaj, odejmij, WERSJA_API } from "./utils/matematyka";

// Możesz jednak nałożyć alias (zmianę nazwy), jeśli nazwa koliduje w obecnym pliku:
import { dodaj as dodajLiczby } from "./utils/matematyka";
```

**3. Co jeszcze można importować w React?** W ekosystemie React za pomocą instrukcji `import` możemy wciągać nie tylko JavaScript! Narzędzia takie jak Webpack czy Vite pozwalają na:

- **Import CSS:** `import "./styles.css";` (aby podłączyć style globalnie dla komponentu)

- **Import Obrazów:** `import logoImg from "../logo.png";` (daje nową powiastkę ze ścieżką do grafiki, co uodparnia na błędy ścieżek względem folderu public!)
- **Import bibliotek:** `import "bootstrap/dist/css/bootstrap.css";`

### 3.17. Konwersje typów

Konstrukcję „Konwersje typów” najlepiej przećwiczyć na danych, które realnie trafiają do komponentu: tekście z inputa, obiekcie użytkownika albo tablicy produktów. Przygotuj wynik w JavaScriptcie przed JSX, a w widoku zostaw tylko odczyt gotowej wartości.

W JavaScript ważne jest rozumienie konwersji typów, szczególnie przy formularzach:

```
// String → Number
const tekst = "42";
const liczba = Number(tekst);           // 42
const liczba2 = parseInt(tekst);        // 42 (tylko całkowite)
const liczba3 = parseFloat("3.14");     // 3.14
const liczba4 = +"42";                   // 42 (skrótowa konwersja)

// Number → String
const num = 42;
const str = String(num);                 // "42"
const str2 = num.toString();              // "42"

// Uwaga na NaN – Not a Number
console.log(Number("abc"));               // NaN
console.log(isNaN(Number("abc")));        // true

// Konwersja na Boolean
Boolean(0)           // false
Boolean("")          // false
Boolean(null)        // false
Boolean(undefined)   // false
Boolean("tekst")     // true
Boolean(42)          // true
Boolean([])          // true (pusta tablica to truthy!)
!!""                 // false (podwójna negacja – skrótowa konwersja na boolean)

// Zaokrąglanie
console.log(Math.round(3.7));             // 4
console.log(Math.floor(3.7));             // 3 (zaokrąglenie w dół)
console.log(Math.ceil(3.2));              // 4 (zaokrąglenie w górę)
console.log(3.14159.toFixed(2));          // "3.14" (zwraca string!)
```

**Ważne w formularzu React:** `input` zawsze zwraca wartość jako `string`. Jeżeli potrzebujesz `number`, musisz go jawnie skonwertować:

```
// e.target.value to ZAWSZE string, nawet dla input type="number"
const handleChange = (e) => {
  setWiek(Number(e.target.value)); // Konwersja na liczbę
};
```

### 3.18. Metody napisów

Metody napisów są praktyczne przy wyszukiwarkach i walidacji. Najczęstszy zestaw to `trim()` do usunięcia spacji, `toLowerCase()` do porównywania bez wielkości liter i `includes()` do filtrowania wyników.

Metody napisów (stringów) są często potrzebne przy walidacji formularzy i przetwarzaniu tekstu:

```
const tekst = "  Witaj w React!  ";

// Usunięcie białych znaków z początku i końca
console.log(tekst.trim());           // "Witaj w React!"

// Zamiana na wielkie/małe litery
console.log(tekst.trim().toUpperCase()); // "WITAJ W REACT!"
console.log(tekst.trim().toLowerCase()); // "witaj w react!"

// Sprawdzenie zawartości
console.log("email@example.com".includes("@")); // true
console.log("abc123".startsWith("abc"));        // true
console.log("plik.pdf".endsWith(".pdf"));        // true

// Wycinanie fragmentu
const imie = "Jan Kowalski";
console.log(imie.slice(0, 3));    // "Jan"
console.log(imie.slice(4));       // "Kowalski"

// Podział tekstu na tablicę
const csv = "Jan,Kowalski,25";
const czesci = csv.split(",");
// ["Jan", "Kowalski", "25"]

// Zamiana fragmentu
const nowy = "Cześć świecie".replace("świecie", "React");
// "Cześć React"

// Zamiana WSZYSTKICH wystąpień
"la la la".replaceAll("la", "da"); // "da da da"

// Długość napisu
console.log("React".length); // 5

// Powtórzenie napisu
"ha".repeat(3) // "hahaha"

// Dopełnienie do określonej długości
"5".padStart(3, "0") // "005"
"42".padStart(5, " ") // "   42"
"hi".padEnd(5, ".")   // "hi..."

// Dostęp do znaku
"React".charAt(0)      // "R"
"React".at(-1)         // "t" (od końca)

// Kod znaku (przydatne w szyfrach, np. Cezara)
```

```
"A".charCodeAt(0) // 65
String.fromCharCode(65) // "A"
```

### 3.19. Truthy i falsy

Truthy i falsy wpływają bezpośrednio na warunki w JSX. Pusty string, `0`, `null`, `undefined` i `false` zachowują się inaczej niż zwykły tekst czy niepusta tablica. Dlatego przy listach lepiej pisać `items.length > 0` niż samo `items.length`.

W JavaScriptcie każda wartość może być potraktowana jako `true` (truthy) lub `false` (falsy) w kontekście logicznym. To kluczowe przy renderowaniu warunkowym w React.

Wartości falsy (traktowane jako `false`):

| WARTOŚĆ                        | TYP       |
|--------------------------------|-----------|
| <code>false</code>             | boolean   |
| <code>0</code>                 | number    |
| <code>-0</code>                | number    |
| <code>0n</code>                | BigInt    |
| <code>""</code> (pusty string) | string    |
| <code>null</code>              | null      |
| <code>undefined</code>         | undefined |
| <code>NaN</code>               | number    |

Wszystko inne jest truthy — w tym `" "` (spacja), `[]` (pusta tablica), `{}` (pusty obiekt).

```
// Przykłady
if ("React") {
  console.log("Prawda — niepusty string");
}

if (0) {
  console.log("Ten kod się NIE wykona — 0 jest falsy");
}

// W React — operator && do warunkowego renderowania
// {bład && <p className="text-danger">{bład}</p>}
// Jeśli bład jest "" (falsy), <p> się nie wyświetli
// Jeśli bład jest "Pole wymagane" (truthy), <p> się wyświetli
```

Uwaga na pułapkę z `0`:

```
// BŁĘDNE — jeśli items.length === 0, React wyświetli "0" na stronie
{items.length && <ul>...</ul>}

// POPRAWNE — jawna konwersja na boolean
{items.length > 0 && <ul>...</ul>}
```



### 3.20. Konsola przeglądarki — console.log()

Konstrukcję „Konsola przeglądarki — console.log()” najlepiej przećwiczyć na danych, które realnie trafiają do komponentu: tekście z inputa, obiekcie użytkownika albo tablicy produktów. Przygotuj wynik w JavaScriptcie przed JSX, a w widoku zostaw tylko odczyt gotowej wartości.

Zanim zaczniesz budować skomplikowane UI, musisz wiedzieć, w jaki sposób komunikować się z przepływem danych w aplikacji. Konsola przeglądarki (klawisz **F12** lub **Prawy przycisk → Zbadaj → zakładka Console**) to podstawowe narzędzie diagnostyczne. Pozwala Ci ona "zajrzeć pod maskę" każdego komponentu React.

**Główne stopnie powiadomień (Severity):**

```
console.log("Cześć, to podstawowa wiadomość informacyjna");
console.warn("Hej, uważaj - niegroźne, ale ważne ostrzeżenie! (wyświetla się na żółto)");
console.error("Błąd! Backend zawiódł! (wyświetla się na czerwono ze ścieżką błędów)");
console.info("Informacja o działaniu procesu");
```

**Złota zasada wstawiania logów w React:** Jeśli komponent ci nie działa i nie odświeża poprawnie danych... powinieneś wstawić **console.log()** zaraz **PO** zadeklarowaniu stanów i **PRZED** blokiem **return ()**. Odpali on zmienne wirtualnie zaraz przed nowym wyrenderowaniem ekranu!

```
import { useState } from "react";

function Koszyk() {
  const [ilosc, setIlosc] = useState(0);

  // GENIALNE do weryfikacji. Za każdym kliknięciem przycisku, zobaczysz nową wartość
  // bez odświeżania całej wielkiej i powolnej aplikacji.
  console.log("-- Koszyk renderuje się! Aktualna Ilość:", ilosc);

  return (
    <div>
      <button onClick={() => setIlosc(ilosc + 1)}>Dodaj Winyłową Płytę</button>
    </div>
  );
}
```

**Bardziej czytelne API: console.table()** Zwykły **console.log()** bywa trudny w obsłudze dla długich tablic lub obiektów. Używaj tablic:

```
const userzy = [
  { id: 10, nazwa: "Janek", rola: "Admin" },
  { id: 11, nazwa: "Krystyna", rola: "User" }
];

console.table(userzy);

// W terminalu F12 wytworzy to przepiękną, sortowalną tabelę ułatwiającą wgląd w atrybuty!
```

### 3.21. Instrukcja warunkowa switch

**switch** bywa czytelny, gdy masz kilka stanów widoku: **loading**, **success**, **error**, **empty**. W takim przypadku funkcja może zwrócić odpowiedni komponent dla każdego statusu, zamiast tworzyć długi łańcuch operatorów trójargumentowych.

Instrukcja **switch** to alternatywa dla wielu warunków **if / else if**. Jest często używana w reducerach (**useReducer**) lub gdy mamy wiele precyzyjnych wartości do sprawdzenia.

```
const rola = "ADMIN";

switch (rola) {
  case "ADMIN":
    console.log("Pełen dostęp");
    break; // Pamiętaj o break, inaczej wykona się też kolejny case!
  case "MODERATOR":
    console.log("Dostęp ograniczony");
    break;
  case "USER":
    console.log("Tylko odczyt");
    break;
  default:
    console.log("Rola nieznana");
}
```

### Użycie switch w React — renderowanie warunkowe:

```
function StatusIkona({ status }) {
  switch (status) {
    case "sukces":
      return <span className="text-success">✓ Gotowe</span>;
    case "blad":
      return <span className="text-danger">✗ Błąd</span>;
    case "ladowanie":
      return <span className="text-warning">⚡ Ładowanie... </span>;
    default:
      return <span className="text-muted">– Brak statusu</span>;
  }
}

// Użycie:
<StatusIkona status="sukces" />
```

### Switch z wieloma case'ami dla tej samej akcji (fall-through):

```
const dzien = new Date().getDay(); // 0 = niedziela, 6 = sobota

switch (dzien) {
  case 1:
  case 2:
  case 3:
  case 4:
  case 5:
    console.log("Dzień roboczy");
    break;
  case 0:
  case 6:
    console.log("Weekend!");
}
```

```
break;
}
```

### 3.22. Pętle (for, while, do...while) i iteracja

W React rzadko używa się pętli `for` bezpośrednio w JSX, ale przydają się do przygotowania danych przed renderowaniem. Do wyświetlania listy w widoku zwykle wybieraj `map()`, bo zwraca nową tablicę elementów JSX.

Chociaż w React zazwyczaj używamy metody `.map()` do renderowania list, klasyczne pętle wciąż są ważne w logice i algorytmach.

1. Pętla `for` - Używana gdy wiemy dokładnie, ile razy chcemy powtórzyć operację.

```
for (let i = 0; i < 5; i++) {
  console.log(`Wykonanie numer ${i}`);
}
```

2. Pętla `while` - Używana, gdy nie wiemy ile razy pętla się wykona, zależy to od jakiegoś warunku, który jest sprawdzany na początku.

```
let licznik = 0;
while (licznik < 3) {
  console.log("Licznik w while:", licznik);
  licznik++;
}
```

3. Pętla `do...while` - Wykona się ZAWSZE przynajmniej raz, ponieważ warunek sprawdzany jest na końcu.

```
let x = 10;
do {
  console.log("Zawsze wykona się przynajmniej raz");
} while (x < 5); // Warunek nie jest spełniony, pętla się kończy
```

4. Pętla `for...of` oraz `for...in`

- `for...of` - Najlepsze do iterowania po tablicach i stringach.
- `for...in` - Używane do iterowania po kluczach obiektów (choć w React częściej używa się `Object.keys()`).

```
const kolory = ["czerwony", "zielony", "niebieski"];
for (const kolor of kolory) {
  console.log(kolor);
}

const osoba = { imie: "Anna", wiek: 22 };
for (const klucz in osoba) {
  console.log(`${klucz}: ${osoba[klucz]}`);
}
```

5. `break` i `continue` — sterowanie pętlą

```
// break - natychmiast przerywa pętlę
for (let i = 0; i < 10; i++) {
```

```

if (i === 5) break; // Pętla kończy się przy i = 5
console.log(i); // 0, 1, 2, 3, 4
}

// continue – pomija bieżącą iterację i przechodzi do następnej
for (let i = 0; i < 6; i++) {
  if (i === 3) continue; // Pomija 3
  console.log(i); // 0, 1, 2, 4, 5
}

```

## 6. Praktyczne użycie pętli w React (poza JSX):

W JSX do renderowania list używamy `map()`, ale pętle klasyczne przydają się w logice:

```

// Generowanie hasła – pętla for w funkcji
function generujHaslo(dlugosc) {
  const znaki = "abcdefghijklmnopqrstuvwxyz0123456789";
  let haslo = "";
  for (let i = 0; i < dlugosc; i++) {
    haslo += znaki[Math.floor(Math.random() * znaki.length)];
  }
  return haslo;
}

// Tworzenie tablicy N elementów
function stworzKosci(ile) {
  const kosci = [];
  for (let i = 0; i < ile; i++) {
    kosci.push({ id: i + 1, wartosc: Math.floor(Math.random() * 6) + 1 });
  }
  return kosci;
}

// Walidacja – while do szukania błędu
function znajdzPierwszyBlad(pola) {
  let i = 0;
  while (i < pola.length) {
    if (pola[i].trim() === "") return `Pole ${i + 1} jest puste`;
    i++;
  }
  return null; // Brak błędów
}

```

### 3.23. Asynchroniczność (Promises, async/await, try/catch)

Przy `async/await` zawsze zakładaj, że operacja może się nie udać. W komponencie najczęściej ustawiasz osobny stan dla ładowania, błędu i danych.

```

try {
  const response = await fetch(url);
  if (!response.ok) throw new Error("Błąd odpowiedzi");
  const data = await response.json();
}

```

```
} catch (error) {  
  console.error(error);  
}
```

W dzisiejszym web developmentcie komunikacja z API jest oparta o asynchroniczność. Oznacza to, że Twój kod nie czeka w miejscu na pobranie danych z serwera, ale idzie dalej i wraca do obsługi danych, kiedy są one gotowe.

#### Podejście klasyczne: Promises (.then / .catch)

```
fetch("https://jsonplaceholder.typicode.com/users/1")  
  .then(response => response.json()) // Przekształcamy na obiekt JS  
  .then(data => console.log(data))   // Otrzymujemy dane  
  .catch(error => console.error("Błąd pobierania:", error));
```

**Podejście nowoczesne: async / await + try / catch** To jest rekomendowany sposób pobierania danych w React. Składnia przypomina kod synchroniczny, dzięki czemu jest bardzo czytelna.

```
// Słowo 'async' przed funkcją pozwala używać 'await' w jej środku  
async function pobierzDane() {  
  try {  
    // await zatrzymuje wykonanie TYLKO w obrębie tej funkcji, do czasu odpowiedzi  
    const response = await fetch("https://jsonplaceholder.typicode.com/users/1");  
    if (!response.ok) {  
      throw new Error(`HTTP error! status: ${response.status}`);  
    }  
    const data = await response.json();  
    console.log("Pobrane dane:", data);  
  } catch (error) {  
    console.error("Coś poszło nie tak:", error.message);  
  } finally {  
    console.log("Wykonuje się zawsze, niezależnie od sukcesu czy błędu");  
  }  
}  
  
pobierzDane();
```

#### Użycie async/await w React (z useEffect):

```
import { useState, useEffect } from "react";  
  
function ListaUzytkownikow() {  
  const [uzytkownicy, setUzytkownicy] = useState([]);  
  const [ladowanie, setLadowanie] = useState(true);  
  const [blad, setBlad] = useState(null);  
  
  useEffect(() => {  
    // Definiujemy async funkcję WEWNĄTRZ useEffect  
    async function pobierz() {  
      try {  
        const res = await fetch("https://jsonplaceholder.typicode.com/users");  
        if (!res.ok) throw new Error("Błąd sieci");  

```

```

    const data = await res.json();
    setUserkownicy(data);
  } catch (err) {
    setBlad(err.message);
  } finally {
    setLadowanie(false);
  }
}

pobierz(); // Wywołujemy ją
}, []); // Pusta tablica = tylko raz przy montowaniu

if (ladowanie) return <p>Ładowanie...</p>;
if (blad) return <p className="text-danger">Błąd: {blad}</p>;

return (
  <ul>
    {uzytkownicy.map(u => <li key={u.id}>{u.name}</li>)}
  </ul>
);
}

```

**Ważne:** Nie można przekazać async funkcji bezpośrednio do useEffect ( `useEffect(async () => {...})` ). Trzeba zdefiniować async funkcję wewnątrz i ją wywołać.

### 3.24. Dodatkowe metody tablic — `forEach`, `some`, `every`, `slice`, `splice`, `concat`

Konstrukcję „Dodatkowe metody tablic — `forEach`, `some`, `every`, `slice`, `splice`, `concat`” najlepiej przećwiczyć na danych, które realnie trafiają do komponentu: tekście z inputa, obiekcie użytkownika albo tablicy produktów. Przygotuj wynik w JavaScriptcie przed JSX, a w widoku zostaw tylko odczyt gotowej wartości.

Oprócz `map`, `filter`, `find` i `reduce` istnieje wiele innych przydatnych metod tablicowych.

#### `FOREACH()` — WYKONANIE OPERACJI NA KAŻDYM ELEMENCIE (BEZ ZWRACANIA NOWEJ TABLICY)

```

const owoce = ["jabłko", "banan", "gruszka"];

owoce.forEach((owoc, index) => {
  console.log(`${index + 1}. ${owoc}`);
});
// 1. jabłko
// 2. banan
// 3. gruszka

```

Różnica `forEach` vs `map`: `forEach` nic nie zwraca (`undefined`), `map` zwraca nową tablicę. W React do renderowania list używamy `map`, nie `forEach`.

#### `SOME()` — CZY JAKIKOLWIEK ELEMENT SPEŁNIA WARUNEK

```

const oceny = [3, 4, 2, 5, 3];
const maOceneCelujaca = oceny.some(o => o === 6); // false
const maNiedostateczna = oceny.some(o => o === 1); // false
const maDwojke = oceny.some(o => o === 2); // true

```

## EVERY() — CZY WSZYSTKIE ELEMENTY SPEŁNIAJĄ WARUNEK

```
const oceny = [4, 5, 4, 5, 5];  
const wszystkieZaliczone = oceny.every(o => o >= 3); // true  
const wszystkieCelujace = oceny.every(o => o === 6); // false
```

Przydatne w React np. do walidacji:

```
const pola = [imie, email, haslo];  
const formularzWypełniony = pola.every(pole => pole.trim() !== "");
```

## SLICE() — WYCIECIE FRAGMENTU TABLICY (BEZ MUTACJI)

```
const liczby = [10, 20, 30, 40, 50];  
  
liczby.slice(1, 3) // [20, 30] — od indeksu 1 do 3 (bez 3)  
liczby.slice(2)    // [30, 40, 50] — od indeksu 2 do końca  
liczby.slice(-2)   // [40, 50] — ostatnie 2 elementy
```

## SPLICE() — MODYFIKACJA TABLICY W MIEJSCU (MUTUJE!)

```
const kolory = ["czerwony", "zielony", "niebieski"];  
  
// Usunięcie 1 elementu od indeksu 1  
kolory.splice(1, 1); // kolory = ["czerwony", "niebieski"]  
  
// Wstawienie elementu na pozycji 1  
kolory.splice(1, 0, "żółty"); // kolory = ["czerwony", "żółty", "niebieski"]  
  
// Zamiana elementu na pozycji 0  
kolory.splice(0, 1, "pomarańczowy"); // kolory = ["pomarańczowy", "żółty", "niebieski"]
```

**Uwaga:** W React NIE używamy `splice` na stanie — mutuje tablicę. Zamiast tego używamy `filter` (usuwanie) lub `map` (zamiana).

## CONCAT() — ŁĄCZENIE TABLIC (BEZ MUTACJI)

```
const a = [1, 2];  
const b = [3, 4];  
const c = a.concat(b); // [1, 2, 3, 4]  
  
// Alternatywa ze spread (częściej używana w React):  
const d = [...a, ...b]; // [1, 2, 3, 4]
```

## FLAT() — SPŁASZCZANIE ZAGNIEŻDŻONYCH TABLIC

```
const zagniezdzona = [[1, 2], [3, 4], [5]];  
const płaska = zagniezdzona.flat(); // [1, 2, 3, 4, 5]
```

## INDEXOF() | FINDINDEX()

```
const kursy = ["HTML", "CSS", "React"];
kursy.indexOf("CSS"); // 1
kursy.indexOf("Angular"); // -1 (nie znaleziono)

const produkty = [{ id: 1, nazwa: "A" }, { id: 2, nazwa: "B" }];
produkty.findIndex(p => p.id === 2); // 1
```

## JOIN() — ŁĄCZENIE ELEMENTÓW W STRING

```
const slowa = ["Cześć", "świecie"];
slowa.join(" "); // "Cześć świecie"
slowa.join(", "); // "Cześć, świecie"
slowa.join(""); // "Cześćświecie"
```

## REVERSE() — ODWRÓCENIE KOLEJNOŚCI (MUTUJE!)

```
const liczby = [1, 2, 3, 4, 5];
const odwrocone = [...liczby].reverse(); // [5, 4, 3, 2, 1] — kopia, by nie mutować
```

## SET — USUWANIE DUPLIKATÓW Z TABLICY

**Set** to struktura danych przechowująca tylko unikalne wartości:

```
const zDuplikatami = ["jabłko", "banan", "jabłko", "gruszka", "banan"];

// Usunięcie duplikatów — najczęstszy wzorec
const unikalne = [...new Set(zDuplikatami)];
// ["jabłko", "banan", "gruszka"]

// Wyciągnięcie unikalnych kategorii z tablicy obiektów (częste w React!)
const produkty = [
  { id: 1, kategoria: "Elektronika" },
  { id: 2, kategoria: "Odzież" },
  { id: 3, kategoria: "Elektronika" },
];
const kategorie = [...new Set(produkty.map(p => p.kategoria))];
// ["Elektronika", "Odzież"]
```

## ARRAY.FROM — TWORZENIE TABLICY Z CZEGOŚ ITEROWALNEGO

```
// Tablica N elementów
Array.from({ length: 5 }, (_, i) => i + 1) // [1, 2, 3, 4, 5]

// Tablica 10 zer
Array.from({ length: 10 }, () => 0) // [0, 0, 0, ..., 0]
```



```
// String na tablicę znaków
Array.from("React") // ["R", "e", "a", "c", "t"]
```

### 3.25. Obiekt Math — losowanie, zaokrąglanie, min/max

Konstrukcję „Obiekt Math — losowanie, zaokrąglanie, min/max” najlepiej przećwiczyć na danych, które realnie trafiają do komponentu: tekście z inputa, obiekcie użytkownika albo tablicy produktów. Przygotuj wynik w JavaScriptcie przed JSX, a w widoku zostaw tylko odczyt gotowej wartości.

Obiekt `Math` zawiera stałe i metody matematyczne. Nie trzeba go importować.

```
// Zaokrąglanie
Math.round(3.5) // 4 - do najbliższej całkowitej
Math.round(3.4) // 3
Math.floor(3.9) // 3 - zawsze w dół
Math.ceil(3.1) // 4 - zawsze w górę
Math.trunc(3.9) // 3 - obcięcie części dziesiętnej

// Wartość bezwzględna
Math.abs(-7) // 7

// Potęgowanie i pierwiastek
Math.pow(2, 3) // 8 (to samo co 2 ** 3)
Math.sqrt(16) // 4

// Minimum i maksimum
Math.min(3, 1, 7, 2) // 1
Math.max(3, 1, 7, 2) // 7

// Min/max z tablicy - użyj spread
const oceny = [3, 5, 2, 4];
Math.min(...oceny) // 2
Math.max(...oceny) // 5

// Stała PI
Math.PI // 3.141592653589793
```

### LOSOWANIE LICZB — MATH.RANDOM()

`Math.random()` zwraca losową liczbę z zakresu [0, 1) — od 0 (włącznie) do 1 (wyłącznie).

```
// Losowa liczba 0-1
Math.random() // np. 0.7234...

// Losowa liczba całkowita od 0 do 9
Math.floor(Math.random() * 10)

// Losowa liczba całkowita od 1 do 6 (kość do gry)
Math.floor(Math.random() * 6) + 1

// Losowa liczba całkowita z zakresu [min, max]
function losujZZakresu(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
```

```

}

losujZZakresu(10, 20) // np. 14

// Losowy element z tablicy
const kolory = ["red", "green", "blue", "yellow"];
const losowy = kolory[Math.floor(Math.random() * kolory.length)];

```

### 3.26. Obiekt Date — data i czas

Konstrukcję „Obiekt Date — data i czas” najlepiej przećwiczyć na danych, które realnie trafiają do komponentu: tekście z inputa, obiekcie użytkownika albo tablicy produktów. Przygotuj wynik w JavaScriptcie przed JSX, a w widoku zostaw tylko odczyt gotowej wartości.

```

// Aktualna data i czas
const teraz = new Date();
console.log(teraz); // np. "2025-06-15T10:30:00.000Z"

// Pobieranie składowych
teraz.getFullYear() // 2025
teraz.getMonth()    // 5 (miesiące od 0! Styczeń = 0, Czerwiec = 5)
teraz.getDate()     // 15 (dzień miesiąca)
teraz.getDay()      // 0 (dzień tygodnia: 0 = niedziela, 1 = poniedziałek)
teraz.getHours()    // 10
teraz.getMinutes()  // 30
teraz.getSeconds()  // 0

// Tworzenie konkretnej daty
const sylwester = new Date(2025, 11, 31); // Grudzień 31 (miesiąc od 0!)
const zStringa = new Date("2025-06-15");

// Timestamp — milisekundy od 1 stycznia 1970
Date.now() // np. 1718451000000 — przydatne jako unikalne ID

// Formatowanie daty
teraz.toLocaleDateString("pl-PL") // "15.06.2025"
teraz.toLocaleTimeString("pl-PL") // "10:30:00"
teraz.toLocaleString("pl-PL")     // "15.06.2025, 10:30:00"

// Porównywanie dat
const data1 = new Date("2025-01-01");
const data2 = new Date("2025-06-01");
console.log(data1 < data2); // true

```

Typowe użycie w React:

```

function Stopka() {
  const rok = new Date().getFullYear();
  return <footer>&copy; {rok} Moja Aplikacja</footer>;
}

```

### 3.27. setTimeout i setInterval — opóźnienia i interwały

Konstrukcję „setTimeout i setInterval — opóźnienia i interwały” najlepiej przećwiczyć na danych, które realnie trafiają do komponentu: tekście z inputa, obiekcie użytkownika albo tablicy produktów. Przygotuj wynik w JavaScriptcie przed JSX, a w widoku zostaw tylko odczyt gotowej wartości.

#### SETTIMEOUT — JEDNORAZOWE OPÓŹNIENIE

```
// Wykonaj funkcję PO 2 sekundach (2000 ms)
setTimeout(() => {
  console.log("Minęły 2 sekundy!");
}, 2000);

// Anulowanie timera
const timer = setTimeout(() => {
  console.log("To się nie wykona");
}, 5000);
clearTimeout(timer); // Anulujemy przed wykonaniem
```

#### SETINTERVAL — POWTARZANIE CO X MILLISEKUND

```
// Wykonuj co 1 sekundę
const interwał = setInterval(() => {
  console.log("Tik...");
}, 1000);

// Zatrzymanie interwału
clearInterval(interwał);
```

#### UŻYCIE W REACT (Z USEEFFECT):

```
import { useState, useEffect } from "react";

function Stoper() {
  const [sekundy, setSekundy] = useState(0);

  useEffect(() => {
    const id = setInterval(() => {
      setSekundy(prev => prev + 1);
    }, 1000);

    // Cleanup – zatrzymanie przy odmontowaniu komponentu
    return () => clearInterval(id);
  }, []);

  return <p>Czas: {sekundy}s</p>;
}
```

### 3.28. Operator ?? (nullish coalescing) i ?. (optional chaining)

Konstrukcję „Operator ?? (nullish coalescing) i ?. (optional chaining)” najlepiej przećwiczyć na danych, które realnie trafiają do komponentu: tekście z inputa, obiekcie użytkownika albo tablicy produktów. Przygotuj wynik w JavaScriptcie przed JSX, a w widoku zostaw tylko odczyt gotowej wartości.

### ?? — WARTOŚĆ DOMYŚLNA DLA NULL/UNDEFINED

Operator `??` zwraca prawą stronę TYLKO gdy lewa jest `null` lub `undefined`. Różni się od `||`, który reaguje na wszystkie wartości fałszy (0, "", false).

```
const imie = null;
const wyswietlane = imie ?? "Anonim"; // "Anonim"

const liczba = 0;
const a = liczba || 10; // 10 - bo 0 jest fałszy!
const b = liczba ?? 10; // 0 - bo 0 nie jest null/undefined

const tekst = "";
const c = tekst || "domyślny"; // "domyślny" - bo "" jest fałszy
const d = tekst ?? "domyślny"; // "" - bo "" nie jest null/undefined
```

### ?. — BEZPIECZNY DOSTĘP DO ZAGNIEJDŻONYCH PÓL

Operator `?.` sprawdza, czy wartość po lewej nie jest `null` / `undefined` zanim spróbuje odczytać pole. Jeśli jest — zwraca `undefined` zamiast rzucać błąd.

```
const user = { imie: "Jan", adres: { miasto: "Kraków" } };

// Bez optional chaining - błąd jeśli adres nie istnieje
// user.adres.ulica -> undefined
// user.kontakt.email -> TypeError: Cannot read property 'email' of undefined

// Z optional chaining - bezpieczne
user.adres?.miasto // "Kraków"
user.adres?.ulica // undefined (bez błędu)
user.kontakt?.email // undefined (bez błędu)

// Łączenie z ??
const miasto = user.adres?.miasto ?? "Nieznane"; // "Kraków"
const email = user.kontakt?.email ?? "brak"; // "brak"
```

Przydatne w React przy danych z API:

```
function ProfilUsera({ user }) {
  return (
    <div>
      <p>Imię: {user?.imie ?? "Ładowanie..."}</p>
      <p>Miasto: {user?.adres?.miasto ?? "Nie podano"}</p>
    </div>
  );
}
```

### 3.29. Obsługa błędów — try / catch / finally

Konstrukcję „Obsługa błędów — try / catch / finally” najlepiej przećwiczyć na danych, które realnie trafiają do komponentu: tekście z inputa, obiekcie użytkownika albo tablicy produktów. Przygotuj wynik w JavaScriptcie przed JSX, a w widoku zostaw tylko odczyt gotowej wartości.

Blok `try/catch` pozwala przechwycić błędy bez zatrzymywania całej aplikacji.

```
// Podstawowa składnia
try {
  // Kod, który może rzucić błąd
  const dane = JSON.parse("to nie jest JSON");
} catch (error) {
  // Obsługa błędu
  console.error("Błąd parsowania:", error.message);
} finally {
  // Opcjonalnie – wykona się ZAWSZE (niezależnie od błędu)
  console.log("Koniec operacji");
}
```

## RZUCANIE WŁASNYCH BŁĘDÓW — `THROW`

```
function podziel(a, b) {
  if (b === 0) {
    throw new Error("Nie można dzielić przez zero!");
  }
  return a / b;
}

try {
  const wynik = podziel(10, 0);
} catch (error) {
  console.error(error.message); // "Nie można dzielić przez zero!"
}
```

## TYPOWE UŻYCIE W REACT — WALIDACJA I FETCH:

```
function handleSubmit(e) {
  e.preventDefault();
  try {
    if (wiek < 0 || wiek > 150) {
      throw new Error("Nieprawidłowy wiek");
    }
    console.log("Formularz OK");
  } catch (error) {
    setBład(error.message);
  }
}
```

### 3.30. Wyrażenia regularne (RegExp) — podstawy

Wyrażenia regularne są przydatne w walidacji, ale nie powinny być jedyną ochroną formularza. Prosty regex może sprawdzić format, np. kod pocztowy, ale nadal warto pokazać użytkownikowi zrozumiały komunikat błędu.

Wyrażenia regularne (regex) służą do wyszukiwania wzorców w tekście. Przydatne przy walidacji formularzy.

```
// Tworzenie wyrażenia regularnego
const wzorzec = /abc/; // Literał
const wzorzec2 = new RegExp("abc"); // Konstruktor

// Testowanie – czy string pasuje do wzorca
/abc/.test("abcdef") // true
/xyz/.test("abcdef") // false

// Flagi
/abc/i // i = case-insensitive (ignoruje wielkość liter)
/abc/g // g = global (znajdź wszystkie wystąpienia)
```

## NAJCZĘSTSZE WZORCE DO WALIDACJI:

```
// Email (uproszczony)
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
emailRegex.test("jan@mail.pl") // true
emailRegex.test("niepoprawny") // false

// Tylko cyfry
/^\d+$/ .test("12345") // true
/^\d+$/ .test("123abc") // false

// Tylko litery (polskie też)
/[a-zA-ZąęłńóśżźĄĆĘŁŃÓŚŻŻ]+$/ .test("Kraków") // true

// Minimum 8 znaków, przynajmniej 1 cyfra i 1 duża litera
/^(?=.*[A-Z])(?=.*\d).\{8,\}$/ .test("Hasło123") // true

// Kod pocztowy (XX-XXX)
/^\d{2}-\d{3}$/ .test("30-001") // true

// Numer telefonu (9 cyfr)
/^\d{9}$/ .test("123456789") // true
```

## METODY STRINGÓW Z REGEX:

```
const tekst = "Mam 3 koty i 2 psy";

// match – znajdź pasujące fragmenty
tekst.match(/\d+/g) // ["3", "2"]

// replace z regex
tekst.replace(/\d+/g, "X") // "Mam X koty i X psy"

// split z regex
"jabłko, banan; gruszka".split(/[;,]\s*/) // ["jabłko", "banan", "gruszka"]
```

## UŻYCIE W REACT — WALIDACJA FORMULARZA:

```
function walidujEmail(email) {
  const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!regex.test(email)) {
    return "Nieprawidłowy format email";
  }
  return "";
}

function walidujHaslo(haslo) {
  if (haslo.length < 8) return "Minimum 8 znaków";
  if (!/[A-Z]/.test(haslo)) return "Wymagana duża litera";
  if (!/d/.test(haslo)) return "Wymagana cyfra";
  return "";
}
```

## 4. JSX — składnia widoku

JSX wygląda jak HTML, ale nadal jest JavaScriptem. Oznacza to, że w atrybutach i treści możesz używać wartości z komponentu, a każda klamra `{ }` przełącza Cię z trybu znaczników do trybu wyrażeń JavaScript.

```
function Naglowek({ tytul, liczba }) {
  return (
    <header className="page-header">
      <h1>{tytul}</h1>
      <p>Liczba elementów: {liczba}</p>
    </header>
  );
}
```

### 4.1. Czym jest JSX

W części „Czym jest JSX” najważniejsze jest utrzymanie JSX jako czytelnej struktury widoku. Jeśli źródło pliku, atrybut albo warunek wymaga kilku operacji, przygotuj zmienną przed `return`, a w znaczniku zostaw prostą wartość.

JSX (JavaScript XML) to rozszerzenie składni JavaScript, które pozwala pisać kod wyglądający jak HTML bezpośrednio w plikach JavaScript. JSX nie jest HTML-em — jest tylko **składnią**, która jest kompilowana do wywołań `React.createElement()`.

```
// To co piszesz (JSX):
const element = <h1>Witaj, React!</h1>;

// To na co JSX jest kompilowany (pod spodem):
const element = React.createElement("h1", null, "Witaj, React!");
```

Nie musisz znać formy skompilowanej — wystarczy, że piszesz w JSX. Babel (kompilator w CRA) dokonuje tej transformacji automatycznie.

## 4.2. Wstawianie wartości JavaScript w JSX

W klamrach JSX możesz umieszczać wyrażenia, czyli coś, co zwraca wartość. Nie umieszczaj tam deklaracji zmiennych ani instrukcji `if`. Te rzeczy przygotuj wyżej.

```
const fullName = `${user.firstName} ${user.lastName}`;  
return <h1>{fullName}</h1>;
```

Wartości JavaScript wstawiamy w JSX za pomocą **nawiasów klamrowych** `{ }`:

```
// Plik: src/App.js  
function App() {  
  const tytuł = "Lista kursów";  
  const liczba = 4;  
  const imie = "Jan";  
  
  return (  
    <section>  
      {/* Zmienna tekstowa */}  
      <h2>{tytuł}</h2>  
  
      {/* Zmienna liczbowa */}  
      <p>Liczba kursów: {liczba}</p>  
  
      {/* Wyrażenie matematyczne */}  
      <p>Następny numer: {liczba + 1}</p>  
  
      {/* Wywołanie metody */}  
      <p>Imię wielkimi literami: {imie.toUpperCase()}</p>  
  
      {/* Operator trójargumentowy */}  
      <p>{liczba > 3 ? "Dużo kursów" : "Mało kursów"}</p>  
    </section>  
  );  
}  
  
export default App;
```

## PORÓWNANIE NAJCZĘSTSZYCH RÓŻNIC MIĘDZY HTML A JSX

Poniższa tabela zbiera w jednym miejscu **najważniejsze różnice**, na które należy zwrócić uwagę przy przechodzeniu z klasycznego HTML na składnię JSX w React. Każda z tych różnic wynika z tego, że JSX jest tak naprawdę kodem JavaScript — dlatego pewne nazwy i konwencje muszą być dostosowane do reguł tego języka.



| CECHA               | HTML                                                            | JSX                                                                                              |
|---------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Klasy CSS           | <code>class="btn"</code>                                        | <code>className="btn"</code> — słowo <code>class</code> jest zarezerwowane w JS                  |
| Etykiety formularzy | <code>&lt;label for="email"&gt;</code>                          | <code>&lt;label htmlFor="email"&gt;</code> — słowo <code>for</code> jest zarezerwowane w JS      |
| Style inline        | <code>style="color: red; font-size: 14px"</code><br>(string)    | <code>style={{ color: "red", fontSize: "14px" }}</code> (obiekt JS)                              |
| Zamykanie tagów     | Opcjonalne — <code>&lt;br&gt;</code> , <code>&lt;img&gt;</code> | Wymagane — <code>&lt;br /&gt;</code> , <code>&lt;img /&gt;</code> — każdy tag musi być zamknięty |
| Komentarze          | <code>&lt;!-- komentarz --&gt;</code>                           | <code>{/* komentarz */}</code> — składnia komentarza JS w klamrach                               |
| Atrybuty boolean    | <code>&lt;input checked&gt;</code> (samo słowo)                 | <code>&lt;input checked={true} /&gt;</code> — jawna wartość <code>true</code> lub skrócona forma |
| Obsługa zdarzeń     | <code>onClick="handleClick()"</code> (string, małe litery)      | <code>onClick={handleClick}</code> (referencja do funkcji, camelCase)                            |

**Wskazówka:** Większość błędów początkujących programistów React wynika właśnie z powyższych różnic. Jeśli Twój komponent się nie renderuje lub widzisz ostrzeżenie w konsoli, pierwszym krokiem powinno być sprawdzenie, czy nie użyłeś przypadkiem nazwy atrybutu z czystego HTML zamiast jego odpowiednika JSX.

Podczas pisania kodu w JSX musisz pamiętać, że pod maską jest to JavaScript, a nie zwykły HTML. Z tego powodu twórcy Reacta musieli wprowadzić pewne zmiany w nazewnictwie atrybutów, aby nie kolidowały one ze słowami kluczowymi języka JavaScript (np. `class` czy `for`). Poniższa tabela przedstawia najważniejsze różnice, o których musisz pamiętać przenosząc kod HTML do Reacta.

| CECHA               | HTML                                                                 | JSX                                                                    |
|---------------------|----------------------------------------------------------------------|------------------------------------------------------------------------|
| Klasy CSS           | <code>class="przycisk"</code>                                        | <code>className="przycisk"</code>                                      |
| Etykiety formularzy | <code>for="email"</code>                                             | <code>htmlFor="email"</code>                                           |
| Style inline        | <code>style="color: red;"</code>                                     | <code>style={{ color: 'red' }}</code>                                  |
| Zamykanie tagów     | Opcjonalne (np. <code>&lt;img&gt;</code> , <code>&lt;br&gt;</code> ) | Wymagane (np. <code>&lt;img /&gt;</code> , <code>&lt;br /&gt;</code> ) |
| Komentarze          | <code>&lt;!-- Komentarz --&gt;</code>                                | <code>{/* Komentarz */}</code>                                         |
| Atrybuty boolean    | <code>checked</code> , <code>disabled</code>                         | <code>checked={true}</code> , <code>disabled={false}</code>            |
| Obsługa zdarzeń     | <code>onClick="funkcja()"</code>                                     | <code>onClick={funkcja}</code>                                         |

4.3. Atrybuty HTML vs JSX

Najczęstsze różnice między HTML i JSX dotyczą nazw atrybutów. `class` zmienia się na `className`, `for` na `htmlFor`, a style inline przyjmują obiekt.

```
<label htmlFor="email" className="form-label">E-mail</label>
<input id="email" style={{ borderColor: "red" }} />
```

W JSX kilka atrybutów HTML ma inne nazwy, ponieważ oryginalne nazwy kolidują ze słowami kluczowymi JavaScript:

| HTML                   | JSX                    | POWÓD                                          |
|------------------------|------------------------|------------------------------------------------|
| <code>class</code>     | <code>className</code> | <code>class</code> to słowo zarezerwowane w JS |
| <code>for</code>       | <code>htmlFor</code>   | <code>for</code> to słowo zarezerwowane w JS   |
| <code>onclick</code>   | <code>onClick</code>   | React używa camelCase                          |
| <code>onchange</code>  | <code>onChange</code>  | camelCase                                      |
| <code>onsubmit</code>  | <code>onSubmit</code>  | camelCase                                      |
| <code>tabindex</code>  | <code>tabIndex</code>  | camelCase                                      |
| <code>readonly</code>  | <code>readOnly</code>  | camelCase                                      |
| <code>maxLength</code> | <code>maxLength</code> | camelCase                                      |

```

{/* Poprawny JSX */}
<label htmlFor="email" className="form-label">
  Email
</label>
<input
  id="email"
  className="form-control"
  type="text"
  readOnly
  tabIndex={1}
/>

```

#### 4.4. Zasada jednego elementu nadrzędnego

W części „Zasada jednego elementu nadrzędnego” najważniejsze jest utrzymanie JSX jako czytelnej struktury widoku. Jeśli źródło pliku, atrybut albo warunek wymaga kilku operacji, przygotuj zmienną przed `return`, a w znaczniku zostaw prostą wartość.

Komponent musi zwrócić **jeden główny element**. Nie można zwrócić dwóch sąsiednich elementów bez wspólnego rodzica:

```

// błąd – dwa elementy obok siebie bez wspólnego rodzica
function App() {
  return (
    <h1>Tytuł</h1>
    <p>Opis</p>
  );
}

// POPRAWNIE – owinięte w jeden element
function App() {
  return (
    <div>
      <h1>Tytuł</h1>
      <p>Opis</p>
    </div>
  );
}

// POPRAWNIE – element semantyczny

```

```
function App() {
  return (
    <main>
      <h1>Tytuł</h1>
      <p>Opis</p>
    </main>
  );
}
```

#### 4.5. Fragmenty — puste znaczniki

W części „Fragmenty — puste znaczniki” najważniejsze jest utrzymanie JSX jako czytelnej struktury widoku. Jeśli źródło pliku, atrybut albo warunek wymaga kilku operacji, przygotuj zmienną przed `return`, a w znaczniku zostaw prostą wartość.

Jeśli nie chcesz dodawać dodatkowego elementu HTML (np. `div`), możesz użyć **Fragmentu**. Fragment to pusty znacznik `<>`, który nie dodaje żadnego elementu do DOM:

```
// Fragment — nie dodaje żadnego elementu HTML
function App() {
  return (
    <>
      <h1>Tytuł</h1>
      <p>Opis</p>
    </>
  );
}

// Alternatywna pełna forma
import { Fragment } from "react";

function App() {
  return (
    <Fragment>
      <h1>Tytuł</h1>
      <p>Opis</p>
    </Fragment>
  );
}
```

#### 4.6. Komentarze w JSX

W części „Komentarze w JSX” najważniejsze jest utrzymanie JSX jako czytelnej struktury widoku. Jeśli źródło pliku, atrybut albo warunek wymaga kilku operacji, przygotuj zmienną przed `return`, a w znaczniku zostaw prostą wartość.

Komentarze w JSX muszą być w nawiasach klamrowych i mieć składnię JavaScriptu:

```
function App() {
  return (
    <div>
      {/* To jest komentarz w JSX */}
      <h1>Tytuł</h1>

      {/* Komentarz wieloliniowy
        w JSX */}
    </div>
  );
}
```

```

    <p>Opis</p>
  </div>

  );
}

```

#### 4.7. Atrybuty boolean

Atrybut boolean bez wartości oznacza `true`. Jeśli wartość zależy od stanu, podaj ją w klamrach. To często pojawia się przy `disabled`, `checked`, `required` i `readOnly`.

```

<button disabled={!formValid}>Zapisz</button>
<input type="checkbox" checked={accepted} onChange={handleChange} />

```

Atrybuty logiczne (boolean) w JSX mogą być podawane bez wartości — wtedy oznaczają `true`:

```

{/* disabled bez wartości = disabled={true} */}
<button disabled>Nieaktywny</button>

{/* Dynamiczny atrybut boolean */}
<button disabled={czyWyslane}>Wyślij</button>

{/* Checkbox zaznaczony */}
<input type="checkbox" checked={zaznaczony} onChange={handleChange} />

{/* Pole tylko do odczytu */}
<input type="text" readOnly value="Nie można edytować" />

```

#### 4.8. Co można wstawiać w klamrach — podsumowanie

W części „Co można wstawiać w klamrach — podsumowanie” najważniejsze jest utrzymanie JSX jako czytelnej struktury widoku. Jeśli źródło pliku, atrybut albo warunek wymaga kilku operacji, przygotuj zmienną przed `return`, a w znaczniku zostaw prostą wartość.

| MOŻNA WSTAWIĆ                    | PRZYKŁAD                                                 | UWAGI                              |
|----------------------------------|----------------------------------------------------------|------------------------------------|
| Zmienną                          | <code>{imie}</code>                                      | Najprostszy przypadek              |
| Liczbę                           | <code>{wiek}</code>                                      | React zamieni ją na tekst          |
| Wyrażenie                        | <code>{cena * ilosc}</code>                              | Krótkie obliczenia                 |
| Wynik funkcji                    | <code>{formatuj(cena)}</code>                            | Funkcja nie powinna zmieniać stanu |
| Operator warunkowy               | <code>{x &gt; 5 ? "Tak" : "Nie"}</code>                  | Do komunikatów                     |
| Operator <code>&amp;&amp;</code> | <code>{blad &amp;&amp; &lt;p&gt;{blad}&lt;/p&gt;}</code> | Warunkowe wyświetlanie             |

#### Czego NIE można wstawić:

- Instrukcji `if/else` (to nie jest wyrażenie)
- Obiektów (React nie wie, jak je wyświetlić)
- Pętli `for` (użyj `map()`)

```

// błąd - if nie jest wyrażeniem
{if (x > 5) { return "Tak" }}

```

```
// POPRAWNIE – operator trójargumentowy
{x > 5 ? "Tak" : "Nie"}
```

#### 4.9. Tagi samozamykające z HTML w JSX (Zasada zamknięcia)

W części „Tagi samozamykające z HTML w JSX (Zasada zamknięcia)” najważniejsze jest utrzymanie JSX jako czytelnej struktury widoku. Jeśli źródło pliku, atrybut albo warunek wymaga kilku operacji, przygotuj zmienną przed `return`, a w znaczniku zostaw prostą wartość.

Gdy przychodzi praca z tagami `HTML`, bardzo często zapomina się o podstawowej regule JSX: **Każdy znacznik musi być zamknięty**. W klasycznym wczesnym HTML pisaliśmy np. `<br>`, `<img>` lub `<input>`. W React (dzięki rygorowi składni XML) coś takiego wywoła od razu potężny błąd kompilacji na czerwono.

Musisz **natychmiast** postawić zamykający ukośnik przez ostatecznym plusem tagu:

```
// Błędny kod w React (nie skompiluje się środowisko, wyrzuci syntax error):
// 
// <input type="text">
// <br>
// <hr>

// Prawidłowy działający kod w React:

<input type="text" />
<br />
<hr />
```

#### 4.10. Multimedia ze źródłem (Audio, Soundplayery i Wideo)

Przy audio i wideo najważniejsze są ścieżki oraz atrybuty sterujące odtwarzaniem. Pliki z `public` podajesz jako `/folder/plik.mp3`, a kontrolę nad wyborem utworu możesz zrobić stanem, np. `currentTrack`. Jeśli źródło się zmienia, czasem warto zmienić też `key`, aby odtwarzacz przeładował media.

Praca z plikami multimedialnymi to klasyk i genialny krok do pierwszych wielkich sukcesów małych interaktywnych apek (np. tworzenie prostej playlisty czy domowego centrum filmowego). Tagi `<audio>` oraz `<video>` wprowadzie są klasycznymi znacznikami HTML, to jednak perfekcyjnie dają się wprawiać w ruch przez mechanizm stanu w JSX i ścieżki (src).

**Podstawowy Odtwarzacz Audio:** Przeglądarki budują pod niego swój własny (nie do podrobienia za lekko) panel playera z głośnością, przesuwakiem time i prędkością - jeżeli tylko nakarmisz tag dopiskiem `controls`.

```
// Aby plik audio zadziałał poprawnie bez problematycznego weebpacka "importu pliku", przenieś swoje pliki mp3
// bezpośrednio do twojego głównego folderu `public/dzwieki/muzyka.mp3`
function PodstawowyAudioPlayer() {
  return (
    <div>
      <h2>Rozdział Odtwarzacza</h2>
      {/*
        controls - absolutnie najważniejszy parametr! To on wyświetla panel odtwarzania z osią czasu, play'em i głosem.
        autoPlay - wymusza autostart (najczęściej blokowany przez przeglądarki Chrome do czasu jak użytkownik sam nie kliknie
        czegokolwiek).
        loop - decyduje by muzyka odtwarzała się w nieskończoność.
      */}
      <audio controls loop>
        <source src="/dzwieki/muzyka.mp3" type="audio/mpeg" />
        Twoja przeglądarka nie obsługuje nowoczesnego tagu audio w HTML5 :(.
```

```

    </audio>
  </div>
);
}

```

**Podstawowe Video:** Tag `video` daje tak samo olbrzymią fantazję z mediami co audio. Jeżeli chcesz utworzyć popularny dzisiaj efekt - odtwarzać z tła wyciszony film loopujący w niekończoność (jak w portfolio agencji czy netflixie), użyj `muted`, `autoPlay` i `loop`:

```

function VideoWyjasnienia() {
  return (
    <div style={{ maxWidth: "600px", margin: "auto" }}>

      <h3>Kino - Film z plakatem startowym</h3>
      /* video jest bardzo czułe, polecam zawsze zapiąć szerokości / max width żeby player się nie wysknał bokiem na monitorze
      */
      <video controls width="100%" poster="/grafika/placeholder.jpg">
        <source src="/filmy/zapowiedz.mp4" type="video/mp4" />
      </video>
      <hr />

      <h3>Film w Tle (nie reagujący, leci jak zwariowany animowany GIF)</h3>
      /* Autostartujący, całkowicie cichy film do bycia interaktywnym żywym tłem - muted! */
      <video autoPlay loop muted width="100%">
        <source src="/filmy/animacja.mp4" type="video/mp4" />
      </video>

    </div>
  );
}

```

**Złota Rada:** W dodanym atrybucie `poster=""` wklepujesz po prostu lokalizację do jpega z miniaturką przed wciśnięciu PLAY przez konsumenta.

#### 4.11. Elementy osadzone: Iframe (Mapy, Embedy z YouTube)

Iframe w JSX wymaga nazw atrybutów zgodnych z Reactem: `frameBorder` zastępuje się zwykłym stylem, `allowFullScreen` zapisuje jako camelCase, a każdy iframe powinien mieć `title`. Dla map i embedów najczęstszy błąd to wklejenie HTML bez dostosowania atrybutów do JSX.

Na stronach "kontaktowych" i w rozbudowanych projektach - niesamowicie cenne jest osadzić aplikacje od zewnętrznych dostawców. Iframe to dosłownie okno w Twojej stronie wyświetlające zasoby z innych serwerów (np. Google).

```

// Zwykle kopiując kod z google maps dostaniesz z internetu kod z czerwonymi ostrzeżeniami jak "frameborder".
// To wina braku zgodności z restrykcyjnym strictmode JSX (pisałismy o tym u góry), trzeba je lekko przespeliować:

function LokalizacjaFirmaWidok() {
  return (
    <iframe
      src="https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d111... itd i tak dalej"
      width="100%"
      height="450"
      style={{ border: 0, borderRadius: "10px" }} // Zamiast np. frameborder="0"
      allowFullScreen={true} // W JSX booleanowe true powinno lecieć dla tych właściwości!
      loading="lazy"
    >

```

```

    referrerPolicy="no-referrer-when-downgrade"
    title="Mapa Firmy Google" // Bezwzględny w React, inaczej dostajesz błąd.
  />
);
}

```

Zawsze pilnuj czy atrybuty CSS skopiowane z Google / YT - np. puste stringi `style="border: 0"` nie są przypisywane do Twoich obiektów style.

## 5. Komponenty

Komponent powinien mieć jasno określoną odpowiedzialność. Jeżeli komponent pobiera dane, filtruje je, obsługuje formularz i jeszcze renderuje rozbudowany układ, to z czasem będzie trudny do utrzymania. Dobrą praktyką jest rozdzielanie komponentów według roli: kontener danych, formularz, lista, pojedynczy element listy, komunikat.

```

function ProductItem({ product }) {
  return <li>{product.name} – {product.price} zł</li>;
}

function ProductList({ products }) {
  return products.map((product) => (
    <ProductItem key={product.id} product={product} />
  ));
}

```

### 5.1. Czym jest komponent

Komponent warto projektować tak, aby dało się go nazwać jednym rzeczownikiem: `Header`, `ProductCard`, `LoginForm`. Jeżeli nazwa zaczyna brzmieć jak opis kilku zadań naraz, komponent prawdopodobnie robi za dużo.

Komponent to absolutny fundament Reacta. Wyobraź sobie stronę internetową nie jako jeden wielki plik HTML, ale jako budowlę z **klocków LEGO**. Każdy klocek to osobny "komponent". Masz klocek-Nawigację, klocek-Przycisk, klocek-Stopkę.

Z technicznego punktu widzenia komponent to zwykła **funkcja JavaScript**, która różni się od innych funkcji tylko dwiema rzeczami:

1. **Zawsze zaczyna się wielką literą** (np. `Nawigacja`, `Przycisk`, a nie `nawigacja`). React używa wielkiej litery, żeby odróżnić własne komponenty od zwykłych tagów HTML (jak `<div>` czy `<span>`).
2. **Zwraca kod JSX** (czyli wyglądający jak HTML kod definiujący, co zobaczy użytkownik).

Dzięki podzieleniu aplikacji na komponenty, możesz:

- Użyć tego samego przycisku w 10 różnych miejscach w kodzie.
- Edytować wygląd Przycisku tylko w jednym pliku, a zmieni się on wszędzie.
- Znacznie łatwiej czytać kod i nim zarządzać.

### 5.2. Pierwszy komponent funkcyjny

Pierwszy komponent powinien być możliwie prosty: jeden `return`, poprawnie domknięty JSX i eksport na końcu pliku. Dopiero gdy ten szkielec działa, dokładaj propsy, stan i zdarzenia.

```

// Plik: src/App.js
function App() {
  return (
    <main>
      <h1>Moja pierwsza aplikacja React</h1>
    </main>
  );
}

```

```

    <p>Projekt działa poprawnie.</p>
  </main>
);
}

export default App;

```

Każdy komponent:

1. Jest **funkcją** (nazwa z wielkiej litery).
2. **Zwraca** JSX (widok).
3. Jest **eksportowany** (`export default`) — aby inne pliki mogły go użyć.

### 5.3. Komponent statyczny — bez stanu

Komponent statyczny jest dobry dla stopki, nagłówka, pustego komunikatu albo elementu informacyjnego. Nie każdy komponent musi mieć `useState`; jeśli widok nie zmienia się po akcji użytkownika, zwykła funkcja zwracająca JSX wystarczy.

Komponent nie musi mieć stanu. Może po prostu wyświetlać statyczny widok:

```

// Plik: src/components/Footer.js
function Footer() {
  const rok = new Date().getFullYear();

  return (
    <footer style={{ textAlign: "center", marginTop: "2rem", color: "gray" }}>
      <p>&copy; {rok} Moja Aplikacja. Wszelkie prawa zastrzeżone.</p>
    </footer>
  );
}

export default Footer;

```

```

// Plik: src/components/InfoBox.js
function InfoBox() {
  return (
    <div className="alert alert-info">
      <h4>Informacja</h4>
      <p>To jest statyczny komponent informacyjny.</p>
    </div>
  );
}

export default InfoBox;

```

### 5.4. Kompozycja — komponenty w komponentach

Kompozycja oznacza, że większy ekran składasz z mniejszych elementów. Rodzic decyduje o układzie, a dzieci odpowiadają za własny fragment. To ułatwia wymianę jednego komponentu bez przepisywania całej strony.

Siła Reacta polega na składaniu komponentów jak klocków. Jeden komponent może zawierać inne:



```
// Plik: src/components/Header.js

function Header() {
  return (
    <header>
      <h1>Aplikacja Kursów</h1>
      <p>Znajdź idealny kurs dla siebie</p>
    </header>
  );
}

export default Header;
```

```
// Plik: src/components/Footer.js

function Footer() {
  return (
    <footer>
      <p>&copy; 2025 Aplikacja Kursów</p>
    </footer>
  );
}

export default Footer;
```

```
// Plik: src/App.js — składanie komponentów

import Header from './components/Header';
import Footer from './components/Footer';

function App() {
  return (
    <div>
      <Header />
      <main className="container mt-4">
        <p>Treść głównej aplikacji</p>
      </main>
      <Footer />
    </div>
  );
}

export default App;
```

### 5.5. Podział na pliki — osobne komponenty

Podział na pliki ma największy sens wtedy, gdy komponent jest używany w kilku miejscach albo jego kod zaczyna zastępować logikę rodzica. Dobrym progiem jest moment, gdy nazwa fragmentu JSX sama prosi się o osobny komponent.

Każdy komponent zazwyczaj ma własny plik. Konwencja nazewnictwa:

- Nazwa pliku = nazwa komponentu
- PascalCase (każde słowo z wielkiej litery)

- Rozszerzenie `.js` (lub `.jsx`)

```
src/
├─ App.js
├─ App.css
├─ index.js
├─ index.css
└─ components/
    ├─ Header.js
    ├─ Footer.js
    ├─ KursLista.js
    └─ KursKarta.js
```

## CZYM SĄ PROPS — WPROWADZENIE TEORETYCZNE

Props (skrót od "properties", czyli właściwości) to fundamentalny mechanizm Reacta pozwalający na przekazywanie danych do komponentów. Działają one dokładnie tak samo, jak argumenty przekazywane do zwykłych funkcji w języku JavaScript, jednak w React przekazujemy je z zewnątrz w postaci atrybutów (podobnie jak w HTML). Kluczową cechą propsów jest to, że są one **tylko do odczytu (readonly)**. Komponent-dziecko, który otrzymuje propsy, w żadnym wypadku nie może ich modyfikować. To zawsze rodzic (komponent wyżej w hierarchii) decyduje o tym, jakie konkretnie wartości zostaną przekazane w dół. Dzięki wykorzystaniu propsów możemy tworzyć wysoce uniwersalne i reużywalne komponenty. Jeden i ten sam komponent wizualny (np. przycisk lub karta profilu) może zostać wywołany wielokrotnie na stronie, za każdym razem z zupełnie innymi danymi wejściowymi.

### 5.6. Props — przekazywanie danych do komponentu

Propsy powinny być nazwane tak, aby komponent dało się zrozumieć bez zaglądania do rodzica. Jeżeli przekazujesz funkcję, nazwa zaczynająca się od `on` dobrze pokazuje, że dziecko tylko zgłasza zdarzenie.

```
<TaskItem task={task} onToggle={toggleTask} onRemove={removeTask} />
```

Props (skrót od "properties", czyli właściwości) to dane przekazywane z komponentu rodzica do komponentu dziecka. Działają dokładnie tak samo jak **parametry funkcji**.

Wróćmy do analogii z klockami LEGO: masz gotowy klocek "Przycisk", ale chcesz, by na jednej stronie był czerwony z napisem "Usuń", a na innej zielony z napisem "Zapisz". Nie budujesz dwóch osobnych komponentów! Zamiast tego do jednego, uniwersalnego komponentu Przycisk przekazujesz odpowiednie **propsy** (np. kolor i tekst).

```
// Plik: src/components/Powitanie.js
// Komponent odbiera obiekt 'props' jako swój pierwszy parametr
function Powitanie(props) {
  return <h2>Cześć, {props.imie}! Masz {props.wiek} lat.</h2>;
}

export default Powitanie;
```

```
// Plik: src/App.js — użycie komponentu z propsami
import Powitanie from "../components/Powitanie";

function App() {
  return (
    <main>
      <Powitanie imie="Jan" wiek={25} />
    </main>
  );
}
```

```

    <Powitanie imie="Anna" wiek={30} />
    <Powitanie imie="Piotr" wiek={22} />
  </main>
);
}

export default App;

```

### Zasady propsów:

- Props są **tylko do odczytu** — dziecko NIE może ich modyfikować.
- Tekst ( `string` ) podajemy w cudzysłowach: `imie="Jan"`.
- Liczby, zmienne i wyrażenia podajemy w klamrach: `wiek={25}`.
- Boolean `true` — wystarczy sam atrybut: `aktywny = aktywny={true}`.

### 5.7. Props — destrukuryzacja

W tym podrozdziale zwróć uwagę na granicę odpowiedzialności komponentu: jakie dane przychodzą z zewnątrz, co komponent wyświetla i jakie akcje zgłasza rodzicowi. To ważniejsze niż sama składnia funkcji.

Zamiast odwoływać się do `props.imie`, `props.wiek` itd., można użyć destrukuryzacji:

```

// Plik: src/components/KursKarta.js
// Destrukuryzacja propsów bezpośrednio w parametrze
function KursKarta({ nazwa, opis, cena }) {
  return (
    <div className="card mb-3">
      <div className="card-body">
        <h5 className="card-title">{nazwa}</h5>
        <p className="card-text">{opis}</p>
        <p className="card-text"><strong>Cena: {cena} zł</strong></p>
      </div>
    </div>
  );
}

export default KursKarta;

```

### 5.8. Props — wartości domyślne

W komponentach funkcyjnych wartości domyślne najczytelniej podać podczas destrukuryzacji argumentu. Dzięki temu komponent działa nawet wtedy, gdy rodzic nie przekaże wszystkich propsów.

```

function Badge({ label, variant = "secondary" }) {
  return <span className={`badge text-bg-${variant}`}>{label}</span>;
}

```

```

// Plik: src/components/Przycisk.js
function Przycisk({ tekst = "Kliknij", kolor = "primary" }) {
  return (
    <button className={`btn btn-${kolor}`}>
      {tekst}
    </button>
  );
}

```

```

    </button>

  );
}

export default Przycisk;

```

```

// Użycie
<Przycisk />                                {/* tekst="Kliknij", kolor="primary" */}
<Przycisk tekst="Zapisz" />                  {/* kolor nadal "primary" */}
<Przycisk tekst="Usuń" kolor="danger" />    {/* wszystkie nadpisane */}

```

## 5.9. Children — zawartość między znacznikami

`children` przydaje się, gdy komponent ma opakowywać dowolną treść: kartę, modal, panel albo layout. Dzięki temu komponent nie musi znać dokładnej zawartości, którą wyświetli.

```

function Panel({ title, children }) {
  return <section><h2>{title}</h2>{children}</section>;
}

```

Specjalny prop `children` zawiera to, co zostanie umieszczone między otwierającym a zamykającym znacznikiem komponentu:

```

// Plik: src/components/Panel.js
function Panel({ tytuł, children }) {
  return (
    <div className="card mb-3">
      <div className="card-header">
        <h5>{tytuł}</h5>
      </div>
      <div className="card-body">
        {children}
      </div>
    </div>
  );
}

export default Panel;

```

```

// Plik: src/App.js — użycie komponentu Panel
import Panel from "../components/Panel";

function App() {
  return (
    <main className="container mt-4">
      <Panel tytuł="0 aplikacji">
        <p>To jest moja aplikacja React.</p>
        <p>Używam komponentu Panel jako kontenera.</p>
      </Panel>
    </main>
  );
}

```

```
    <Panel tytuł="Kontakt">
      <p>Email: jan@example.com</p>
    </Panel>
  </main>
);
}

export default App;
```

## 5.10. Kiedy dzielić komponent na mniejsze

Dziel komponent, gdy widzisz powtarzalny fragment, osobną odpowiedzialność albo zbyt długi `return`. Nie dziel tylko po to, aby mieć więcej plików; podział ma skracać czytanie, a nie rozpraszać kod.

Komponent warto podzielić, gdy:

- Ma więcej niż ~100 linii JSX.
- Powtarza się w wielu miejscach (np. karta produktu).
- Ma wyraźnie oddzielne odpowiedzialności (np. formularz + lista wyników).
- Chcesz przekazywać mu różne dane przez props.

W prostych aplikacjach często wystarczy jeden komponent `App`. Nie musisz na siłę dzielić interfejsu, jeśli widok jest mały.

## 6. Stylowanie

Stylowanie w React można prowadzić na kilka sposobów, ale najważniejsze jest zachowanie czytelności. Style globalne są dobre dla resetu, typografii i ogólnego układu strony. Style komponentu są lepsze dla elementów powtarzalnych. Style inline zostawiaj dla wartości naprawdę dynamicznych, np. szerokości paska postępu albo koloru wybranego przez użytkownika.

| RODZAJ STYLU        | NAJLEPSZE ZASTOSOWANIE                 |
|---------------------|----------------------------------------|
| globalny CSS        | layout strony, typografia, zmienne CSS |
| plik CSS komponentu | karta, formularz, panel, element listy |
| klasy dynamiczne    | aktywny element, błąd, zaznaczenie     |
| inline style        | wartości wyliczane w JavaScript        |

### 6.1. CSS w projekcie React (CRA)

Globalny CSS jest dobry dla ustawień bazowych: `body`, czcionek, tła strony i zmiennych CSS. Style konkretnej karty, formularza albo listy lepiej trzymać bliżej komponentu, żeby łatwiej znaleźć ich użycie.

W CRA pliki CSS importujesz bezpośrednio w plikach JavaScript:

```
// Plik: src/App.js
import './App.css'; // Import pliku CSS

function App() {
  return <h1 className="tytuł-główny">Witaj!</h1>;
}

export default App;
```

```
/* Plik: src/App.css */
.tytul-glowny {
  color: #333;
  font-size: 2rem;
  text-align: center;
  margin-top: 2rem;
}
```

## 6.2. className zamiast class

W części „className zamiast class” rozdziel stałe style od wariantów zależnych od danych. Stałe reguły przenieś do CSS, a w komponencie zostaw tylko decyzję, która klasa pasuje do aktualnego stanu.

W JSX atrybut HTML `class` zamieniony jest na `className`:

```
{/* BŁĘDNE – class to słowo zarezerwowane w JS */}
<div class="container">

{/* POPRAWNIE – className */}
<div className="container">
```

### KIEDY UŻYWAĆ STYLÓW INLINE, A KIEDY KLAS CSS (CLASSNAME)?

W React masz do dyspozycji dwa główne sposoby stylowania komponentów: **klasy CSS** (przez atrybut `className`) oraz **style inline** (przez atrybut `style`). Style inline w React różnią się od tych znanych z czystego HTML — nie są zapisywane jako string (np. `"color: red"`), lecz jako **obiekt JavaScript**, w którym nazwy właściwości CSS muszą być w notacji **camelCase** (np. `backgroundColor` zamiast `background-color`). Dodatkowo wartości liczbowe (np. `fontSize: 20`) automatycznie otrzymują jednostkę `px`, więc nie trzeba jej dopisywać ręcznie. Z reguły klasy CSS są preferowane w większości przypadków — są bardziej wydajne, wspierają pseudo-klasy, media queries i responsywność.

| CECHA                   | KLASY CSS ( <code>CLASSNAME</code> )                                                    | STYLE INLINE ( <code>STYLE</code> )                                                      |
|-------------------------|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Składnia                | <code>className="btn btn-primary"</code>                                                | <code>style={{ color: "red", fontSize: 20 }}</code>                                      |
| Nazewnictwo właściwości | Standardowe CSS: <code>background-color</code>                                          | camelCase: <code>backgroundColor</code>                                                  |
| Kiedy preferować        | Dla stałych, powtarzalnych stylów i złożonych layoutów                                  | Dla dynamicznych, obliczanych w locie wartości (np. <code>width</code> zależny od stanu) |
| Responsywność           | Pełne wsparcie (media queries, pseudo-klasy <code>:hover</code> , <code>:focus</code> ) | Brak wsparcia dla media queries i pseudo-klas                                            |

W React mamy do dyspozycji różne podejścia do stylowania elementów. Najpopularniejszym sposobem jest używanie zewnętrznych arkuszy stylów przypinanych za pomocą atrybutu `className`, ale czasem zachodzi potrzeba użycia stylów wbudowanych (inline). Style inline w React różnią się jednak znacząco od tych z HTML. Zamiast przekazywać je jako zwykły ciąg znaków, musisz użyć obiektu JavaScript. Oznacza to, że nazwy wszystkich właściwości CSS piszemy z użyciem tzw. camelCase (np. `backgroundColor` zamiast `background-color`), a wartości liczbowe domyślnie traktowane są jako piksele (`px`). Style inline przydają się najbardziej, gdy chcemy płynnie obliczać wartość w zależności od stanu (np. szerokość paska postępu), jednak do globalnego wyglądu strony rekomendowane są klasy CSS.

| CECHA                   | KLASY CSS ( <code>CLASSNAME</code> )      | STYLE INLINE                                      |
|-------------------------|-------------------------------------------|---------------------------------------------------|
| Składnia                | <code>&lt;div className="box"&gt;</code>  | <code>&lt;div style={{ color: 'red' }}&gt;</code> |
| Nazewnictwo właściwości | Zwykły CSS (np. <code>margin-top</code> ) | camelCase (np. <code>marginTop</code> )           |
| Kiedy preferować        | Główne stylowanie aplikacji               | Dynamiczne i obliczane wartości                   |
| Responsywność           | Obsługuje Media Queries i Hover           | Brak Media Queries i pseudo-klas                  |

### 6.3. Style inline w JSX

W części „Style inline w JSX” rozdziel stałe style od wariantów zależnych od danych. Stałe reguły przenieś do CSS, a w komponencie zostaw tylko decyzję, która klasa pasuje do aktualnego stanu.

Style inline w JSX zapisywane są jako **obiekt JavaScript** (nie string jak w HTML):

```
// W HTML: style="color: red; font-size: 20px;"
// W JSX:
<p style={{ color: "red", fontSize: "20px" }}>
  Tekst czerwony
</p>

// Podwójne klamry:
// Zewnętrzne {} – wstawiamy JavaScript do JSX
// Wewnętrzne {} – definiujemy obiekt JavaScript
```

### Różnice między CSS a stylem inline w JSX:

| CSS                                 | JSX INLINE                          |
|-------------------------------------|-------------------------------------|
| <code>font-size: 20px;</code>       | <code>fontSize: "20px"</code>       |
| <code>background-color: red;</code> | <code>backgroundColor: "red"</code> |
| <code>margin-top: 10px;</code>      | <code>marginTop: "10px"</code>      |
| <code>border-radius: 5px;</code>    | <code>borderRadius: "5px"</code>    |
| <code>text-align: center;</code>    | <code>textAlign: "center"</code>    |

Zasada: nazwy CSS z myślnikami zamieniamy na **camelCase**.

### 6.4. Dynamiczne klasy CSS

Dynamiczne klasy najlepiej budować z małych, czytelnych warunków. Jeśli warunków jest dużo, przygotuj tablicę klas i połącz ją przez `join(" ")`.

```
const classes = ["alert", error ? "alert-danger" : "alert-success"];
return <div className={classes.join(" ")}>{message}</div>;
```

Klasy CSS można ustawiać dynamicznie na podstawie stanu:

```
// Plik: src/App.js
import { useState } from "react";
import "./App.css";

function App() {
```

```

const [aktywny, setAktywny] = useState(false);

return (
  <div>
    {/* Dynamiczna klasa za pomocą operatora trójargumentowego */}
    <p className={aktywny ? "tekst-zielony" : "tekst-czerwony"}>
      Status: {aktywny ? "Aktywny" : "Nieaktywny"}
    </p>

    {/* łączenie stałej klasy z dynamiczną */}
    <button
      className={`btn ${aktywny ? "btn-success" : "btn-danger"}`}
      onClick={() => setAktywny(!aktywny)}
    >
      Przełącz
    </button>
  </div>
);
}

export default App;

```

```

/* Plik: src/App.css */
.tekst-zielony {
  color: green;
  font-weight: bold;
}

.tekst-czerwony {
  color: red;
  font-weight: bold;
}

```

## 6.5. Dynamiczne style inline

Style inline są dobre dla wartości obliczanych w JavaScript, np. procentu postępu. Nie powinny zastępować zwykłego CSS dla całego wyglądu komponentu.

```

<div className="progress-bar" style={{ width: `${percent}%` }} />

```

```

// Plik: src/App.js
import { useState } from "react";

function App() {
  const [rozmiar, setRozmiar] = useState(16);
  const [kolor, setKolor] = useState("#333333");

  return (
    <div className="container mt-4">
      <p style={{ fontSize: `${rozmiar}px`, color: kolor }}>

```



```

    Tekst z dynamicznym stylem
  </p>

  <label>
    Rozmiar: {rozmiar}px
    <input
      type="range"
      min="10"
      max="50"
      value={rozmiar}
      onChange={(e) => setRozmiar(Number(e.target.value))}
    />
  </label>
</div>
);
}

export default App;

```

## 6.6. Organizacja plików CSS

Dla małego projektu wystarczy `App.css` i `index.css`, ale przy większej aplikacji warto grupować style według komponentów. Jeżeli usuwasz komponent, łatwiej wtedy usunąć także jego nieużywany CSS.

W prostych projektach wystarczy:

- `src/index.css` — style globalne
- `src/App.css` — style komponentu App

W większych projektach:

- Każdy komponent może mieć własny plik CSS: `Header.css`, `Footer.css`
- Import CSS w pliku komponentu: `import './Header.css';`

```

src/
├── components/
│   ├── Header.js
│   ├── Header.css
│   ├── KursKarta.js
│   └── KursKarta.css
├── App.js
├── App.css
├── index.js
└── index.css

```

## 7. Zdarzenia (Events)

Zdarzenia w React są głównym sposobem komunikacji użytkownika z aplikacją. Handler nie powinien tylko „robić czegoś w DOM”, ale najczęściej powinien zmienić stan, wywołać funkcję przekazaną przez props albo uruchomić logikę pomocniczą.

```

function CounterButton({ onAdd }) {
  return <button onClick={() => onAdd(1)}>Dodaj 1</button>;
}

```

```
}
```

## SYSTEM ZDARZEN W REACT — TEORIA

Mechanizm obsługi zdarzeń w React różni się nieco od klasycznego JavaScriptu i HTML. React nie korzysta bezpośrednio z natywnych zdarzeń DOM, lecz tworzy nad nimi własną, wysoce zoptymalizowaną warstwę abstrakcji zwaną **Synthetic Events** (zdarzenia syntetyczne). Gwarantuje to, że zdarzenia będą zachowywać się dokładnie tak samo, niezależnie od tego, jakiej przeglądarki używa użytkownik, eliminując typowe dla starszych przeglądarek błędy kompatybilności. Wszystkie nazwy zdarzeń w React zapisywane są zgodnie z notacją camelCase, dlatego używamy `onClick` i `onChange` zamiast klasycznych `onclick` czy `onchange`. Bardzo ważną zasadą jest również to, że zdarzenie nigdy nie jest wywoływane w momencie renderowania komponentu — do atrybutu przekazujemy jedynie **referencję** do funkcji (nasłuchiвач), a nie wywołujemy jej od razu (brak nawiasów okrągłych przy nazwie funkcji).

### 7.1. onClick — obsługa kliknięcia

Dla zdarzenia „onClick — obsługa kliknięcia” nazwij handler zgodnie z akcją użytkownika i trzymaj go krótko. Jeśli obsługa obejmuje walidację, przygotowanie danych i zapis, każdą część wydziel do osobnej funkcji.

Zdarzenie `onClick` reaguje na kliknięcie elementu (najczęściej przycisku):

```
// Plik: src/App.js
function App() {
  // Sposób 1: Osobna funkcja obsługi (handler)
  function handleKliknij() {
    console.log("Przycisk został kliknięty!");
  }

  // Sposób 2: Handler z parametrem
  function handlePowitaj(imie) {
    console.log(`Cześć, ${imie}!`);
  }

  return (
    <div className="container mt-4">
      {/* Przekazanie referencji do funkcji (BEZ nawiasów) */}
      <button onClick={handleKliknij}>Kliknij mnie</button>

      {/* Funkcja strzałkowa inline */}
      <button onClick={() => console.log("Kliknięto!")}>
        Kliknij
      </button>

      {/* Przekazanie argumentu — wymaga strzałki */}
      <button onClick={() => handlePowitaj("Jan")}>
        Powitaj Jana
      </button>
    </div>
  );
}

export default App;
```

**Ważne — częsty błąd:**

```
{/* błąd – funkcja WYKONA SIĘ od razu przy renderze */}
<button onClick={handlePowitaj("Jan")}>Kliknij</button>

{/* POPRAWNIE – funkcja wykona się dopiero po kliknięciu */}
<button onClick={() => handlePowitaj("Jan")}>Kliknij</button>
```

## 7.2. onChange — zmiana wartości pola

`onChange` w React odpala się przy każdej zmianie wartości pola. Najczęściej pobierasz `event.target.value` dla tekstu i `event.target.checked` dla checkboxa. Te dwie właściwości są częstym źródłem pomyłek.

Zdarzenie `onChange` reaguje na zmianę wartości pola formularza. Jest kluczowe w formularzach kontrolowanych:

```
// Plik: src/App.js
import { useState } from "react";

function App() {
  const [imie, setImie] = useState("");

  function handleZmiana(event) {
    // event.target.value zawiera aktualną wartość pola
    setImie(event.target.value);
  }

  return (
    <div className="container mt-4">
      <input
        type="text"
        value={imie}
        onChange={handleZmiana}
        placeholder="Wpisz imię"
      />
      <p>Wpisałeś: {imie}</p>
    </div>
  );
}

export default App;
```

## 7.3. onSubmit — wysłanie formularza

Przy `onSubmit` prawie zawsze potrzebujesz `event.preventDefault()`, bo domyślne zachowanie formularza odświeża stronę. Dopiero po zatrzymaniu formularza wykonaj walidację i zapis danych.

```
function handleSubmit(event) {
  event.preventDefault();
  saveForm();
}
```

Zdarzenie `onSubmit` reaguje na wysłanie formularza. **Zawsze** należy wywołać `event.preventDefault()`, aby zapobiec przeładowaniu strony:

```
// Plik: src/App.js
import { useState } from "react";

function App() {
  const [imie, setImie] = useState("");
  const [wiadomosc, setWiadomosc] = useState("");

  function handleSubmit(event) {
    event.preventDefault(); // Zapobiegamy przeładowaniu strony!
    console.log("Wysłano formularz:", imie);
    setWiadomosc(`Cześć, ${imie}!`);
    setImie(""); // Czyszczenie pola po wysłaniu
  }

  return (
    <div className="container mt-4">
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          value={imie}
          onChange={(e) => setImie(e.target.value)}
          placeholder="Wpisz imię"
          className="form-control mb-2"
        />
        <button type="submit" className="btn btn-primary">
          Wyślij
        </button>
      </form>

      {wiadomosc && <p className="mt-3">{wiadomosc}</p>}
    </div>
  );
}

export default App;
```

#### 7.4. onBlur — utrata fokusa

**onBlur** przydaje się do walidacji po opuszczeniu pola. Dzięki temu nie musisz pokazywać błędu od pierwszego znaku, ale możesz zareagować, gdy użytkownik skończy edycję konkretnego inputa.

Zdarzenie **onBlur** reaguje, gdy pole traci fokus (kursor go opuści). Przydatne do walidacji:

```
// Plik: src/App.js
import { useState } from "react";

function App() {
  const [email, setEmail] = useState("");
  const [blad, setBlad] = useState("");

  function walidujEmail() {
    if (!email.includes("@")) {
      setBlad("Email musi zawierać znak @");
    }
  }
}
```

```

    } else {
      setBlad("");
    }
  }

  return (
    <div className="container mt-4">
      <input
        type="text"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        onBlur={walidujEmail}
        placeholder="Wpisz email"
        className="form-control"
      />
      {blad && <p style={{ color: "red" }}>{blad}</p>}
    </div>
  );
}

export default App;

```

### 7.5. Przekazywanie argumentów do handlera

Argument do handlera przekazuj przez funkcję strzałkową. Bez niej wywołasz funkcję od razu podczas renderowania, a nie dopiero po kliknięciu.

Gdy chcesz przekazać argument do funkcji obsługi zdarzenia, musisz użyć funkcji strzałkowej:

```

// Plik: src/App.js
function App() {
  function handleUsun(id) {
    console.log("Usuwa element o id:", id);
  }

  const elementy = [
    { id: 1, nazwa: "Element A" },
    { id: 2, nazwa: "Element B" },
    { id: 3, nazwa: "Element C" },
  ];

  return (
    <ul>
      {elementy.map((el) => (
        <li key={el.id}>
          {el.nazwa}
          {/* Strzałka opakowuje wywołanie z argumentem */}
          <button onClick={() => handleUsun(el.id)}>Usuń</button>
        </li>
      ))}
    </ul>
  );
}

```

```
export default App;
```

## 7.6. Obiekt zdarzenia (event)

`event.target` wskazuje element, który faktycznie wywołał zdarzenie, a `event.currentTarget` element, do którego przypięto handler. Różnica jest ważna przy kliknięciach wewnątrz złożonych przycisków lub kart.

```
function handleClick(event) {  
  console.log(event.currentTarget.dataset.id);  
}
```

Każdy handler otrzymuje obiekt zdarzenia (event) jako pierwszy argument:

```
function App() {  
  function handleKliknij(event) {  
    console.log("Typ zdarzenia:", event.type);      // "click"  
    console.log("Element docelowy:", event.target); // <button>...</button>  
    console.log("Tekst elementu:", event.target.textContent);  
  }  
  
  function handleZmiana(event) {  
    console.log("Wartość pola:", event.target.value);  
    console.log("Nazwa pola:", event.target.name);  
  }  
  
  return (  
    <div>  
      <button onClick={handleKliknij}>Kliknij</button>  
      <input name="imie" onChange={handleZmiana} />  
    </div>  
  );  
}
```

## 7.7. Najczęstsze zdarzenia — tabela

Dla zdarzenia „Najczęstsze zdarzenia — tabela” nazwij handler zgodnie z akcją użytkownika i trzymaj go krótko. Jeśli obsługa obejmuje walidację, przygotowanie danych i zapis, każdą część wydziel do osobnej funkcji.

| ZDARZENIE                 | ELEMENT                           | KIEDY SIĘ URUCHAMIA           |
|---------------------------|-----------------------------------|-------------------------------|
| <code>onClick</code>      | Przycisk, link, dowolny element   | Po kliknięciu                 |
| <code>onChange</code>     | Input, select, textarea, checkbox | Gdy zmieni się wartość        |
| <code>onSubmit</code>     | Form                              | Gdy formularz jest wysyłany   |
| <code>onBlur</code>       | Input, select, textarea           | Gdy pole traci fokus          |
| <code>onFocus</code>      | Input, select, textarea           | Gdy pole uzyskuje fokus       |
| <code>onKeyDown</code>    | Input                             | Gdy klawisz jest wciśnięty    |
| <code>onKeyUp</code>      | Input                             | Gdy klawisz jest puszczoney   |
| <code>onMouseEnter</code> | Dowolny element                   | Gdy kursor wjeżdża na element |
| <code>onMouseLeave</code> | Dowolny element                   | Gdy kursor opuszcza element   |

## 8. Stan komponentu — useState

Stan to pamięć komponentu. Każda wartość, która ma wpływać na widok i może zmienić się w czasie działania aplikacji, zwykle powinna być stanem. Wartości, które można wyliczyć z istniejącego stanu, często nie powinny być osobnym stanem.

```
const [items, setItems] = useState([]);
const count = items.length; // wyliczone, nie osobny useState
```

Stan (ang. *state*) to jeden z najważniejszych konceptów w programowaniu reaktywnym, na którym opiera się cała filozofia Reacta. Można go rozumieć jako **dane, które żyją wewnątrz komponentu i mogą się zmieniać w czasie** — np. wartość licznika, tekst wpisany w pole formularza, informacja o tym, czy menu jest otwarte, czy zamknięte. W klasycznym, imperatywnym programowaniu używamy zwykłych zmiennych (`let`, `var`) do przechowywania wartości, które się zmieniają. Problem polega na tym, że zmiana zwykłej zmiennej wewnątrz komponentu React **nie powoduje ponownego wyrenderowania widoku** — przeglądarka po prostu nie wie, że coś się zmieniło, i dalej wyświetla stary HTML. Stan w React rozwiązuje ten problem: kiedy wywołujemy funkcję aktualizującą stan (np. `setLicznik`), React automatycznie **ponownie renderuje komponent** z nową wartością i aktualizuje DOM. Dzięki temu interfejs użytkownika jest zawsze zsynchronizowany z danymi — to właśnie oznacza "reaktywność". Stan jest prywatny dla komponentu — każda instancja komponentu ma własną, niezależną kopię stanu. Co więcej, stan **przetrwa pomiędzy kolejnymi renderami** — w przeciwieństwie do zwykłych zmiennych, które przy każdym re-renderze są deklarowane od nowa i tracą poprzednią wartość. Dlatego `useState` jest absolutnym fundamentem budowania interaktywnych aplikacji w React.

| CECHA                | ZMIENNA ( <code>LET</code> )                                 | STAN ( <code>USESTATE</code> )                                      |
|----------------------|--------------------------------------------------------------|---------------------------------------------------------------------|
| Deklaracja           | <code>let licznik = 0;</code>                                | <code>const [licznik, setLicznik] = useState(0);</code>             |
| Zmiana wartości      | <code>licznik = licznik + 1;</code>                          | <code>setLicznik(licznik + 1);</code>                               |
| Re-render po zmianie | ❌ Nie — widok się nie aktualizuje                            | ✅ Tak — React automatycznie przerysowuje komponent                  |
| Przetrwa re-render   | ❌ Nie — zmienna jest tworzona od nowa z wartością początkową | ✅ Tak — React zapamiętuje wartość między renderami                  |
| Typowe użycie        | Tymczasowe obliczenia wewnątrz funkcji, zmienne pomocnicze   | Dane wpływające na widok: liczniki, formularze, przełączniki, listy |

### 8.1. Po co jest stan

Stan jest potrzebny wtedy, gdy komponent ma pamiętać zmianę między renderami: wpisany tekst, wybraną kartę, otwarty panel, pobrane dane. Zmienna lokalna znika przy następnym renderze, więc nie nadaje się do danych widocznych w UI.

W podstawowym JavaScriptcie, gdy chcemy przechować rosnącą liczbę kliknięć posłużylibyśmy się słówkiem `let`, a następnie zmienili tę wartość poprzez standardowe przypisanie (np. `naszaZmienna = 5`). W React, budując aplikacje Single Page Application wymagamy, by w reakcji na zaistniałą operację nasz ekran natychmiast odświeżał bloki odpowiedzialne w HTML i JSX za dany zmienny widok.

Zwykle zadeklarowanie `let` nie wystarcza – o ile system zaktualizowałby wpis matematycznie w obrębie pamięci wewnętrznej uśpionego układu – ekran HTML w przeglądarce internauty powtarzałby uparcie cyfrę wyświetloną jednorazowo wejścia przy renderze. Stan, czyli `state`, i z dedykowana do powoływania go instrukcja tj: `useState`, wymusza aktualizowanie od zera wskazanego wyrenderowanego komponentu zawsze, gdy tylko przypiszemy z wykorzystaniem go jakiegokolwiek pożądane, ewoluujące dane.

```
// BŁĘDNIIE - Dlaczego 'let' na ekranie nam w UI nie zadziałała
function Komponent0party0LetBledny() {
  let zepsutyLicznik = 0;

  function przyspieszLiczenie() {
    zepsutyLicznik = zepsutyLicznik + 1;
    console.log("Ten licznik zmienia wartość, ale HTML o tym nie wie", zepsutyLicznik);
  }

  return (
    <div>
      <p>Licznik zatrzymany i na wpust martwy wizualnie: {zepsutyLicznik}</p>
      <button onClick={przyspieszLiczenie}>Wykonaj. Żadna graficzna animacja DOM HTML na podstronie nie wystąpi.</button>
    </div>
  );
}
```

```
// POPRAWNIE - W oparciu o naturalne środowisko i Hook z React → State
import { useState } from "react";

function DziałającyReakcyjnyNaszZliczającyWidok() {
  // useState wyrównuje dwu-częściową Tablicę po dekonstrukcji:
  // Pierwszy wpis [liczbaWPolu] to zwykły uchwyt pozwalający ODCZYTYWAĆ wartość z systemu dla HTML.
  // Drugi wpis [setLiczbaWPolu], zwany w żargonie profesjonalnym `setterem`, służy systemowi TYLKO i WYŁĄCZNIE do modyfikowania
  // owej liczby. Po wpisaniu tam danych "w locie" po wciśnięciu guzika, set... zarządzi błyskawicznie "re-render" byś od razu po spacji
  // zobaczyć efekty na oknie HTML. W argumentcie do (..) useState - daliśmy 0 odpowiada za domyślny status startu do wglądu.

  const [liczbaWPolu, setLiczbaWPolu] = useState(0);

  function wezwijRozkazReRenderuZPodbiciemPrawa() {
    setLiczbaWPolu(liczbaWPolu + 1);
  }

  return (
    <div>
      <p>Licznik powiązany ze sprawnie działającym aktualizatorem State w oknie render: {liczbaWPolu}</p>
      <button onClick={wezwijRozkazReRenderuZPodbiciemPrawa}>Naciśnij guzik! Skrypt uaktualni i odmaluje re-renderowane ramy na
      nowe wyniki.</button>
    </div>
  );
}
```



```
export default DziałającyReakcyjnyNaszZliczającyWidok;
```

## 8.2. Składnia useState

`useState` zwraca tablicę dwóch elementów: aktualną wartość i funkcję ustawiającą nową wartość. Nazwa settera powinna odpowiadać nazwie stanu, np. `email` i `setEmail`, bo to ułatwia czytanie komponentu.

```
import { useState } from "react";

// Składnia:
const [wartość, setWartość] = useState(wartośćPoczątkowa);
```

| ELEMENT                        | ZNACZENIE                                                     |
|--------------------------------|---------------------------------------------------------------|
| <code>wartość</code>           | Aktualna wartość stanu                                        |
| <code>setWartość</code>        | Funkcja do zmiany stanu (konwencja: <code>set</code> + nazwa) |
| <code>wartośćPoczątkowa</code> | Wartość przy pierwszym renderze                               |

```
// Przykłady typów stanu
const [licznik, setLicznik] = useState(0); // number
const [imie, setImie] = useState(""); // string
const [aktywny, setAktywny] = useState(false); // boolean
const [lista, setLista] = useState([]); // tablica
const [formularz, setFormularz] = useState({ // obiekt
  imie: "",
  email: "",
});
```

## 8.3. Stan liczbowy — licznik

Stan liczbowy często wymaga pilnowania zakresu. Przy licznikach, ocenach i ilościach warto dodać `Math.max` albo blokadę przycisku, żeby użytkownik nie zszedł poniżej zera.

```
// Plik: src/App.js
import { useState } from "react";

function App() {
  const [licznik, setLicznik] = useState(0);

  return (
    <div className="container mt-4 text-center">
      <h1>Licznik: {licznik}</h1>
      <button className="btn btn-success me-2" onClick={() => setLicznik(licznik + 1)}>
        +1
      </button>
      <button className="btn btn-danger me-2" onClick={() => setLicznik(licznik - 1)}>
        -1
      </button>
      <button className="btn btn-secondary" onClick={() => setLicznik(0)}>

```

```

        Reset
      </button>
    </div>
  );
}

export default App;

```

#### 8.4. Stan tekstowy

Przy stanie tekstowym często przydają się wartości pochodne: długość tekstu, wersja po `trim()` albo informacja, czy pole jest puste. Nie muszą być osobnym stanem, bo można je obliczyć przy renderowaniu.

```

// Plik: src/App.js
import { useState } from "react";

function App() {
  const [imie, setImie] = useState("");

  return (
    <div className="container mt-4">
      <label htmlFor="imie" className="form-label">Twoje imię:</label>
      <input
        id="imie"
        type="text"
        className="form-control"
        value={imie}
        onChange={(e) => setImie(e.target.value)}
        placeholder="Wpisz imię"
      />
      <p className="mt-2">Wpisałeś: <strong>{imie || "(nic)"}</strong></p>
      <p>Liczba znaków: {imie.length}</p>
    </div>
  );
}

export default App;

```

#### 8.5. Stan boolean — przełącznik

Stan boolean dobrze opisuje przełączniki: otwarte/zamknięte, widoczne/ukryte, aktywne/nieaktywne. Do zmiany na przeciwną wartość używaj formy funkcyjnej, gdy zależy od poprzedniego stanu.

```

// Plik: src/App.js
import { useState } from "react";

function App() {
  const [widoczny, setWidoczny] = useState(true);

  return (
    <div className="container mt-4">
      <button
        className="btn btn-primary"

```

```

    onClick={() => setWidoczny(!widoczny)}
  >
    {widoczny ? "Ukryj" : "Pokaż"} treść
  </button>

  {widoczny && (
    <div className="alert alert-info mt-3">
      <p>To jest treść, którą można pokazać lub ukryć.</p>
    </div>
  )}
</div>
);
}

export default App;

```

## 8.6. Aktualizacja na podstawie poprzedniego stanu

Gdy nowy stan zależy od poprzedniego, użyj funkcji aktualizującej. To chroni przed błędami przy kilku aktualizacjach wykonywanych blisko siebie.

```

setCount((current) => current + 1);
setItems((current) => [newItem, ...current]);

```

Jeśli nowa wartość stanu zależy od poprzedniej, używaj **formy funkcyjnej**:

```

// Forma zwykła – OK w prostych przypadkach
setLicznik(licznik + 1);

// Forma funkcyjna – ZALECANA gdy nowa wartość zależy od poprzedniej
setLicznik((prev) => prev + 1);

```

Forma funkcyjna jest ważna, gdy wiele aktualizacji może nastąpić szybko po sobie:

```

function dodajTrzy() {
  // błąd – wszystkie trzy odczytają ten sam „stary” licznik
  setLicznik(licznik + 1);
  setLicznik(licznik + 1);
  setLicznik(licznik + 1);
  // Wynik: licznik wzrośnie tylko o 1!

  // POPRAWNIE – każda aktualizacja bazuje na aktualnej wartości
  setLicznik((prev) => prev + 1);
  setLicznik((prev) => prev + 1);
  setLicznik((prev) => prev + 1);
  // Wynik: licznik wzrośnie o 3
}

```

## 8.7. Reset stanu

Reset powinien przywracać dokładnie ten sam kształt danych, z którym komponent startował. Przy formularzach warto mieć stałą `initialForm`, żeby nie przepisywać pustych wartości w kilku miejscach.

Reset stanu polega na ustawieniu wartości początkowej:

```
const [imie, setImie] = useState("");
const [wiek, setWiek] = useState(0);

function handleReset() {
  setImie("");
  setWiek(0);
}
```

## 8.8. Stan nie aktualizuje się natychmiast

Po wywołaniu settera nie zakładaj, że zmienna stanu od razu ma nową wartość w tej samej funkcji. Jeśli chcesz zareagować na zmianę, użyj wartości obliczonej lokalnie albo efektu zależnego od tego stanu.

```
const nextCount = count + 1;
setCount(nextCount);
console.log(nextCount);
```

Funkcja `setState` jest asynchroniczna — nowa wartość nie jest dostępna od razu w tej samej linii kodu:

```
function handleKliknij() {
  setLicznik(licznik + 1);
  console.log(licznik); // Nadal STARA wartość!
}
```

Jeśli potrzebujesz nowej wartości do obliczeń w tym samym handlerze, oblicz ją przed ustawieniem stanu:

```
function handleKliknij() {
  const nowaWartosc = licznik + 1;
  setLicznik(nowaWartosc);
  console.log(nowaWartosc); // Teraz masz nową wartość
}
```

## 8.9. Lazy initial state

Lazy initial state przydaje się, gdy początkowa wartość wymaga kosztownego obliczenia albo odczytu z `localStorage`. Funkcja zostanie wykonana tylko przy pierwszym renderze komponentu.

```
const [theme, setTheme] = useState(() => {
  return localStorage.getItem("theme") ?? "light";
});
```

Jeśli obliczenie wartości początkowej jest kosztowne, przekaż funkcję do `useState`:

```
// Funkcja zostanie wywołana TYLKO przy pierwszym renderze
const [dane, setDane] = useState(() => {
  const zapisane = localStorage.getItem("dane");
  return zapisane ? JSON.parse(zapisane) : [];
});
```

## 8.10. Zmienna lokalna vs stan — różnica

Zmienna lokalna jest dobra dla wartości pomocniczej obliczanej podczas renderowania, ale nie dla danych, które użytkownik zmienia. Jeśli zmiana ma być widoczna w UI, użyj stanu.

| CECHA               | ZMIENNA LOKALNA ( <code>LET</code> ) | STAN ( <code>USESTATE</code> )        |
|---------------------|--------------------------------------|---------------------------------------|
| Zmiana wartości     | Odbywa się po cichu                  | Powoduje re-render komponentu         |
| Widok (JSX)         | Nie aktualizuje się                  | Aktualizuje się automatycznie         |
| Przetwra re-render? | Nie — resetuje się                   | Tak — React zapamiętuje wartość       |
| Użycie w React      | Zmienne pomocnicze, obliczenia       | Dane interaktywne (formularze, listy) |

## 9. Formularze kontrolowane

Formularz kontrolowany oznacza, że React zna aktualną wartość pola. Dzięki temu można walidować dane na bieżąco, blokować przycisk zapisu, wyświetlać komunikaty i przygotować jeden obiekt do wysłania.

```
const [email, setEmail] = useState("");
const emailPoprawny = email.includes("@");
```

### 9.1. Czym jest formularz kontrolowany

W polu „Czym jest formularz kontrolowany” sprawdź, jaka właściwość zdarzenia jest właściwa: tekst pobierasz z `value`, checkbox z `checked`, a liczby zwykle wymagają `Number()`. Dzięki temu stan formularza ma oczekiwany typ.

Zrozumienie **formularza kontrolowanego (controlled forms)** jest jedną z absolutnie najważniejszych umiejętności react-developera. Jeżeli przychodzisz z czystego HTML'a lub PHP, pamiętasz że kliknięcie przycisku "Submit" (

) domyślnie powodowało odświeżenie całej strony (i np. wysłanie żądania do serwera dopisując parametry do paska adresu URL typu `?name=adam`).

**W aplikacjach Single Page Application (React) takie zdarzenie (przeładowanie karty) to gigantyczny błąd! Aplikacja nigdy nie powinna mrugać odświeżając okno przeglądarki!** Właśnie dlatego całkowicie "przejmujesz kontrolę" nad zachowaniem formularza z wykorzystaniem **stanu (useState)**.

Wprowadzamy mechanizm z 3 fundamentalnymi filarami:

- Pamięć Aplikacji (Stan):** Deklarujemy zmienną `useState` np. `[hasło, setHasło] = useState("")`.
- Kierunek w dół (value):** Nakazujemy inputowi (pole wpisywania), aby zawsze pokazywał to co kryje się w the w stanie: `value={hasło}`. Oznacza to, że input bez zgłoszenia przez React sam z siebie absolutnie nie pokaże tekstu jeśli klikniemy klawisz! W tym momencie jest uwięziony ("kontrolowany" przez stary stan).
- Kierunek w górę (onChange):** Reagujemy na fizyczne wstukiwanie klawiszy we wpisie przez event: `onChange={(event) => setHasło(event.target.value)}`. Event ten powołuje nowy stan z nową literką - a punkt drugi każe inputowi to przerysować.

```
// Złoty, najważniejszy wzorzec formularza basic:
import { useState } from "react";
```

```
function FormularzPodstawowy() {
  const [szukanaFraza, setSzukanaFraza] = useState("");

  // Funkcja blokująca odświeżenie strony po wciśnięciu Enter!
  const obslugaWyslania = (event) => {
    event.preventDefault(); // ← KRÓL REAGOWANIA W REACT (Zatrzymanie domyślnego zachowania)
    console.log("Znaleziono wpisaną przez użytkownika frazę: ", szukanaFraza);

    // Na przykład tutaj wysyłałoby się zapytanie do serwera...
  };

  return (
    <form onSubmit={obslugaWyslania}>
      <label>Szukaj u nas:</label>
      <input
        type="text"
        value={szukanaFraza} // 2. Połączenie 1 stronne (blokada tekstu z zewnątrz)
        onChange={(e) => setSzukanaFraza(e.target.value)} // 3. Uwolnienie tekstów i reakcja
      />
      <button type="submit">Szukaj teraz</button>
    </form>
  );
}
```

Zastosowanie `e.preventDefault()` jest krytyczne podczas budowy aplikacji wykorzystujących klasyczne formularze webowe!

## 9.2. Input text

W polu „Input text” sprawdź, jaka właściwość zdarzenia jest właściwa: tekst pobierasz z `value`, checkbox z `checked`, a liczby zwykle wymagają `Number()`. Dzięki temu stan formularza ma oczekiwany typ.

```
// Plik: src/App.js
import { useState } from "react";

function App() {
  const [imie, setImie] = useState("");

  return (
    <div className="container mt-4">
      <div className="mb-3">
        <label htmlFor="imie" className="form-label">Imię:</label>
        <input
          id="imie"
          type="text"
          className="form-control"
          value={imie}
          onChange={(e) => setImie(e.target.value)}
          placeholder="Wpisz swoje imię"
        />
      </div>
      <p>Wartość: {imie}</p>
    </div>
  );
}
```

```
}  
  
export default App;
```

### 9.3. Input number

Pole numeryczne nadal zwraca tekst. Jeśli puste pole jest dozwolone, nie zamieniaj go od razu na `0`, bo użytkownik straci możliwość wygodnej edycji.

```
const amountNumber = amount === "" ? 0 : Number(amount);
```

```
// Plik: src/App.js  
import { useState } from "react";  
  
function App() {  
  const [wiek, setWiek] = useState(0);  
  
  return (  
    <div className="container mt-4">  
      <div className="mb-3">  
        <label htmlFor="wiek" className="form-label">Wiek:</label>  
        <input  
          id="wiek"  
          type="number"  
          className="form-control"  
          value={wiek}  
          onChange={(e) => setWiek(Number(e.target.value))}  
          min="0"  
          max="150"  
        />  
      </div>  
      <p>Wiek: {wiek} lat</p>  
      <p>{wiek ≥ 18 ? "Pełnoletni" : "Niepełnoletni"}</p>  
    </div>  
  );  
}
```

```
export default App;
```

**Uwaga:** `e.target.value` zawsze zwraca `string`, nawet dla `type="number"`. Musisz skonwertować na liczbę za pomocą `Number()`.

### 9.4. Input password

Pole hasła działa jak zwykły input kontrolowany, ale warto dodać walidację długości i opcjonalny przełącznik podglądu hasła. Sam `type="password"` ukrywa tekst, ale nie waliduje jakości hasła.

```
<div className="mb-3">  
  <label htmlFor="hasło" className="form-label">Hasło:</label>  
  <input  
    id="hasło"
```

```

    type="password"
    className="form-control"
    value={haslo}
    onChange={(e) => setHaslo(e.target.value)}
    placeholder="Wpisz hasło"
  />
</div>

```

## 9.5. Select — lista rozwijana

W `select` wartością stanu jest zwykle `value` wybranej opcji, a nie jej etykieta. Dobrze sprawdza się pusta opcja startowa typu `value=""`, która wymusza świadomy wybór użytkownika.

```

// Plik: src/App.js
import { useState } from "react";

function App() {
  const [rodzaj, setRodzaj] = useState("");

  const rodzaje = ["Sensacyjny", "Komedia", "Horror", "Dramat", "Sci-Fi"];

  return (
    <div className="container mt-4">
      <div className="mb-3">
        <label htmlFor="rodzaj" className="form-label">Rodzaj filmu:</label>
        <select
          id="rodzaj"
          className="form-select"
          value={rodzaj}
          onChange={(e) => setRodzaj(e.target.value)}
        >
          <option value="">-- Wybierz rodzaj --</option>
          {rodzaje.map((r) => (
            <option key={r} value={r}>{r}</option>
          ))}
        </select>
      </div>
      {rodzaj && <p>Wybrany rodzaj: {rodzaj}</p>}
    </div>
  );
}

export default App;

```

## 9.6. Textarea

W polu „Textarea” sprawdź, jaka właściwość zdarzenia jest właściwa: tekst pobierasz z `value`, checkbox z `checked`, a liczby zwykle wymagają `Number()`. Dzięki temu stan formularza ma oczekiwany typ.

Textarea w React działa tak samo jak input — przez `value` i `onChange`:

```

<div className="mb-3">
  <label htmlFor="opis" className="form-label">Opis:</label>

```



```

<textarea
  id="opis"
  className="form-control"
  rows="4"
  value={opis}
  onChange={(e) => setOpis(e.target.value)}
  placeholder="Wpisz opis..."
/>
</div>

```

## 9.7. Checkbox

W polu „Checkbox” sprawdź, jaka właściwość zdarzenia jest właściwa: tekst pobierasz z `value`, checkbox z `checked`, a liczby zwykle wymagają `Number()`. Dzięki temu stan formularza ma oczekiwany typ.

Checkbox używa `checked` zamiast `value` i `onChange` z `e.target.checked`:

```

// Plik: src/App.js
import { useState } from "react";

function App() {
  const [zgoda, setZgoda] = useState(false);

  return (
    <div className="container mt-4">
      <div className="form-check mb-3">
        <input
          id="zgoda"
          type="checkbox"
          className="form-check-input"
          checked={zgoda}
          onChange={(e) => setZgoda(e.target.checked)}
        />
        <label htmlFor="zgoda" className="form-check-label">
          Akceptuję regulamin
        </label>
      </div>
      <p>Zgoda: {zgoda ? "Tak" : "Nie"}</p>
    </div>
  );
}

export default App;

```

## 9.8. Checkbox jako switch (Bootstrap)

Switch Bootstrapa nadal jest zwykłym checkboxem pod względem logiki Reacta. Różni się klasami CSS, ale wartość nadal odczytujesz przez `checked`, a nie przez `value`.

```

<div className="form-check form-switch mb-3">
  <input
    id="tryb"
    type="checkbox"

```

```

        className="form-check-input"
        checked={trybCiemny}
        onChange={(e) => setTrybCiemny(e.target.checked)}
      />
      <label htmlFor="tryb" className="form-check-label">
        Tryb ciemny
      </label>
    </div>

```

## 9.9. Radio — wybór jednej opcji

Grupa radio powinna mieć wspólny atrybut `name`, a stan powinien przechowywać jedną wybraną wartość. Każda opcja porównuje swoje `value` z tym stanem.

```

// Plik: src/App.js
import { useState } from "react";

function App() {
  const [plec, setPlec] = useState("");

  return (
    <div className="container mt-4">
      <p>Płeć:</p>
      <div className="form-check">
        <input
          id="kobieta"
          type="radio"
          className="form-check-input"
          name="plec"
          value="kobieta"
          checked={plec === "kobieta"}
          onChange={(e) => setPlec(e.target.value)}
        />
        <label htmlFor="kobieta" className="form-check-label">Kobieta</label>
      </div>
      <div className="form-check">
        <input
          id="meczczyna"
          type="radio"
          className="form-check-input"
          name="plec"
          value="meczczyna"
          checked={plec === "meczczyna"}
          onChange={(e) => setPlec(e.target.value)}
        />
        <label htmlFor="meczczyna" className="form-check-label">Mężczyzna</label>
      </div>
      {plec && <p className="mt-2">Wybrano: {plec}</p>}
    </div>
  );
}

```

```
export default App;
```

### 9.10. Range — suwak

Suwak `range` zwraca tekst, tak samo jak inne inputy. Jeśli wynik ma być liczbą, konwertuj przez `Number`, szczególnie przy obliczeniach i porównaniach.

```
// Plik: src/App.js
import { useState } from "react";

function App() {
  const [glosnosc, setGlosnosc] = useState(50);

  return (
    <div className="container mt-4">
      <label htmlFor="glosnosc" className="form-label">
        Głośność: {glosnosc}%
      </label>
      <input
        id="glosnosc"
        type="range"
        className="form-range"
        min="0"
        max="100"
        value={glosnosc}
        onChange={(e) => setGlosnosc(Number(e.target.value))}
      />
    </div>
  );
}

export default App;
```

### 9.11. Formularz jako jeden obiekt stanu

Jeden obiekt stanu upraszcza formularze z wieloma polami. Handler może używać atrybutu `name`, aby aktualizować odpowiednie pole bez pisania osobnej funkcji dla każdego inputa.

```
function handleChange(event) {
  const { name, value } = event.target;
  setForm((current) => ({ ...current, [name]: value }));
}
```

Zamiast tworzyć osobny `useState` dla każdego pola, możesz trzymać cały formularz w jednym obiekcie:

```
// Plik: src/App.js
import { useState } from "react";

function App() {
  const [formularz, setFormularz] = useState({
```

```

    imie: "",
    email: "",
    wiek: 0,
  });

  // Uniwersalna funkcja obsługi zmiany – działa dla każdego pola
  function handleChange(e) {
    const { name, value, type } = e.target;
    setFormularz((prev) => ({
      ...prev,
      // Jeśli pole jest liczbowe, konwertuj na Number
      [name]: type === "number" ? Number(value) : value,
    }));
  }

  function handleSubmit(e) {
    e.preventDefault();
    console.log("Dane formularza:", formularz);
  }

  return (
    <div className="container mt-4">
      <form onSubmit={handleSubmit}>
        <div className="mb-3">
          <label htmlFor="imie" className="form-label">Imię:</label>
          <input
            id="imie"
            name="imie"
            type="text"
            className="form-control"
            value={formularz.imie}
            onChange={handleChange}
          />
        </div>
        <div className="mb-3">
          <label htmlFor="email" className="form-label">Email:</label>
          <input
            id="email"
            name="email"
            type="text"
            className="form-control"
            value={formularz.email}
            onChange={handleChange}
          />
        </div>
        <div className="mb-3">
          <label htmlFor="wiek" className="form-label">Wiek:</label>
          <input
            id="wiek"
            name="wiek"
            type="number"
            className="form-control"
            value={formularz.wiek}
            onChange={handleChange}

```

```

    />
  </div>

  <button type="submit" className="btn btn-primary">Wyślij</button>
</form>
</div>

);
}

export default App;

```

**Kluczowy mechanizm:** `[name]: value` — dynamiczny klucz obiektu. Jeśli `name="imie"`, to `[name]` staje się polem `imie` w obiekcie. Dzięki temu **jedna funkcja** `handleChange` obsługuje **wszystkie pola**.

#### WZORZEC FORMULARZA KONTROLOWANEGO (CONTROLLED COMPONENT)

W klasycznym HTML-u to **przeglądarka** zarządza wartością pola `<input>` — użytkownik wpisuje tekst, przeglądarka aktualizuje DOM, a programista odczytuje tę wartość dopiero wtedy, gdy jest mu potrzebna (np. przy wysłaniu formularza). W React podejście jest odwrotne: to **React jest jedynym źródłem prawdy** (ang. *single source of truth*). Wartość inputa nie "żyje" w DOM-ie — ona jest przechowywana w stanie komponentu (`useState`), a input jedynie ją **wyświetla**. Kiedy użytkownik wpisuje coś w pole, przeglądarka generuje zdarzenie `onChange`, które wywołuje handler, a ten z kolei aktualizuje stan przez `setState`. Dopiero po aktualizacji stanu React ponownie renderuje komponent, a input wyświetla nową wartość. Oznacza to, że **input ZAWSZE pokazuje dokładnie to, co jest zapisane w stanie** — nigdy więcej, nigdy mniej. Taki wzorec daje programiście pełną kontrolę nad danymi formularza: można łatwo walidować, formatować i transformować wartość przy każdym naciśnięciu klawisza.

#### Przepływ danych w formularzu kontrolowanym:

1. Użytkownik wpisuje znak w pole `<input>`
2. Przeglądarka generuje zdarzenie `onChange`
3. Handler zdarzenia odczytuje `event.target.value` (nową wartość inputa)
4. Handler wywołuje `setState(nowaWartość)` — aktualizuje stan komponentu
5. React wykrywa zmianę stanu i uruchamia **re-render** komponentu
6. Input zostaje wyrenderowany z nowym atrybutem `value={stanKomponentu}`
7. Użytkownik widzi zaktualizowaną wartość na ekranie

### 9.12. Walidacja formularza

Walidacja powinna zwracać konkretne komunikaty, a nie tylko `true` albo `false`. Dzięki temu możesz pokazać użytkownikowi dokładnie, które pole wymaga poprawy.

```

const errors = {};

if (!email.includes("@")) errors.email = "Podaj poprawny e-mail";

if (password.length < 8) errors.password = "Minimum 8 znaków";

```

```

// Plik: src/App.js
import { useState } from "react";

function App() {
  const [imie, setImie] = useState("");
  const [email, setEmail] = useState("");
  const [bledy, setBledy] = useState({});

  function waliduj() {
    const noweBledy = {};

```

```

    if (imie.trim() === "") {
      noweBledy.imie = "Imię jest wymagane";
    } else if (imie.trim().length < 2) {
      noweBledy.imie = "Imię musi mieć co najmniej 2 znaki";
    }

    if (email.trim() === "") {
      noweBledy.email = "Email jest wymagany";
    } else if (!email.includes("@")) {
      noweBledy.email = "Email musi zawierać @";
    }

    return noweBledy;
  }

function handleSubmit(e) {
  e.preventDefault();
  const noweBledy = waliduj();

  if (Object.keys(noweBledy).length > 0) {
    setBledy(noweBledy);
    return; // Przerywamy – formularz jest niepoprawny
  }

  setBledy({}); // Czyścimy błędy
  console.log("Wysłano:", { imie, email });
}

return (
  <div className="container mt-4">
    <form onSubmit={handleSubmit}>
      <div className="mb-3">
        <label htmlFor="imie" className="form-label">Imię:</label>
        <input
          id="imie"
          type="text"
          className={`form-control ${bledy.imie ? "is-invalid" : ""}`}
          value={imie}
          onChange={(e) => setImie(e.target.value)}
        />
        {bledy.imie && <div className="invalid-feedback">{bledy.imie}</div>}
      </div>
      <div className="mb-3">
        <label htmlFor="email" className="form-label">Email:</label>
        <input
          id="email"
          type="text"
          className={`form-control ${bledy.email ? "is-invalid" : ""}`}
          value={email}
          onChange={(e) => setEmail(e.target.value)}
        />
        {bledy.email && <div className="invalid-feedback">{bledy.email}</div>}
      </div>
      <button type="submit" className="btn btn-primary">Wyślij</button>
    </form>
  </div>
);

```

```

    </form>
  </div>

  );
}

export default App;

```

### 9.13. Reset formularza

Reset formularza powinien czyścić nie tylko wartości pól, ale też błędy i komunikat sukcesu. Inaczej użytkownik może zobaczyć stary błąd przy już pustym formularzu.

```

function handleReset() {
  setImie("");
  setEmail("");
  setWiek(0);
  setBledy({});
}

// Dla formularza obiektowego:
function handleReset() {
  setFormularz({ imie: "", email: "", wiek: 0 });
}

```

```

<button type="button" className="btn btn-secondary" onClick={handleReset}>
  Wyczyść
</button>

```

## 10. Renderowanie warunkowe

Renderowanie warunkowe powinno być czytelne. Jeżeli warunek jest prosty, można użyć operatora `&&` albo ternary. Jeżeli wariantów jest kilka, często lepiej przygotować zmienną przed `return` albo wydzielić osobny komponent.

| SYTUACJA                      | DOBRY ZAPIS                                                   |
|-------------------------------|---------------------------------------------------------------|
| pokaż albo ukryj fragment     | <code>warunek &amp;&amp; &lt;Element /&gt;</code>             |
| pokaż jeden z dwóch wariantów | <code>warunek ? &lt;A /&gt; : &lt;B /&gt;</code>              |
| wiele wariantów               | <code>if</code> przed <code>return</code> albo osobna funkcja |

Renderowanie warunkowe jest jedną z kluczowych technik w React, która wynika bezpośrednio z filozofii tej biblioteki. W klasycznym HTML strona jest **statyczna** — raz wyrenderowana treść nie zmienia się sama z siebie. Aby coś ukryć lub pokazać, trzeba ręcznie manipulować DOM-em za pomocą JavaScript (np. `element.style.display = "none"`). W React widok jest **funkcją stanu** — komponent to funkcja, która na podstawie aktualnych danych zwraca odpowiedni JSX. Skoro dane (stan) mogą się zmieniać, to naturalną konsekwencją jest to, że chcemy wyświetlać **różne elementy w zależności od stanu**. Na przykład: inny widok dla zalogowanego i niezalogowanego użytkownika, komunikat o błędzie tylko gdy wystąpi błąd, spinner ładowania tylko gdy dane się wczytują. React oferuje kilka technik realizacji renderowania warunkowego, z których każda sprawdza się w innym scenariuszu. Nie ma jednej "najlepszej" metody — wybór zależy od złożoności warunku i tego, czy chcemy pokazać alternatywny widok, czy po prostu ukryć element.

| TECHNIKA                                  | SKŁADNIA                                                            | KIEDY UŻYWAĆ                                                                                                                       |
|-------------------------------------------|---------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <code>if</code> przed <code>return</code> | <pre>if (warunek) { return &lt;A /&gt;; } return &lt;B /&gt;;</pre> | Gdy chcesz zwrócić <b>zupełnie inny widok</b> w zależności od warunku (np. ekran logowania vs panel użytkownika)                   |
| Operator trójargumentowy                  | <pre>{warunek ? &lt;A /&gt; : &lt;B /&gt;}</pre>                    | Gdy chcesz w jednym miejscu JSX wybrać <b>jeden z dwóch elementów</b> do wyświetlenia (np. tekst "Aktywny" vs "Nieaktywny")        |
| Operator <code>&amp;&amp;</code>          | <pre>{warunek &amp;&amp; &lt;A /&gt;}</pre>                         | Gdy chcesz <b>pokazać element albo nic</b> — nie ma alternatywy, element po prostu się pojawia lub znika (np. komunikat o błędzie) |

### 10.1. if przed return

`if` przed `return` jest najlepszy przy dużych wariantach widoku, np. loading, error i success. Dzięki temu główny JSX nie jest zagnieżdżony w wielu warunkach.

Najprostszy sposób — warunkowe zwrócenie innego widoku:

```
function App() {
  const [zalogowany, setZalogowany] = useState(false);

  if (!zalogowany) {
    return (
      <div className="container mt-4">
        <p>Musisz się zalogować.</p>
        <button className="btn btn-primary" onClick={() => setZalogowany(true)}>
          Zaloguj
        </button>
      </div>
    );
  }

  return (
    <div className="container mt-4">
      <p>Witaj! Jesteś zalogowany.</p>
      <button className="btn btn-danger" onClick={() => setZalogowany(false)}>
        Wyloguj
      </button>
    </div>
  );
}
```

### 10.2. Operator trójargumentowy w JSX

Warunek z części „Operator trójargumentowy w JSX” powinien mieć czytelny stan wejściowy, np. `hasError`, `isEmpty` albo `canSubmit`. Nazwana zmienna przed `return` często jest czytelniejsza niż długi warunek bezpośrednio w JSX.

Do krótkich warunków w JSX:

```
<p>{aktywny ? "Status: Aktywny" : "Status: Nieaktywny"}</p>
<button className={`btn ${aktywny ? "btn-success" : "btn-danger"}`}>
  {aktywny ? "Wyłącz" : "Włącz"}
</button>
```



### 10.3. Operator && — warunkowe wyświetlanie

Najczęstsza pułapka operatora `&&` w JSX to renderowanie zera. Jeśli lewa strona ma wartość `0`, React może pokazać `0` na ekranie. Porównuj jawnie długość tablicy.

```
{items.length > 0 && <List items={items} />}
```

Wyświetla element **tylko gdy** warunek jest prawdziwy:

```
{blad && <p className="text-danger">{blad}</p>}
{lista.length > 0 && <ul>{lista.map(e1 => <li key={e1}>{e1}</li>)}</ul>}
{zalogowany && <button className="btn btn-danger">Wyloguj</button>}
```

### 10.4. Komunikaty błędów walidacji

Komunikat błędu powinien być blisko pola albo akcji, której dotyczy. W formularzach najlepiej trzymać błędy w obiekcie, np. `errors.email`, aby łatwo pokazać komunikat przy konkretnym inputcie.

```
{bledy.imie && (
  <div className="text-danger small mt-1">{bledy.imie}</div>
)}
```

### 10.5. Obsługa pustej listy

Warunek z części „Obsługa pustej listy” powinien mieć czytelny stan wejściowy, np. `hasError`, `isEmpty` albo `canSubmit`. Nazwana zmienna przed `return` często jest czytelniejsza niż długi warunek bezpośrednio w JSX.

```
function ListaKursow({ kursy }) {
  if (kursy.length === 0) {
    return <p className="text-muted">Brak kursów do wyświetlenia.</p>;
  }

  return (
    <ul className="list-group">
      {kursy.map((kurs) => (
        <li key={kurs.id} className="list-group-item">{kurs.nazwa}</li>
      ))}
    </ul>
  );
}
```

## 11. Tablice i renderowanie list

Listy w React prawie zawsze łączą trzy rzeczy: tablicę danych, metodę `map()` oraz stabilny atrybut `key`. Jeżeli lista ma wyszukiwanie, sortowanie albo filtrowanie, najpierw przygotuj nową tablicę, a dopiero potem ją renderuj.

```
const widoczne = produkty
  .filter((produkt) => produkt.aktywny)
```

```
.sort((a, b) => a.nazwa.localeCompare(b.nazwa));
```

Praca z tablicami w React wymaga zrozumienia jednej fundamentalnej zasady: **niemutowalności** (ang. *immutability*). W zwykłym JavaScript jesteśmy przyzwyczajeni do metod takich jak `.push()`, `.splice()` czy `.sort()`, które **modyfikują oryginalną tablicę** w miejscu. W React takie podejście jest **niedopuszczalne** przy pracy ze stanem. Dlaczego? Ponieważ React decyduje o tym, czy ponownie wyrenderować komponent, porównując **referencje** (adresy w pamięci) obiektów, a nie ich zawartość. Jeśli wywołamy `tablica.push(element)`, tablica zmieni swoją zawartość, ale jej referencja (adres w pamięci) **pozostanie taka sama**. Dla Reacta to oznacza: "nic się nie zmieniło, nie trzeba ponownie renderować". Dlatego zamiast mutować istniejącą tablicę, **zawsze tworzymy nową** — za pomocą metod takich jak `.map()`, `.filter()`, operator spread `[...tablica]` czy `.concat()`. Te metody zwracają **nowy obiekt tablicy** z nową referencją, co React poprawnie interpretuje jako zmianę i uruchamia re-render. Ta sama zasada dotyczy sortowania — `sort()` mutuje tablicę, więc najpierw tworzymy kopię (`[...tablica]`), a dopiero na niej sortujemy. Zapamiętaj prostą regułę: **w stanie React nigdy nie zmieniasz, zawsze twórz nowe**.

| OPERACJA NA TABLICY   | METODA MUTUJĄCA ❌ (ZŁA)                   | METODA NIEMUTUJĄCA ✅ (DOBRA)                                                        |
|-----------------------|-------------------------------------------|-------------------------------------------------------------------------------------|
| Dodawanie             | <code>tablica.push(element)</code>        | <code>[...tablica, element]</code> lub <code>tablica.concat(element)</code>         |
| Usuwanie              | <code>tablica.splice(index, 1)</code>     | <code>tablica.filter((el) =&gt; el.id !== id)</code>                                |
| Aktualizacja elementu | <code>tablica[index] = nowaWartość</code> | <code>tablica.map((el) =&gt; el.id === id ? {...el, pole: nowaWartość} : el)</code> |
| Sortowanie            | <code>tablica.sort(fn)</code>             | <code>[...tablica].sort(fn)</code>                                                  |

### 11.1. Renderowanie tablicy przez map()

W części „Renderowanie tablicy przez map()” przygotuj osobną tablicę wynikową przed renderowaniem. Dzięki temu możesz najpierw filtrować, sortować albo aktualizować dane, a potem w JSX wykonać prosty `map` z poprawnym `key`.

`map()` to główny sposób wyświetlania list w React:

```
// Plik: src/App.js
function App() {
  const kursy = [
    "Programowanie w C#",
    "Angular dla początkujących",
    "React od podstaw",
    "Bazy danych SQL",
  ];

  return (
    <div className="container mt-4">
      <h2>Dostępne kursy ({kursy.length})</h2>
      <ol>
        {kursy.map((kurs, index) => (
          <li key={index}>{kurs}</li>
        ))}
      </ol>
    </div>
  );
}

export default App;
```

## 11.2. Atrybut key — dlaczego jest wymagany

`key` nie jest ozdobą ani sposobem na usunięcie ostrzeżenia. To informacja dla Reacta, który element listy jest tym samym elementem po zmianie kolejności, filtrze albo usunięciu rekordu.

```
{users.map((user) => <UserRow key={user.id} user={user} />)}
```

Każdy element generowany przez `map()` musi mieć atrybut `key` — unikalny identyfikator, który pozwala Reactowi śledzić, który element się zmienił:

```
// błąd - brak key (ostrzeżenie w konsoli)
{elementy.map((el) => <li>{el.nazwa}</li>)}

// POPRAWNIE - key z unikalnego id
{elementy.map((el) => <li key={el.id}>{el.nazwa}</li>)}

// DOPUSZCZALNE - index jako key (tylko gdy lista się nie zmienia)
{elementy.map((el, index) => <li key={index}>{el.nazwa}</li>)}
```

### Zasady key:

- Key musi być **unikalny** wśród rodzeństwa.
- Najlepszy key to **id** z danych (np. z bazy danych, `Date.now()`).
- **Nie używaj indeksu** (`index`) jako key, jeśli kolejność elementów może się zmieniać (przy usuwaniu, sortowaniu).

## 11.3. Lista numerowana

W części „Lista numerowana” przygotuj osobną tablicę wynikową przed renderowaniem. Dzięki temu możesz najpierw filtrować, sortować albo aktualizować dane, a potem w JSX wykonać prosty `map` z poprawnym `key`.

```
function App() {
  const kursy = ["HTML", "CSS", "JavaScript", "React"];

  return (
    <div className="container mt-4">
      <h2>Dostępnych kursów: {kursy.length}</h2>
      <ol>
        {kursy.map((kurs, index) => (
          <li key={index}>{kurs}</li>
        ))}
      </ol>
    </div>
  );
}
```

## 11.4. Dodawanie elementu do tablicy stanu

Dodawanie do tablicy w stanie nie powinno używać `push`, bo `push` modyfikuje istniejącą tablicę. Użyj `spread`, aby utworzyć nową tablicę z nowym elementem na początku albo na końcu.

W React **nigdy nie mutujemy** stanu. Zamiast `push()` tworzymy nową tablicę za pomocą `spread`:

```
// Plik: src/App.js
import { useState } from "react";

function App() {
  const [zadania, setZadania] = useState([]);
  const [tekst, setTekst] = useState("");

  function handleDodaj(e) {
    e.preventDefault();
    if (tekst.trim() === "") return;

    // POPRAWNIE – tworzenie nowej tablicy z dodanym elementem
    const noweZadanie = { id: Date.now(), tekst: tekst };
    setZadania((prev) => [...prev, noweZadanie]);
    setTekst(""); // Czyszczenie pola
  }

  return (
    <div className="container mt-4">
      <form onSubmit={handleDodaj}>
        <input
          type="text"
          className="form-control mb-2"
          value={tekst}
          onChange={(e) => setTekst(e.target.value)}
          placeholder="Nowe zadanie"
        />
        <button type="submit" className="btn btn-success">Dodaj</button>
      </form>
      <ul className="list-group mt-3">
        {zadania.map((z) => (
          <li key={z.id} className="list-group-item">{z.tekst}</li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

### 11.5. Usuwanie elementu z tablicy stanu

Usuwanie elementu najczytelniej zapisać przez `filter`. Funkcja zostawia tylko te elementy, których identyfikator nie pasuje do usuwanego rekordu.

Usuwanie odbywa się przez `filter()` — tworzymy nową tablicę bez elementu o podanym id:

```
function handleUsun(id) {
  setZadania((prev) => prev.filter((z) => z.id !== id));
}

// W JSX:
{zadania.map((z) => (
```

```

<li key={z.id} className="list-group-item d-flex justify-content-between">
  {z.tekst}
  <button className="btn btn-danger btn-sm" onClick={() => handleUsun(z.id)}>
    Usuń
  </button>
</li>
)}}

```

## 11.6. Aktualizacja jednego elementu w tablicy

Aktualizacja jednego elementu listy zwykle oznacza `map()`: element pasujący do identyfikatora zastępujesz kopią ze zmianą, a pozostałe zwracasz bez zmian.

```

setTasks((tasks) => tasks.map((task) =>
  task.id === id ? { ...task, done: !task.done } : task
));

```

Aktualizacja jednego elementu odbywa się przez `map()` — tworzymy nową tablicę, a element o podanym id zastępujemy zmodyfikowaną kopią:

```

function zwiekszPobrania(id) {
  setZdjecia((prev) =>
    prev.map((zdjecie) =>
      zdjecie.id === id
        ? { ...zdjecie, pobrania: zdjecie.pobrania + 1 }
        : zdjecie
    )
  );
}

```

Wyjaśnienie krok po kroku:

1. `map()` przechodzi przez każdy element tablicy.
2. Jeśli `id` elementu pasuje — tworzymy **kopię** z zmienionym polem (`{ ...zdjecie, pobrania: zdjecie.pobrania + 1 }`).
3. Jeśli `id` nie pasuje — zwracamy element bez zmian.
4. `map()` zwraca **nową tablicę** — stara nie jest mutowana.

## 11.7. Sortowanie tablicy w stanie

`sort()` modyfikuje tablicę, dlatego przed sortowaniem stanu zrób kopię: `[...items].sort(...)`. Bez kopii możesz przypadkiem zmienić istniejący stan.

```

function handleSortuj() {
  setElementy((prev) =>
    [...prev].sort((a, b) => a.nazwa.localeCompare(b.nazwa))
  );
}

// Sortowanie liczbowe
function handleSortujPoCenie() {
  setProdukty((prev) =>
    [...prev].sort((a, b) => a.cena - b.cena)
  );
}

```

```
);  
}
```

**Ważne:** `sort()` mutuje tablicę, dlatego najpierw tworzymy kopię `[...prev]`, a dopiero na niej wywołujemy `sort()`.

## 12. Obiekty w stanie

Obiekty w stanie są wygodne, ale wymagają ostrożnego kopiowania. Aktualizując jedno pole obiektu, nie nadpisuj całej reszty przypadkiem. Najczęściej używa się operatora spread, a dla zagnieżdżonych danych tworzy się kopię na każdym poziomie.

```
setUser((current) => ({  
  ...current,  
  address: { ...current.address, city: "Kraków" },  
}));
```

Obiekty w stanie React podlegają **dokładnie tym samym zasadom niemutowalności** co tablice — nigdy nie modyfikujemy obiektu bezpośrednio, zawsze tworzymy nową kopię ze zmienionymi polami. Kluczowym narzędziem do pracy z obiektami jest **operator spread** (`{...obiekt}`), który tworzy **płytką kopię** (ang. *shallow copy*) obiektu. Oznacza to, że kopiowane są wartości wszystkich pól na pierwszym poziomie zagnieżdżenia, ale jeśli pole zawiera zagnieżdżony obiekt lub tablicę, kopiowana jest jedynie **referencja** do tego obiektu, a nie jego zawartość. W praktyce, gdy aktualizujemy obiekt w stanie, najpierw rozprzestrzenia (spread) cały istniejący obiekt, a potem nadpisujemy tylko te pola, które chcemy zmienić: `{...staryObiekt, zmienionePole: nowaWartość}`. Dzięki temu reszta pól pozostaje nienaruszona, a React widzi nową referencję i prawidłowo uruchamia re-render.

### 12.1. Model danych — tablica obiektów

Model danych powinien mieć stabilne identyfikatory i przewidywalne nazwy pól. Jeśli potem renderujesz listę obiektów, `id` przyda się jako `key`, a jasne nazwy pól ograniczą liczbę komentarzy w JSX.

W React dane najczęściej modelujemy jako tablicę obiektów:

```
const zdjecia = [  
  { id: 1, nazwa: "kwiat.jpg", kategoria: "kwiaty", pobrania: 12 },  
  { id: 2, nazwa: "gora.jpg", kategoria: "krajobrazy", pobrania: 34 },  
  { id: 3, nazwa: "roza.jpg", kategoria: "kwiaty", pobrania: 7 },  
  { id: 4, nazwa: "miasto.jpg", kategoria: "miasto", pobrania: 21 },  
];
```

### 12.2. Kopiowanie obiektu — spread

Spread przy obiekcie tworzy nowy obiekt tylko na pierwszym poziomie. Jeśli pole jest zagnieżdżonym obiektem, ono nadal wskazuje na tę samą referencję, dopóki też go nie skopiujesz.

```
setProfile((profile) => ({  
  ...profile,  
  settings: { ...profile.settings, newsletter: true },  
}));
```

W React stan jest niezmienny (immutable). Przy aktualizacji obiektu **nigdy nie modyfikujemy** go bezpośrednio — tworzymy kopię:

```
const [osoba, setOsoba] = useState({ imie: "Jan", wiek: 25, miasto: "Kraków" });

// błąd – mutacja
osoba.wiek = 26;
setOsoba(osoba); // React NIE widzi zmiany (ta sama referencja)

// POPRAWNIE – kopia ze zmienioną wartością
setOsoba({ ...osoba, wiek: 26 });

// Z formą funkcyjną (zalecane)
setOsoba((prev) => ({ ...prev, wiek: 26 }));
```

### 12.3. Formularz jako obiekt stanu

Formularz jako obiekt stanu jest wygodny, gdy pól jest kilka i mają wspólny cykl życia: reset, walidacja, zapis. Używaj atrybutu `name`, żeby jeden handler aktualizował różne pola.

Patrz sekcja 9.11.

### 12.4. Dane z pliku przepisane do kodu

Dane przepisane z pliku warto od razu zamienić na tablicę obiektów o jednolitym kształcie. Lepiej zrobić to raz na początku niż później dopisywać warunki dla różnych formatów w JSX.

Często surowe dane, pochodzące np. z pliku `dane.txt`, możemy przenieść bezpośrednio do kodu jako tablicę obiektów:

```
// Plik: src/App.js
// dane.txt zawierał:
// kwiat.jpg;kwiaty;12
// gora.jpg;krajobrazy;34
// roza.jpg;kwiaty;7

function App() {
  const [zdjecia, setZdjecia] = useState([
    { id: 1, nazwa: "kwiat.jpg", kategoria: "kwiaty", pobrania: 12 },
    { id: 2, nazwa: "gora.jpg", kategoria: "krajobrazy", pobrania: 34 },
    { id: 3, nazwa: "roza.jpg", kategoria: "kwiaty", pobrania: 7 },
  ]);

  // ... reszta komponentu
}
```

## 13. Bootstrap w React — skrót

W dokumentacji Reacta Bootstrap powinien być traktowany jako narzędzie do stylowania komponentów, a nie jako osobny temat dominujący cały plik. Pełny opis Bootstrapa znajduje się w `bootstrap.md`, a tutaj zostaje tylko najważniejsze połączenie klas CSS z JSX, stanem i formularzami.

Bootstrap jest frameworkiem CSS, który dostarcza gotowe klasy, siatkę responsywną, style formularzy oraz zestaw popularnych komponentów interfejsu. W aplikacji React Bootstrap najczęściej pełni rolę warstwy wizualnej: React odpowiada za komponenty, stan i zdarzenia, a Bootstrap za wygląd, odstępy, układ i podstawowe warianty elementów.

Ten rozdział jest skróconym opisem użycia Bootstrapa w React. Pełna dokumentacja Bootstrapa została wydzielona do osobnego pliku: bootstrap.md. Tam znajdują się szersze opisy siatki, formularzy, utilities, komponentów, JavaScriptu Bootstrapa, motywów oraz większe przykłady praktyczne.

### 13.1. Rola Bootstrapa w aplikacji React

W części „Rola Bootstrapa w aplikacji React” Bootstrap dostarcza klasy wyglądu, ale React nadal decyduje o stanie. W praktyce oznacza to, że najpierw wyliczasz stan komponentu, a dopiero potem dobierasz klasy typu `is-invalid`, `active` albo `btn-primary`.

Bootstrap nie zastępuje Reacta. Nie zarządza stanem aplikacji, nie tworzy komponentów JSX i nie decyduje o przepływie danych. Dostarcza natomiast gotowy język klas CSS, dzięki któremu można szybko budować spójny interfejs.

Typowy podział odpowiedzialności wygląda tak:

| OBSZAR        | REACT                                            | BOOTSTRAP                                                                      |
|---------------|--------------------------------------------------|--------------------------------------------------------------------------------|
| Dane          | stan, propsy, tablice, obiekty                   | brak odpowiedzialności                                                         |
| Logika        | funkcje obsługi zdarzeń, warunki, mapowanie list | brak odpowiedzialności                                                         |
| Struktura     | komponenty JSX                                   | klasy układu, np. <code>container</code> , <code>row</code> , <code>col</code> |
| Wygląd        | ewentualnie własne komponenty i style            | przyciski, formularze, karty, alerty, tabele                                   |
| Responsywność | warunkowe renderowanie, jeśli potrzebne          | breakpointy, np. <code>col-md-6</code> , <code>d-lg-flex</code>                |

Najważniejsza zasada: w React nadal piszesz komponenty i logikę po reactowemu, a klasy Bootstrapa traktujesz jako gotowy zestaw stylów.

### 13.2. Instalacja i import stylów

Import Bootstrapa najlepiej umieścić raz w pliku startowym aplikacji, a nie w każdym komponentcie. Własny CSS importuj po Bootstrapie, żeby mógł nadpisać domyślne style.

```
import "bootstrap/dist/css/bootstrap.min.css";
import "./index.css";
```

W projekcie React najczęściej instaluje się Bootstrapa przez npm:

```
npm install bootstrap
```

Następnie importuje się arkusz CSS w głównym pliku aplikacji, na przykład w `src/main.jsx` albo `src/index.js`:

```
import React from "react";
import ReactDOM from "react-dom/client";

import "bootstrap/dist/css/bootstrap.min.css";
import "./index.css";

import App from "./App.jsx";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```



Kolejność importów jest istotna. Najpierw importuje się Bootstrapa, a dopiero potem własny plik CSS. Dzięki temu własne style mogą nadpisać domyślne reguły Bootstrapa.

Jeżeli korzystasz tylko z klas CSS, przycisków, formularzy, kart, siatki i tabel, import CSS wystarczy. Jeżeli chcesz używać komponentów Bootstrapa opartych o JavaScript, takich jak modal, dropdown, tooltip, popover, collapse albo offcanvas, potrzebny jest dodatkowy import bundle:

```
import "bootstrap/dist/js/bootstrap.bundle.min.js";
```

W React często lepiej sterować takimi elementami stanem komponentu zamiast bezpośrednio polegać na atrybutach `data-bs-*`. Dotyczy to szczególnie modali, paneli rozwijanych i zakładek, bo ich widoczność zwykle zależy od danych aplikacji.

13.3. Klasy Bootstrapa w JSX

Klasy Bootstrapa traktuj jak opis wyglądu aktualnego stanu. Gdy stan oznacza błąd, możesz dodać `is-invalid`; gdy element jest aktywny, dodajesz `active`. Sama logika nadal powinna wynikać ze stanu Reacta.

W HTML używa się atrybutu `class`, ale w JSX trzeba używać `className`.

```
function ProductCard() {
  return (
    <article className="card shadow-sm h-100">
      <div className="card-body">
        <span className="badge text-bg-success mb-2">Dostępny</span>
        <h2 className="h5 card-title">Klawiatura mechaniczna</h2>
        <p className="card-text text-secondary">
          Kompaktowa klawiatura z podświetleniem i przełącznikami liniowymi.
        </p>
        <button className="btn btn-primary">Dodaj do koszyka</button>
      </div>
    </article>
  );
}
```

Najczęściej używane grupy klas:

| GRUPA      | PRZYKŁADY                                                                                    | ZASTOSOWANIE                 |
|------------|----------------------------------------------------------------------------------------------|------------------------------|
| Layout     | <code>container</code> , <code>row</code> , <code>col-md-6</code>                            | układ strony i siatka        |
| Spacing    | <code>mt-3</code> , <code>mb-4</code> , <code>p-3</code> , <code>gap-2</code>                | marginesy, paddingi, odstępy |
| Flex       | <code>d-flex</code> , <code>align-items-center</code> , <code>justify-content-between</code> | wyrównanie elementów         |
| Kolory     | <code>text-primary</code> , <code>text-secondary</code> , <code>bg-light</code>              | tekst i tła                  |
| Przyciski  | <code>btn</code> , <code>btn-primary</code> , <code>btn-outline-danger</code>                | akcje użytkownika            |
| Formularze | <code>form-control</code> , <code>form-select</code> , <code>form-check</code>               | pola formularzy              |
| Komponenty | <code>card</code> , <code>alert</code> , <code>badge</code> , <code>navbar</code>            | gotowe elementy interfejsu   |

Przykład responsywnego układu kart:

```
function ProductGrid({ products }) {
  return (
    <div className="container py-4">
```

```

<div className="row g-3">
  {products.map((product) => (
    <div className="col-12 col-md-6 col-xl-4" key={product.id}>
      <article className="card h-100 shadow-sm">
        <div className="card-body">
          <h3 className="h5">{product.name}</h3>
          <p className="text-secondary mb-3">{product.description}</p>
          <strong>{product.price} zł</strong>
        </div>
      </article>
    </div>
  ))}
</div>
</div>
);
}

```

Klasy `col-12 col-md-6 col-xl-4` oznaczają: pełna szerokość na małych ekranach, dwie kolumny od breakpointu `md` i trzy kolumny od breakpointu `xl`.

#### 13.4. Komponenty zależne od stanu Reacta

W części „Komponenty zależne od stanu Reacta” Bootstrap dostarcza klasy wyglądu, ale React nadal decyduje o stanie. W praktyce oznacza to, że najpierw wyliczasz stan komponentu, a dopiero potem dobierasz klasy typu `is-invalid`, `active` albo `btn-primary`.

Bootstrap dobrze łączy się ze stanem Reacta, jeżeli traktujesz klasy jako wynik danych. Przykład: alert może zmieniać wariant zależnie od typu komunikatu.

```

function StatusAlert({ status }) {
  const variants = {
    success: "alert-success",
    warning: "alert-warning",
    error: "alert-danger",
    info: "alert-info",
  };

  const alertClassName = "alert " + (variants[status.type] ?? "alert-secondary");

  return (
    <div className={alertClassName}>
      <strong>{status.title}</strong>
      <p className="mb-0">{status.message}</p>
    </div>
  );
}

```

Podobnie można sterować klasami przycisków, kart, pól formularzy i widoczności sekcji:

```

function SaveButton({ isSaving, isDirty }) {
  return (
    <button className="btn btn-primary" disabled={isSaving || !isDirty}>
      {isSaving ? "Zapisywanie..." : "Zapisz zmiany"}
    </button>
  );
}

```

```

    </button>

  );
}

```

Dla formularzy najważniejsze jest połączenie kontrolowanych pól Reacta z klasami Bootstrapa:

```

function EmailField({ value, onChange }) {
  const isInvalid = value.length > 0 && !value.includes("@");
  const inputClassName = isInvalid ? "form-control is-invalid" : "form-control";

  return (
    <div className="mb-3">
      <label htmlFor="email" className="form-label">E-mail</label>
      <input
        id="email"
        type="email"
        className={inputClassName}
        value={value}
        onChange={(event) => onChange(event.target.value)}
      />
      {isInvalid && (
        <div className="invalid-feedback">Adres e-mail musi zawierać znak @.</div>
      )}
    </div>
  );
}

```

### 13.5. React-Bootstrap czy zwykłe klasy

Klasy Bootstrapa traktuj jak opis wyglądu aktualnego stanu. Gdy stan oznacza błąd, możesz dodać `is-invalid`; gdy element jest aktywny, dodasz `active`. Sama logika nadal powinna wynikać ze stanu Reacta.

Są dwa popularne sposoby używania Bootstrapa w React:

| PODEJŚCIE               | NA CZYM POLEGA                                                                                       | KIEDY UŻYWAĆ                                                       |
|-------------------------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Zwykłe klasy Bootstrapa | piszesz JSX i dodajesz <code>className</code>                                                        | gdy chcesz pełnej kontroli nad strukturą HTML                      |
| React-Bootstrap         | używasz gotowych komponentów React, np. <code>&lt;Button /&gt;</code> , <code>&lt;Modal /&gt;</code> | gdy chcesz komponentów zgodnych z React i mniej ręcznej obsługi JS |

Instalacja React-Bootstrap:

```
npm install react-bootstrap bootstrap
```

Przykład:

```

import Button from "react-bootstrap/Button";
import Card from "react-bootstrap/Card";

function UserCard() {
  return (

```

```

<Card className="shadow-sm">
  <Card.Body>
    <Card.Title>Anna Kowalska</Card.Title>
    <Card.Text className="text-secondary">
      Konto aktywne, ostatnie logowanie dzisiaj.
    </Card.Text>
    <Button variant="primary">Szczegóły</Button>
  </Card.Body>
</Card>
);
}

```

React-Bootstrap nie jest wymagany. Jeżeli wystarczają Ci klasy CSS, możesz zostać przy zwykłym Bootstrapie. Jeżeli często używasz modali, dropdownów, zakładek lub komponentów wymagających interakcji, React-Bootstrap może uprościć kod.

### 13.6. Mini przykład: formularz i karta

W części „Mini przykład: formularz i karta” Bootstrap dostarcza klasy wyglądu, ale React nadal decyduje o stanie. W praktyce oznacza to, że najpierw wyliczasz stan komponentu, a dopiero potem dobierasz klasy typu `is-invalid`, `active` albo `btn-primary`.

Poniższy przykład pokazuje typowy sposób łączenia Reacta i Bootstrapa: stan formularza jest w React, a wygląd pól, przycisków, listy i komunikatów pochodzi z klas Bootstrapa.

```

import { useMemo, useState } from "react";

const initialTasks = [
  { id: 1, title: "Przygotować strukturę komponentów", done: true },
  { id: 2, title: "Dodać formularz kontaktowy", done: false },
  { id: 3, title: "Sprawdzić widok mobilny", done: false },
];

function App() {
  const [tasks, setTasks] = useState(initialTasks);
  const [title, setTitle] = useState("");

  const remainingCount = useMemo(
    () => tasks.filter((task) => !task.done).length,
    [tasks]
  );

  function addTask(event) {
    event.preventDefault();

    const trimmedTitle = title.trim();
    if (trimmedTitle.length < 3) {
      return;
    }

    const nextTask = {
      id: Date.now(),
      title: trimmedTitle,
      done: false,
    };
  }

```

```

    setTasks((currentTasks) => [nextTask, ...currentTasks]);
    setTitle("");
  }

function toggleTask(taskId) {
  setTasks((currentTasks) =>
    currentTasks.map((task) =>
      task.id === taskId ? { ...task, done: !task.done } : task
    )
  );
}

return (
  <main className="bg-light min-vh-100 py-5">
    <div className="container">
      <div className="row justify-content-center">
        <div className="col-12 col-lg-8 col-xl-6">
          <section className="card shadow-sm border-0">
            <div className="card-body p-4">
              <div className="d-flex justify-content-between align-items-start gap-3 mb-4">
                <div>
                  <h1 className="h3 mb-1">Lista zadań</h1>
                  <p className="text-secondary mb-0">
                    Pozostało do wykonania: {remainingCount}
                  </p>
                </div>
                <span className="badge text-bg-primary rounded-pill">
                  {tasks.length} razem
                </span>
              </div>
              <form className="row g-2 mb-4" onSubmit={addTask}>
                <div className="col-12 col-md">
                  <label htmlFor="taskTitle" className="visually-hidden">
                    Treść zadania
                  </label>
                  <input
                    id="taskTitle"
                    className="form-control"
                    value={title}
                    onChange={(event) => setTitle(event.target.value)}
                    placeholder="Nowe zadanie"
                  />
                </div>
                <div className="col-12 col-md-auto">
                  <button className="btn btn-primary w-100" type="submit">
                    Dodaj
                  </button>
                </div>
              </form>

              {tasks.length === 0 ? (
                <div className="alert alert-info mb-0">
                  Lista jest pusta. Dodaj pierwsze zadanie.
                </div>
              ) : null}
            </div>
          </section>
        </div>
      </div>
    </div>
  </main>
);

```

```

    </div>
  ) : (
    <ul className="list-group list-group-flush">
      {tasks.map((task) => (
        <li
          className="list-group-item d-flex align-items-center gap-3 px-0"
          key={task.id}
        >
          <input
            className="form-check-input mt-0"
            type="checkbox"
            checked={task.done}
            onChange={() => toggleTask(task.id)}
          />
          <span className={task.done ? "text-decoration-line-through text-secondary" : ""}>
            {task.title}
          </span>
        </li>
      ))}
    </ul>
  )}
</div>
</section>
</div>
</div>
</div>
</main>
);
}

export default App;

```

W przykładzie Bootstrap odpowiada za układ strony, kartę, formularz, listę, badge, alert i responsywność. React odpowiada za dodawanie elementów, zmianę stanu checkboxa, przeliczanie liczby pozostałych zadań i warunkowe renderowanie pustej listy.

## 14. Obrazy i zasoby statyczne

Zasoby statyczne są częstym źródłem błędów, bo ścieżka do pliku zależy od miejsca jego przechowywania. Plik z `public` odwołuje się przez ścieżkę URL, a plik z `src` zwykle importuje się jak moduł.

```

import logo from "./assets/logo.png";

function Header() {
  return <img src={logo} alt="Logo aplikacji" />;
}

```

W projekcie React istnieją **dwa główne miejsca**, w których możemy przechowywać obrazy i inne zasoby statyczne (ikony, czcionki, pliki SVG): folder `public/` oraz folder `src/`. Każde z tych miejsc działa inaczej i jest przeznaczone do innych scenariuszy. Obrazy umieszczone w folderze `public/` są serwowane statycznie przez serwer deweloperski — nie przechodzą przez żaden proces budowania ani optymalizacji. Są dostępne dokładnie pod taką nazwą, jaką im nadaliśmy, np. `/logo.png`. Z kolei obrazy umieszczone w folderze `src/` i importowane za pomocą instrukcji `import` przechodzą przez Webpack (lub Vite) podczas budowania aplikacji. Webpack nadaje im **unikalne nazwy hashowane** (np. `logo.a1b2c3d4.png`), co rozwiązuje problem cache przeglądarki — gdy

zmienimy obraz, jego hash się zmieni i przeglądarka pobierze nową wersję zamiast wyświetlać starą z pamięci podręcznej. Dodatkowo, Webpack ostrzeże nas podczas kompilacji, jeśli importowany plik nie istnieje, co eliminuje ryzyko zepsutych obrazów na produkcji. Dla małych obrazów (poniżej 10 KB) Webpack automatycznie zamieni je na format Base64 i osadzi bezpośrednio w kodzie JavaScript, co zmniejszy liczbę żądań HTTP.

| CECHA                          | FOLDER <code>PUBLIC/</code>                                                                                | FOLDER <code>SRC/</code> (IMPORT)                             |
|--------------------------------|------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Ścieżka w kodzie               | Bezpośrednia, np. <code>"/Logo.png"</code>                                                                 | Przez import: <code>import logo from "./Logo.png"</code>      |
| Optymalizacja Webpack          | Brak — plik serwowany jak jest                                                                             | Tak — hashowanie, minifikacja, Base64 dla małych plików       |
| Ostrzeżenie o brakującym pliku | Brak — błąd dopiero w przeglądarce (404)                                                                   | Tak — błąd kompilacji, aplikacja się nie zbuduje              |
| Zmiana nazwy pliku w buildzie  | Nie — zawsze ta sama nazwa                                                                                 | Tak — np. <code>logo.a1b2c3.png</code> (cache busting)        |
| Kiedy używać                   | Pliki dynamiczne (ścieżka zależy od zmiennej), <code>favicon.ico</code> , pliki <code>manifest.json</code> | Obrazy używane bezpośrednio w komponentach, ikony, ilustracje |

#### 14.1. Obrazy z folderu public

Folder `public` jest dobry dla plików, które mają stały adres URL albo których nazwa jest znana dopiero w czasie działania aplikacji. Minusem jest brak kontroli kompilatora nad błędną ścieżką.

```

```

Obrazy umieszczone w folderze `public/` są dostępne bezpośrednio po ścieżce:

```
{/* Obraz: public/zdjecia/kwiat.jpg */}


{/* Obraz: public/logo.png */}

```

Zalety: prostota, brak importu. Wady: brak optymalizacji przez Webpack.

#### 14.2. Obrazy z folderu src — import

Import z `src` daje kontrolę narzędzi budujących projekt. Jeśli plik nie istnieje, aplikacja zgłosi błąd podczas kompilacji, a nie dopiero jako brakujący obraz w przeglądarce.

```
import avatar from "../assets/avatar.png";
return <img src={avatar} alt="Avatar użytkownika" />;
```

```
// Plik: src/App.js
import kwiatImg from "../zdjecia/kwiat.jpg";

function App() {
  return <img src={kwiatImg} alt="Kwiat" />;
}
```

Zalety: Webpack optymalizuje obraz, ostrzeże, jeśli plik nie istnieje.

### 14.3. Obraz zależny od stanu

Przy obrazach zdecyduj, czy ścieżka ma być kontrolowana przez bundler. Import z `src` jest bezpieczniejszy dla obrazów używanych w komponentach, a `public` przydaje się dla plików wybieranych dynamicznie.

```
// Plik: src/App.js
import { useState } from "react";

function App() {
  const [kategoria, setKategoria] = useState("kwiaty");

  // Obraz z folderu public
  const sciezka = `/zdjecia/${kategoria}.jpg`;

  return (
    <div className="container mt-4">
      <img src={sciezka} alt={kategoria} style={{ maxWidth: "300px" }} />
      <div className="mt-2">
        <button className="btn btn-primary me-2" onClick={() => setKategoria("kwiaty")}>Kwiaty</button>
        <button className="btn btn-primary me-2" onClick={() => setKategoria("krajobrazy")}>Krajobrazy</button>
        <button className="btn btn-primary" onClick={() => setKategoria("miasto")}>Miasto</button>
      </div>
    </div>
  );
}

export default App;
```

### 14.4. Obrazy w kolekcjach (tablicach obiektów)

Przy obrazach zdecyduj, czy ścieżka ma być kontrolowana przez bundler. Import z `src` jest bezpieczniejszy dla obrazów używanych w komponentach, a `public` przydaje się dla plików wybieranych dynamicznie.

```
const zdjecia = [
  { id: 1, plik: "kwiat.jpg", opis: "Kwiat" },
  { id: 2, plik: "gora.jpg", opis: "Góra" },
  { id: 3, plik: "morze.jpg", opis: "Morze" },
];

// Renderowanie – obrazy z folderu public/zdjecia/
{zdjecia.map((z) => (
  <div key={z.id} className="card" style={{ width: "200px" }}>
    <img src={`/${zdjecia}/${z.plik}`} className="card-img-top" alt={z.opis} />
    <div className="card-body">
      <p className="card-text">{z.opis}</p>
    </div>
  </div>
))}
```



## 14.5. Atrybut alt — dostępność

`alt` powinien opisywać sens obrazu, a nie zaczynać się od „obrazek przedstawia”. Jeśli grafika jest tylko dekoracją, pusty `alt=""` jest lepszy niż powtarzanie niepotrzebnej informacji.

Atrybut `alt` jest wymagany na obrazkach. Opisuje zawartość obrazu dla czytników ekranu i wyświetla się, gdy obraz nie może być załadowany:

```
{/* Poprawnie */}


{/* Dla obrazów dekoracyjnych – pusty alt */}

```

## 15. Przepływ danych — props w górę i w dół

Przepływ danych w React jest jednokierunkowy. Rodzic przekazuje dane do dziecka przez propsy, a dziecko może zgłosić akcję przez funkcję również przekazaną w propsach. To porządkuje aplikację i ułatwia znalezienie miejsca, w którym zmienia się stan.

```
<TaskItem task={task} onToggle={toggleTask} />
```

Przepływ danych w React to **najważniejsza koncepcja architekuralna**, którą musisz zrozumieć, by tworzyć poprawne aplikacje. React stosuje wzorec **jednokierunkowego przepływu danych** (ang. *one-way data flow* lub *unidirectional data flow*). Oznacza to, że dane **ZAWSZE płyną z góry na dół** — od komponentu rodzica do komponentu dziecka — za pośrednictwem propsów. Dziecko nigdy nie może bezpośrednio zmodyfikować danych rodzica ani wysłać mu czegokolwiek w górę. Gdy dziecko musi powiedzieć coś rodzicowi (np. że użytkownik kliknął przycisk lub wpisał tekst), robi to **pośrednio** — wywołując funkcję zwrrotną (callback), którą rodzic wcześniej przekazał mu jako prop. Ta funkcja callback, po wywołaniu przez dziecko, zmienia stan w rodzicu za pomocą `setState`. React wykrywa zmianę stanu, automatycznie re-renderuje rodzica, a nowe dane spływają ponownie w dół do wszystkich dzieci jako zaktualizowane propsy. Ten cykl jest przewidywalny i łatwy do debugowania, ponieważ zawsze wiadomo, **skąd** dane przychodzą i **kto** jest odpowiedzialny za ich zmianę. W przeciwieństwie do dwukierunkowego bindingu (ang. *two-way binding*) znanego z Angular, jednokierunkowy przepływ eliminuje sytuacje, w których trudno ustalić, co zmieniło dane.

### Schemat przepływu danych w React:

```
1. Stan żyje w RODZICU (np. useState)
    |
    v
2. Props lecą W DOL do dziecka
    |
    v
3. Dziecko WYWOŁUJE callback (np. onClick, onSubmit)
    |
    v
4. Callback ZMIENIA stan rodzica (setState)
    |
    v
5. React RE-RENDERUJE rodzica i dzieci z nowymi danymi
    |
    +----- powrot do kroku 2 -----+
```

### 15.1. Dane płyną z góry na dół (top-down)

Jeśli kilka komponentów musi znać tę samą wartość, nie duplikuj jej. Przenieś stan do wspólnego rodzica i przekaz w dół dane oraz funkcje zmieniające.

W React dane (props) płyną **zawsze z rodzica do dziecka** — nigdy odwrotnie. Rodzic przekazuje dane jako props, a dziecko je odbiera i wyświetla:

```
// Plik: src/App.js (rodzic)
import KursKarta from "../components/KursKarta";

function App() {
  const kursy = [
    { id: 1, nazwa: "React", cena: 199 },
    { id: 2, nazwa: "JavaScript", cena: 149 },
  ];

  return (
    <div className="container mt-4">
      {kursy.map((kurs) => (
        <KursKarta key={kurs.id} nazwa={kurs.nazwa} cena={kurs.cena} />
      ))}
    </div>
  );
}

export default App;
```

```
// Plik: src/components/KursKarta.js (dziecko)
function KursKarta({ nazwa, cena }) {
  return (
    <div className="card mb-2">
      <div className="card-body">
        <h5>{nazwa}</h5>
        <p>Cena: {cena} zł</p>
      </div>
    </div>
  );
}

export default KursKarta;
```

### 15.2. Callback — dziecko zgłasza zdarzenie rodzicowi

Callback przekazywany do dziecka powinien opisywać zdarzenie z perspektywy dziecka, np. `onRemove`, `onSelect`, `onToggle`. Rodzic decyduje, co ta akcja realnie zmieni w stanie.

```
<ProductCard product={product} onSelect={setSelectedProductId} />
```

Dziecko nie może bezpośrednio zmienić stanu rodzica. Zamiast tego rodzic **przekazuje funkcję** (callback) jako prop, a dziecko ją wywołuje:

```
// Plik: src/App.js (rodzic)

import { useState } from "react";
import DodajKurs from "../components/DodajKurs";

function App() {
  const [kursy, setKursy] = useState(["React", "JavaScript"]);

  // Ta funkcja jest przekazywana jako prop do dziecka
  function handleDodaj(nowyKurs) {
    setKursy((prev) => [...prev, nowyKurs]);
  }

  return (
    <div className="container mt-4">
      <DodajKurs onDodaj={handleDodaj} />
      <ul className="list-group mt-3">
        {kursy.map((kurs, i) => (
          <li key={i} className="list-group-item">{kurs}</li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

```
// Plik: src/components/DodajKurs.js (dziecko)

import { useState } from "react";

function DodajKurs({ onDodaj }) {
  const [tekst, setTekst] = useState("");

  function handleSubmit(e) {
    e.preventDefault();
    if (tekst.trim() === "") return;
    onDodaj(tekst); // Wywołanie callbacka z rodzica
    setTekst(""); // Czyszczenie pola
  }

  return (
    <form onSubmit={handleSubmit}>
      <div className="input-group">
        <input
          type="text"
          className="form-control"
          value={tekst}
          onChange={(e) => setTekst(e.target.value)}
          placeholder="Nazwa kursu"
        />
        <button type="submit" className="btn btn-success">Dodaj</button>
      </div>
    </form>
  );
}
```

```

    </form>

  );
}

export default DodajKurs;

```

### 15.3. Lifting state up — podnoszenie stanu

Lifting state up stosuj wtedy, gdy dwa komponenty muszą widzieć tę samą wartość albo reagować na tę samą zmianę. Stan przenosi się do najbliższego wspólnego rodzica, a dzieci dostają dane i callbacki.

```

<SearchInput value={query} onChange={setQuery} />
<Results query={query} />

```

Gdy dwa komponenty muszą dzielić ten sam stan, stan przenosi się do ich **wspólnego rodzica**. Rodzic trzyma stan i przekazuje go do obu dzieci:

```

// Plik: src/App.js – rodzic trzyma stan
import { useState } from "react";
import Formularz from "./components/Formularz";
import Podglad from "./components/Podglad";

function App() {
  // Stan jest w rodzicu – oba dzieci mają do niego dostęp
  const [imie, setImie] = useState("");

  return (
    <div className="container mt-4">
      <Formularz imie={imie} onZmiana={setImie} />
      <Podglad imie={imie} />
    </div>
  );
}

export default App;

```

```

// Plik: src/components/Formularz.js – dziecko edytujące
function Formularz({ imie, onZmiana }) {
  return (
    <input
      type="text"
      className="form-control mb-3"
      value={imie}
      onChange={(e) => onZmiana(e.target.value)}
      placeholder="Wpisz imię"
    />
  );
}

```

```
export default Formularz;
```

```
// Plik: src/components/Podglad.js — dziecko wyświetlające
function Podglad({ imie }) {
  return <p>Podgląd: <strong>{imie || "(puste)"}</strong></p>;
}

export default Podglad;
```

#### 15.4. Pełny przykład wieloplikowy z przepływem danych

Jeśli kilka komponentów musi znać tę samą wartość, nie duplikuj jej. Przenieś stan do wspólnego rodzica i przekaz w dół dane oraz funkcje zmieniające.

Diagram przepływu danych:

```
App (stan: zadania)
├─ ZadanieFormularz (callback: onDodaj)
└─ ZadanieLista (props: zadania, callback: onUsun)
   └─ ZadanieElement (props: zadanie, callback: onUsun)
```

- **Dół (Props):** App → ZadanieLista → ZadanieElement (dane płyną w dół).
- **Góra (Callbacks):** ZadanieFormularz → App (dziecko zgłasza zdarzenie do rodzica).
- **Odświeżenie:** Gdy rodzic zmieni stan, React automatycznie re-renderuje wszystkie dzieci z nowymi danymi.

## 16. useEffect i efekty uboczne

`useEffect` służy do synchronizacji komponentu ze światem zewnętrznym: API, timerem, localStorage, subskrypcją lub tytułem dokumentu. Nie powinien zastępować zwykłych obliczeń, które można wykonać bezpośrednio podczas renderowania.

```
useEffect(() => {
  document.title = `Elementy: ${items.length}`;
}, [items.length]);
```

Aby zrozumieć `useEffect`, musimy najpierw zrozumieć pojęcie **efektu ubocznego** (ang. *side effect*). W programowaniu funkcyjnym istnieje koncepcja **czystej funkcji** (*pure function*) — to funkcja, która na podstawie tych samych danych wejściowych **ZAWSZE** zwraca dokładnie ten sam wynik i nie zmienia niczego poza sobą (nie modyfikuje zmiennych globalnych, nie wysyła żądań sieciowych, nie zapisuje do pliku). Komponent React z założenia powinien zachowywać się jak czysta funkcja — na podstawie otrzymanych propsów i aktualnego stanu zawsze powinien zwracać ten sam JSX. Jeśli prześlemy mu `nazwa="React"` i `cena={199}`, to za każdym razem powinien wyrenderować identyczny fragment interfejsu. Ale w prawdziwych aplikacjach potrzebujemy operacji, które wykraczają poza czyste renderowanie — musimy pobrać dane z serwera (`fetch`), zmienić tytuł zakładki przeglądarki (`document.title`), ustawić timer (`setInterval`), zapisać coś do `localStorage` czy nasłuchiwać na zdarzenia okna (`window.addEventListener`). Wszystkie te operacje to właśnie **efekty uboczne** — czynności, które dotyczą świata zewnętrznego poza samym komponentem. Hook `useEffect` jest specjalnie stworzony do obsługi takich operacji — pozwala nam powiedzieć Reactowi: po wyrenderowaniu komponentu, wykonaj jeszcze tę dodatkową operację. Dzięki temu logika renderowania (czysta) jest oddzielona od logiki efektów ubocznych (nieczysta).

| CZYSTA OPERACJA (BEZPOŚREDNIO W CIELE KOMPONENTU)                                              | EFEKT UBOCZNY (WYMAGA <code>USEEFFECT</code> )                                 |
|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Obliczenie sumy: <code>const suma = ceny.reduce((a, b) =&gt; a + b, 0)</code>                  | Pobranie danych z API: <code>fetch("https://api.example.com/dane")</code>      |
| Filtrowanie tablicy: <code>const aktywne = zadania.filter(z =&gt; !z.done)</code>              | Zmiana tytułu strony: <code>document.title = "Nowy tytuł"</code>               |
| Formatowanie daty: <code>new Date().toLocaleDateString("pl-PL")</code>                         | Ustawienie timera: <code>setInterval(() =&gt; ..., 1000)</code>                |
| Warunkowe renderowanie: <code>{zaLogowany &amp;&amp; &lt;Panel /&gt;}</code>                   | Zapis do localStorage: <code>localStorage.setItem("klucz", wartosc)</code>     |
| Mapowanie danych na JSX: <code>lista.map(el =&gt; &lt;Li key={el.id}&gt;...&lt;/Li&gt;)</code> | Nasłuchiwanie zdarzeń okna: <code>window.addEventListener("resize", fn)</code> |
| Laczenie stringów: tekst z template literal                                                    | Subskrypcja WebSocket: <code>socket.on("message", handler)</code>              |

### 16.1. Po co jest useEffect

Efekt powinien mieć konkretny powód: pobranie danych, zapis do localStorage, timer, subskrypcję albo zmianę tytułu dokumentu. Jeśli nie synchronizujesz się z czymś zewnętrznym, zwykle nie potrzebujesz efektu.

`useEffect` to hook do wykonywania **efektów ubocznych** — operacji, które nie dotyczą bezpośrednio wyniku renderowania. Przykłady:

- Pobranie danych z API lub localStorage
- Ustawienie tytułu strony
- Ustawienie timera

```
import { useEffect } from "react";
```

### 16.2. useEffect przy starcie aplikacji

Efekt powinien mieć konkretny powód: pobranie danych, zapis do localStorage, timer, subskrypcję albo zmianę tytułu dokumentu. Jeśli nie synchronizujesz się z czymś zewnętrznym, zwykle nie potrzebujesz efektu.

```
// Plik: src/App.js
import { useState, useEffect } from "react";

function App() {
  const [dane, setDane] = useState([]);

  // useEffect z pustą tablicą zależności [] – wykona się RAZ po pierwszym renderze
  useEffect(() => {
    console.log("Komponent się zamontował!");
    document.title = "Moja Aplikacja";
  }, []);

  return (
    <div className="container mt-4">
      <h1>Aplikacja</h1>
    </div>
  );
}

export default App;
```

### 16.3. Tablica zależności

Tablica zależności mówi Reactowi, kiedy efekt ma zostać uruchomiony ponownie. Pusta tablica oznacza start komponentu, brak tablicy oznacza każdy render, a konkretne wartości oznaczają reakcję na ich zmianę.

```
useEffect(() => {
  fetchUser(userId);
}, [userId]);
```

Tablica zależności `[]` kontroluje, **kiedy** efekt się uruchomi:

```
// Wykonuje się po KAŻDYM renderze (brak tablicy zależności)
useEffect(() => {
  console.log("Render!");
});

// Wykonuje się TYLKO RAZ — po pierwszym renderze (pusta tablica)
useEffect(() => {
  console.log("Montowanie!");
}, []);

// Wykonuje się gdy zmieni się wartość 'szukaj'
useEffect(() => {
  console.log("Wyszukiwanie:", szukaj);
}, [szukaj]);
```

| TABLICA ZALEŻNOŚCI  | KIEDY SIĘ URUCHOMI                               |
|---------------------|--------------------------------------------------|
| Brak                | Po każdym renderze                               |
| <code>[]</code>     | Tylko raz — po pierwszym renderze                |
| <code>[a, b]</code> | Gdy zmieni się <code>a</code> lub <code>b</code> |

### 16.4. Cleanup — sprzątanie efektu

Cleanup jest konieczny przy timerach, subskrypcjach i nasłuchiowaniu zdarzeń. Bez niego komponent może zostawić działający kod po odmontowaniu.

```
useEffect(() => {
  const id = setInterval(tick, 1000);
  return () => clearInterval(id);
}, []);
```

Efekt może zwrócić funkcję czyszczącą (cleanup), która wykona się przed następnym uruchomieniem efektu lub gdy komponent się odmontowuje:

```
useEffect(() => {
  const timer = setInterval(() => {
    console.log("Tykanie...");
  }, 1000);

  // Cleanup — sprzątanie przy odmontowywaniu
```

```
return () => {
  clearInterval(timer);
};
}, []);
```

## 16.5. localStorage — zapis i odczyt danych

Przy localStorage pamiętaj, że zapisuje tekst. Obiekty i tablice trzeba zamienić przez `JSON.stringify`, a przy odczycie odtworzyć przez `JSON.parse`.

```
localStorage.setItem("tasks", JSON.stringify(tasks));
const saved = JSON.parse(localStorage.getItem("tasks") ?? "[]");
```

`localStorage` pozwala zapisywać dane w przeglądarce, które przetrwają odświeżenie strony:

```
// Plik: src/App.js
import { useState, useEffect } from "react";

function App() {
  // Odczyt z localStorage przy starcie
  const [zadania, setZadania] = useState(() => {
    const zapisane = localStorage.getItem("zadania");
    return zapisane ? JSON.parse(zapisane) : [];
  });

  // Zapis do localStorage przy każdej zmianie zadań
  useEffect(() => {
    localStorage.setItem("zadania", JSON.stringify(zadania));
  }, [zadania]);

  // ... reszta komponentu
  return (
    <div className="container mt-4">
      <p>Liczba zadań: {zadania.length}</p>
    </div>
  );
}

export default App;
```

### Kluczowe funkcje localStorage:

| METODA                                                | OPIS                                   |
|-------------------------------------------------------|----------------------------------------|
| <code>localStorage.setItem("klucz", "wartość")</code> | Zapisuje dane (string)                 |
| <code>localStorage.getItem("klucz")</code>            | Odczytuje dane (string lub null)       |
| <code>localStorage.removeItem("klucz")</code>         | Usuwa dane                             |
| <code>JSON.stringify(obiekt)</code>                   | Zamienia obiekt/tablicę na string JSON |
| <code>JSON.parse(tekst)</code>                        | Zamienia string JSON na obiekt/tablicę |



## 16.6. Typowe pułapki useEffect

Typowa pułapka `useEffect` to dopisanie efektu tam, gdzie wystarczy zwykłe obliczenie. Druga pułapka to brak zależności w tablicy, przez co efekt korzysta ze starej wartości.

```
// PUŁAPKA 1: Brak tablicy zależności – nieskończona pętla!
useEffect(() => {
  setLicznik(licznik + 1); // Zmiana stanu → re-render → efekt → zmiana stanu → ...
}); // ← brak [] to powtarzanie w nieskończoność

// PUŁAPKA 2: Zapomnienie o zależności
useEffect(() => {
  console.log("Szukaj:", szukaj); // Używasz szukaj, ale nie masz go w []
}, []); // ← efekt uruchomi się tylko raz, nie reagując na zmiany szukaj

// POPRAWNIE:
useEffect(() => {
  console.log("Szukaj:", szukaj);
}, [szukaj]); // ← efekt uruchomi się gdy zmieni się szukaj
```

## 17. useRef — referencje do elementów DOM

`useRef` jest przydatny, gdy potrzebujesz zapamiętać wartość bez wywoływania ponownego renderu albo odwołać się do rzeczywistego elementu DOM. Nie używaj go jako zamiennika stanu dla danych, które mają być widoczne na ekranie.

### 17.1. Czym jest useRef

Ref nie jest zamiennikiem stanu. Używaj go do dostępu do DOM albo przechowania wartości technicznej, która nie musi odświeżać widoku.

`useRef` to hook, który tworzy „pojemnik” na wartość, która **nie powoduje re-renderu** przy zmianie. Najczęściej używany do uzyskania referencji do elementu DOM:

```
import { useRef } from "react";
```

### 17.2. Ustawianie fokusa na polu

Focus przez `useRef` jest dobrym przykładem pracy z DOM, której nie da się wygodnie zrobić samym JSX. Najczęściej używa się go po kliknięciu przycisku albo po pokazaniu formularza.

```
const inputRef = useRef(null);

<button onClick={() => inputRef.current?.focus()}>Ustaw fokus</button>
```

```
// Plik: src/App.js
import { useState, useRef } from "react";

function App() {
  const [imie, setImie] = useState("");
  const inputRef = useRef(null); // Tworzymy referencję

  function handleReset() {
```

```

    setImie("");
    inputRef.current.focus(); // Ustawiamy fokus na polu
  }

  return (
    <div className="container mt-4">
      <input
        ref={inputRef} // Przypisujemy referencję do elementu
        type="text"
        className="form-control mb-2"
        value={imie}
        onChange={(e) => setImie(e.target.value)}
        placeholder="Wpisz imię"
      />
      <button className="btn btn-secondary" onClick={handleReset}>
        Wyczyść i ustaw fokus
      </button>
    </div>
  );
}

export default App;

```

### 17.3. useRef vs useState

Ref nie jest zamiennikiem stanu. Używaj go do dostępu do DOM albo przechowania wartości technicznej, która nie musi odświeżać widoku.

| CECHA                              | USESTATE                  | USEREF                    |
|------------------------------------|---------------------------|---------------------------|
| Zmiana wartości powoduje re-render | Tak                       | Nie                       |
| Typowe użycie                      | Dane wyświetlane w widoku | Referencje do DOM, timery |
| Dostęp do wartości                 | zmienna                   | ref.current               |

## 18. Dane lokalne, JSON i fetch

Dane w aplikacji mogą pochodzić z kodu, pliku JSON, folderu `public` albo z API. Niezależnie od źródła, komponent powinien mieć jasne stany: ładowanie, sukces, błąd i brak danych. Dzięki temu interfejs nie zależy od przypadku.

```

if (loading) return <p>Ładowanie...</p>;
if (error) return <p>Nie udało się pobrać danych.</p>;
if (items.length === 0) return <p>Brak danych.</p>;

```

### 18.1. Tablice danych w kodzie

Tablice wpisane w kodzie są dobre dla danych startowych i ćwiczeń. Trzymaj je poza komponentem, jeśli nie zależą od stanu, żeby nie tworzyć tej samej tablicy przy każdym renderze.

Najprostszy sposób — dane wpisane bezpośrednio w pliku:

```

// Plik: src/data/kursy.js
const kursy = [

```

```
{ id: 1, nazwa: "HTML i CSS", cena: 99 },
{ id: 2, nazwa: "JavaScript", cena: 149 },
{ id: 3, nazwa: "React", cena: 199 },
];

export default kursy;
```

```
// Plik: src/App.js
import kursy from "../data/kursy";

function App() {
  return (
    <ul>
      {kursy.map((k) => (
        <li key={k.id}>{k.nazwa} – {k.cena} zł</li>
      ))}
    </ul>
  );
}
```

## 18.2. Import pliku JSON

Import JSON jest wygodny, gdy dane są częścią projektu i nie zmieniają się po wdrożeniu. Jeśli dane mają być edytowane przez użytkowników albo serwer, lepszy będzie fetch z API.

Można bezpośrednio importować plik JSON:

```
// Plik: src/data/filmy.json
[
  { "id": 1, "tytuł": "Matrix", "rok": 1999 },
  { "id": 2, "tytuł": "Incepcja", "rok": 2010 },
  { "id": 3, "tytuł": "Titanic", "rok": 1997 }
]
```

```
// Plik: src/App.js
import filmy from "../data/filmy.json";

function App() {
  return (
    <ul>
      {filmy.map((f) => (
        <li key={f.id}>{f.tytuł} ({f.rok})</li>
      ))}
    </ul>
  );
}
```

### 18.3. Fetch z folderu public

Dla danych w części „Fetch z folderu public” zaplanuj osobne zachowanie dla ładowania, błędu, pustej odpowiedzi i poprawnych danych. Dopiero wtedy komponent jest odporny na realne odpowiedzi z pliku albo API.

Pliki JSON umieszczone w `public/` można pobrać za pomocą `fetch`:

```
// Plik: src/App.js
import { useState, useEffect } from "react";

function App() {
  const [dane, setDane] = useState([]);
  const [ladowanie, setLadowanie] = useState(true);

  useEffect(() => {
    fetch("/dane.json") // Plik w public/dane.json
      .then((response) => response.json())
      .then((data) => {
        setDane(data);
        setLadowanie(false);
      })
      .catch((error) => {
        console.error("Błąd pobierania:", error);
        setLadowanie(false);
      });
  }, []);

  if (ladowanie) {
    return <p>Ładowanie...</p>;
  }

  return (
    <ul>
      {dane.map((el) => (
        <li key={el.id}>{el.nazwa}</li>
      ))}
    </ul>
  );
}

export default App;
```

### 18.4. Parsowanie danych tekstowych

Parsowanie tekstu warto zamknąć w osobnej funkcji, która zwraca tablicę obiektów. Dzięki temu komponent nie musi wiedzieć, czy dane przyszły z CSV, TXT czy ręcznie wpisanej listy.

Gdy dane z pliku `dane.txt` trzeba przetworzyć:

```
// Dane w formacie: "nazwa;kategoria;liczba"
const daneRaw = `kwiat.jpg;kwiaty;12
gora.jpg;krajobrazy;34
roza.jpg;kwiaty;7`;

const zdjecia = daneRaw.split("\n").map((linia, index) => {
```

```

const [nazwa, kategoria, pobrania] = linia.split(";");
return {
  id: index + 1,
  nazwa: nazwa,
  kategoria: kategoria,
  pobrania: Number(pobrania),
};
});

console.log(zdjecia);

// [
//   { id: 1, nazwa: "kwiat.jpg", kategoria: "kwiaty", pobrania: 12 },
//   { id: 2, nazwa: "gora.jpg", kategoria: "krajobrazy", pobrania: 34 },
//   { id: 3, nazwa: "roza.jpg", kategoria: "kwiaty", pobrania: 7 },
// ]

```

### 18.5. Czym jest API

API to umowa między aplikacją a serwerem: pod jaki adres wysłać żądanie, jaką metodą, z jakimi danymi i jakiej odpowiedzi oczekiwać. W React interesuje Cię głównie moment pobrania danych i sposób zapisania ich w stanie.

**API** (Application Programming Interface) to sposób komunikacji między aplikacjami. W React najczęściej oznacza to pobieranie lub wysyłanie danych do serwera przez HTTP.

Przykład z życia:

- React wyświetla formularz logowania
- użytkownik wpisuje dane
- aplikacja wysyła dane do API
- serwer odsyła odpowiedź: sukces, błąd albo dodatkowe dane

W aplikacji frontendowej React zwykle odpowiada za:

- pokazanie danych użytkownikowi
- przechowywanie danych w stanie (`useState`)
- pobranie danych po starcie lub po akcji użytkownika (`useEffect`, eventy)
- pokazanie stanu ładowania i błędu

API zwykle odpowiada za:

- zapis danych w bazie
- sprawdzenie poprawności danych
- logowanie i uprawnienia
- zwrócenie danych w formacie JSON

### 18.6. Endpoint, metoda HTTP i status odpowiedzi

Dla danych w części „Endpoint, metoda HTTP i status odpowiedzi” zaplanuj osobne zachowanie dla ładowania, błędu, pustej odpowiedzi i poprawnych danych. Dopiero wtedy komponent jest odporny na realne odpowiedzi z pliku albo API.

Adres, pod który wysyłamy zapytanie, nazywa się **endpointem**.

```
https://jsonplaceholder.typicode.com/users
```

Najczęstsze metody HTTP:

| METODA      | DO CZEGO SŁUŻY          | PRZYKŁAD                  |
|-------------|-------------------------|---------------------------|
| GET         | Pobieranie danych       | lista użytkowników        |
| POST        | Dodawanie nowych danych | nowy formularz kontaktowy |
| PUT / PATCH | Aktualizacja danych     | zmiana profilu            |
| DELETE      | Usuwanie danych         | usunięcie zadania         |

Status odpowiedzi mówi, czy operacja się udała:

| STATUS | ZNACZENIE                      |
|--------|--------------------------------|
| 200    | OK — udało się                 |
| 201    | Created — utworzono nowy zasób |
| 400    | Bad Request — błędne dane      |
| 401    | Unauthorized — brak logowania  |
| 404    | Not Found — nie znaleziono     |
| 500    | Server Error — błąd serwera    |

Ważne: `fetch()` nie rzuca błędu dla statusu `404` albo `500`. Trzeba samodzielnie sprawdzić `response.ok`.

```
const response = await fetch("https://jsonplaceholder.typicode.com/users");

if (!response.ok) {
  throw new Error(`Błąd HTTP: ${response.status}`);
}

const data = await response.json();
```

## 18.7. Pobieranie danych z zewnętrznego API

Przy pobieraniu danych obsłuż też przerwanie żądania, jeśli komponent zniknie z ekranu. Do tego służy `AbortController`.

```
useEffect(() => {
  const controller = new AbortController();
  fetch(url, { signal: controller.signal });
  return () => controller.abort();
}, [url]);
```

Dane z API najczęściej pobieramy w `useEffect`, bo pobranie danych jest **efektem ubocznym** — nie jest zwykłym renderowaniem JSX.

```
// Plik: src/App.js
import { useEffect, useState } from "react";

function App() {
  const [uzytkownicy, setUzytkownicy] = useState([]);

  useEffect(() => {
    async function pobierzUzytkownikow() {
```

```

    const response = await fetch("https://jsonplaceholder.typicode.com/users");
    const data = await response.json();
    setUzytkownicy(data);
  }

  pobierzUzytkownikow();
}, []);

return (
  <div className="container mt-4">
    <h1>Użytkownicy</h1>

    <ul className="list-group">
      {uzytkownicy.map((user) => (
        <li className="list-group-item" key={user.id}>
          {user.name} – {user.email}
        </li>
      ))}
    </ul>
  </div>
);
}

export default App;

```

### 18.8. Loading, błąd i pusta lista

Dla danych w części „Loading, błąd i pusta lista” zaplanuj osobne zachowanie dla ładowania, błędu, pustej odpowiedzi i poprawnych danych. Dopiero wtedy komponent jest odporny na realne odpowiedzi z pliku albo API.

W realnej aplikacji nie wystarczy samo `setDane(data)`. Użytkownik powinien wiedzieć:

- czy dane się jeszcze ładują
- czy wystąpił błąd
- czy API zwróciło pustą listę

```

import { useEffect, useState } from "react";

function App() {
  const [posty, setPosty] = useState([]);
  const [ladowanie, setLadowanie] = useState(true);
  const [blad, setBlad] = useState("");

  useEffect(() => {
    async function pobierzPosty() {
      try {
        setLadowanie(true);
        setBlad("");

        const response = await fetch("https://jsonplaceholder.typicode.com/posts");

        if (!response.ok) {
          throw new Error(`Błąd HTTP: ${response.status}`);
        }
      }
    }
  });
}

```

```

        const data = await response.json();
        setPosty(data.slice(0, 5));
    } catch (error) {
        setBlad("Nie udało się pobrać postów.");
    } finally {
        setLadowanie(false);
    }
}

pobierzPosty();
}, []);

if (ladowanie) return <p>Ładowanie danych...</p>;
if (blad) return <p className="text-danger">{blad}</p>;
if (posty.length === 0) return <p>Brak danych do wyświetlenia.</p>;

return (
    <div className="container mt-4">
        <h1>Posty</h1>

        {posty.map((post) => (
            <article className="border rounded p-3 mb-3" key={post.id}>
                <h2 className="h5">{post.title}</h2>
                <p>{post.body}</p>
            </article>
        ))}
    </div>
);
}

```

## 18.9. Wysyłanie danych metodą POST

Przy POST najczęściej wysyłasz JSON. Ustaw nagłówek `Content-Type`, zamień obiekt przez `JSON.stringify` i sprawdź status odpowiedzi tak samo jak przy GET.

```

await fetch("/api/orders", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(order),
});

```

Do wysyłania danych używamy `fetch()` z dodatkowymi opcjami:

- `method` — metoda HTTP
- `headers` — informacje o formacie danych
- `body` — dane zamienione na JSON

```

async function dodajPost() {
    const nowyPost = {
        title: "Nowy wpis",
        body: "Treść wpisu",
    };
}

```



```

    userId: 1,
  });

  const response = await fetch("https://jsonplaceholder.typicode.com/posts", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(nowyPost),
  });

  if (!response.ok) {
    throw new Error("Nie udało się zapisać danych");
  }

  const zapisanyPost = await response.json();
  console.log(zapisanyPost);
}

```

W prawdziwym API serwer zapisuje dane w bazie. `jsonplaceholder.typicode.com` tylko udaje zapis, ale dzięki temu dobrze nadaje się do ćwiczeń.

#### 18.10. Parametry w adresie URL

Dla danych w części „Parametry w adresie URL” zaplanuj osobne zachowanie dla ładowania, błędu, pustej odpowiedzi i poprawnych danych. Dopiero wtedy komponent jest odporny na realne odpowiedzi z pliku albo API.

Często API pozwala filtrować dane przez parametry w adresie.

```
https://jsonplaceholder.typicode.com/posts?userId=1
```

W React warto budować taki adres za pomocą `URLSearchParams`, zamiast sklejać tekst ręcznie.

```

const params = new URLSearchParams({
  userId: "1",
});

const url = `https://jsonplaceholder.typicode.com/posts?${params.toString()}`;
const response = await fetch(url);

```

Przykład z wyszukiwaniem:

```

const [szukaj, setSzukaj] = useState("");

const wyniki = uzytkownicy.filter((user) =>
  user.name.toLowerCase().includes(szukaj.toLowerCase())
);

```

Jeśli API obsługuje wyszukiwanie po stronie serwera, wtedy zamiast filtrować tablicę w React, wysyłamy zapytanie z parametrem, np. `q=react`.

### 18.11. Dobre praktyki przy pracy z API

Dla danych w części „Dobre praktyki przy pracy z API” zaplanuj osobne zachowanie dla ładowania, błędu, pustej odpowiedzi i poprawnych danych. Dopiero wtedy komponent jest odporny na realne odpowiedzi z pliku albo API.

#### 1. Zawsze obsługuj błąd i ładowanie

Bez tego użytkownik widzi pustą stronę i nie wie, czy aplikacja działa.

#### 2. Sprawdzaj `response.ok`

Sam `fetch()` nie wystarczy do poprawnej obsługi błędów HTTP.

#### 3. Nie zakładaj, że dane zawsze istnieją

Przy danych z API przydaje się optional chaining:

```
<p>Miasto: {user?.address?.city ?? "Brak miasta"}</p>
```

#### 4. Nie trzymaj sekretów w kodzie frontendu

Klucze prywatne, hasła i tokeny administracyjne nie powinny być wpisane w pliku React. Kod frontendu trafia do przeglądarki użytkownika, więc można go podejrzec.

#### 5. Uważaj na CORS

Jeśli przeglądarka blokuje zapytanie komunikatem o CORS, problem zwykle leży po stronie konfiguracji serwera API, a nie w samym komponencie React.

#### 6. Przy kilku zapytaniach używaj `Promise.all`

Gdy dwa zapytania są niezależne, można pobrać je równolegle:

```
const [usersResponse, postsResponse] = await Promise.all([
  fetch("https://jsonplaceholder.typicode.com/users"),
  fetch("https://jsonplaceholder.typicode.com/posts"),
]);
```

## 19. Logika aplikacji poza JSX

Im więcej logiki da się przenieść poza JSX, tym łatwiej czytać komponent. Funkcje formatujące, filtrujące, sortujące i walidujące mogą działać jako zwykły JavaScript, bez zależności od Reacta.

```
export function formatPrice(value) {
  return `_${value.toFixed(2)} zł`;
}
```

Jedną z kluczowych zasad dobrego programowania jest **Separation of Concerns** (rozdzielenie odpowiedzialności). Oznacza to, że każdy fragment kodu powinien być odpowiedzialny za **jedną, konkretną rzecz**. W kontekście React oznacza to, że komponent powinien zajmować się **wyłącznie wyświetlaniem interfejsu użytkownika** (renderowaniem JSX) i obsługą interakcji (kliknięcia, wpisywanie tekstu). Logika biznesowa — obliczenia matematyczne, walidacja danych, formatowanie tekstu, transformacje tablic — powinna być wyciągnięta do **osobnych plików pomocniczych** w folderze `utils/` lub `helpers/`. Dane początkowe (listy, konfiguracje, stałe) powinny żyć w folderze `data/`. Dzięki temu komponenty są krótkie i czytelne, a logikę można łatwo testować jednostkowo bez konieczności renderowania całego komponentu. Dodatkowo, te same funkcje pomocnicze mogą być współdzielone przez wiele komponentów, co eliminuje duplikację kodu i ułatwia utrzymanie aplikacji.

### 19.1. Funkcje pomocnicze

Funkcja pomocnicza powinna być czysta, jeśli to możliwe: dla tych samych argumentów zwraca ten sam wynik i nie zmienia danych z zewnątrz. Dzięki temu łatwo jej użyć w kilku komponentach.

```
export function countCompleted(tasks) {  
  return tasks.filter((task) => task.done).length;  
}
```

Funkcje, które nie potrzebują stanu, można definiować **poza komponentem** lub w osobnych plikach:

```
// Plik: src/App.js  
  
// Funkcja pomocnicza – poza komponentem  
function formatujCene(cena) {  
  return `${cena.toFixed(2)} zł`;  
}  
  
function czyPoprawnyEmail(email) {  
  return email.includes("@") && email.includes(".");  
}  
  
function App() {  
  return (  
    <div>  
      <p>Cena: {formatujCene(29.9)}</p>  
      <p>Email: {czyPoprawnyEmail("jan@mail.pl") ? "OK" : "Błąd"}</p>  
    </div>  
  );  
}  
  
export default App;
```

### 19.2. Osobne moduły z logiką

Jeśli logikę da się zapisać jako czystą funkcję, przenieś ją poza JSX. Komponent stanie się krótszy, a funkcję łatwiej użyć w innym miejscu.

```
// Plik: src/utils/walidacja.js  
export function czyPuste(wartosc) {  
  return wartosc.trim() === "";  
}  
  
export function czyWZakresie(liczba, min, max) {  
  return liczba ≥ min && liczba ≤ max;  
}  
  
export function czyPoprawnyEmail(email) {  
  return email.includes("@") && email.includes(".");  
}
```

```
// Plik: src/App.js
import { czyPuste, czyPoprawnyEmail } from "../utils/walidacja";

function App() {
  // ... użycie funkcji walidacyjnych
}
```

### 19.3. Oddzielenie UI od obliczeń

Jeśli logikę da się zapisać jako czystą funkcję, przenieś ją poza JSX. Komponent stanie się krótszy, a funkcję łatwiej użyć w innym miejscu.

Dobra praktyka: logika obliczeniowa **poza komponentem**, widok **w komponencie**:

```
// Plik: src/utils/koszyk.js
export function obliczSume(produkty) {
  return produkty.reduce((suma, p) => suma + p.cena * p.ilosc, 0);
}

export function obliczRabat(suma, procentRabatu) {
  return suma * (procentRabatu / 100);
}
```

```
// Plik: src/App.js
import { obliczSume, obliczRabat } from "../utils/koszyk";

function App() {
  const produkty = [
    { id: 1, nazwa: "Kurs React", cena: 199, ilosc: 1 },
    { id: 2, nazwa: "Kurs JS", cena: 149, ilosc: 2 },
  ];

  const suma = obliczSume(produkty);
  const rabat = obliczRabat(suma, 10);

  return (
    <div className="container mt-4">
      <p>Suma: {suma.toFixed(2)} zł</p>
      <p>Rabat 10%: -{rabat.toFixed(2)} zł</p>
      <p><strong>Do zapłaty: {(suma - rabat).toFixed(2)} zł</strong></p>
    </div>
  );
}
```

## 20. Organizacja projektu

Organizacja projektu powinna rosnąć razem z aplikacją. Na początku wystarczy kilka plików, ale gdy pojawiają się formularze, listy, dane i funkcje pomocnicze, warto rozdzielić je na foldery według odpowiedzialności.

```
src/
├─ components/
├─ data/
├─ hooks/
├─ utils/
└─ App.jsx
```

Organizacja plików i folderów w projekcie React może wydawać się nieistotna na początku, ale staje się **kluczowa**, gdy aplikacja zaczyna rosnąć. W małym projekcie składającym się z 1-3 komponentów wystarczy umieścić wszystkie pliki bezpośrednio w folderze `src/` — dodatkowe foldery byłyby nadmiarowe. Jednak gdy projekt rozrasta się do 10 i więcej komponentów, brak przemyślanej struktury prowadzi do chaosu — trudno znaleźć właściwy plik, trudno zrozumieć zależności między komponentami. Dobrze zorganizowany projekt przyspiesza pracę zespołową, ułatwia wdrażanie nowych programistów i minimalizuje ryzyko błędów.

| ROZMIAR PROJEKTU          | ZALECANA STRUKTURA                                                                                                                                                             |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mały (1-3 komponenty)     | Wszystkie pliki w <code>src/</code> — bez dodatkowych folderów                                                                                                                 |
| Średni (4-10 komponentów) | Folder <code>src/components/</code> na komponenty + <code>src/data/</code> na dane                                                                                             |
| Duży (10+ komponentów)    | Pełna struktura: <code>components/</code> , <code>utils/</code> , <code>data/</code> , <code>hooks/</code> , <code>styles/</code> , ewentualnie podział na moduły funkcjonalne |

## 20.1. Nazewnictwo plików i komponentów

Dobre nazwy skracają czas szukania błędu. Komponenty zapisuj wielką literą, a funkcje pomocnicze czasownikiem opisującym działanie.

```
ProductCard.jsx
formatPrice.js
filterProducts.js
```

| KONWENCJA  | PRZYKŁAD                                                     | DOTYCZY                    |
|------------|--------------------------------------------------------------|----------------------------|
| PascalCase | <code>KursKarta.js</code> , <code>ZadanieFormularz.js</code> | Pliki komponentów          |
| camelCase  | <code>walidacja.js</code> , <code>formatowanie.js</code>     | Pliki z logiką/narzędziami |
| camelCase  | <code>handleSubmit</code> , <code>setImie</code>             | Funkcje i zmienne          |

**Zasada:** Nazwa pliku komponentu = nazwa komponentu:

- Plik: `KursKarta.js` → Komponent: `function KursKarta() { ... }`

## 20.2. Folder components

`components` trzymaj dla elementów UI wielokrotnego użytku: kart, przycisków, formularzy, list. Jeżeli komponent jest pełną stroną routingu, lepiej pasuje do folderu `pages` albo `views`.

```
src/
└─ components/
    ├── Header.js
    ├── Footer.js
    ├── KursKarta.js
    ├── KursLista.js
    └─ ZadanieFormularz.js
```

### 20.3. Folder data

`data` jest dobre dla statycznych tablic: kategorii, przykładowych produktów, pytań quizu. Jeżeli dane zaczynają być pobierane z API, folder może zostać, ale pliki zmienią rolę na dane testowe lub fallback.

```
src/
└─ data/
  │ └─ kursy.js      # Tablica obiektów
  └─ filmy.json     # Dane JSON
```

### 20.4. Folder utils

`utils` powinien zawierać funkcje bez JSX i bez hooków. Jeśli funkcja używa `useState` albo `useEffect`, nie jest zwykłym utilsem, tylko hookiem albo częścią komponentu.

```
src/
└─ utils/
  │ └─ walidacja.js  # Funkcje walidacyjne
  │ └─ formatowanie.js # Formatowanie tekstu, cen
  └─ algorytmy.js   # Funkcje algorytmiczne
```

### 20.5. Przykładowa struktura projektu

Struktura projektu powinna odpowiadać temu, jak szukasz kodu. Jeśli poprawiasz formularz, powinno być jasne, gdzie jest komponent, gdzie walidacja, a gdzie dane pomocnicze.

```
src/
├─ components/
│   ├── Header.js
│   ├── Footer.js
│   ├── KursFormularz.js
│   └─ KursLista.js
├─ data/
│   └─ kursy.js
├─ utils/
│   └─ walidacja.js
├─ App.js
├─ App.css
├─ index.js
└─ index.css
```

## 21. Debugowanie

Debugowanie w React zaczyna się od ustalenia, co jest niepoprawne: dane wejściowe, stan, propsy, warunek renderowania czy samo JSX. Najszybsza metoda to sprawdzenie błędu w konsoli, wypisanie wartości i zawężenie problemu do jednego komponentu.

### 21.1. Konsola przeglądarki

Konsola nie służy tylko do `console.log`. Przy tablicach obiektów często lepsze jest `console.table`, a przy śledzeniu kolejności wywołań można użyć `console.group`.

```
console.table(products);
console.group("submit");
console.log(form);
console.groupEnd();
```

```
function handleSubmit(e) {
  e.preventDefault();
  console.log("Wartości formularza:", { imie, email, wiek });
  console.log("Typ imie:", typeof imie);
  console.log("Długość listy:", lista.length);
}
```

## 21.2. React DevTools

React DevTools pozwala zobaczyć drzewo komponentów, propsy, stan i przyczynę ponownego renderowania. To szczególnie przydatne, gdy dziecko dostaje inną wartość niż zakładasz.

React DevTools to rozszerzenie przeglądarki (Chrome / Firefox), które pozwala:

- Przeglądać drzewo komponentów
- Podglądać props i stan (state) każdego komponentu
- Śledzić re-rendery

Instalacja: wyszukaj „React Developer Tools” w sklepie rozszerzeń przeglądarki.

## 21.3. Typowe błędy składni

Błędy składni zwykle wskazują linię, ale prawdziwa przyczyna może być kilka linijek wyżej: brakujący nawias, niedomknięty tag albo źle wstawiony komentarz JSX.

| BŁĄD                                  | PRZYCZYNA                                                | ROZWIĄZANIE                                                 |
|---------------------------------------|----------------------------------------------------------|-------------------------------------------------------------|
| Adjacent JSX elements must be wrapped | Dwa elementy bez wspólnego rodzica                       | Owinąć w <code>&lt;div&gt;</code> lub <code>&lt;&gt;</code> |
| 'class' is not a valid attribute      | Użycie <code>class</code> zamiast <code>className</code> | Zamień na <code>className</code>                            |
| 'for' is not a valid attribute        | Użycie <code>for</code> zamiast <code>htmlFor</code>     | Zamień na <code>htmlFor</code>                              |
| Expected a ')' to match '('           | Brakujący nawias                                         | Sprawdź nawiasy w JSX                                       |

## 21.4. Typowe błędy stanu

Przy błędach stanu sprawdź, czy używasz settera, czy tworzysz kopię tablicy lub obiektu i czy nie oczekujesz nowej wartości natychmiast po wywołaniu settera.

| OBJAW                       | PRZYCZYNA                                             | ROZWIĄZANIE                                                       |
|-----------------------------|-------------------------------------------------------|-------------------------------------------------------------------|
| Widok nie aktualizuje się   | Użycie <code>let</code> zamiast <code>useState</code> | Zamień na <code>useState</code>                                   |
| Widok nie aktualizuje się   | Mutowanie stanu (push, bezpośrednia zmiana)           | Tworzenie kopii (spread)                                          |
| Stara wartość w console.log | Stan jest asynchroniczny                              | Loguj przed <code>setState</code> lub użyj <code>useEffect</code> |

## 21.5. Typowe błędy formularzy

Przy błędach formularzy najpierw sprawdź, czy input ma `value` i `onChange`, czy `name` zgadza się z polem w obiekcie stanu oraz czy `preventDefault()` działa w submit.

| OBJAW                                 | PRZYCZYNA                            | ROZWIĄZANIE                                               |
|---------------------------------------|--------------------------------------|-----------------------------------------------------------|
| Strona się przeładowuje               | Brak <code>e.preventDefault()</code> | Dodaj na początku <code>handleSubmit</code>               |
| Pole nie reaguje na pisanie           | Brak <code>onChange</code>           | Dodaj <code>onChange</code> z <code>setState</code>       |
| Pole wyświetla <code>undefined</code> | Brak wartości początkowej stanu      | <code>useState("")</code> zamiast <code>useState()</code> |
| <code>NaN</code> w polu liczbowym     | Brak konwersji <code>Number()</code> | <code>setWiek(Number(e.target.value))</code>              |

## 22. Najczęstsze pułapki i jak ich unikać

Pułapki w React zwykle wynikają z naruszenia kilku zasad: nie mutuj stanu, nie odczytuj stanu tak, jakby aktualizował się natychmiast, nie wywołuj handlera podczas renderowania i nie ukrywaj błędów w zbyt dużych komponentach.

| OBJAW                         | NAJCZĘSTSZA PRZYCZYNA                                         |
|-------------------------------|---------------------------------------------------------------|
| lista nie odświeża się        | mutacja tablicy zamiast kopii                                 |
| formularz przeładowuje stronę | brak <code>event.preventDefault()</code>                      |
| funkcja uruchamia się od razu | <code>onClick={fn()}</code> zamiast <code>onClick={fn}</code> |

### 22.1. Brak key w pętli map()

Objaw braku dobrego `key` to dziwne zachowanie listy po usuwaniu, sortowaniu albo edycji elementów. Indeks tablicy działa tylko dla list statycznych, które nigdy nie zmieniają kolejności.

```
// BŁĄD (ostrzeżenie w konsoli)
{items.map((item) => <li>{item.text}</li>)}

// POPRAWNIE
{items.map((item) => <li key={item.id}>{item.text}</li>)}
```

### 22.2. Mutowanie stanu zamiast tworzenia kopii

Mutacja stanu często wygląda niewinnie, ale React może nie zauważyć zmiany, jeśli referencja tablicy lub obiektu pozostaje ta sama. Zawsze twórz nową tablicę lub obiekt.

```
// źle
items.push(newItem);
setItems(items);

// dobrze
setItems((items) => [...items, newItem]);
```

```
// błąd – React nie widzi zmiany referencji
tasks.push(newTask);
setTasks(tasks);

// POPRAWNIE – nowa tablica
setTasks([...tasks, newTask]);

// błąd – mutacja obiektu
```



```
osoba.wiek = 26;
setOsoba(osoba);

// POPRAWNIE – nowy obiekt
setOsoba({ ...osoba, wiek: 26 });
```

### 22.3. Odczyt stanu zaraz po ustawieniu

Objaw tego błędu: `console.log` pokazuje starą wartość zaraz po `useState`. To normalne, bo aktualizacja zostanie uwzględniona przy kolejnym renderze. Jeśli potrzebujesz nowej wartości od razu, policz ją w zmiennej lokalnej.

```
// błąd – stara wartość
setCount(count + 1);
console.log(count); // Nadal stara!

// ROZWIĄZANIE – oblicz przed
const nowy = count + 1;
setCount(nowy);
console.log(nowy); // Nowa wartość
```

### 22.4. Brak event.preventDefault() w formularzu

Objaw braku `preventDefault()` to przeładowanie strony i utrata stanu formularza. Jeśli po kliknięciu submit aplikacja wraca do początku, najpierw sprawdź tę linię.

```
// błąd – strona się przeładowuje
function handleSubmit(e) {
  console.log("Dane:", imie);
}

// POPRAWNIE
function handleSubmit(e) {
  e.preventDefault(); // Najważniejsza linia!
  console.log("Dane:", imie);
}
```

### 22.5. Zapomnienie o import useState

Objaw braku importu hooka to błąd typu `useState is not defined`. Sprawdź pierwsze linie pliku i upewnij się, że importujesz hook z `react`, a nie z innego pakietu.

```
// błąd – useState is not defined
function App() {
  const [x, setX] = useState(0);
}

// POPRAWNIE
import { useState } from "react";
function App() {
  const [x, setX] = useState(0);
}
```

## 22.6. Wywołanie funkcji zamiast przekazania referencji

W JSX przekazujesz funkcję, a nie wynik jej wykonania. Nawiasy po nazwie funkcji uruchamiają ją natychmiast podczas renderowania.

```
<button onClick={save}>Zapisz</button>
<button onClick={() => save(id)}>Zapisz element</button>
```

```
// błąd - funkcja WYKONA SIĘ natychmiast przy renderze
<button onClick={handleKliknij()}>Kliknij</button>

// POPRAWNIE - przekazanie referencji (bez nawiasów)
<button onClick={handleKliknij}>Kliknij</button>

// POPRAWNIE - z argumentem (strzałka)
<button onClick={() => handleUsun(id)}>Usuń</button>
```

## 23. Build i publikacja projektu

Build produkcyjny różni się od trybu deweloperskiego. Kod jest minifikowany, importy są przetwarzane, a ścieżki do zasobów mogą wyglądać inaczej. Dlatego przed publikacją warto uruchomić build i sprawdzić wynik lokalnie.

### 23.1. npm run build

`npm run build` tworzy wersję produkcyjną: mniejszą, zoptymalizowaną i gotową do publikacji. Jeśli build się nie udaje, nie publikuj projektu, tylko najpierw napraw błąd kompilacji.

```
npm run build
```

To polecenie tworzy zoptymalizowaną wersję produkcyjną w folderze `build/`. Pliki są minifikowane (skompresowane), co zapewnia szybsze ładowanie.

### 23.2. Co zawiera folder build

Folder produkcyjny zawiera już przetworzone pliki. Nie edytuj ich ręcznie; poprawiaj kod w `src`, uruchamiaj build ponownie i dopiero tę nową wersję publikuj.

```
build/
├─ index.html      # Główny plik HTML
├─ static/
│   ├─ css/        # Skompresowane pliki CSS
│   │   └─ main.abc123.css
│   ├─ js/         # Skompresowane pliki JavaScript
│   │   └─ main.xyz789.js
│   └─ media/       # Obrazy importowane z src/
└─ favicon.ico
```

### 23.3. Typowe problemy przy buildzie

Problemy przy buildzie często wynikają z wielkości liter w nazwach plików. macOS może tolerować różnice typu `Logo.png` i `logo.png`, ale serwer produkcyjny już niekoniecznie.

| PROBLEM                              | PRZYCZYNA                             | ROZWIĄZANIE                                                                                          |
|--------------------------------------|---------------------------------------|------------------------------------------------------------------------------------------------------|
| Ostrzeżenia o nieużywanych zmiennych | Zadeklarowane, ale nieużywane zmienne | Usuń nieużywane zmienne i importy                                                                    |
| Białe plamy zamiast obrazów          | Zła ścieżka do obrazu                 | Użyj <code>import</code> dla obrazów z <code>src/</code> lub <code>/</code> dla <code>public/</code> |
| Błąd kompilacji                      | Błąd składni w kodzie                 | Napraw błąd wskazany w terminalu                                                                     |

## 24. Dobre praktyki UI i dostępność

Dostępność nie jest osobnym dodatkiem na końcu projektu. Semantyczne znaczniki, etykiety formularzy, poprawne przyciski i teksty alternatywne wpływają jednocześnie na jakość kodu, obsługę klawiatury i zrozumiałość interfejsu.

```
<label htmlFor="email">E-mail</label>
<input id="email" type="email" />
```

### 24.1. Typ przycisku — button vs submit

Przycisk w formularzu domyślnie ma typ `submit`. Jeśli ma tylko otwierać panel, czyścić filtr albo przełączać widok, ustaw `type="button"`, żeby przypadkiem nie wysłał formularza.

```
<button type="button" onClick={clearFilters}>Wyczyść filtry</button>
<button type="submit">Zapisz</button>
```

```
{/* Przycisk wysyłający formularz */}
<button type="submit">Wyślij</button>

{/* Przycisk NIE wysyłający formularza – np. reset, anuluj */}
<button type="button" onClick={handleAnuluj}>Anuluj</button>
```

Jeśli nie podasz `type`, przycisk wewnątrz `<form>` domyślnie jest `type="submit"` i może spowodować niechciane wysłanie formularza.

### 24.2. Label i htmlFor

`label` powiązany z polem zwiększa obszar kliknięcia i ułatwia obsługę czytnikom ekranu. W JSX używaj `htmlFor`, bo `for` jest słowem zarezerwowanym w JavaScript.

```
<label htmlFor="password">Hasło</label>
<input id="password" type="password" />
```

Każde pole formularza powinno mieć etykietę `<label>` powiązaną z polem przez `htmlFor`:

```
<label htmlFor="email" className="form-label">Email:</label>
<input id="email" type="text" className="form-control" />
```

Kliknięcie etykiety automatycznie przenosi fokus na powiązane pole — to ułatwia obsługę, szczególnie na urządzeniach mobilnych.

### 24.3. Semantyczny układ strony

Dostępność najlepiej dodawać od początku. Poprawny znacznik, etykieta i typ przycisku zwykle kosztują jedną linijkę, a znacząco poprawiają jakość interfejsu.

```
function App() {
  return (
    <div>
      <header>
        <h1>Moja Aplikacja</h1>
      </header>
      <main className="container mt-4">
        { /* Główna treść */ }
      </main>
      <footer className="text-center mt-4">
        <p>&copy; 2025</p>
      </footer>
    </div>
  );
}
```

## 25. Routing i Nawigacja w SPA (react-router-dom)

Routing w SPA nie przeladowuje całej strony. Zmienia się adres w pasku przeglądarki, a React renderuje odpowiedni komponent. Dlatego linki wewnętrzne powinny korzystać z `<Link>`, a nie ze zwykłego `<a href>`, jeśli nie wychodzisz poza aplikację.

### 25.1. Czym jest Client-Side Routing?

Routing powinien opisywać strukturę widoków. Jeśli parametr w adresie identyfikuje rekord, komponent powinien odczytać go z URL i na tej podstawie znaleźć dane.

Tak jak wspomnieliśmy w rozdziale wprowadzającym do SPA, aplikacje Reactowe z reguły ładują tylko jeden plik `index.html`. Aby zasymulować przechodzenie między podstronami (np. z `/` na `/profil` czy `/logowanie`) bez odświeżania całej przeglądarki, używamy tzw. "Client-Side Routingu". W ekosystemie React najpopularniejszym do tego narzędziem jest biblioteka **React Router**.

Zanim zaczniesz, musisz ją zainstalować:

```
npm install react-router-dom
```

### 25.2. BrowserRouter, Routes i Route

`BrowserRouter` powinien obejmować całą część aplikacji korzystającą z routing. `Routes` wybiera pasującą trasę, a `Route` mapuje ścieżkę na element JSX.

Aby routing zadziałał, musimy obudować naszą aplikację (zwykle w pliku `src/index.js` lub `src/App.js`) odpowiednimi komponentami dostarczonymi przez bibliotekę `react-router-dom`.

- `BrowserRouter` - Główny "wrapper" (owijka), który włącza nasłuchiwanie na zmiany adresu URL w przeglądarce.
- `Routes` - Kontener na nasze ścieżki. Sprawdza obecny URL i wybiera *tylko jeden*, najlepiej pasujący `Route`.
- `Route` - Definiuje konkretną ścieżkę. Przyjmuje atrybut `path` (ścieżka w URL) oraz `element` (komponent JSX, który ma zostać wyświetlony).

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
```

```
import EkranDomowy from "../pages/EkranDomowy";
import LogowanieForm from "../pages/LogowanieForm";
import Bład404 from "../pages/Bład404";

function App() {
  return (
    <BrowserRouter>
      {/* Elementy umieszczone poza <Routes> (np. nawigacja, stopka) będą widoczne na KAZDEJ podstronie */}
      <nav>Menu Główne dla Sklepu - Stały widżet Nav</nav>

      <Routes>
        {/* Strona główna */}
        <Route path="/" element={<EkranDomowy />} />

        {/* Podstrona logowania */}
        <Route path="/login" element={<LogowanieForm />} />

        {/* Specjalna ścieżka "*" wyłapuje wszystkie nieznane adresy (Bład 404) */}
        <Route path="*" element={<Bład404 />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

### 25.3. Linkowanie pomiędzy podstronami używając <Link>

**Link** zmienia adres wewnątrz aplikacji bez pełnego przeładowania dokumentu. Zwykłego **<a>** używaj do linków zewnętrznych, pobierania plików albo przejścia poza aplikację.

```
<Link to="/produkty">Produkty</Link>
<a href="https://react.dev">Dokumentacja React</a>
```

Gdybyśmy do nawigacji użyli standardowego znacznika HTML **<a href="/login">**, przeglądarka pobrałaby stronę na nowo z serwera, co spowodowałoby reset całego stanu Reacta. Aby temu zapobiec, używamy komponentu **<Link>**.

```
import { Link } from "react-router-dom";

function TopNavigation() {
  return (
    <nav>
      {/* Zamiast <a href="/"> używamy <Link to="/"> */}
      <Link to="/">Strona Główna</Link>
      <Link to="/login">Zaloguj się</Link>
    </nav>
  );
}
```

Dzięki **<Link>**, React zmienia zawartość ekranu "w locie", co jest błyskawiczne i zachowuje stan aplikacji.

## 25.4. Nawigacja z poziomu kodu (useNavigate)

`useNavigate` służy do przejścia wywołanego logiką, np. po poprawnym logowaniu albo zapisaniu formularza. Do zwykłej nawigacji widocznej na stronie lepszy jest `Link`.

Często musimy przenieść użytkownika na inną stronę w wyniku jakiejś akcji (np. po udanym zalogowaniu lub po wysłaniu formularza). Nie używamy do tego kliknięcia w `<Link>`, lecz hooka `useNavigate`.

```
import { useNavigate } from "react-router-dom";
import { useState } from "react";

function LogowanieForm() {
  const [login, setLogin] = useState("");
  const navigate = useNavigate(); // Inicjalizacja hooka

  function handleSubmit(e) {
    e.preventDefault();

    // Tutaj np. wysyłamy zapytanie do serwera. Jeśli sukces:
    if (login === "admin") {
      // Przenieś programatycznie na stronę panelu
      navigate("/panel");
    } else {
      alert("Błędny login!");
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={login} onChange={e => setLogin(e.target.value)} />
      <button type="submit">Zaloguj</button>
    </form>
  );
}
```

## 25.5. Parametry w ścieżkach (useParams)

Parametry ścieżki są dobre dla identyfikatorów, np. `/products/15`. Po odczytaniu przez `useParams` pamiętaj, że wartość jest tekstem, więc do porównań liczbowych trzeba ją przekonwertować.

```
const { id } = useParams();
const productId = Number(id);
```

Często ścieżki są dynamiczne, np. profil konkretnego użytkownika `/user/123` lub strona produktu `/produkt/5`. W React Router definiujemy to przy pomocy dwukropka `:id`. Następnie w komponencie możemy ten parametr odczytać używając hooka `useParams`.

Definicja w App.js:

```
<Route path="/produkt/:id" element={<ProduktSzczegoly />} />
```

Odczyt w komponencie:

```
import { useParams } from "react-router-dom";

function ProduktSzczegoly() {
  // useParams zwraca obiekt z parametrami z paska adresu
  const { id } = useParams();

  return (
    <div>
      <h1>Szczegóły produktu numer: {id}</h1>
      {/* Tutaj możemy użyć id np. do pobrania danych o tym konkretnym produkcie z API */}
    </div>
  );
}
```

## 26. Wzorce praktyczne

Wzorce praktyczne warto traktować jak katalog gotowych rozwiązań, ale nie jak kod do bezmyślnego kopiowania. Najpierw rozpoznaj mechanizm: stan, formularz, lista, filtr, obliczenia, API albo routing. Potem przenieś sam wzorec do własnego komponentu.

| TYP WZORCA | CZEGO SZUKAĆ W KODZIE                                      |
|------------|------------------------------------------------------------|
| formularz  | kontrolowane pola, walidacja, submit                       |
| kalkulator | stan wejściowy, obliczenia, formatowanie wyniku            |
| lista      | tablica, <code>map</code> , <code>key</code> , filtrowanie |
| gra        | stan rundy, wynik, reset                                   |
| API        | <code>useEffect</code> , loading, error, dane              |

### 26.1. Formularz rejestracji

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Formularz rejestracji” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Formularz rejestracji” dodaj co najmniej dwa realne błędy walidacji, np. zbyt krótki tekst i brak zaznaczenia zgody. Po poprawnym wysłaniu pokaż podsumowanie danych oraz wyczyść zarówno wartości pól, jak i obiekt błędów.

Prosty formularz z walidacją — jeden z najczęstszych wzorców w aplikacjach webowych.

```
// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  // Stan dla każdego pola formularza
  const [imie, setImie] = useState("");
  const [email, setEmail] = useState("");
  const [haslo, setHaslo] = useState("");
  const [regulamin, setRegulamin] = useState(false);

  // Stan dla błędów walidacji
  const [bledy, setBledy] = useState({});

  // Stan dla komunikatu sukcesu
```

```
const [sukces, setSukces] = useState("");

// Funkcja walidacyjna – sprawdza dane i zwraca obiekt z błędami
function waliduj() {
  const noweBledy = {};

  // Walidacja imienia
  if (imie.trim() === "") {
    noweBledy.imie = "Imię jest wymagane";
  } else if (imie.trim().length < 2) {
    noweBledy.imie = "Imię musi mieć co najmniej 2 znaki";
  }

  // Walidacja emaila
  if (email.trim() === "") {
    noweBledy.email = "Email jest wymagany";
  } else if (!email.includes("@") || !email.includes(".")) {
    noweBledy.email = "Email musi zawierać @ i .";
  }

  // Walidacja hasła
  if (haslo === "") {
    noweBledy.haslo = "Hasło jest wymagane";
  } else if (haslo.length < 6) {
    noweBledy.haslo = "Hasło musi mieć co najmniej 6 znaków";
  }

  // Walidacja regulaminu
  if (!regulamin) {
    noweBledy.regulamin = "Musisz zaakceptować regulamin";
  }

  return noweBledy;
}

// Obsługa wysłania formularza
function handleSubmit(e) {
  e.preventDefault(); // Zapobiegamy przeładowaniu strony

  const noweBledy = waliduj();

  // Jeśli są błędy – wyświetlamy je i przerywamy
  if (Object.keys(noweBledy).length > 0) {
    setBledy(noweBledy);
    setSukces("");
    return;
  }

  // Brak błędów – wyświetlamy sukces
  setBledy({});
  setSukces(`Zarejestrowano: ${imie} (${email})`);
  console.log("Dane rejestracji:", { imie, email, haslo });

  // Reset formularza
```



```
setImie("");
setEmail("");
setHaslo("");
setRegulamin(false);
}

return (
  <main className="container mt-4" style={{ maxWidth: "500px" }}>
    <h1 className="mb-4">Rejestracja</h1>

    {/* Komunikat sukcesu */}
    {sukces && <div className="alert alert-success">{sukces}</div>}

    <form onSubmit={handleSubmit}>
      {/* Pole: Imię */}
      <div className="mb-3">
        <label htmlFor="imie" className="form-label">Imię:</label>
        <input
          id="imie"
          type="text"
          className={`form-control ${bledy.imie ? "is-invalid" : ""}`}
          value={imie}
          onChange={(e) => setImie(e.target.value)}
          placeholder="Wpisz imię"
        />
        {bledy.imie && <div className="invalid-feedback">{bledy.imie}</div>}
      </div>

      {/* Pole: Email */}
      <div className="mb-3">
        <label htmlFor="email" className="form-label">Email:</label>
        <input
          id="email"
          type="text"
          className={`form-control ${bledy.email ? "is-invalid" : ""}`}
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          placeholder="jan@example.com"
        />
        {bledy.email && <div className="invalid-feedback">{bledy.email}</div>}
      </div>

      {/* Pole: Hasło */}
      <div className="mb-3">
        <label htmlFor="haslo" className="form-label">Hasło:</label>
        <input
          id="haslo"
          type="password"
          className={`form-control ${bledy.haslo ? "is-invalid" : ""}`}
          value={haslo}
          onChange={(e) => setHaslo(e.target.value)}
          placeholder="Minimum 6 znaków"
        />
        {bledy.haslo && <div className="invalid-feedback">{bledy.haslo}</div>}
      </div>
    </form>
  </main>
);
```

```

    </div>

    {/* Checkbox: Regulamin */}
    <div className="form-check mb-3">
      <input
        id="regulamin"
        type="checkbox"
        className={`form-check-input ${bledy.regulamin ? "is-invalid" : ""}`}
        checked={regulamin}
        onChange={(e) => setRegulamin(e.target.checked)}
      />
      <label htmlFor="regulamin" className="form-check-label">
        Akceptuję regulamin
      </label>
      {bledy.regulamin && <div className="invalid-feedback">{bledy.regulamin}</div>}
    </div>

    <button type="submit" className="btn btn-primary w-100">
      Zarejestruj się
    </button>
  </form>
</main>
);
}

export default App;

```

## 26.2. Zapisy na kurs

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Zapisy na kurs” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Zapisy na kurs” dodaj co najmniej dwa realne błędy walidacji, np. zbyt krótki tekst i brak zaznaczenia zgody. Po poprawnym wysłaniu pokaż podsumowanie danych oraz wyczyść zarówno wartości pól, jak i obiekt błędów.

Przykład połączenia prostego formularza z widokiem powiązany z tablicą danych.

### Wymagania:

- Tablica nazw kursów
- Wyświetlenie liczby kursów
- Lista numerowana kursów
- Formularz z imieniem, nazwiskiem i numerem kursu
- Wyświetlenie w konsoli imienia, nazwiska i nazwy kursu lub komunikatu „Nieprawidłowy numer kursu”

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  // Tablica kursów – dane stałe
  const kursy = [
    "Programowanie w C#",

```

```

    "Angular dla początkujących",
    "Sieci komputerowe",
    "Bazy danych SQL",
  ];

  // Stan formularza
  const [imie, setImie] = useState("");
  const [nazwisko, setNazwisko] = useState("");
  const [numerKursu, setNumerKursu] = useState("");

  // Obsługa przycisku "Zapisz do kursu"
  function handleZapisz() {
    const numer = Number(numerKursu);

    // Walidacja – czy numer mieści się w zakresie tablicy
    if (numer ≥ 1 && numer ≤ kursy.length) {
      // Numer jest poprawny – wyświetlamy dane w konsoli
      console.log(`${imie} ${nazwisko}`);
      console.log(`Kurs: ${kursy[numer - 1]}`);
    } else {
      // Numer jest niepoprawny
      console.log(`${imie} ${nazwisko}`);
      console.log("Nieprawidłowy numer kursu");
    }
  }

  return (
    <main className="container mt-4" style={{ maxWidth: "600px" }}>
      <h1>Zapisy na kursy</h1>

      {/* Wyświetlenie liczby kursów */}
      <p>Dostępnych kursów: <strong>{kursy.length}</strong></p>

      {/* Lista numerowana kursów – renderowana dynamicznie */}
      <ol>
        {kursy.map((kurs, index) => (
          <li key={index}>{kurs}</li>
        ))}
      </ol>

      <hr />

      {/* Formularz zapisu */}
      <div className="mb-3">
        <label htmlFor="imie" className="form-label">Imię:</label>
        <input
          id="imie"
          type="text"
          className="form-control"
          value={imie}
          onChange={(e) => setImie(e.target.value)}
        />
      </div>
    </main>
  );
}

```

```

<div className="mb-3">
  <label htmlFor="nazwisko" className="form-label">Nazwisko:</label>
  <input
    id="nazwisko"
    type="text"
    className="form-control"
    value={nazwisko}
    onChange={(e) => setNazwisko(e.target.value)}
  />
</div>

<div className="mb-3">
  <label htmlFor="numer" className="form-label">Numer kursu:</label>
  <input
    id="numer"
    type="number"
    className="form-control"
    value={numerKursu}
    onChange={(e) => setNumerKursu(e.target.value)}
  />
</div>

<button className="btn btn-primary" onClick={handleZapisz}>
  Zapisz do kursu
</button>
</main>
);
}

export default App;

```

#### Kluczowe mechaniki:

- Tablica danych stałych ( `const kursy = [...]` )
- Renderowanie listy numerowanej przez `map()`
- Odczyt wartości pól formularza ze stanu
- Mapowanie numeru na element tablicy ( `kursy[numer - 1]` )
- Prosty warunek walidacyjny
- Wynik trafia do `console.log()` , nie na stronę

### 26.3. Formularz filmu

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Formularz filmu” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Formularz filmu” dodaj co najmniej dwa realne błędy walidacji, np. zbyt krótki tekst i brak zaznaczenia zgody. Po poprawnym wysłaniu pokaż podsumowanie danych oraz wyczyść zarówno wartości pól, jak i obiekt błędów.

Kolejny przykład wprowadzania kontrolowanych struktur danych – tym razem na liście rozwijanej.

#### Wymagania:

- Formularz z polem „Tytuł filmu”
- Lista rozwijana (select) z rodzajem filmu

- Przycisk „Dodaj”
- Startowo puste pola
- Wypisanie danych formularza do konsoli po kliknięciu

```
// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  // Stan formularza – startowo puste
  const [tytuł, setTytuł] = useState("");
  const [rodzaj, setRodzaj] = useState("");

  // Lista rodzajów filmów do selecta
  const rodzaje = ["Sensacyjny", "Komedia", "Horror", "Dramat", "Sci-Fi"];

  // Obsługa kliknięcia "Dodaj"
  function handleDodaj(e) {
    e.preventDefault(); // Zapobiegamy przeładowaniu

    // Wypisanie danych do konsoli
    console.log(`Tytuł: ${tytuł}`);
    console.log(`Rodzaj: ${rodzaj}`);
  }

  return (
    <main className="container mt-4" style={{ maxWidth: "500px" }}>
      <h1>Dodaj film</h1>

      <form onSubmit={handleDodaj}>
        {/} Pole: Tytuł filmu {/}
        <div className="mb-3">
          <label htmlFor="tytuł" className="form-label">Tytuł filmu:</label>
          <input
            id="tytuł"
            type="text"
            className="form-control"
            value={tytuł}
            onChange={(e) => setTytuł(e.target.value)}
            placeholder="Wpisz tytuł filmu"
          />
        </div>

        {/} Pole: Rodzaj filmu (select) {/}
        <div className="mb-3">
          <label htmlFor="rodzaj" className="form-label">Rodzaj:</label>
          <select
            id="rodzaj"
            className="form-select"
            value={rodzaj}
            onChange={(e) => setRodzaj(e.target.value)}
          >
            <option value="">-- Wybierz rodzaj --</option>
          </select>
        </div>
      </form>
    </main>
  );
}
```

```

        {rodzaje.map((r) => (
            <option key={r} value={r}>{r}</option>
        ))}
    </select>
</div>

    <button type="submit" className="btn btn-primary">
        Dodaj
    </button>
</form>
</main>
);
}

export default App;

```

#### Kluczowe mechaniki:

- Kontrolowane pole tekstowe ( `input text` )
- Kontrolowany `select` z dynamicznie generowanymi opcjami
- Obsługa `onSubmit` z `preventDefault()`
- Składanie danych do jednego wyniku w konsoli

## 26.4. Formularz zamówienia pizzy

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Formularz zamówienia pizzy” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Formularz zamówienia pizzy” dodaj co najmniej dwa realne błędy walidacji, np. zbyt krótki tekst i brak zaznaczenia zgody. Po poprawnym wysłaniu pokaż podsumowanie danych oraz wyczyść zarówno wartości pól, jak i obiekt błędów.

Rozbudowany formularz łączący wiele typów pól: radio (rozmiar), checkboxy (składniki z cenami), select (sos), range (ostrość), textarea (uwagi), dynamiczne obliczanie ceny, walidację i podsumowanie zamówienia w formie paragonu.

**Smaczki:** dynamiczna cena aktualizowana w czasie rzeczywistym, limit składników (max 5), wizualny wskaźnik ostrości z emoji, animowany paragon po złożeniu zamówienia, przycisk "Zamów ponownie" przywracający ostatnie zamówienie z localStorage.

```

// Plik: src/App.js
import { useState, useEffect } from "react";
import "bootstrap/dist/css/bootstrap.css";

// Dane cennika – poza komponentem (stałe)
const ROZMIARY = [
    { id: "mała", nazwa: "Mała (25cm)", cena: 22 },
    { id: "średnia", nazwa: "Średnia (32cm)", cena: 28 },
    { id: "duża", nazwa: "Duża (40cm)", cena: 35 },
];

const SKŁADNIKI = [
    { id: "ser", nazwa: "Podwójny ser", cena: 4 },
    { id: "szynka", nazwa: "Szynka", cena: 5 },
    { id: "pieczarki", nazwa: "Pieczarki", cena: 3 },
    { id: "oliwki", nazwa: "Oliwki", cena: 4 },

```

```

{ id: "papryka", nazwa: "Papryka", cena: 3 },
{ id: "sałami", nazwa: "Sałami pikantne", cena: 6 },
{ id: "rukola", nazwa: "Rukola", cena: 3 },
{ id: "ananas", nazwa: "Ananas", cena: 4 },
];

const SOSY = ["Bez sosu", "Czosnkowy", "Pomidorowy", "Ostry", "Barbecue"];

const MAX_SKLADNIKOW = 5;

// Funkcja pomocnicza – emoji ostrości
function emotikonOstrosci(poziom) {
  if (poziom === 0) return "😌 Łagodna";
  if (poziom ≤ 2) return "😋 Lekko pikantna";
  if (poziom ≤ 4) return "😋😋 Pikantna";
  return "😋😋😋 Piekiełna!";
}

function App() {
  // Stan formularza
  const [rozmiar, setRozmiar] = useState("srednia");
  const [wybraneSkładniki, setWybraneSkładniki] = useState([]);
  const [sos, setSos] = useState("Bez sosu");
  const [ostrosc, setOstrosc] = useState(0);
  const [uwagi, setUwagi] = useState("");
  const [imie, setImie] = useState("");

  // Stan UI
  const [zamowienie, setZamowienie] = useState(null); // Złożone zamówienie (paragon)
  const [bledy, setBledy] = useState({});

  // Odczyt ostatniego zamówienia z localStorage
  const [ostatnie, setOstatnie] = useState(() => {
    const zapis = localStorage.getItem("ostatniePizza");
    return zapis ? JSON.parse(zapis) : null;
  });

  // Obliczanie ceny – dynamicznie przy każdym renderze
  const cenaRozmiaru = ROZMIARY.find((r) => r.id === rozmiar)?.cena || 0;
  const cenaSkładnikow = wybraneSkładniki.reduce((suma, id) => {
    const składnik = SKLADNIKI.find((s) => s.id === id);
    return suma + (składnik?.cena || 0);
  }, 0);
  const cenaCałkowita = cenaRozmiaru + cenaSkładnikow;

  // Obsługa checkboxów składników
  function handleSkładnik(id) {
    setWybraneSkładniki((prev) => {
      if (prev.includes(id)) {
        // Odznaczenie – usuwamy
        return prev.filter((s) => s !== id);
      }
      // Zaznaczenie – dodajemy (z limitem)
      if (prev.length ≥ MAX_SKLADNIKOW) return prev;
    });
  }

```

```

        return [...prev, id];
    });
}

// Walidacja
function waliduj() {
    const noweBledy = {};
    if (imie.trim().length < 2) noweBledy.imie = "Podaj imię (min. 2 znaki)";
    if (wybraneSkładniki.length === 0) noweBledy.składniki = "Wybierz przynajmniej 1 składnik";
    return noweBledy;
}

// Złożenie zamówienia
function handleSubmit(e) {
    e.preventDefault();
    const noweBledy = waliduj();
    if (Object.keys(noweBledy).length > 0) {
        setBledy(noweBledy);
        return;
    }
    setBledy({});

    const dane = {
        imie,
        rozmiar: ROZMIARY.find((r) => r.id === rozmiar).nazwa,
        składniki: wybraneSkładniki.map(
            (id) => SKŁADNIKI.find((s) => s.id === id).nazwa
        ),
        sos,
        ostrosc,
        uwagi,
        cena: cenaCalkowita,
        czas: new Date().toLocaleTimeString("pl-PL"),
    };

    setZamowienie(dane);

    // Zapis do localStorage
    localStorage.setItem("ostatniePizza", JSON.stringify({ rozmiar, wybraneSkładniki, sos, ostrosc }));
    setOstatnie({ rozmiar, wybraneSkładniki, sos, ostrosc });
}

// Przywrócenie ostatniego zamówienia
function handlePrzywroc() {
    if (!ostatnie) return;
    setRozmiar(ostatnie.rozmiar);
    setWybraneSkładniki(ostatnie.wybraneSkładniki);
    setSos(ostatnie.sos);
    setOstrosc(ostatnie.ostrosc);
    setZamowienie(null);
}

// Nowe zamówienie – reset
function handleReset() {

```



```

    setRozmiar("srednia");
    setWybraneSkladniki([]);
    setSos("Bez sosu");
    setOstrostc(0);
    setUwagi("");
    setImie("");
    setZamowienie(null);
    setBledy({});
}

// Widok paragonu po złożeniu zamówienia
if (zamowienie) {
    return (
        <main className="container mt-4" style={{ maxWidth: "500px" }}>
            <div className="card border-success shadow">
                <div className="card-header bg-success text-white text-center">
                    <h4 className="mb-0">🍕 Zamówienie złożone!</h4>
                </div>
                <div className="card-body" style={{ fontFamily: "monospace" }}>
                    <p className="text-muted text-center mb-3">--- PARAGON ---</p>
                    <p><strong>Klient:</strong> {zamowienie.imie}</p>
                    <p><strong>Rozmiar:</strong> {zamowienie.rozmiar}</p>
                    <p><strong>Składniki:</strong></p>
                    <ul className="mb-2">
                        {zamowienie.skladniki.map((s) => (
                            <li key={s}>{s}</li>
                        ))}
                    </ul>
                    <p><strong>Sos:</strong> {zamowienie.sos}</p>
                    <p><strong>Ostrość:</strong> {emotikonOstrosci(zamowienie.ostrostc)}</p>
                    {zamowienie.uwagi && <p><strong>Uwagi:</strong> {zamowienie.uwagi}</p>}
                    <hr />
                    <p className="fs-4 text-center fw-bold text-success">
                        Do zapłaty: {zamowienie.cena.toFixed(2)} zł
                    </p>
                    <p className="text-muted text-center small">Godzina: {zamowienie.czas}</p>
                </div>
                <div className="card-footer d-flex gap-2">
                    <button className="btn btn-primary flex-grow-1 onClick={handleReset}>
                        Nowe zamówienie
                    </button>
                    <button className="btn btn-outline-secondary flex-grow-1 onClick={handlePrzywroc}>
                        Zamów to samo
                    </button>
                </div>
            </div>
        </main>
    );
}

return (
    <main className="container mt-4" style={{ maxWidth: "600px" }}>
        <h1 className="text-center mb-4">🍕 Zamów Pizzę</h1>
    </main>
);

```

```

{/* Pasek ceny – przyklejony na górze */}
<div className="alert alert-warning d-flex justify-content-between align-items-center sticky-top">
  <span>Twoja pizza:</span>
  <span className="fs-4 fw-bold">{cenaCalkowita.toFixed(2)} zł</span>
</div>

<form onSubmit={handleSubmit}>
  {/* Imię */}
  <div className="mb-3">
    <label htmlFor="imie" className="form-label">Twoje imię:</label>
    <input
      id="imie"
      type="text"
      className={`form-control ${bledy.imie ? "is-invalid" : ""}`}
      value={imie}
      onChange={(e) => setImie(e.target.value)}
      placeholder="Na kogo zamówienie?"
    />
    {bledy.imie && <div className="invalid-feedback">{bledy.imie}</div>}
  </div>

  {/* Rozmiar – radio */}
  <fieldset className="mb-3">
    <legend className="fs-6 fw-bold">Rozmiar:</legend>
    {ROZMIARY.map((r) => (
      <div key={r.id} className="form-check">
        <input
          id={`rozmiar-${r.id}`}
          type="radio"
          className="form-check-input"
          name="rozmiar"
          value={r.id}
          checked={rozmiar === r.id}
          onChange={(e) => setRozmiar(e.target.value)}
        />
        <label htmlFor={`rozmiar-${r.id}`} className="form-check-label">
          {r.nazwa} – <strong>{r.cena} zł</strong>
        </label>
      </div>
    ))}
  </fieldset>

  {/* Składniki – checkboxy z limitem */}
  <fieldset className="mb-3">
    <legend className="fs-6 fw-bold">
      Składniki ({wybraneSkładniki.length}/{MAX_SKLADNIKOW}):
      {bledy.składniki && (
        <span className="text-danger small ms-2">{bledy.składniki}</span>
      )}
    </legend>
    <div className="row">
      {SKLADNIKI.map((s) => {
        const zaznaczony = wybraneSkładniki.includes(s.id);
        const zablokowany = !zaznaczony && wybraneSkładniki.length ≥ MAX_SKLADNIKOW;

```

```

        return (
          <div key={s.id} className="col-6">
            <div className="form-check">
              <input
                id={`skladnik-${s.id}`}
                type="checkbox"
                className="form-check-input"
                checked={zaznaczony}
                disabled={zablokowany}
                onChange={() => handleSkladnik(s.id)}
              />
              <label
                htmlFor={`skladnik-${s.id}`}
                className={`form-check-label ${zablokowany ? "text-muted" : ""}`}
              >
                {s.nazwa} (+{s.cena} zł)
              </label>
            </div>
          </div>
        );
      })}
    </div>
  </fieldset>

  {/* Sos - select */}
  <div className="mb-3">
    <label htmlFor="sos" className="form-label fw-bold">Sos do pizzy:</label>
    <select
      id="sos"
      className="form-select"
      value={sos}
      onChange={(e) => setSos(e.target.value)}
    >
      {SOSY.map((s) => (
        <option key={s} value={s}>{s}</option>
      ))}
    </select>
  </div>

  {/* Ostrość - range z emoji */}
  <div className="mb-3">
    <label htmlFor="ostrosc" className="form-label fw-bold">
      Ostrość: {emotikonOstrosci(ostrosc)}
    </label>
    <input
      id="ostrosc"
      type="range"
      className="form-range"
      min="0"
      max="5"
      value={ostrosc}
      onChange={(e) => setOstrosc(Number(e.target.value))}
    />
    <div className="d-flex justify-content-between text-muted small">

```

```

        <span>Łagodna</span>
        <span>Piekiełna</span>
      </div>
    </div>

    {/* Uwagi - textarea */}
    <div className="mb-4">
      <label htmlFor="uwagi" className="form-label fw-bold">
        Uwagi do zamówienia:
      </label>
      <textarea
        id="uwagi"
        className="form-control"
        rows="2"
        value={uwagi}
        onChange={(e) => setUwagi(e.target.value)}
        placeholder="Np. bez cebuli, pokrojona na 8 kawałków..."
        maxLength={200}
      />
      <small className="text-muted">{uwagi.length}/200 znaków</small>
    </div>

    {/* Przyciski */}
    <div className="d-flex gap-2">
      <button type="submit" className="btn btn-success flex-grow-1 btn-lg">
        🛒 Zamów ({cenaCałkowita.toFixed(2)} zł)
      </button>
      {ostatnie && (
        <button type="button" className="btn btn-outline-primary" onClick={handlePrzywroc}>
          ↶ Ostatnie
        </button>
      )}
    </div>
  </form>
</main>

);
}

export default App;

```

#### Kluczowe mechaniki w tym wzorcu:

- **Radio** — wybór jednej opcji (rozmiar) z dynamiczną ceną
- **Checkboxy z limitem** — `disabled` gdy osiągnięto max, wizualne wygaszenie
- **Select** — lista rozwijana (sos)
- **Range z wizualizacją** — suwak ostrości + emoji zależne od wartości
- **Textarea z licznikiem znaków** — `maxLength` + wyświetlanie `uwagi.length`
- **Dynamiczna cena** — obliczana przy każdym renderze (nie w stanie!)
- **localStorage** — zapamiętywanie ostatniego zamówienia
- **Warunkowy render całego widoku** — `if (zamowienie)` zwraca paragon zamiast formularza
- **fieldset + legend** — semantyczne grupowanie pól formularza
- **Sticky element** — pasek ceny przyklejony na górze (`sticky-top`)

## 26.5. Formularz wyceny ubezpieczenia OC pojazdu

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Formularz wyceny ubezpieczenia OC pojazdu” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Formularz wyceny ubezpieczenia OC pojazdu” dodaj co najmniej dwa realne błędy walidacji, np. zbyt krótki tekst i brak zaznaczenia zgody. Po poprawnym wystąpieniu pokaż podsumowanie danych oraz wyczyść zarówno wartości pól, jak i obiekt błędów.

Prosty kalkulator składek bazujący na wieku kierowcy i historii bezszkodowej jazdy.

```
// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [wiek, setWiek] = useState("");
  const [pojemnosc, setPojemnosc] = useState("");
  const [bezszkodowa, setBezszkodowa] = useState(true);
  const [cena, setCena] = useState(null);

  function obliczOC(e) {
    e.preventDefault();
    let baza = 500;

    const wiekNum = Number(wiek);
    const pojNum = Number(pojemnosc);

    if (wiekNum < 18) {
      alert("Kierowca musi mieć ukończone 18 lat!");
      return;
    }

    // Zwyżki i zniżki
    if (wiekNum < 25) baza += 300;
    if (pojNum > 2000) baza += 200;
    else if (pojNum > 1400) baza += 100;

    if (bezszkodowa) baza *= 0.8; // Zniżka 20%

    setCena(Math.round(baza));
  }

  return (
    <main className="container mt-4" style={{ maxWidth: "500px" }}>
      <h1>Kalkulator OC</h1>
      <form onSubmit={obliczOC}>
        <div className="mb-3">
          <label className="form-label">Wiek kierowcy:</label>
          <input type="number" className="form-control" value={wiek} onChange={(e) => setWiek(e.target.value)} required min="18" max="100" />
        </div>
        <div className="mb-3">
          <label className="form-label">Pojemność silnika (cm3):</label>
          <input type="number" className="form-control" value={pojemnosc} onChange={(e) => setPojemnosc(e.target.value)} required
```

```

min="500" max="6000" />
</div>
<div className="form-check mb-3">
  <input type="checkbox" className="form-check-input" id="bezzskodowa" checked={bezzskodowa} onChange={(e) =>
setBezzskodowa(e.target.checked)} />
  <label className="form-check-label" htmlFor="bezzskodowa">Bezzskodowa jazda (zniżka 20%)</label>
</div>
<button type="submit" className="btn btn-primary w-100">Oblicz składkę</button>
</form>
{cena !== null && (
  <div className="alert alert-success mt-3">
    Szacowana roczna składka: <strong>{cena}</strong> PLN</div>
  </div>
)}
</main>
);
}

export default App;

```

## 26.6. Formularz rezerwacji wizyty lekarskiej

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Formularz rezerwacji wizyty lekarskiej” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Formularz rezerwacji wizyty lekarskiej” dodaj co najmniej dwa realne błędy walidacji, np. zbyt krótki tekst i brak zaznaczenia zgody. Po poprawnym wystąpieniu pokaż podsumowanie danych oraz wyczyść zarówno wartości pól, jak i obiekt błędów.

Formularz, w którym wybór specjalizacji filtruje listę dostępnych lekarzy.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

const lekarze = [
  { id: 1, nazwa: "Jan Kowalski", spec: "Kardiolog" },
  { id: 2, nazwa: "Anna Nowak", spec: "Okulista" },
  { id: 3, nazwa: "Piotr Wiśniewski", spec: "Kardiolog" },
  { id: 4, nazwa: "Maria Wójcik", spec: "Ortopeda" }
];

const specjalizacje = [...new Set(lekarze.map(l => l.spec))];

function App() {
  const [spec, setSpec] = useState("");
  const [lekarz, setLekarz] = useState("");
  const [data, setData] = useState("");
  const [potwierdzenie, setPotwierdzenie] = useState("");

  const dostepniLekarze = lekarze.filter(l => l.spec === spec);

  function handleSubmit(e) {
    e.preventDefault();
  }

```

```

const wybranyLekarz = lekarze.find(l => l.id === Number(lekarz));
setPotwierdzenie(`Zarezerwowano wizytę u: ${wybranyLekarz.nazwa} (${spec}) na termin: ${data}`);
}

return (
  <main className="container mt-4" style={{ maxWidth: "500px" }}>
    <h1>Rezerwacja wizyty</h1>
    {potwierdzenie ? (
      <div className="alert alert-success">{potwierdzenie}</div>
    ) : (
      <form onSubmit={handleSubmit}>
        <div className="mb-3">
          <label className="form-label">Specjalizacja:</label>
          <select className="form-select" value={spec} onChange={(e) => { setSpec(e.target.value); setLekarz(""); }} required>
            <option value="">Wybierz specjalizację...</option>
            {specjalizacje.map(s => <option key={s} value={s}>{s}</option>)}
          </select>
        </div>
        <div className="mb-3">
          <label className="form-label">Lekarz:</label>
          <select className="form-select" value={lekarz} onChange={(e) => setLekarz(e.target.value)} required disabled={!spec}>
            <option value="">Wybierz lekarza...</option>
            {dostepniLekarze.map(l => <option key={l.id} value={l.id}>{l.nazwa}</option>)}
          </select>
        </div>
        <div className="mb-3">
          <label className="form-label">Data wizyty:</label>
          <input type="date" className="form-control" value={data} onChange={(e) => setData(e.target.value)} required />
        </div>
        <button type="submit" className="btn btn-primary w-100">Zarezerwuj</button>
      </form>
    )}
  </main>
);
}

export default App;

```

## 26.7. Generator i podgląd CV (Live CV Builder)

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Generator i podgląd CV (Live CV Builder)” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Generator i podgląd CV (Live CV Builder)” dodaj co najmniej dwa realne błędy walidacji, np. zbyt krótki tekst i brak zaznaczenia zgody. Po poprawnym wystąpieniu pokaż podsumowanie danych oraz wyczyść zarówno wartości pól, jak i obiekt błędów.

Aplikacja, która natychmiastowo aktualizuje wizualny podgląd CV podczas wprowadzania danych, z funkcją wydruku.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {

```

```

const [imie, setImie] = useState("Jan Kowalski");
const [stanowisko, setStanowisko] = useState("Frontend Developer");
const [telefon, setTelefon] = useState("123-456-789");
const [email, setEmail] = useState("jan@example.com");
const [umiejetnosci, setUmiejetnosci] = useState("React, JavaScript, CSS");

return (
  <main className="container-fluid mt-4">
    <div className="row">
      {/* Formularz - lewa kolumna */}
      <div className="col-md-5 d-print-none">
        <h2>Wprowadź dane</h2>
        <div className="mb-3">
          <label className="form-label">Imię i nazwisko</label>
          <input className="form-control" value={imie} onChange={e => setImie(e.target.value)} />
        </div>
        <div className="mb-3">
          <label className="form-label">Stanowisko</label>
          <input className="form-control" value={stanowisko} onChange={e => setStanowisko(e.target.value)} />
        </div>
        <div className="mb-3">
          <label className="form-label">Telefon</label>
          <input className="form-control" value={telefon} onChange={e => setTelefon(e.target.value)} />
        </div>
        <div className="mb-3">
          <label className="form-label">Email</label>
          <input className="form-control" value={email} onChange={e => setEmail(e.target.value)} />
        </div>
        <div className="mb-3">
          <label className="form-label">Umiejętności (po przecinku)</label>
          <input className="form-control" value={umiejetnosci} onChange={e => setUmiejetnosci(e.target.value)} />
        </div>
        <button className="btn btn-success" onClick={() => window.print()}>Drukuj CV</button>
      </div>

      {/* Podgląd CV - prawa kolumna */}
      <div className="col-md-7">
        <div className="p-4 border rounded shadow-sm bg-light" style={{ minHeight: "800px" }}>
          <h1 className="text-primary">{imie} || "Imię i Nazwisko"</h1>
          <h4 className="text-secondary">{stanowisko} || "Twoje Stanowisko"</h4>
          <hr />
          <p><strong>Kontakt:</strong> {telefon} | {email}</p>
          <h5 className="mt-4">Umiejętności</h5>
          <ul>
            {umiejetnosci.split(",").map((um, i) => (
              um.trim() ? <li key={i}>{um.trim()}</li> : null
            ))}
          </ul>
        </div>
      </div>
    </div>
  </main>
);
}

```



```
export default App;
```

## 26.8. Formularz ankiety z oceną gwiazdkową

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Formularz ankiety z oceną gwiazdkową” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Formularz ankiety z oceną gwiazdkową” dodaj co najmniej dwa realne błędy walidacji, np. zbyt krótki tekst i brak zaznaczenia zgody. Po poprawnym wystąpieniu pokaż podsumowanie danych oraz wyczyść zarówno wartości pól, jak i obiekt błędów.

Zastosowanie klikalnych przycisków symulujących gwiazdki i warunkowa walidacja (jeśli ocena jest niska, wymagany jest komentarz).

```
// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [ocena, setOcena] = useState(0);
  const [komentarz, setKomentarz] = useState("");
  const [wyslano, setWyslano] = useState(false);

  function handleSubmit(e) {
    e.preventDefault();
    if (ocena === 0) {
      alert("Proszę wybrać ocenę.");
      return;
    }
    if (ocena ≤ 3 && komentarz.trim() === "") {
      alert("Przy ocenie 3 lub niższej prosimy o krótki komentarz, co poszło nie tak.");
      return;
    }
    setWyslano(true);
  }

  if (wyslano) {
    return <div className="container mt-4 alert alert-success">Dziękujemy za opinię!</div>;
  }

  return (
    <main className="container mt-4" style={{ maxWidth: "500px" }}>
      <h1>Oceń usługę</h1>
      <form onSubmit={handleSubmit}>
        <div className="mb-4">
          <label className="form-label d-block">Twoja ocena (1-5):</label>
          {[1, 2, 3, 4, 5].map((gwiazdka) => (
            <button
              key={gwiazdka}
              type="button"
              className={`btn me-1 ${ocena ≥ gwiazdka ? "btn-warning" : "btn-outline-secondary"}`}
              onClick={() => setOcena(gwiazdka)}
            >
              ★
            </button>
          ))}
        </div>
        <div className="form-label">
          <input type="text" value={komentarz} />
        </div>
      </form>
    </main>
  );
}
```

```

        </button>
      )}
    </div>

    <div className="mb-3">
      <label className="form-label">Komentarz:</label>
      <textarea
        className="form-control"
        rows="3"
        value={komentarz}
        onChange={(e) => setKomentarz(e.target.value)}
        placeholder={ocena > 0 && ocena ≤ 3 ? "Wymagany komentarz..." : "Opcjonalny komentarz..."}
      />
    </div>

    <button type="submit" className="btn btn-primary w-100">Wyślij ankietę</button>
  </form>
</main>
);
}

export default App;

```

## 26.9. Kalkulator wyceny szafy na wymiar

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Kalkulator wyceny szafy na wymiar” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Kalkulator wyceny szafy na wymiar” dodaj obsługę pustego pola, wartości ujemnych i zaokrąglenia wyniku. Funkcję obliczającą trzymaj poza JSX, np. `obliczWynik(dane)`, a w komponencie zostaw tylko pobieranie danych i prezentację rezultatu.

Aplikacja z kilkoma zależnościami - zmiana wymiarów lub materiału wpływa na cenę całkowitą.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [szerokosc, setSzerokosc] = useState(100);
  const [wysokosc, setWysokosc] = useState(200);
  const [material, setMaterial] = useState("plyta");
  const [montaz, setMontaz] = useState(false);

  // Zastosowanie prostego algorytmu wyceny
  const pole = (szerokosc / 100) * (wysokosc / 100); // w metrach kwadratowych

  let cenaZaMetr = 0;
  if (material === "plyta") cenaZaMetr = 150;
  else if (material === "drewno") cenaZaMetr = 450;
  else if (material === "szklo") cenaZaMetr = 600;

  let suma = pole * cenaZaMetr * 2; // uproszczone wyliczenie na fronty i korpus
  if (montaz) suma += 300; // stała cena montażu

```

```

return (
  <main className="container mt-4" style={{ maxWidth: "500px" }}>
    <h1>Konfigurator szafy</h1>

    <div className="mb-3">
      <label className="form-label">Szerokość (cm): {szerokosc}</label>
      <input type="range" className="form-range" min="50" max="300" step="10" value={szerokosc} onChange={(e) =>
setSzerokosc(Number(e.target.value))} />
    </div>

    <div className="mb-3">
      <label className="form-label">Wysokość (cm): {wysokosc}</label>
      <input type="range" className="form-range" min="100" max="250" step="10" value={wysokosc} onChange={(e) =>
setWysokosc(Number(e.target.value))} />
    </div>

    <div className="mb-3">
      <label className="form-label">Materiał frontów:</label>
      <select className="form-select" value={material} onChange={(e) => setMaterial(e.target.value)}>
        <option value="plyta">Płyta laminowana</option>
        <option value="drewno">Drewno lite</option>
        <option value="szklo">Szkło / Lustro</option>
      </select>
    </div>

    <div className="form-check mb-4">
      <input type="checkbox" className="form-check-input" id="montaz" checked={montaz} onChange={(e) =>
setMontaz(e.target.checked)} />
      <label className="form-check-label" htmlFor="montaz">Z usługą montażu (+300 zł)</label>
    </div>

    <div className="alert alert-info">
      <strong>Szacowany koszt: {Math.round(suma)} PLN</strong>
    </div>
  </main>
);
}

export default App;

```

## 26.10. Kalkulator BMI

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Kalkulator BMI” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Kalkulator BMI” dodaj obsługę pustego pola, wartości ujemnych i zaokrąglenia wyniku. Funkcję obliczającą trzymaj poza JSX, np. `obliczWynik(dane)`, a w komponencie zostaw tylko pobieranie danych i prezentację rezultatu.

Formularz z obliczeniem BMI i interpretacją wyniku.

```

// Plik: src/App.js
import { useState } from "react";

```

```
import "bootstrap/dist/css/bootstrap.css";

// Funkcja pomocnicza – interpretacja BMI (poza komponentem)
function interpretujBMI(bmi) {
  if (bmi < 18.5) return { tekst: "Niedowaga", kolor: "warning" };
  if (bmi < 25) return { tekst: "Waga prawidłowa", kolor: "success" };
  if (bmi < 30) return { tekst: "Nadwaga", kolor: "warning" };
  return { tekst: "Otyłość", kolor: "danger" };
}

function App() {
  const [waga, setWaga] = useState("");
  const [wzrost, setWzrost] = useState("");
  const [wynik, setWynik] = useState(null);
  const [blad, setBlad] = useState("");

  function handleOblicz(e) {
    e.preventDefault();

    // Walidacja
    const wagaNum = Number(waga);
    const wzrostNum = Number(wzrost);

    if (wagaNum ≤ 0 || wzrostNum ≤ 0) {
      setBlad("Podaj poprawne wartości wagi i wzrostu");
      setWynik(null);
      return;
    }

    // Obliczenie BMI: waga(kg) / wzrost(m)^2
    const wzrostM = wzrostNum / 100; // cm → m
    const bmi = wagaNum / (wzrostM * wzrostM);

    setWynik(bmi);
    setBlad("");
    console.log(`Waga: ${wagaNum}kg, Wzrost: ${wzrostNum}cm, BMI: ${bmi.toFixed(2)}`);
  }

  return (
    <main className="container mt-4" style={{ maxWidth: "400px" }}>
      <h1>Kalkulator BMI</h1>

      <form onSubmit={handleOblicz}>
        <div className="mb-3">
          <label htmlFor="waga" className="form-label">Waga (kg):</label>
          <input
            id="waga"
            type="number"
            className="form-control"
            value={waga}
            onChange={(e) => setWaga(e.target.value)}
            placeholder="np. 70"
          />
        </div>
      </form>
    </main>
  );
}
```

```

    <div className="mb-3">
      <label htmlFor="wzrost" className="form-label">Wzrost (cm):</label>
      <input
        id="wzrost"
        type="number"
        className="form-control"
        value={wzrost}
        onChange={(e) => setWzrost(e.target.value)}
        placeholder="np. 175"
      />
    </div>

    <button type="submit" className="btn btn-primary w-100">
      Oblicz BMI
    </button>
  </form>

  {/* Błąd walidacji */}
  {blad && <div className="alert alert-danger mt-3">{blad}</div>}

  {/* Wynik BMI */}
  {wynik !== null && (
    <div className={`alert alert-${interpretujBMI(wynik).kolor} mt-3`}>
      <h4>BMI: {wynik.toFixed(2)}</h4>
      <p>{interpretujBMI(wynik).tekst}</p>
    </div>
  )}
</main>
);
}

export default App;

```

## 26.11. Przelicznik walut

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Przelicznik walut” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Przelicznik walut” dodaj obsługę pustego pola, wartości ujemnych i zaokrąglenia wyniku. Funkcję obliczającą trzymaj poza JSX, np. `obliczWynik(dane)`, a w komponencie zostaw tylko pobieranie danych i prezentację rezultatu.

Dwukierunkowy przelicznik walut z dynamiczną tabelą kursów, historią przeliczeń, zamianą walut jednym kliknięciem i wizualnym wskaźnikiem siły waluty.

**Smaczki:** przeliczanie w obie strony (zmiana jednego inputa aktualizuje drugi), przycisk zamiany walut ( $\rightleftharpoons$ ), historia ostatnich 5 przeliczeń, flagi krajów przy walutach, formatowanie liczb z separatorami tysięcy, podświetlenie waluty mocniejszej/słabszej.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

// Kursy walut względem PLN (mockowane – w prawdziwej apce z API)

```

```
const WALUTY = [
  { kod: "PLN", nazwa: "Polski złoty", flaga: "🇵🇱", kursDoPLN: 1 },
  { kod: "EUR", nazwa: "Euro", flaga: "🇪🇺", kursDoPLN: 4.32 },
  { kod: "USD", nazwa: "Dolar amerykański", flaga: "🇺🇸", kursDoPLN: 3.95 },
  { kod: "GBP", nazwa: "Funt brytyjski", flaga: "🇬🇧", kursDoPLN: 5.05 },
  { kod: "CHF", nazwa: "Frank szwajcarski", flaga: "🇨🇭", kursDoPLN: 4.48 },
  { kod: "JPY", nazwa: "Jen japoński", flaga: "🇯🇵", kursDoPLN: 0.026 },
  { kod: "CZK", nazwa: "Korona czeska", flaga: "🇨🇪", kursDoPLN: 0.17 },
  { kod: "UAH", nazwa: "Hrywna ukraińska", flaga: "🇺🇦", kursDoPLN: 0.096 },
];
```

```
// Funkcja przeliczająca
```

```
function przelicz(kwota, walutaZ, walutaDo) {
  if (!kwota || isNaN(kwota)) return "";
  // Najpierw na PLN, potem na walutę docelową
  const wPLN = kwota * walutaZ.kursDoPLN;
  const wynik = wPLN / walutaDo.kursDoPLN;
  return wynik;
}
```

```
// Formatowanie kwoty z separatorami
```

```
function formatujKwote(liczba, kod) {
  if (liczba === "" || isNaN(liczba)) return "-";
  return new Intl.NumberFormat("pl-PL", {
    minimumFractionDigits: 2,
    maximumFractionDigits: 2,
  }).format(liczba) + " " + kod;
}
```

```
function App() {
```

```
  const [kwotaZ, setKwotaZ] = useState("100");
  const [kwotaDo, setKwotaDo] = useState("");
  const [walutaZId, setWalutaZId] = useState("PLN");
  const [walutaDoId, setWalutaDoId] = useState("EUR");
  const [historia, setHistoria] = useState([]);
  const [ostatniaEdycja, setOstatniaEdycja] = useState("z"); // "z" lub "do"
```

```
  const walutaZ = WALUTY.find((w) => w.kod === walutaZId);
  const walutaDo = WALUTY.find((w) => w.kod === walutaDoId);
```

```
  // Kurs wymiany między wybranymi walutami
```

```
  const kursWymiany = walutaZ.kursDoPLN / walutaDo.kursDoPLN;
  const kursOdwrotny = walutaDo.kursDoPLN / walutaZ.kursDoPLN;
```

```
  // Przeliczenie przy zmianie kwoty "z"
```

```
  function handleKwotaZ(e) {
    const val = e.target.value;
    setKwotaZ(val);
    setOstatniaEdycja("z");
    if (val === "" || isNaN(val)) {
      setKwotaDo("");
    } else {
      setKwotaDo(przelicz(Number(val), walutaZ, walutaDo).toFixed(2));
    }
  }
```

```

}

// Przeliczenie przy zmianie kwoty "do" (dwukierunkowe!)
function handleKwotaDo(e) {
  const val = e.target.value;
  setKwotaDo(val);
  setOstatniaEdycja("do");
  if (val === "" || isNaN(val)) {
    setKwotaZ("");
  } else {
    setKwotaZ(przelicz(Number(val), walutaDo, walutaZ).toFixed(2));
  }
}

// Zmiana waluty źródłowej – przelicz ponownie
function handleZmianaWalutyZ(e) {
  const nowaWaluta = WALUTY.find((w) => w.kod === e.target.value);
  setWalutaZId(e.target.value);
  if (kwotaZ && !isNaN(kwotaZ)) {
    setKwotaDo(przelicz(Number(kwotaZ), nowaWaluta, walutaDo).toFixed(2));
  }
}

// Zmiana waluty docelowej – przelicz ponownie
function handleZmianaWalutyDo(e) {
  const nowaWaluta = WALUTY.find((w) => w.kod === e.target.value);
  setWalutaDoId(e.target.value);
  if (kwotaZ && !isNaN(kwotaZ)) {
    setKwotaDo(przelicz(Number(kwotaZ), walutaZ, nowaWaluta).toFixed(2));
  }
}

// Zamiana walut (≡)
function handleZamien() {
  setWalutaZId(walutaDoId);
  setWalutaDoId(walutaZId);
  setKwotaZ(kwotaDo);
  setKwotaDo(kwotaZ);
}

// Zapisanie do historii
function handleZapisz() {
  if (!kwotaZ || isNaN(kwotaZ) || Number(kwotaZ) === 0) return;
  const wpis = {
    id: Date.now(),
    z: `${formatujKwote(Number(kwotaZ), walutaZId)}`,
    do: `${formatujKwote(Number(kwotaDo), walutaDoId)}`,
    kurs: kursWymiany.toFixed(4),
    czas: new Date().toLocaleTimeString("pl-PL"),
  };
  setHistoria((prev) => [wpis, ...prev].slice(0, 5)); // Max 5 wpisów
}

return (

```

```

<main className="container mt-4" style={{ maxWidth: "600px" }}>
  <h1 className="text-center mb-4">Przelicznik Walut</h1>

  {/* Kurs wymiany */}
  <div className="alert alert-info text-center py-2">
    <span>{walutaZ.flaga} 1 {walutaZId}</span>
    <strong className="mx-2">=</strong>
    <span>{walutaDo.flaga} {kursWymiany.toFixed(4)} {walutaDoId}</span>
  </div>

  {/* Formularz przeliczania */}
  <div className="card shadow-sm mb-4">
    <div className="card-body">

      {/* Waluta źródłowa */}
      <div className="row align-items-end mb-3">
        <div className="col-7">
          <label htmlFor="kwotaZ" className="form-label small text-muted">Mam:</label>
          <input
            id="kwotaZ"
            type="number"
            className="form-control form-control-lg"
            value={kwotaZ}
            onChange={handleKwotaZ}
            placeholder="0.00"
            min="0"
            step="0.01"
          />
        </div>
        <div className="col-5">
          <select
            className="form-select form-select-lg"
            value={walutaZId}
            onChange={handleZmianaWalutyZ}
          >
            {WALUTY.map((w) => (
              <option key={w.kod} value={w.kod}>
                {w.flaga} {w.kod}
              </option>
            ))}
          </select>
        </div>
      </div>

      {/* Przycisk zamiany */}
      <div className="text-center mb-3">
        <button
          type="button"
          className="btn btn-outline-secondary btn-sm rounded-circle"
          onClick={handleZamien}
          title="Zamień waluty"
          style={{ width: "40px", height: "40px", fontSize: "1.2rem" }}
        >
      </div>
    </div>
  </div>

```



```

    </button>
  </div>

  { /* Waluta docelowa */ }
  <div className="row align-items-end mb-3">
    <div className="col-7">
      <label htmlFor="kwotaDo" className="form-label small text-muted">Otrzymam:</label>
      <input
        id="kwotaDo"
        type="number"
        className="form-control form-control-lg"
        value={kwotaDo}
        onChange={handleKwotaDo}
        placeholder="0.00"
        min="0"
        step="0.01"
      />
    </div>
    <div className="col-5">
      <select
        className="form-select form-select-lg"
        value={walutaDoId}
        onChange={handleZmianaWalutyDo}
      >
        {WALUTY.map((w) => (
          <option key={w.kod} value={w.kod}>
            {w.flaga} {w.kod}
          </option>
        ))}
      </select>
    </div>
  </div>

  { /* Przycisk zapisu */ }
  <button
    type="button"
    className="btn btn-primary w-100"
    onClick={handleZapisz}
    disabled={!kwotaZ || isNaN(kwotaZ) || Number(kwotaZ) === 0}
  >
     Zapisz przeliczenie
  </button>
</div>

{ /* Tabela kursów */ }
<details className="mb-4">
  <summary className="fw-bold mb-2" style={{ cursor: "pointer" }}>
     Tabela kursów (względem PLN)
  </summary>
  <table className="table table-sm table-striped">
    <thead className="table-dark">
      <tr>
        <th>Waluta</th>

```

```

        <th className="text-end">Kurs (1 = X PLN)</th>
        <th className="text-end">Siła</th>
      </tr>
    </thead>
    <tbody>
      {WALUTY.filter((w) => w.kod !== "PLN")
        .sort((a, b) => b.kursDoPLN - a.kursDoPLN)
        .map((w) => (
          <tr key={w.kod}>
            <td>{w.flaga} {w.kod} – {w.nazwa}</td>
            <td className="text-end">{w.kursDoPLN.toFixed(3)}</td>
            <td className="text-end">
              {w.kursDoPLN > 4 ? (
                <span className="badge bg-success">Mocna</span>
              ) : w.kursDoPLN > 1 ? (
                <span className="badge bg-warning text-dark">Średnia</span>
              ) : (
                <span className="badge bg-secondary">Słaba</span>
              )}
            </td>
          </tr>
        ))}
    </tbody>
  </table>
</details>

{/* Historia przeliczeń */}
{historia.length > 0 && (
  <div className="card">
    <div className="card-header d-flex justify-content-between align-items-center">
      <span className="fw-bold">📊 Historia ({historia.length}/5)</span>
      <button
        className="btn btn-sm btn-outline-danger"
        onClick={() => setHistoria([])}
      >
        Wyczyść
      </button>
    </div>
    <ul className="list-group list-group-flush">
      {historia.map((h) => (
        <li key={h.id} className="list-group-item d-flex justify-content-between">
          <span>{h.z} → {h.do}</span>
          <small className="text-muted">{h.czas}</small>
        </li>
      ))}
    </ul>
  </div>
)}
</main>
);
}

```

```
export default App;
```

### Kluczowe mechaniki w tym wzorcu:

- **Dwukierunkowe przeliczanie** — zmiana jednego inputa aktualizuje drugi (i odwrotnie)
- **Dynamiczny przelicznik** — zmiana waluty w select natychmiast przelicza kwotę
- **Zamiana walut ( $\rightleftharpoons$ )** — jednym kliknięciem zamienia źródło z celem
- **Historia z limitem** — `slice(0, 5)` ogranicza do 5 ostatnich wpisów
- **Intl.NumberFormat** — profesjonalne formatowanie liczb z separatorami
- `<details>` / `<summary>` — natywny HTML do rozwijania tabeli kursów (bez stanu!)
- **Warunkowe badge** — zagnieżdżony ternary do wyświetlania siły waluty
- **Disabled button** — przycisk nieaktywny gdy brak danych
- **Obliczenia poza stanem** — `kursWymiany` obliczany przy renderze, nie trzymany w useState

## 26.12. Kalkulator spalania paliwa i kosztów podróży

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Kalkulator spalania paliwa i kosztów podróży” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Kalkulator spalania paliwa i kosztów podróży” dodaj obsługę pustego pola, wartości ujemnych i zaokrąglenia wyniku. Funkcję obliczającą trzymaj poza JSX, np. `obliczWynik(dane)`, a w komponencie zostaw tylko pobieranie danych i prezentację rezultatu.

Prosta aplikacja kalkulująca koszty przejazdu na podstawie odległości, spalania i ceny paliwa.

```
// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [dystans, setDystans] = useState("");
  const [spalanie, setSpalanie] = useState("");
  const [cena, setCena] = useState("");
  const [pasazerowie, setPasazerowie] = useState("1");
  const [wynik, setWynik] = useState(null);

  function oblicz(e) {
    e.preventDefault();
    const d = Number(dystans);
    const s = Number(spalanie);
    const c = Number(cena);
    const p = Number(pasazerowie);

    if (d > 0 && s > 0 && c > 0 && p > 0) {
      const zuzycie = (d / 100) * s;
      const kosztCalkowity = zuzycie * c;
      const kosztNaOsobe = kosztCalkowity / p;

      setWynik({
        zuzycie: zuzycie.toFixed(2),
        koszt: kosztCalkowity.toFixed(2),
        naOsobe: kosztNaOsobe.toFixed(2)
      });
    }
  }
}
```

```

return (
  <main className="container mt-4" style={{ maxWidth: "400px" }}>
    <h1>Koszty podróży</h1>
    <form onSubmit={oblicz}>
      <div className="mb-2">
        <label className="form-label">Dystans (km):</label>
        <input type="number" className="form-control" value={dystans} onChange={e => setDystans(e.target.value)} required />
      </div>
      <div className="mb-2">
        <label className="form-label">Średnie spalanie (l/100km):</label>
        <input type="number" step="0.1" className="form-control" value={spalanie} onChange={e => setSpalanie(e.target.value)}
        required />
      </div>
      <div className="mb-2">
        <label className="form-label">Cena paliwa (zł/l):</label>
        <input type="number" step="0.01" className="form-control" value={cena} onChange={e => setCena(e.target.value)} required
        />
      </div>
      <div className="mb-3">
        <label className="form-label">Liczba osób:</label>
        <input type="number" className="form-control" value={pasazerowie} min="1" onChange={e => setPasazerowie(e.target.value)}
        required />
      </div>
      <button type="submit" className="btn btn-primary w-100">Oblicz</button>
    </form>

    {wynik && (
      <div className="mt-4 p-3 bg-light rounded border">
        <p>Zużycie paliwa: <strong>{wynik.zuzycie} l</strong></p>
        <p>Całkowity koszt: <strong>{wynik.koszt} zł</strong></p>
        <p>Koszt na osobę: <strong className="text-success">{wynik.naOsobe} zł</strong></p>
      </div>
    )}
  </main>
);
}

export default App;

```

### 26.13. Kalkulator rat kredytu (symulator)

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Kalkulator rat kredytu (symulator)” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Kalkulator rat kredytu (symulator)” dodaj obsługę pustego pola, wartości ujemnych i zaokrąglenia wyniku. Funkcję obliczającą trzymaj poza JSX, np. `obliczWynik(dane)`, a w komponencie zostaw tylko pobieranie danych i prezentację rezultatu.

Aplikacja wykorzystująca wzór matematyczny do obliczania stałej raty kredytowej.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

```

```

function App() {
  const [kwota, setKwota] = useState("");
  const [oprocentowanie, setOprocentowanie] = useState("");
  const [okres, setOkres] = useState("");
  const [wynik, setWynik] = useState(null);

  function obliczRate(e) {
    e.preventDefault();
    const K = Number(kwota); // Kapitał
    const p = Number(oprocentowanie) / 100 / 12; // Miesięczne oprocentowanie
    const n = Number(okres); // Liczba rat (miesięcy)

    if (K > 0 && p > 0 && n > 0) {
      // Wzór na ratę stałą
      const rata = (K * p * Math.pow(1 + p, n)) / (Math.pow(1 + p, n) - 1);
      const suma = rata * n;
      const odsetki = suma - K;

      setWynik({
        rata: rata.toFixed(2),
        suma: suma.toFixed(2),
        odsetki: odsetki.toFixed(2)
      });
    }
  }

  return (
    <main className="container mt-4" style={{ maxWidth: "450px" }}>
      <h1>Kalkulator Kredytowy</h1>
      <form onSubmit={obliczRate}>
        <div className="mb-3">
          <label className="form-label">Kwota kredytu (zł):</label>
          <input type="number" className="form-control" value={kwota} onChange={e => setKwota(e.target.value)} required />
        </div>
        <div className="mb-3">
          <label className="form-label">Oprocentowanie roczne (%):</label>
          <input type="number" step="0.1" className="form-control" value={oprocentowanie} onChange={e =>
setOprocentowanie(e.target.value)} required />
        </div>
        <div className="mb-4">
          <label className="form-label">Okres spłaty (w miesiącach):</label>
          <input type="number" className="form-control" value={okres} onChange={e => setOkres(e.target.value)} required />
        </div>
        <button type="submit" className="btn btn-primary w-100">Oblicz ratę</button>
      </form>

      {wynik && (
        <div className="mt-4 alert alert-info">
          <h5>Miesięczna rata: <strong>{wynik.rata} zł</strong></h5>
          <hr />
          <p className="mb-1">Całkowita kwota do spłaty: {wynik.suma} zł</p>
          <p className="mb-0 text-danger">Z tego odsetki: {wynik.odsetki} zł</p>
        </div>
      )}
    </main>
  );
}

```

```

    })
  </main>
);
}

export default App;

```

#### 26.14. Kalkulator zapotrzebowania kalorycznego (BMR i TDEE)

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Kalkulator zapotrzebowania kalorycznego (BMR i TDEE)” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Kalkulator zapotrzebowania kalorycznego (BMR i TDEE)” dodaj obsługę pustego pola, wartości ujemnych i zaokrąglenia wyniku. Funkcję obliczającą trzymaj poza JSX, np. `obliczWynik(dane)`, a w komponencie zostaw tylko pobieranie danych i prezentację rezultatu.

Kalkulator używający wzoru Harrisa-Benedicta oraz mnożników aktywności z radio buttons i selectem.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [płeć, setPłeć] = useState("k");
  const [waga, setWaga] = useState("");
  const [wzrost, setWzrost] = useState("");
  const [wiek, setWiek] = useState("");
  const [aktywnosc, setAktywnosc] = useState("1.2");
  const [wynik, setWynik] = useState(null);

  function obliczKcal(e) {
    e.preventDefault();
    const w = Number(waga);
    const h = Number(wzrost);
    const a = Number(wiek);

    let bmr = 0;
    // Wzór Mifflina-St Jeor (dokładniejszy dla ogółu)
    if (płeć === "m") {
      bmr = (10 * w) + (6.25 * h) - (5 * a) + 5;
    } else {
      bmr = (10 * w) + (6.25 * h) - (5 * a) - 161;
    }

    const tdee = bmr * Number(aktywnosc);
    setWynik({ bmr: Math.round(bmr), tdee: Math.round(tdee) });
  }

  return (
    <main className="container mt-4" style={{ maxWidth: "500px" }}>
      <h1>Zapotrzebowanie Kcal</h1>
      <form onSubmit={obliczKcal}>
        <div className="mb-3">

```

```

        <label className="me-3">Płeć:</label>
        <div className="form-check form-check-inline">
            <input className="form-check-input" type="radio" name="plec" value="k" checked={płeć === "k"} onChange={e =>
setPłeć(e.target.value)} />
            <label className="form-check-label">Kobieta</label>
        </div>
        <div className="form-check form-check-inline">
            <input className="form-check-input" type="radio" name="plec" value="m" checked={płeć === "m"} onChange={e =>
setPłeć(e.target.value)} />
            <label className="form-check-label">Mężczyzna</label>
        </div>
    </div>
    <div className="row mb-3">
        <div className="col">
            <label>Waga (kg)</label>
            <input type="number" className="form-control" value={waga} onChange={e => setWaga(e.target.value)} required />
        </div>
        <div className="col">
            <label>Wzrost (cm)</label>
            <input type="number" className="form-control" value={wzrost} onChange={e => setWzrost(e.target.value)} required />
        </div>
        <div className="col">
            <label>Wiek</label>
            <input type="number" className="form-control" value={wiek} onChange={e => setWiek(e.target.value)} required />
        </div>
    </div>
    <div className="mb-4">
        <label>Poziom aktywności fizycznej:</label>
        <select className="form-select" value={aktywnosc} onChange={e => setAktywnosc(e.target.value)}>
            <option value="1.2">Brak aktywności (praca siedząca)</option>
            <option value="1.375">Niska aktywność (1-3 razy w tyg.)</option>
            <option value="1.55">Średnia aktywność (3-5 razy w tyg.)</option>
            <option value="1.725">Wysoka aktywność (codziennie)</option>
        </select>
    </div>
    <button type="submit" className="btn btn-success w-100">Oblicz</button>
</form>

{wynik && (
    <div className="alert alert-success mt-4">
        <p className="mb-1">BMR (Podstawowa przemiana materii): <strong>{wynik.bmr}</strong> kcal</p>
        <p className="mb-0">TDEE (Całkowite zapotrzebowanie): <strong>{wynik.tdee}</strong> kcal</p>
    </div>
)}
</main>
);
}

export default App;

```

## 26.15. Kalkulator wieku psa (ludzkie lata)

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Kalkulator wieku psa (ludzkie lata)” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Kalkulator wieku psa (ludzkie lata)” dodaj obsługę pustego pola, wartości ujemnych i zaokrąglenia wyniku. Funkcję obliczającą trzymaj poza JSX, np. `obliczWynik(dane)`, a w komponencie zostaw tylko pobieranie danych i prezentację rezultatu.

Prosty kalkulator z suwakiem i warunkowym mnożnikiem uzależnionym od rozmiaru zwierzęcia.

```
// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [wiekPsa, setWiekPsa] = useState(5);
  const [rozmiar, setRozmiar] = useState("mały");

  let ludzkieLata = 0;
  if (wiekPsa === 1) ludzkieLata = 15;
  else if (wiekPsa === 2) ludzkieLata = 24;
  else {
    const baza = 24;
    const resztaLat = wiekPsa - 2;
    if (rozmiar === "mały") ludzkieLata = baza + (resztaLat * 4);
    else if (rozmiar === "średni") ludzkieLata = baza + (resztaLat * 5);
    else if (rozmiar === "duży") ludzkieLata = baza + (resztaLat * 6);
  }

  return (
    <main className="container mt-4" style={{ maxWidth: "450px" }}>
      <h1>Ile lat ma Twój pies?</h1>
      <div className="card p-4 shadow-sm mt-3">
        <div className="mb-3">
          <label className="form-label">Wiek psa w latach kalendarzowych: <strong>{wiekPsa}</strong></label>
          <input
            type="range"
            className="form-range"
            min="1" max="20"
            value={wiekPsa}
            onChange={(e) => setWiekPsa(Number(e.target.value))}
          />
        </div>

        <div className="mb-4">
          <label className="form-label">Rozmiar psa:</label>
          <select className="form-select" value={rozmiar} onChange={(e) => setRozmiar(e.target.value)}>
            <option value="mały">Mały (do 10 kg)</option>
            <option value="średni">Średni (10 - 25 kg)</option>
            <option value="duży">Duży (powyżej 25 kg)</option>
          </select>
        </div>

        <div className="text-center">
          <p className="text-muted mb-1">Wiek w przełożeniu na ludzkie lata:</p>

```



```

        <h2 className="text-primary display-4">{ludzkieLata}</h2>
      </div>
    </div>
  </main>
);
}

export default App;

```

## 26.16. Kalkulator czasu pracy i wynagrodzenia

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Kalkulator czasu pracy i wynagrodzenia” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Kalkulator czasu pracy i wynagrodzenia” dodaj obsługę pustego pola, wartości ujemnych i zaokrąglenia wyniku. Funkcję obliczającą trzymaj poza JSX, np. `obliczWynik(dane)`, a w komponencie zostaw tylko pobieranie danych i prezentację rezultatu.

Narzędzie przeliczające stawkę godzinową na pensję brutto i przybliżone netto w zależności od wybranej umowy.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [stawka, setStawka] = useState("");
  const [godziny, setGodziny] = useState("");
  const [umowa, setUmowa] = useState("zlecenie");
  const [wynik, setWynik] = useState(null);

  function obliczWyplate(e) {
    e.preventDefault();
    const brutto = Number(stawka) * Number(godziny);

    let netto = 0;
    // Bardzo uproszczone kalkulacje podatkowe na potrzeby przykładu
    if (umowa === "zlecenie") netto = brutto * 0.85; // ok. 15% potrąceń (status studenta itp.)
    else if (umowa === "dzieło") netto = brutto * 0.91; // Koszty uzyskania przychodu
    else if (umowa === "etat") netto = brutto * 0.73; // Pełny ZUS i podatek

    setWynik({
      brutto: brutto.toFixed(2),
      netto: netto.toFixed(2)
    });
  }

  return (
    <main className="container mt-4" style={{ maxWidth: "450px" }}>
      <h1>Kalkulator Wypłaty</h1>
      <form onSubmit={obliczWyplate} className="card p-3 bg-light">
        <div className="mb-3">
          <label>Stawka godzinowa brutto (zł):</label>
          <input type="number" className="form-control" value={stawka} onChange={e => setStawka(e.target.value)} required />
        </div>

```

```

    <div className="mb-3">
      <label>Przepracowane godziny:</label>
      <input type="number" className="form-control" value={godziny} onChange={e => setGodziny(e.target.value)} required />
    </div>
    <div className="mb-3">
      <label>Rodzaj umowy:</label>
      <select className="form-select" value={umowa} onChange={e => setUmowa(e.target.value)}>
        <option value="zlecenie">Umowa Zlecenie</option>
        <option value="dzieło">Umowa o Dzieło</option>
        <option value="etat">Umowa o Pracę (Etat)</option>
      </select>
    </div>
    <button type="submit" className="btn btn-dark">Wylicz wynagrodzenie</button>
  </form>

  {wynik && (
    <div className="alert alert-success mt-3">
      <p className="mb-1">Wynagrodzenie brutto: <strong>{wynik.brutto} zł</strong></p>
      <p className="mb-0">Szacowane netto na rękę: <strong>{wynik.netto} zł</strong></p>
    </div>
  )}
</main>
);
}

export default App;

```

## 26.17. Konwerter systemów liczbowych

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Konwerter systemów liczbowych” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Konwerter systemów liczbowych” dodaj obsługę pustego pola, wartości ujemnych i zaokrąglenia wyniku. Funkcję obliczającą trzymaj poza JSX, np. `obliczWynik(dane)`, a w komponencie zostaw tylko pobieranie danych i prezentację rezultatu.

Aplikacja z dynamicznym przetwarzaniem wprowadzanej liczby dziesiętnej na system dwójkowy, ósemkowy i szesnastkowy.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [dziesiętna, setDziesiętna] = useState("");

  const num = parseInt(dziesiętna, 10);
  const isOk = !isNaN(num);

  return (
    <main className="container mt-4" style={{ maxWidth: "500px" }}>
      <h1>Konwerter systemów liczbowych</h1>

      <div className="mb-4 mt-3">
        <label className="form-label">Wprowadź liczbę dziesiętną (DEC):</label>

```

```

    <input
      type="number"
      className="form-control form-control-lg"
      value={dziesietna}
      onChange={(e) => setDziesietna(e.target.value)}
      placeholder="Np. 255"
    />
  </div>

  <div className="card">
    <div className="card-header bg-dark text-white">Wyniki konwersji</div>
    <ul className="list-group list-group-flush">
      <li className="list-group-item d-flex justify-content-between align-items-center">
        <strong>System Dwójkowy (BIN)</strong>
        <span className="badge bg-primary rounded-pill fs-6">
          {isOk ? num.toString(2) : "---"}
        </span>
      </li>
      <li className="list-group-item d-flex justify-content-between align-items-center">
        <strong>System Ósemkowy (OCT)</strong>
        <span className="badge bg-success rounded-pill fs-6">
          {isOk ? num.toString(8) : "---"}
        </span>
      </li>
      <li className="list-group-item d-flex justify-content-between align-items-center">
        <strong>System Szesnastkowy (HEX)</strong>
        <span className="badge bg-danger rounded-pill fs-6">
          {isOk ? num.toString(16).toUpperCase() : "---"}
        </span>
      </li>
    </ul>
  </div>
</main>
);
}

export default App;

```

## 26.18. Generator hasła

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Generator hasła” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Generator hasła” dodaj jeden konkretny wariant nieidealny: brak danych, błędną wartość, reset albo komunikat po wykonaniu akcji. Taki wariant pokazuje, czy stan komponentu jest zaprojektowany kompletnie.

Aplikacja generująca losowe hasło na podstawie ustawień użytkownika.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {

```

```

const [dlugosc, setDlugosc] = useState(12);
const [duzeLinetry, setDuzeLinetry] = useState(true);
const [cyfry, setCyfry] = useState(true);
const [specjalne, setSpecjalne] = useState(false);
const [haslo, setHaslo] = useState("");

function generujHaslo() {
  let znaki = "abcdefghijklmnopqrstuvwxyz";
  if (duzeLinetry) znaki += "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
  if (cyfry) znaki += "0123456789";
  if (specjalne) znaki += "!@#$%^&*()_+~[]{}|;:,.<>?";

  let wynik = "";
  for (let i = 0; i < dlugosc; i++) {
    const losowy = Math.floor(Math.random() * znaki.length);
    wynik += znaki[losowy];
  }

  setHaslo(wynik);
}

return (
  <main className="container mt-4" style={{ maxWidth: "500px" }}>
    <h1>Generator hasła</h1>

    {/}* Suwak długości */}
    <div className="mb-3">
      <label className="form-label">Długość: {dlugosc}</label>
      <input
        type="range"
        className="form-range"
        min="4"
        max="32"
        value={dlugosc}
        onChange={(e) => setDlugosc(Number(e.target.value))}
      />
    </div>

    {/}* Checkboxy opcji */}
    <div className="form-check mb-2">
      <input
        id="duze"
        type="checkbox"
        className="form-check-input"
        checked={duzeLinetry}
        onChange={(e) => setDuzeLinetry(e.target.checked)}
      />
      <label htmlFor="duze" className="form-check-label">Duże litery (A-Z)</label>
    </div>

    <div className="form-check mb-2">
      <input
        id="cyfry"
        type="checkbox"

```

```

        className="form-check-input"
        checked={cyfry}
        onChange={(e) => setCyfry(e.target.checked)}
      />
      <label htmlFor="cyfry" className="form-check-label">Cyfry (0-9)</label>
    </div>

    <div className="form-check mb-3">
      <input
        id="specjalne"
        type="checkbox"
        className="form-check-input"
        checked={specjalne}
        onChange={(e) => setSpecjalne(e.target.checked)}
      />
      <label htmlFor="specjalne" className="form-check-label">Znaki specjalne (!@#$)</label>
    </div>

    <button className="btn btn-primary mb-3" onClick={generujHaslo}>
      Generuj hasło
    </button>

    {/* Wynik */}
    {haslo && (
      <div className="alert alert-success">
        <strong>Wygenerowane hasło:</strong>
        <code className="d-block mt-1" style={{ fontSize: "1.2rem" }}>
          {haslo}
        </code>
      </div>
    )}
  </main>
);
}

export default App;

```

## 26.19. Kości do gry z blokowaniem

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Kości do gry z blokowaniem” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Kości do gry z blokowaniem” dodaj historię rund, licznik punktów i reset całej rozgrywki. Dane konfiguracyjne, takie jak lista pytań, możliwe ruchy albo ściany kostki, trzymaj poza komponentem.

Gra w kości — rzut 5 kośćmi, możliwość blokowania wybranych kości przy ponownym rzucie.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

// Funkcja losująca wartość kości (1-6)

```

```

function losujKosc() {
  return Math.floor(Math.random() * 6) + 1;
}

function App() {
  // Stan kości – tablica 5 obiektów z wartością i stanem blokady
  const [kosci, setKosci] = useState([
    { id: 1, wartosc: losujKosc(), zablockowana: false },
    { id: 2, wartosc: losujKosc(), zablockowana: false },
    { id: 3, wartosc: losujKosc(), zablockowana: false },
    { id: 4, wartosc: losujKosc(), zablockowana: false },
    { id: 5, wartosc: losujKosc(), zablockowana: false },
  ]);

  const [liczbaRzutow, setLiczbaRzutow] = useState(1);

  // Rzut kośćmi – tylko niezablockowane
  function handleRzut() {
    setKosci((prev) =>
      prev.map((k) =>
        k.zablockowana ? k : { ...k, wartosc: losujKosc() }
      )
    );
    setLiczbaRzutow((prev) => prev + 1);
  }

  // Blokowanie/odblokowanie kości
  function handleBlokuj(id) {
    setKosci((prev) =>
      prev.map((k) =>
        k.id === id ? { ...k, zablockowana: !k.zablockowana } : k
      )
    );
  }

  // Nowa gra – reset
  function handleNowaGra() {
    setKosci(
      kosci.map((k) => ({ ...k, wartosc: losujKosc(), zablockowana: false }))
    );
    setLiczbaRzutow(1);
  }

  // Suma kości
  const suma = kosci.reduce((acc, k) => acc + k.wartosc, 0);

  return (
    <main className="container mt-4 text-center" style={{ maxWidth: "500px" }}>
      <h1>Kości do gry</h1>
      <p>Rzut numer: {liczbaRzutow} | Suma: {suma}</p>

      {/* Wyświetlanie kości */}
      <div className="d-flex justify-content-center gap-2 mb-4">
        {kosci.map((k) => (

```

```

        <button
          key={k.id}
          className={`btn btn-lg ${
            k.zablokowana ? "btn-danger" : "btn-outline-dark"
          }`}
          style={{ width: "60px", height: "60px", fontSize: "1.5rem" }}
          onClick={() => handleBlokuj(k.id)}
        >
          {k.wartosc}
        </button>
      )}
    </div>

    <p className="text-muted small">
      Kliknij kość, aby ją zablokować (czerwona = zablokowana)
    </p>

    <div className="d-flex gap-2 justify-content-center">
      <button className="btn btn-primary" onClick={handleRzut}>
        Rzuć kośćmi
      </button>
      <button className="btn btn-secondary" onClick={handleNowaGra}>
        Nowa gra
      </button>
    </div>
  </main>
);
}

export default App;

```

## 26.20. Gra w zgadywanie liczb (Za dużo / Za mało)

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Gra w zgadywanie liczb (Za dużo / Za mało)” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Gra w zgadywanie liczb (Za dużo / Za mało)” dodaj historię rund, licznik punktów i reset całej rozgrywki. Dane konfiguracyjne, takie jak lista pytań, możliwe ruchy albo ściany kostki, trzymaj poza komponentem.

Prosta gra w zgadywanie liczby wylosowanej przez komputer z przedziału 1-100 z historią prób.

```

// Plik: src/App.js
import { useState, useEffect } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [cel, setCel] = useState(0);
  const [proba, setProba] = useState("");
  const [historia, setHistoria] = useState([]);
  const [komunikat, setKomunikat] = useState("Zgadnij liczbę od 1 do 100");
  const [wygrana, setWygrana] = useState(false);

```

```

// Losowanie liczby przy pierwszym montowaniu lub po resecie
useEffect(() => {
  losujZnowu();
}, []);

function losujZnowu() {
  setCel(Math.floor(Math.random() * 100) + 1);
  setHistoria([]);
  setProba("");
  setKomunikat("Zgadnij liczbę od 1 do 100");
  setWygrana(false);
}

function sprawdz(e) {
  e.preventDefault();
  if (wygrana) return;

  const zgadywana = Number(proba);
  if (zgadywana < 1 || zgadywana > 100) {
    setKomunikat("Liczba poza zakresem!");
    return;
  }

  const nowaHistoria = [...historia, zgadywana];
  setHistoria(nowaHistoria);
  setProba("");

  if (zgadywana === cel) {
    setKomunikat(`Brawo! Zgadłeś w ${nowaHistoria.length} próbach.`);
    setWygrana(true);
  } else if (zgadywana < cel) {
    setKomunikat("Za mało!");
  } else {
    setKomunikat("Za dużo!");
  }
}

return (
  <main className="container mt-5 text-center" style={{ maxWidth: "400px" }}>
    <h1>Zgadnij Liczbę</h1>
    <h4 className={`my-3 ${wygrana ? 'text-success' : 'text-primary'}`}>
      {komunikat}
    </h4>

    <form onSubmit={sprawdz} className="mb-3">
      <input
        type="number"
        className="form-control mb-2 text-center fs-4"
        value={proba}
        onChange={e => setProba(e.target.value)}
        disabled={wygrana}
        autoFocus
      />
      {!wygrana ? (

```



```

        <button type="submit" className="btn btn-primary w-100">Sprawdź</button>
      ) : (
        <button type="button" className="btn btn-success w-100" onClick={losujZnowu}>Graj jeszcze raz</button>
      )}
    </form>

    {historia.length > 0 && (
      <div className="text-start">
        <h5>Twoje próby:</h5>
        <p>{historia.join(" → ")}</p>
      </div>
    )}
  </main>
);
}

export default App;

```

## 26.21. Kamień, Papier, Nożyce

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Kamień, Papier, Nożyce” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Kamień, Papier, Nożyce” dodaj historię rund, licznik punktów i reset całej rozgrywki. Dane konfiguracyjne, takie jak lista pytań, możliwe ruchy albo ściany kostki, trzymaj poza komponentem.

Klasyczna gra z komputerem wykorzystująca losowanie wartości oraz zarządzanie punktacją.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

const opcje = ["Kamień", "Papier", "Nożyce"];

function App() {
  const [gracz, setGracz] = useState(null);
  const [komputer, setKomputer] = useState(null);
  const [wynikRundy, setWynikRundy] = useState("");
  const [punktyGracza, setPunktyGracza] = useState(0);
  const [punktyKomputera, setPunktyKomputera] = useState(0);

  function zagraj(wyborGracza) {
    const los = Math.floor(Math.random() * 3);
    const wyborKomputera = opcje[los];

    setGracz(wyborGracza);
    setKomputer(wyborKomputera);

    if (wyborGracza === wyborKomputera) {
      setWynikRundy("Remis!");
    } else if (
      (wyborGracza === "Kamień" && wyborKomputera === "Nożyce") ||
      (wyborGracza === "Papier" && wyborKomputera === "Kamień") ||

```

```

    (wyborGracza === "Nożyce" && wyborKomputera === "Papier")
  ) {
    setWynikRundy("Wygrywasz rundę!");
    setPunktyGracza(p => p + 1);
  } else {
    setWynikRundy("Komputer wygrywa rundę!");
    setPunktyKomputera(p => p + 1);
  }
}

return (
  <main className="container mt-5 text-center" style={{ maxWidth: "500px" }}>
    <h1>Kamień, Papier, Nożyce</h1>

    <div className="d-flex justify-content-around my-4">
      <div>
        <h3>Ty</h3>
        <h2>{punktyGracza}</h2>
      </div>
      <div>
        <h3>Komputer</h3>
        <h2>{punktyKomputera}</h2>
      </div>
    </div>

    <div className="mb-4">
      {opcje.map(opcja => (
        <button key={opcja} className="btn btn-outline-dark mx-2 btn-lg" onClick={() => zagraj(opcja)}>
          {opcja}
        </button>
      ))}
    </div>

    {gracz && (
      <div className="card p-3 bg-light">
        <p className="fs-5 mb-1">Twój wybór: <strong>{gracz}</strong></p>
        <p className="fs-5 mb-3">Wybór komputera: <strong>{komputer}</strong></p>
        <h4 className={wynikRundy.includes("Wygrywasz") ? "text-success" : (wynikRundy.includes("Komputer") ? "text-danger" :
"text-muted")}>
          {wynikRundy}
        </h4>
      </div>
    )}
  </main>
);
}

export default App;

```

## 26.22. Rzut monetą ze statystykami i historią

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Rzut monetą ze statystykami i historią” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Rzut monetą ze statystykami i historią” dodaj historię rund, licznik punktów i reset całej rozgrywki. Dane konfiguracyjne, takie jak lista pytań, możliwe ruchy albo ściany kostki, trzymaj poza komponentem.

Symulator rzutu monetą generujący statystyki rzutów z zapisem ostatnich pięciu wyników.

```
// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [wynik, setWynik] = useState(null);
  const [orly, setOrly] = useState(0);
  const [reszki, setReszki] = useState(0);
  const [historia, setHistoria] = useState([]);

  function rzucMoneta() {
    const los = Math.random() < 0.5 ? "Orzeł" : "Reszka";
    setWynik(los);

    if (los === "Orzeł") setOrly(o => o + 1);
    else setReszki(r => r + 1);

    setHistoria(prev => {
      const nowa = [los, ...prev];
      if (nowa.length > 5) nowa.pop(); // Zostaw maksymalnie 5 ostatnich
      return nowa;
    });
  }

  const suma = orly + reszki;
  const procOrzel = suma > 0 ? Math.round((orly / suma) * 100) : 0;
  const procReszka = suma > 0 ? Math.round((reszki / suma) * 100) : 0;

  return (
    <main className="container mt-5 text-center" style={{ maxWidth: "450px" }}>
      <h1>Rzut Monetą</h1>

      <div className="my-4" style={{ height: "100px" }}>
        {wynik ? (
          <h1 className="display-1 fw-bold text-primary">{wynik}</h1>
        ) : (
          <p className="text-muted mt-4 pt-2">Kliknij przycisk, aby rzucić</p>
        )}
      </div>

      <button className="btn btn-warning btn-lg px-5 mb-4" onClick={rzucMoneta}>
        RZUĆ MONETĄ
      </button>

      <div className="row text-center mb-4">
```

```

    <div className="col">
      <p className="mb-0">0rzeł</p>
      <h4>{orly} ({procOrzeł}%)</h4>
    </div>
    <div className="col">
      <p className="mb-0">Suma rzutów</p>
      <h4>{suma}</h4>
    </div>
    <div className="col">
      <p className="mb-0">Reszka</p>
      <h4>{reszki} ({procReszka}%)</h4>
    </div>
  </div>

  {historia.length > 0 && (
    <ul className="list-group">
      <li className="list-group-item bg-light fw-bold">Ostatnie rzuty:</li>
      {historia.map((h, i) => (
        <li key={i} className="list-group-item">{h}</li>
      ))}
    </ul>
  )}
</main>
);
}

export default App;

```

## 26.23. Galeria zdjęć z kategoriami

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Galeria zdjęć z kategoriami” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Galeria zdjęć z kategoriami” dodaj wyszukiwarkę albo filtr kategorii i trzymaj wynik w zmiennej `widoczneElementy`. Dzięki temu logika wyboru danych jest oddzielona od JSX renderującego pojedynczą kartę lub wiersz.

Interaktywna galeria kafelkowa opierająca się o filtry stanów (checkboxy / switchy).

### Wymagania:

- Tablica obiektów zdjęć z `dane.txt`
- Trzy pola switch/checkbox do filtrowania kategorii
- Filtrowanie zdjęć po zaznaczonych kategoriach
- Wyświetlanie bloków zdjęć obok siebie
- Pokazanie liczby pobrań
- Przycisk „Pobierz” zwiększający liczbę pobrań dla klikniętego zdjęcia

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  // Dane zdjęć – przepisane z dane.txt
  const [zdjecia, setZdjecia] = useState([

```

```

    { id: 1, nazwa: "kwiat.jpg", kategoria: "kwiaty", pobrania: 12 },
    { id: 2, nazwa: "gora.jpg", kategoria: "krajobrazy", pobrania: 34 },
    { id: 3, nazwa: "roza.jpg", kategoria: "kwiaty", pobrania: 7 },
    { id: 4, nazwa: "miasto.jpg", kategoria: "miasto", pobrania: 21 },
    { id: 5, nazwa: "jezioro.jpg", kategoria: "krajobrazy", pobrania: 15 },
    { id: 6, nazwa: "tulipan.jpg", kategoria: "kwiaty", pobrania: 3 },
    { id: 7, nazwa: "ulica.jpg", kategoria: "miasto", pobrania: 9 },
  ]);

// Stan checkboxów/switchów – które kategorie są zaznaczone
const [filtry, setFiltry] = useState({
  kwiaty: true,
  krajobrazy: true,
  miasto: true,
});

// Obsługa zmiany switcha – aktualizacja jednego pola w obiekcie filtrów
function handleFiltr(kategoria) {
  setFiltry((prev) => ({
    ...prev,
    [kategoria]: !prev[kategoria],
  }));
}

// Obsługa kliknięcia "Pobierz" – zwiększenie pobrań dla jednego zdjęcia
function handlePobierz(id) {
  setZdjecia((prev) =>
    prev.map((z) =>
      z.id === id ? { ...z, pobrania: z.pobrania + 1 } : z
    )
  );
}

// Filtrowanie zdjęć – pokazujemy tylko te z zaznaczonych kategorii
const przefiltrowane = zdjecia.filter((z) => filtry[z.kategoria]);

return (
  <main className="container mt-4">
    <h1>Galeria zdjęć</h1>

    {/* Switche do filtrowania kategorii */}
    <div className="mb-4">
      {Object.keys(filtry).map((kategoria) => (
        <div key={kategoria} className="form-check form-switch form-check-inline">
          <input
            id={`filtr-${kategoria}`}
            type="checkbox"
            className="form-check-input"
            checked={filtry[kategoria]}
            onChange={() => handleFiltr(kategoria)}
          />
          <label htmlFor={`filtr-${kategoria}`} className="form-check-label">
            {kategoria}
          </label>
        </div>
      ))}
    </div>
  </main>
);

```

```

    </div>
  )}
</div>

{/* Siatka kart ze zdjęciami */}
<div className="row">
  {przefiltrowane.map((zdjecie) => (
    <div key={zdjecie.id} className="col-md-4 mb-3">
      <div className="card">
        {/* Miejsce na obraz – w prawdziwym projekcie byłby tu <img> */}
        <div
          className="card-img-top bg-secondary d-flex align-items-center justify-content-center"
          style={{ height: "150px", color: "white" }}
        >
          {zdjecie.nazwa}
        </div>
        <div className="card-body">
          <p className="card-text">
            Kategoria: <strong>{zdjecie.kategoria}</strong>
          </p>
          <p className="card-text">
            Pobrania: <strong>{zdjecie.pobrania}</strong>
          </p>
          <button
            className="btn btn-success btn-sm"
            onClick={() => handlePobierz(zdjecie.id)}
          >
            Pobierz
          </button>
        </div>
      </div>
    </div>
  )})}
</div>

{/* Komunikat gdy brak zdjęć */}
{przefiltrowane.length === 0 && (
  <p className="text-muted text-center">Brak zdjęć do wyświetlenia.</p>
)}
</main>

);
}

export default App;

```

#### Kluczowe mechaniki:

- Tablica obiektów w stanie ( `useState` )
- Switche/checkboxy kontrolujące filtrowanie
- `filter()` do wyświetlania tylko wybranych kategorii
- `map()` do aktualizacji jednego elementu (zwiększenie licznika pobrań)
- Spread operator ( `...` ) do niemutowalnej aktualizacji
- Grid Bootstrap ( `row` / `col-md-4` ) do układania kart obok siebie

- Warunkowe renderowanie pustej listy

## 26.24. Lista zadań (Todo App) — wieloplikowy

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Lista zadań (Todo App) — wieloplikowy” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Lista zadań (Todo App) — wieloplikowy” dodaj wyszukiwarkę albo filtr kategorii i trzymaj wynik w zmiennej `widoczneElementy`. Dzięki temu logika wyboru danych jest oddzielona od JSX renderującego pojedynczą kartę lub wiersz.

Kompletny przykład aplikacji z podziałem na pliki — wzorec przepływu danych parent-child.

```
src/
├─ App.js           # Rodzic – trzyma stan
├─ components/
│   ├─ TaskForm.js  # Formularz dodawania
│   └─ TaskList.js  # Lista zadań
└─ index.js         # Punkt wejścia
```

```
// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";
import TaskForm from "../components/TaskForm";
import TaskList from "../components/TaskList";

function App() {
  // Stan globalny – lista zadań trzymana w rodzicu
  const [tasks, setTasks] = useState([]);

  // Funkcja dodająca zadanie – przekazywana do TaskForm
  function addTask(taskText) {
    const newTask = {
      id: Date.now(), // Prosty sposób na unikalne ID
      text: taskText,
      completed: false,
    };
    setTasks((prev) => [...prev, newTask]);
  }

  // Funkcja usuwająca zadanie – przekazywana do TaskList
  function deleteTask(id) {
    setTasks((prev) => prev.filter((task) => task.id !== id));
  }

  // Funkcja przełączająca ukończenie – przekazywana do TaskList
  function toggleTask(id) {
    setTasks((prev) =>
      prev.map((task) =>
        task.id === id ? { ...task, completed: !task.completed } : task
      )
    );
  }
}
```

```

    }

    return (
      <main className="container mt-5" style={{ maxWidth: "600px" }}>
        <h1 className="text-center mb-4">Lista Zadań</h1>

        {/* Komponent formularza – otrzymuje callback onAddTask */}
        <TaskForm onAddTask={addTask} />

        {/* Komponent listy – otrzymuje dane i callbacki */}
        <TaskList
          tasks={tasks}
          onDeleteTask={deleteTask}
          onToggleTask={toggleTask}
        />

        {/* Podsumowanie */}
        <div className="mt-3 text-muted text-center">
          Pozostało zadań: {tasks.filter((t) => !t.completed).length} / {tasks.length}
        </div>
      </main>
    );
  }

  export default App;

```

```

// Plik: src/components/TaskForm.js
import { useState } from "react";

function TaskForm({ onAddTask }) {
  const [text, setText] = useState(""); // Stan lokalny – tylko dla tego inputa

  function handleSubmit(e) {
    e.preventDefault();

    // Walidacja – nie dodajemy pustych zadań
    if (text.trim() === "") return;

    // Wywołujemy callback od rodzica
    onAddTask(text);

    // Czyścimy pole
    setText("");
  }

  return (
    <form onSubmit={handleSubmit} className="input-group mb-4">
      <input
        type="text"
        className="form-control"
        placeholder="Co masz do zrobienia?"
        value={text}

```



```

      onChange={(e) => setText(e.target.value)}
    />
    <button className="btn btn-success" type="submit">
      Dodaj
    </button>
  </form>
);
}

export default TaskForm;

```

```

// Plik: src/components/TaskList.js
function TaskList({ tasks, onDeleteTask, onToggleTask }) {
  // Obsługa pustej listy
  if (tasks.length === 0) {
    return <p className="text-center text-muted">Brak zadań. Odpocznij!</p>;
  }

  return (
    <ul className="list-group">
      {tasks.map((task) => (
        <li
          key={task.id}
          className="list-group-item d-flex justify-content-between align-items-center"
        >
          {/* Tekst zadania – przekreślony, jeśli ukończone */}
          <span
            style={{
              textDecoration: task.completed ? "line-through" : "none",
              cursor: "pointer",
            }}
            onClick={() => onToggleTask(task.id)}
          >
            {task.text}
          </span>

          {/* Przycisk usuwania */}
          <button
            className="btn btn-danger btn-sm"
            onClick={() => onDeleteTask(task.id)}
          >
            Usuń
          </button>
        </li>
      ))}
    </ul>
  );
}

export default TaskList;

```

## Podsumowanie przepływu danych:

1. Dół (Props): `App` → `TaskList` (przekazuje `tasks`, `onDeleteTask`, `onToggleTask`)
2. Góra (Callbacks): `TaskForm` → `App` (dziecko wywołuje `onAddTask`)
3. Odświeżenie: Gdy `App` zmieni stan, React odmalowuje `TaskForm` i `TaskList` z nowymi danymi

## 26.25. Widok kart z filtrami i wyszukiwaniem

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Widok kart z filtrami i wyszukiwaniem” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Widok kart z filtrami i wyszukiwaniem” dodaj jeden konkretny wariant nieidealny: brak danych, błędną wartość, reset albo komunikat po wykonaniu akcji. Taki wariant pokazuje, czy stan komponentu jest zaprojektowany kompletnie.

Rozbudowany widok kart z wyszukiwarką tekstową i filtrami kategorii.

```
// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  // Dane stałe – produkty
  const produkty = [
    { id: 1, nazwa: "Laptop Pro", kategoria: "Elektronika", cena: 4500 },
    { id: 2, nazwa: "Mysz bezprzewodowa", kategoria: "Elektronika", cena: 120 },
    { id: 3, nazwa: "Koszulka bawełniana", kategoria: "Odzież", cena: 49 },
    { id: 4, nazwa: "Spodnie jeansowe", kategoria: "Odzież", cena: 180 },
    { id: 5, nazwa: "Rower górski", kategoria: "Sport", cena: 2200 },
    { id: 6, nazwa: "Piłka nożna", kategoria: "Sport", cena: 80 },
    { id: 7, nazwa: "Monitor 27\"", kategoria: "Elektronika", cena: 1500 },
    { id: 8, nazwa: "Buty biegowe", kategoria: "Sport", cena: 350 },
  ];

  // Wyciągnięcie unikalnych kategorii
  const kategorie = [...new Set(produkty.map((p) => p.kategoria))];

  // Stan filtrów
  const [szukaj, setSzukaj] = useState("");
  const [wybranaKategoria, setWybranaKategoria] = useState("Wszystkie");
  const [sortowanie, setSortowanie] = useState("nazwa");

  // Filtrowanie
  let przefiltrowane = produkty;

  // Filtr tekstowy
  if (szukaj.trim() !== "") {
    przefiltrowane = przefiltrowane.filter((p) =>
      p.nazwa.toLowerCase().includes(szukaj.toLowerCase())
    );
  }

  // Filtr kategorii
  if (wybranaKategoria !== "Wszystkie") {

```

```

przefiltrowane = przefiltrowane.filter(
  (p) => p.kategoria === wybranaKategoria
);
}

// Sortowanie
przefiltrowane = [...przefiltrowane].sort((a, b) => {
  if (sortowanie === "nazwa") return a.nazwa.localeCompare(b.nazwa);
  if (sortowanie === "cena-rosnaco") return a.cena - b.cena;
  if (sortowanie === "cena-malejaco") return b.cena - a.cena;
  return 0;
});

return (
  <main className="container mt-4">
    <h1>Katalog produktów</h1>

    {/* Panel filtrów */}
    <div className="row mb-4">
      {/* Wyszukiwarka */}
      <div className="col-md-4 mb-2">
        <input
          type="text"
          className="form-control"
          placeholder="Szukaj produktu..."
          value={szukaj}
          onChange={(e) => setSzukaj(e.target.value)}
        />
      </div>

      {/* Filtr kategorii */}
      <div className="col-md-4 mb-2">
        <select
          className="form-select"
          value={wybranaKategoria}
          onChange={(e) => setWybranaKategoria(e.target.value)}
        >
          <option value="Wszystkie">Wszystkie kategorie</option>
          {kategorie.map((k) => (
            <option key={k} value={k}>{k}</option>
          ))}
        </select>
      </div>

      {/* Sortowanie */}
      <div className="col-md-4 mb-2">
        <select
          className="form-select"
          value={sortowanie}
          onChange={(e) => setSortowanie(e.target.value)}
        >
          <option value="nazwa">Sortuj: Nazwa A-Z</option>
          <option value="cena-rosnaco">Sortuj: Cena rosnaco</option>
          <option value="cena-malejaco">Sortuj: Cena malejaco</option>
        </select>
      </div>
    </div>
  </main>
);

```

```

        </select>
      </div>
    </div>

    {/* Licznik wyników */}
    <p className="text-muted">Znaleziono: {przefiltrowane.length} produktów</p>

    {/* Siatka kart */}
    <div className="row">
      {przefiltrowane.map((produkt) => (
        <div key={produkt.id} className="col-md-4 mb-3">
          <div className="card h-100">
            <div className="card-body">
              <h5 className="card-title">{produkt.nazwa}</h5>
              <p className="card-text">
                <span className="badge bg-secondary">{produkt.kategoria}</span>
              </p>
              <p className="card-text">
                <strong>{produkt.cena}<strong> zł</strong>
              </p>
            </div>
          </div>
        </div>
      ))}
    </div>

    {/* Obsługa braku wyników */}
    {przefiltrowane.length === 0 && (
      <p className="text-center text-muted">
        Brak produktów spełniających kryteria wyszukiwania.
      </p>
    )}
  </main>
);
}

export default App;

```

## 26.26. Algorytmy — sumowanie, zliczanie, filtrowanie

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Algorytmy — sumowanie, zliczanie, filtrowanie” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Algorytmy — sumowanie, zliczanie, filtrowanie” dodaj jeden konkretny wariant nieidealny: brak danych, błędną wartość, reset albo komunikat po wykonaniu akcji. Taki wariant pokazuje, czy stan komponentu jest zaprojektowany kompletnie.

Przykłady typowych operacji algorytmicznych osadzonych w React.

### 26.12.1. SUMA I ŚREDNIA Z TABLICY

```

// Plik: src/App.js
import { useState } from "react";

```

```

function App() {
  const [oceny, setOceny] = useState([5, 4, 3, 5, 4, 2, 5, 3]);
  const [nowaOcena, setNowaOcena] = useState("");

  // Obliczenia – robione przed return, nie w stanie
  const suma = oceny.reduce((acc, o) => acc + o, 0);
  const srednia = oceny.length > 0 ? suma / oceny.length : 0;
  const najwyzsza = oceny.length > 0 ? Math.max(...oceny) : 0;
  const najnizsza = oceny.length > 0 ? Math.min(...oceny) : 0;

  function handleDodaj() {
    const ocena = Number(nowaOcena);
    if (ocena ≥ 1 && ocena ≤ 6) {
      setOceny((prev) => [...prev, ocena]);
      setNowaOcena("");
    }
  }

  return (
    <div className="container mt-4" style={{ maxWidth: "400px" }}>
      <h2>Statystyki ocen</h2>
      <p>Oceny: {oceny.join(", ")}</p>
      <p>Suma: {suma}</p>
      <p>Średnia: {srednia.toFixed(2)}</p>
      <p>Najwyższa: {nawyzsza}</p>
      <p>Najniższa: {najnizsza}</p>
      <p>Liczba ocen: {oceny.length}</p>

      <div className="input-group mt-3">
        <input
          type="number"
          className="form-control"
          value={nowaOcena}
          onChange={(e) => setNowaOcena(e.target.value)}
          placeholder="1-6"
          min="1"
          max="6"
        />
        <button className="btn btn-primary" onClick={handleDodaj}>Dodaj ocenę</button>
      </div>
    </div>
  );
}

export default App;

```

## 26.12.2. ZLICZANIE WYSTĄPIEŃ

```

function App() {
  const dane = ["kot", "pies", "kot", "ryba", "pies", "kot", "papuga"];

  // Zliczanie za pomocą reduce
  const zliczenie = dane.reduce((acc, el) => {

```

```

    acc[e1] = (acc[e1] || 0) + 1;
    return acc;
  }, {});

return (
  <div className="container mt-4">
    <h2>Zliczanie wystąpień</h2>
    <table className="table" style={{ maxWidth: "300px" }}>
      <thead>
        <tr><th>Element</th><th>Ilość</th></tr>
      </thead>
      <tbody>
        {Object.entries(zliczenie).map(([klucz, wartosc]) => (
          <tr key={klucz}>
            <td>{klucz}</td>
            <td>{wartosc}</td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
);
}

```

### 26.12.3. FILTROWANIE PO WIELU KRYTERIACH

```

function App() {
  const [minCena, setMinCena] = useState(0);
  const [maxCena, setMaxCena] = useState(1000);
  const [szukaj, setSzukaj] = useState("");

  const produkty = [
    { id: 1, nazwa: "Laptop", cena: 3000 },
    { id: 2, nazwa: "Mysz", cena: 50 },
    { id: 3, nazwa: "Klawiatura", cena: 200 },
    { id: 4, nazwa: "Monitor", cena: 1500 },
    { id: 5, nazwa: "Słuchawki", cena: 350 },
  ];

  // Filtrowanie z wieloma warunkami jednocześnie
  const wyniki = produkty
    .filter((p) => p.cena ≥ minCena && p.cena ≤ maxCena)
    .filter((p) => p.nazwa.toLowerCase().includes(szukaj.toLowerCase()));

  return (
    <div className="container mt-4">
      <h2>Filtrowanie produktów</h2>

      <div className="row mb-3">
        <div className="col">
          <input type="text" className="form-control" placeholder="Szukaj..."
            value={szukaj} onChange={(e) => setSzukaj(e.target.value)} />
        </div>
      </div>
    </div>
  );
}

```

```

    <div className="col">
      <input type="number" className="form-control" placeholder="Min cena"
        value={minCena} onChange={(e) => setMinCena(Number(e.target.value))} />
    </div>
    <div className="col">
      <input type="number" className="form-control" placeholder="Max cena"
        value={maxCena} onChange={(e) => setMaxCena(Number(e.target.value))} />
    </div>
  </div>

  <p>Znalezione: {wyniki.length}</p>
  <ul className="list-group">
    {wyniki.map((p) => (
      <li key={p.id} className="list-group-item d-flex justify-content-between">
        <span>{p.nazwa}</span>
        <span>{p.cena} zł</span>
      </li>
    ))}
  </ul>
</div>
);
}

```

#### 26.12.4. SZYFR CEZARA

```

import { useState } from "react";

function App() {
  const [tekst, setTekst] = useState("");
  const [przesuniecie, setPrzesuniecie] = useState(3);
  const [wynik, setWynik] = useState("");

  function szyfrujCezar(tekst, przesuniecie) {
    return tekst
      .split("")
      .map((znak) => {
        // Małe litery
        if (znak ≥ "a" && znak ≤ "z") {
          const kod = ((znak.charCodeAt(0) - 97 + przesuniecie) % 26 + 26) % 26 + 97;
          return String.fromCharCode(kod);
        }
        // Duże litery
        if (znak ≥ "A" && znak ≤ "Z") {
          const kod = ((znak.charCodeAt(0) - 65 + przesuniecie) % 26 + 26) % 26 + 65;
          return String.fromCharCode(kod);
        }
        // Inne znaki bez zmian
        return znak;
      })
      .join("");
  }

  function handleSzyfruj() {

```

```

    setWynik(szyfrujCezar(tekst, przesuniecie));
  }

  function handleDeszyfruj() {
    setWynik(szyfrujCezar(tekst, -przesuniecie));
  }

  return (
    <div className="container mt-4" style={{ maxWidth: "500px" }}>
      <h2>Szyfr Cezara</h2>

      <div className="mb-3">
        <label className="form-label">Tekst:</label>
        <input type="text" className="form-control" value={tekst}
          onChange={(e) => setTekst(e.target.value)} placeholder="Wpisz tekst" />
      </div>

      <div className="mb-3">
        <label className="form-label">Przesunięcie: {przesuniecie}</label>
        <input type="range" className="form-range" min="1" max="25"
          value={przesuniecie} onChange={(e) => setPrzesuniecie(Number(e.target.value))} />
      </div>

      <div className="d-flex gap-2 mb-3">
        <button className="btn btn-primary" onClick={handleSzyfruj}>Szyfruj</button>
        <button className="btn btn-secondary" onClick={handleDeszyfruj}>Deszyfruj</button>
      </div>

      {wynik && (
        <div className="alert alert-info">
          <strong>Wynik:</strong> {wynik}
        </div>
      )}
    </div>
  );
}

export default App;

```

## 26.27. Galeria zdjęć z lightboxem i ulubionymi

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Galeria zdjęć z lightboxem i ulubionymi” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Galeria zdjęć z lightboxem i ulubionymi” dodaj wyszukiwarkę albo filtr kategorii i trzymaj wynik w zmiennej `widoczneElementy`. Dzięki temu logika wyboru danych jest oddzielona od JSX renderującego pojedynczą kartę lub wiersz.

Interaktywna galeria zdjęć z filtrowaniem po kategoriach, powiększaniem w lightboxie (modal), nawigacją strzałkami  $\leftarrow/\rightarrow$ , oznaczaniem ulubionych (serduszko), licznikiem wyświetleń i trybem siatki/listy.

**Smaczki:** lightbox z nawigacją klawiaturą ( $\leftarrow \rightarrow$  Esc), animowane serduszko ulubionych, przełącznik widoku siatka/lista, sortowanie po wyświetleniach/nazwie, licznik wyświetleń zwiększany przy otwarciu lightboxa, filtr "Tylko ulubione".



```

// Plik: src/App.js

import { useState, useEffect } from "react";
import "bootstrap/dist/css/bootstrap.css";

// Dane galerii – w prawdziwej apce z API lub JSON
// Obrazy z folderu public/galeria/
const ZDJECIA_POCZATKOWE = [
  { id: 1, plik: "krajobraz1.jpg", tytuł: "Górski poranek", kategoria: "Krajobrazy", wyswietlenia: 0, ulubione: false },
  { id: 2, plik: "miasto1.jpg", tytuł: "Nocne miasto", kategoria: "Miasto", wyswietlenia: 0, ulubione: false },
  { id: 3, plik: "krajobraz2.jpg", tytuł: "Jezioro o świcie", kategoria: "Krajobrazy", wyswietlenia: 0, ulubione: false },
  { id: 4, plik: "zwierze1.jpg", tytuł: "Lis w lesie", kategoria: "Zwierzęta", wyswietlenia: 0, ulubione: false },
  { id: 5, plik: "miasto2.jpg", tytuł: "Most o zachodzie", kategoria: "Miasto", wyswietlenia: 0, ulubione: false },
  { id: 6, plik: "zwierze2.jpg", tytuł: "Sowa na gałęzi", kategoria: "Zwierzęta", wyswietlenia: 0, ulubione: false },
  { id: 7, plik: "krajobraz3.jpg", tytuł: "Polna droga", kategoria: "Krajobrazy", wyswietlenia: 0, ulubione: false },
  { id: 8, plik: "miasto3.jpg", tytuł: "Stare kamienice", kategoria: "Miasto", wyswietlenia: 0, ulubione: false },
  { id: 9, plik: "zwierze3.jpg", tytuł: "Motyl na kwiatku", kategoria: "Zwierzęta", wyswietlenia: 0, ulubione: false },
];

function App() {
  const [zdjecia, setZdjecia] = useState(ZDJECIA_POCZATKOWE);
  const [filtrKategoria, setFiltrKategoria] = useState("Wszystkie");
  const [tylkoUlubione, setTylkoUlubione] = useState(false);
  const [sortowanie, setSortowanie] = useState("nazwa");
  const [widok, setWidok] = useState("siatka"); // "siatka" lub "lista"
  const [lightboxId, setLightboxId] = useState(null); // ID otwartego zdjęcia (null = zamknięty)

  // Wyciągnięcie unikalnych kategorii
  const kategorie = ["Wszystkie", ...new Set(zdjecia.map((z) => z.kategoria))];

  // Filtrowanie i sortowanie
  let przefiltrowane = zdjecia;

  if (filtrKategoria !== "Wszystkie") {
    przefiltrowane = przefiltrowane.filter((z) => z.kategoria === filtrKategoria);
  }

  if (tylkoUlubione) {
    przefiltrowane = przefiltrowane.filter((z) => z.ulubione);
  }

  // Sortowanie
  przefiltrowane = [...przefiltrowane].sort((a, b) => {
    if (sortowanie === "nazwa") return a.tytuł.localeCompare(b.tytuł);
    if (sortowanie === "wyswietlenia") return b.wyswietlenia - a.wyswietlenia;
    return 0;
  });

  // Lightbox – aktualne zdjęcie
  const lightboxZdjecie = zdjecia.find((z) => z.id === lightboxId);
  // Indeks w przefiltrowanej liście (do nawigacji <->)
  const lightboxIndex = przefiltrowane.findIndex((z) => z.id === lightboxId);

  // Otwarcie lightboxa – zwiększa wyświetlenia
  function otworzLightbox(id) {

```

```

    setLightboxId(id);
    setZdjecia((prev) =>
      prev.map((z) => (z.id === id ? { ...z, wyswietlenia: z.wyswietlenia + 1 } : z))
    );
  }
}

```

// Nawigacja w lightboxie

```

function nastepne() {
  if (lightboxIndex < przefiltrowane.length - 1) {
    const noweId = przefiltrowane[lightboxIndex + 1].id;
    otworzLightbox(noweId);
  }
}

```

```

function poprzednie() {
  if (lightboxIndex > 0) {
    const noweId = przefiltrowane[lightboxIndex - 1].id;
    otworzLightbox(noweId);
  }
}

```

```

function zamknijLightbox() {
  setLightboxId(null);
}

```

// Obsługa klawiatury w lightboxie

```

useEffect(() => {
  if (lightboxId === null) return;

  function handleKey(e) {
    if (e.key === "Escape") zamknijLightbox();
    if (e.key === "ArrowRight") nastepne();
    if (e.key === "ArrowLeft") poprzednie();
  }

  window.addEventListener("keydown", handleKey);
  return () => window.removeEventListener("keydown", handleKey);
});

```

// Toggle ulubione

```

function toggleUlubione(id, e) {
  e.stopPropagation(); // Nie otwieraj lightboxa przy kliknięciu serduszka
  setZdjecia((prev) =>
    prev.map((z) => (z.id === id ? { ...z, ulubione: !z.ulubione } : z))
  );
}

```

// Liczba ulubionych

```

const liczbaUlubionych = zdjecia.filter((z) => z.ulubione).length;

```

```

return (
  <main className="container mt-4">
    <h1 className="text-center mb-4">🖼️ Galeria Zdjęć</h1>

```

```

{/* Panel filtrów */}
<div className="row mb-4 g-2 align-items-center">
  {/* Kategoria */}
  <div className="col-md-3">
    <select
      className="form-select"
      value={filtrKategoria}
      onChange={(e) => setFiltrKategoria(e.target.value)}
    >
      {kategorie.map((k) => (
        <option key={k} value={k}>{k}</option>
      ))}
    </select>
  </div>

  {/* Sortowanie */}
  <div className="col-md-3">
    <select
      className="form-select"
      value={sortowanie}
      onChange={(e) => setSortowanie(e.target.value)}
    >
      <option value="nazwa">Sortuj: A-Z</option>
      <option value="wyswietlenia">Sortuj: Popularne</option>
    </select>
  </div>

  {/* Tylko ulubione */}
  <div className="col-md-3">
    <div className="form-check form-switch">
      <input
        id="ulubione-filtr"
        type="checkbox"
        className="form-check-input"
        checked={tylkoUlubione}
        onChange={(e) => setTylkoUlubione(e.target.checked)}
      />
      <label htmlFor="ulubione-filtr" className="form-check-label">
        ❤️ Ulubione ({liczbaUlubionych})
      </label>
    </div>
  </div>

  {/* Przełącznik widoku */}
  <div className="col-md-3 text-end">
    <div className="btn-group" role="group">
      <button
        className={`btn btn-sm ${widok === "siatka" ? "btn-primary" : "btn-outline-primary"}`}
        onClick={() => setWidok("siatka")}
      >
        ■ Siatka
      </button>
      <button
        className={`btn btn-sm ${widok === "lista" ? "btn-primary" : "btn-outline-primary"}`}

```

```
onClick={() => setWidok("lista")}
```

```
</button>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{/* Licznik wyników */}

<p className="text-muted small">

    Wyświetlanie: {przefiltrowane.length} z {zdjecia.length} zdjęć

</p>
```

```
{/* Widok SIATKA */}

{widok === "siatka" && (

    <div className="row">

        {przefiltrowane.map((z) => (

            <div key={z.id} className="col-6 col-md-4 mb-4">

                <div

                    className="card h-100 shadow-sm"

                    style={{ cursor: "pointer" }}

                    onClick={() => otworzLightbox(z.id)}

                >

                    {/* Obraz – w prawdziwej apce: src={` /galeria/${z.plik}`} */}

                    <div

                        className="card-img-top bg-secondary d-flex align-items-center justify-content-center text-white"

                        style={{ height: "180px", fontSize: "0.8rem" }}

                    >

                        🖼️ {z.plik}

                    </div>

                    <div className="card-body p-2">

                        <div className="d-flex justify-content-between align-items-start">

                            <div>

                                <h6 className="card-title mb-1">{z.tytul}</h6>

                                <small className="text-muted">

                                    <span className="badge bg-light text-dark me-1">{z.kategoria}</span>

                                    👁️ {z.wyswietlenia}

                                </small>

                            </div>

                        </div>

                        <div>

                            <small>Przycisk ulubione</small>

                            <button

                                className="btn btn-sm p-0 border-0"

                                onClick={(e) => toggleUlubione(z.id, e)}

                                style={{ fontSize: "1.3rem" }}

                            >

                                {z.ulubione ? "❤️ " : "💜 "}

                            </button>

                        </div>

                    </div>

                </div>

            )})

        </div>

    )}}
```

```
</div>
```

```
)}
```



```

onClick={(e) => e.stopPropagation()}
>
{/* Przycisk zamknięcia */}
<button
  className="btn btn-outline-light position-absolute top-0 end-0 m-3"
  onClick={zamknijLightbox}
>
  x
</button>

{/* Obraz powiększony */}
<div
  className="bg-dark border border-secondary rounded d-flex align-items-center justify-content-center mx-auto mb-3"
  style={{ width: "70vw", height: "60vh", fontSize: "1.5rem" }}
>
  🖼️ {lightboxZdjecie.plik}
  {/* W prawdziwej apce: <img src={`./galeria/${lightboxZdjecie.plik}`} ... /> */}
</div>

{/* Info */}
<h4>{lightboxZdjecie.tytul}</h4>
<p className="text-muted">
  {lightboxZdjecie.kategoria} • 👁️ {lightboxZdjecie.wyswietlenia} wyświetleń
  <button
    className="btn btn-sm border-0 ms-2"
    onClick={() => toggleUlubione(lightboxZdjecie.id, { stopPropagation: () => {} })}
    style={{ fontSize: "1.3rem" }}
  >
    {lightboxZdjecie.ulubione ? "❤️" : "💔"}
  </button>
</p>

{/* Nawigacja <=> */}
<div className="d-flex justify-content-between mt-3" style={{ width: "70vw", margin: "0 auto" }}>
  <button
    className="btn btn-outline-light btn-lg"
    onClick={poprzednie}
    disabled={lightboxIndex <= 0}
  >
    ← Poprzednie
  </button>
  <span className="align-self-center">
    {lightboxIndex + 1} / {przefiltrowane.length}
  </span>
  <button
    className="btn btn-outline-light btn-lg"
    onClick={nastepne}
    disabled={lightboxIndex >= przefiltrowane.length - 1}
  >
    Następne →
  </button>
</div>

<p className="text-muted small mt-3">

```

```

        Nawigacja: ← → strzałki | Esc = zamknij

        </p>
      </div>
    </div>
  )}
</main>
);
}

export default App;

```

#### Kluczowe mechaniki w tym wzorcu:

- **Lightbox (modal)** — pełnoekranowy overlay z `position: fixed`, zamykany kliknięciem tła lub Esc
- **Nawigacja klawiaturą** — `useEffect` z `addEventListener("keydown")` + `cleanup`
- `e.stopPropagation()` — kliknięcie serduszka nie otwiera lightboxa, kliknięcie zdjęcia w modalu nie zamyka go
- **Przełącznik widoku** — `widok === "siatka"` vs `widok === "lista"` renderuje zupełnie inny layout
- **Filtrowanie + sortowanie + ulubione** — trzy niezależne filtry działające razem
- **Licznik wyświetleń** — zwiększany przy otwarciu lightboxa (`otworzLightbox`)
- **Toggle ulubione** — `map` z odwróceniem booleana dla jednego elementu
- `new Set` — wyciąganie unikalnych kategorii z danych
- **Nawigacja w lightboxie** — `findIndex` w przefiltrowanej liście do `←/→`
- **Disabled buttons** — strzałki nieaktywne na krańcach listy
- **Responsywna siatka** — `col-6 col-md-4` (2 kolumny na telefonie, 3 na desktopie)
- **Placeholder zamiast obrazów** — div z emoji symulujący obraz (w prawdziwej apce zamień na `<img>`)

## 26.28. Książka adresowa z wyszukiwarką i tagami

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Książka adresowa z wyszukiwarką i tagami” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Książka adresowa z wyszukiwarką i tagami” dodaj wyszukiwarkę albo filtr kategorii i trzymaj wynik w zmiennej `widoczneElementy`. Dzięki temu logika wyboru danych jest oddzielona od JSX renderującego pojedynczą kartę lub wiersz.

Zarządzanie listą obiektów z jednoczesnym filtrowaniem tekstowym oraz grupowym.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [kontakty, setKontakty] = useState([
    { id: 1, imie: "Anna Nowak", telefon: "111-222-333", grupa: "Rodzina" },
    { id: 2, imie: "Piotr Kowalski", telefon: "444-555-666", grupa: "Praca" },
    { id: 3, imie: "Marek Wiśniewski", telefon: "777-888-999", grupa: "Znajomi" }
  ]);

  const [szukaj, setSzukaj] = useState("");
  const [filtrGrupy, setFiltrGrupy] = useState("Wszystkie");

  const [noweImie, setNoweImie] = useState("");
  const [nowyTelefon, setNowyTelefon] = useState("");
  const [nowaGrupa, setNowaGrupa] = useState("Znajomi");

```

```

function dodajKontakt(e) {
  e.preventDefault();
  if (!noweImie || !nowyTelefon) return;

  const nowy = {
    id: Date.now(),
    imie: noweImie,
    telefon: nowyTelefon,
    grupa: nowaGrupa
  };

  setKontakty([...kontakty, nowy]);
  setNoweImie("");
  setNowyTelefon("");
}

function usunKontakt(id) {
  setKontakty(kontakty.filter(k => k.id !== id));
}

// Logika podwójnego filtrowania
const wyswietlaneKontakty = kontakty.filter(k => {
  const pasujeTekst = k.imie.toLowerCase().includes(szukaj.toLowerCase()) || k.telefon.includes(szukaj);
  const pasujeGrupa = filtrGrupy === "Wszystkie" || k.grupa === filtrGrupy;
  return pasujeTekst && pasujeGrupa;
});

return (
  <main className="container mt-4" style={{ maxWidth: "600px" }}>
    <h1>Książka Adresowa</h1>

    <form onSubmit={dodajKontakt} className="bg-light p-3 border rounded mb-4">
      <h5>Dodaj nowy kontakt</h5>
      <div className="row g-2 mb-2">
        <div className="col-md-5">
          <input type="text" className="form-control" placeholder="Imię i nazwisko" value={noweImie} onChange={e =>
setNoweImie(e.target.value)} required />
        </div>
        <div className="col-md-4">
          <input type="text" className="form-control" placeholder="Telefon" value={nowyTelefon} onChange={e =>
setNowyTelefon(e.target.value)} required />
        </div>
        <div className="col-md-3">
          <select className="form-select" value={nowaGrupa} onChange={e => setNowaGrupa(e.target.value)}>
            <option value="Rodzina">Rodzina</option>
            <option value="Praca">Praca</option>
            <option value="Znajomi">Znajomi</option>
          </select>
        </div>
      </div>
      <button type="submit" className="btn btn-success btn-sm w-100">Dodaj kontakt</button>
    </form>
  </main>
)

```



```

    <div className="row mb-3">
      <div className="col-md-8">
        <input type="text" className="form-control" placeholder="Szukaj po nazwie lub telefonie..." value={szukaj} onChange={e =>
setSzukaj(e.target.value)} />
      </div>
      <div className="col-md-4">
        <select className="form-select" value={filtrGrupy} onChange={e => setFiltrGrupy(e.target.value)}>
          <option value="Wszystkie">Wszystkie grupy</option>
          <option value="Rodzina">Rodzina</option>
          <option value="Praca">Praca</option>
          <option value="Znajomi">Znajomi</option>
        </select>
      </div>
    </div>

    <ul className="list-group">
      {wyswietlaneKontakty.length === 0 && <li className="list-group-item text-muted">Brak wyników</li>}
      {wyswietlaneKontakty.map(k => (
        <li key={k.id} className="list-group-item d-flex justify-content-between align-items-center">
          <div>
            <strong>{k.imie}</strong> - {k.telefon} <br/>
            <span className="badge bg-secondary">{k.grupa}</span>
          </div>
          <button className="btn btn-danger btn-sm" onClick={() => usunKontakt(k.id)}>Usuń</button>
        </li>
      )
    )})
  </ul>
</main>
);
}

export default App;

```

## 26.29. Biblioteczka książek ze statusem przeczytania

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Biblioteczka książek ze statusem przeczytania” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Biblioteczka książek ze statusem przeczytania” dodaj wyszukiwarkę albo filtr kategorii i trzymaj wynik w zmiennej `widoczneElementy`. Dzięki temu logika wyboru danych jest oddzielona od JSX renderującego pojedynczą kartę lub wiersz.

Zarządzanie stanem logicznym (true/false) dla wielu elementów listy po kliknięciu.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [ksiazki, setKsiazki] = useState([
    { id: 1, tytuł: "Władca Pierścieni", autor: "J.R.R. Tolkien", przeczytana: true },
    { id: 2, tytuł: "Wiedźmin", autor: "Andrzej Sapkowski", przeczytana: false }
  ]);

```

```

const [tytuł, setTytuł] = useState("");
const [autor, setAutor] = useState("");
const [tylkoNieprzeczytane, setTylkoNieprzeczytane] = useState(false);

function dodaj(e) {
  e.preventDefault();
  if (!tytuł || !autor) return;

  setKsiążki([...książki, {
    id: Date.now(),
    tytuł,
    autor,
    przeczytana: false
  }]);
  setTytuł("");
  setAutor("");
}

function przełączStatus(id) {
  setKsiążki(książki.map(k =>
    k.id === id ? { ...k, przeczytana: !k.przeczytana } : k
  ));
}

const doWyswietlenia = tylkoNieprzeczytane ? książki.filter(k => !k.przeczytana) : książki;

const procentPrzeczytanych = książki.length > 0
  ? Math.round((książki.filter(k => k.przeczytana).length / książki.length) * 100)
  : 0;

return (
  <main className="container mt-4" style={{ maxWidth: "600px" }}>
    <h1>Moja Biblioteczka</h1>

    <div className="progress mb-4" style={{ height: "25px" }}>
      <div className="progress-bar bg-success" style={{ width: `${procentPrzeczytanych}%` }}>
        Przeczytane: {procentPrzeczytanych}%
      </div>
    </div>

    <form onSubmit={dodaj} className="mb-4 d-flex gap-2">
      <input type="text" className="form-control" placeholder="Tytuł" value={tytuł} onChange={e => setTytuł(e.target.value)}
required />
      <input type="text" className="form-control" placeholder="Autor" value={autor} onChange={e => setAutor(e.target.value)}
required />
      <button type="submit" className="btn btn-primary">Dodaj</button>
    </form>

    <div className="form-check mb-3">
      <input className="form-check-input" type="checkbox" id="filtr" checked={tylkoNieprzeczytane} onChange={e =>
setTylkoNieprzeczytane(e.target.checked)} />
      <label className="form-check-label text-danger" htmlFor="filtr">Pokaż tylko nieprzeczytane</label>
    </div>
  </main>

```

```

    <ul className="list-group">
      {doWyswietlenia.map(k => (
        <li key={k.id} className={`list-group-item d-flex justify-content-between align-items-center ${k.przeczytana ? 'bg-light text-muted' : ''}`}>
          <div>
            <strong style={{ textDecoration: k.przeczytana ? 'line-through' : 'none' }}>{k.tytul}</strong>
            <br/>
            <small>{k.autor}</small>
          </div>
          <button
            className={`btn btn-sm ${k.przeczytana ? 'btn-outline-secondary' : 'btn-success'}`}
            onClick={() => przelaczStatus(k.id)}
          >
            {k.przeczytana ? "Oznac jako nieprzeczytane" : "✓ Przeczytane"}
          </button>
        </li>
      )})}
    </ul>
  </main>
);
}

export default App;

```

## 26.30. Wyszukiwarka przepisów kulinarnych po składnikach

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Wyszukiwarka przepisów kulinarnych po składnikach” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Wyszukiwarka przepisów kulinarnych po składnikach” dodaj wyszukiwarkę albo filtr kategorii i trzymaj wynik w zmiennej `widoczneElementy`. Dzięki temu logika wyboru danych jest oddzielona od JSX renderującego pojedynczą kartę lub wiersz.

Złożone filtrowanie polegające na porównywaniu zaznaczonych elementów z tablicami składników receptur.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

const bazaPrzepisow = [
  { id: 1, nazwa: "Makaron z serem", skladniki: ["Makaron", "Ser"] },
  { id: 2, nazwa: "Kanapka z szynką i serem", skladniki: ["Chleb", "Ser", "Szynka"] },
  { id: 3, nazwa: "Jajecznica z szynką", skladniki: ["Jajka", "Szynka"] },
  { id: 4, nazwa: "Tosty z serem", skladniki: ["Chleb", "Ser"] }
];

const dostepneSkladniki = ["Makaron", "Ser", "Chleb", "Szynka", "Jajka"];

function App() {
  const [mojeSkladniki, setMojeSkladniki] = useState([]);

  function toggleSkladnik(skladnik) {
    if (mojeSkladniki.includes(skladnik)) {
      setMojeSkladniki(mojeSkladniki.filter(s => s !== skladnik));
    }
  }
}

```

```

    } else {
      setMojeSkładniki([...mojeSkładniki, składnik]);
    }
  }
}

// Filtr: przepis jest widoczny tylko wtedy, gdy MAMY WSZYSTKIE wymagane w nim składniki
const mozliweDoZrobienia = bazaPrzepisow.filter(przepis =>
  przepis.składniki.every(wymagany => mojeSkładniki.includes(wymagany))
);

return (
  <main className="container mt-4" style={{ maxWidth: "600px" }}>
    <h1>Co zjem dzisiaj?</h1>
    <p className="text-muted">Zaznacz składniki, które masz w lodówce:</p>

    <div className="mb-4 border p-3 rounded bg-light">
      {dostępneSkładniki.map(skl => (
        <div key={skl} className="form-check form-check-inline">
          <input
            type="checkbox"
            className="form-check-input"
            id={skl}
            checked={mojeSkładniki.includes(skl)}
            onChange={() => toggleSkładnik(skl)}
          />
          <label className="form-check-label" htmlFor={skl}>{skl}</label>
        </div>
      ))}
    </div>

    <h4>Możesz ugotować ({mozliweDoZrobienia.length}):</h4>
    {mozliweDoZrobienia.length === 0 ? (
      <div className="alert alert-warning">Musisz zaznaczyć więcej składników!</div>
    ) : (
      <ul className="list-group">
        {mozliweDoZrobienia.map(p => (
          <li key={p.id} className="list-group-item">
            <strong>{p.nazwa}</strong>
            <div className="text-muted small">Wymaga: {p.składniki.join(", ")}</div>
          </li>
        ))}
      </ul>
    )}
  </main>
);
}

export default App;

```

### 26.31. Dzienniczek ocen z obliczaniem średniej ważonej

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Dzienniczek ocen z obliczaniem średniej ważonej” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Dzienniczek ocen z obliczaniem średniej ważonej” dodaj jeden konkretny wariant nieidealny: brak danych, błędną wartość, reset albo komunikat po wykonaniu akcji. Taki wariant pokazuje, czy stan komponentu jest zaprojektowany kompletnie.

Aplikacja zbierająca liczbowe wartości wraz z ich "wagą" oraz dynamicznie przeliczająca skomplikowaną średnią.

```
// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [oceny, setOceny] = useState([]);
  const [przedmiot, setPrzedmiot] = useState("");
  const [wartosc, setWartosc] = useState("5");
  const [waga, setWaga] = useState("1");

  function dodajOcene(e) {
    e.preventDefault();
    if (!przedmiot.trim()) return;

    setOceny([...oceny, {
      id: Date.now(),
      przedmiot,
      wartosc: Number(wartosc),
      waga: Number(waga)
    }]);
    setPrzedmiot("");
  }

  function usunOcene(id) {
    setOceny(oceny.filter(o => o.id !== id));
  }

  // Obliczanie średniej ważonej: Suma(Ocena * Waga) / Suma(Wag)
  let sumaIloczynow = 0;
  let sumaWag = 0;
  oceny.forEach(o => {
    sumaIloczynow += (o.wartosc * o.waga);
    sumaWag += o.waga;
  });

  const srednia = sumaWag > 0 ? (sumaIloczynow / sumaWag).toFixed(2) : "0.00";
  const ostrzezenie = Number(srednia) < 2.0 && sumaWag > 0;

  return (
    <main className="container mt-4" style={{ maxWidth: "600px" }}>
      <h1>Dzienniczek Ocen</h1>

      <div className={`alert ${ostrzezenie ? 'alert-danger' : 'alert-primary'} text-center`}>
        <h2>Średnia ważona: {srednia}</h2>
      </div>
    </main>
  );
}
```

```

<form onSubmit={dodajOcene} className="row g-2 mb-4">
  <div className="col-md-5">
    <input type="text" className="form-control" placeholder="Przedmiot (np. Matematyka)" value={przedmiot} onChange={e =>
setPrzedmiot(e.target.value)} required />
  </div>
  <div className="col-md-3">
    <label className="small text-muted d-block">Ocena:</label>
    <select className="form-select" value={wartosc} onChange={e => setWartosc(e.target.value)}>
      <option value="6">6 (Celujący)</option>
      <option value="5">5 (Bardzo dobry)</option>
      <option value="4">4 (Dobry)</option>
      <option value="3">3 (Dostateczny)</option>
      <option value="2">2 (Dopuszczający)</option>
      <option value="1">1 (Niedostateczny)</option>
    </select>
  </div>
  <div className="col-md-2">
    <label className="small text-muted d-block">Waga:</label>
    <select className="form-select" value={waga} onChange={e => setWaga(e.target.value)}>
      <option value="1">1</option>
      <option value="2">2</option>
      <option value="3">3</option>
    </select>
  </div>
  <div className="col-md-2 d-flex align-items-end">
    <button type="submit" className="btn btn-success w-100">Dodaj</button>
  </div>
</form>

<ul className="list-group">
  {oceny.length === 0 && <li className="list-group-item text-center">Brak ocen</li>}
  {oceny.map(o => (
    <li key={o.id} className="list-group-item d-flex justify-content-between align-items-center">
      <div>
        <strong>{o.przedmiot}</strong> <br/>
        <span className="text-muted small">Ocena: <span className="fs-5 fw-bold text-dark">{o.wartosc}</span> (waga {o.waga})
      </span>
      </div>
      <button className="btn btn-outline-danger btn-sm" onClick={() => usunOcene(o.id)}>Usuń</button>
    </li>
  ))}
</ul>
</main>
);
}

export default App;

```

## 26.32. Lista zakupów z podziałem na działy

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Lista zakupów z podziałem na działy” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Lista zakupów z podziałem na działy” dodaj wyszukiwarkę albo filtr kategorii i trzymaj wynik w zmiennej

`widoczneElementy`. Dzięki temu logika wyboru danych jest oddzielona od JSX renderującego pojedynczą kartę lub wiersz.

Grupowanie tablicy na podstawie jednej ze zmiennych i warunkowe renderowanie wielu sekcji z koszykiem.

```
// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

const KATEGORIE = ["Owoce/Warzywa", "Nabiał", "Pieczywo", "Chemia"];

function App() {
  const [zakupy, setZakupy] = useState([
    { id: 1, nazwa: "Jabłko", kategoria: "Owoce/Warzywa", kupione: false },
    { id: 2, nazwa: "Chleb", kategoria: "Pieczywo", kupione: true }
  ]);
  const [nazwa, setNazwa] = useState("");
  const [wybranaKategoria, setWybranaKategoria] = useState(KATEGORIE[0]);

  function dodaj(e) {
    e.preventDefault();
    if (!nazwa) return;
    setZakupy([...zakupy, { id: Date.now(), nazwa, kategoria: wybranaKategoria, kupione: false }]);
    setNazwa("");
  }

  function przełączStan(id) {
    setZakupy(zakupy.map(z => z.id === id ? { ...z, kupione: !z.kupione } : z));
  }

  function usunKupione() {
    setZakupy(zakupy.filter(z => !z.kupione));
  }

  return (
    <main className="container mt-4" style={{ maxWidth: "500px" }}>
      <h1>Lista Zakupów</h1>

      <form onSubmit={dodaj} className="d-flex gap-2 mb-4">
        <input type="text" className="form-control" placeholder="Co kupić?" value={nazwa} onChange={e => setNazwa(e.target.value)}
        required />
        <select className="form-select w-auto" value={wybranaKategoria} onChange={e => setWybranaKategoria(e.target.value)}>
          {KATEGORIE.map(k => <option key={k} value={k}>{k}</option>)}
        </select>
        <button type="submit" className="btn btn-primary">Dodaj</button>
      </form>

      {KATEGORIE.map(kat => {
        const produktyWKategorii = zakupy.filter(z => z.kategoria === kat);
        if (produktyWKategorii.length === 0) return null; // Nie renderuj pustych działów
```

```

    return (
      <div key={kat} className="mb-3 border rounded p-2 bg-light">
        <h5 className="text-secondary border-bottom pb-1 mb-2">{kat}</h5>
        <ul className="list-group list-group-flush">
          {produktyWKategorii.map(z => (
            <li
              key={z.id}
              className={`list-group-item list-group-item-action ${z.kupione ? "text-decoration-line-through text-muted" :
""}`>

              <input type="checkbox" className="form-check-input me-2" checked={z.kupione} readOnly />
              {z.nazwa}
            </li>
          ))}
        </ul>
      </div>
    )
  )}

  <button className="btn btn-outline-danger w-100 mt-2" onClick={usunKupione}>
    Usuń oznaczone jako kupione
  </button>
</main>
);
}

export default App;

```

### 26.33. Mixer kolorów RGB

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Mixer kolorów RGB” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Mixer kolorów RGB” dodaj obsługę pustego pola, wartości ujemnych i zaokrąglenia wyniku. Funkcję obliczającą trzymaj poza JSX, np. `obliczWynik(dane)`, a w komponencie zostaw tylko pobieranie danych i prezentację rezultatu.

Trzy suwaki sterujące kolorem tła w czasie rzeczywistym.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [r, setR] = useState(100);
  const [g, setG] = useState(150);
  const [b, setB] = useState(200);

  // Kolor w formacie CSS
  const kolor = `rgb(${r}, ${g}, ${b})`;

```



```
// Kolor w formacie HEX
const hex =

  "#" +

  r.toString(16).padStart(2, "0") +

  g.toString(16).padStart(2, "0") +

  b.toString(16).padStart(2, "0");

return (

  <main className="container mt-4" style={{ maxWidth: "500px" }}>

    <h1>Mixer kolorów RGB</h1>

    {/* Podgląd koloru */}

    <div

      style={{

        width: "100%",

        height: "150px",

        backgroundColor: kolor,

        borderRadius: "10px",

        marginBottom: "1rem",

        border: "1px solid #ccc",

      }}

    />

    {/* Wartości koloru */}

    <p className="text-center">

      <strong>RGB:</strong> {kolor} | <strong>HEX:</strong> {hex.toUpperCase()}

    </p>

    {/* Suwak R (czerwony) */}

    <div className="mb-3">

      <label className="form-label" style={{ color: "red" }}>

        R (czerwony): {r}

      </label>

      <input

        type="range"

        className="form-range"

        min="0"

        max="255"

        value={r}

        onChange={(e) => setR(Number(e.target.value))}

      />

    </div>

    {/* Suwak G (zielony) */}

    <div className="mb-3">

      <label className="form-label" style={{ color: "green" }}>

        G (zielony): {g}

      </label>

      <input

        type="range"

        className="form-range"

        min="0"

        max="255"

        value={g}

      />

    </div>

  </main>

);
```

```

      onChange={(e) => setG(Number(e.target.value))}
    />
  </div>

  /* Suwak B (niebieski) */
  <div className="mb-3">
    <label className="form-label" style={{ color: "blue" }}>
      B (niebieski): {b}
    </label>
    <input
      type="range"
      className="form-range"
      min="0"
      max="255"
      value={b}
      onChange={(e) => setB(Number(e.target.value))}
    />
  </div>

  /* Reset */
  <button
    className="btn btn-secondary w-100"
    onClick={() => { setR(0); setG(0); setB(0); }}
  >
    Reset (czarny)
  </button>
</main>
);
}

export default App;

```

### 26.34. Licznik z historią operacji

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Licznik z historią operacji” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Licznik z historią operacji” dodaj obsługę pustego pola, wartości ujemnych i zaokrąglenia wyniku. Funkcję obliczającą trzymaj poza JSX, np. `obliczWynik(dane)`, a w komponencie zostaw tylko pobieranie danych i prezentację rezultatu.

Licznik, który zapisuje historię wszystkich wykonanych operacji.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [licznik, setLicznik] = useState(0);
  const [historia, setHistoria] = useState([]);

  function wykonajOperacje(operacja, wartosc) {
    let nowaWartosc;

```

```

switch (operacja) {
  case "dodaj":
    nowaWartosc = licznik + wartosc;
    break;
  case "odejmij":
    nowaWartosc = licznik - wartosc;
    break;
  case "pomnoz":
    nowaWartosc = licznik * wartosc;
    break;
  case "reset":
    nowaWartosc = 0;
    break;
  default:
    return;
}

// Zapis do historii
const wpis = {
  id: Date.now(),
  operacja: operacja,
  wartosc: wartosc,
  wynik: nowaWartosc,
  czas: new Date().toLocaleTimeString(),
};

setHistoria((prev) => [wpis, ...prev]); // Najnowsze na górze
setLicznik(nowaWartosc);
}

return (
  <main className="container mt-4" style={{ maxWidth: "500px" }}>
    <h1 className="text-center">Licznik: {licznik}</h1>

    {/* Przyciski operacji */}
    <div className="d-flex gap-2 justify-content-center mb-4">
      <button className="btn btn-success" onClick={() => wykonajOperacje("dodaj", 1)}>+1</button>
      <button className="btn btn-success" onClick={() => wykonajOperacje("dodaj", 5)}>+5</button>
      <button className="btn btn-danger" onClick={() => wykonajOperacje("odejmij", 1)}>-1</button>
      <button className="btn btn-danger" onClick={() => wykonajOperacje("odejmij", 5)}>-5</button>
      <button className="btn btn-info" onClick={() => wykonajOperacje("pomnoz", 2)}>x2</button>
      <button className="btn btn-secondary" onClick={() => wykonajOperacje("reset", 0)}>Reset</button>
    </div>

    {/* Historia operacji */}
    <h5>Historia operacji ({historia.length})</h5>
    {historia.length === 0 ? (
      <p className="text-muted">Brak operacji</p>
    ) : (
      <table className="table table-sm table-striped">
        <thead>
          <tr>
            <th>Czas</th>

```

```

        <th>Operacja</th>
        <th>Wynik</th>
      </tr>
    </thead>
    <tbody>
      {historia.map((w) => (
        <tr key={w.id}>
          <td>{w.czas}</td>
          <td>{w.operacja} {w.wartosc !== 0 ? w.wartosc : ""}</td>
          <td>{w.wynik}</td>
        </tr>
      ))}
    </tbody>
  </table>
)}

{/* Czyszczenie historii */}
{historia.length > 0 && (
  <button
    className="btn btn-outline-danger btn-sm"
    onClick={() => setHistoria([])}
  >
    Wyczyść historię
  </button>
)}
</main>
);
}

export default App;

```

### 26.35. Prosta Playlista Audio (Odtwarzacz ze stanem)

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Prosta Playlista Audio (Odtwarzacz ze stanem)” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Prosta Playlista Audio (Odtwarzacz ze stanem)” dodaj wyszukiwarkę albo filtr kategorii i trzymaj wynik w zmiennej `widoczneElementy`. Dzięki temu logika wyboru danych jest oddzielona od JSX renderującego pojedynczą kartę lub wiersz.

Ten wzorec jest genialnym rozwiązaniem skomplikowanego zadania polegającego na zbudowaniu „Player’a” i manipulowaniu ścieżkami podawanych plików typu `mp3`. Z reguły polega to na wrzuceniu utworów do tablicy (bazy), a po kliknięciu klawisza `<li>` zmienieniu całego odtwarzanego źródła na nową muzykę w hooku (stanie).

```

// Główny lub opcjonalny plik: src/App.js
import React, { useState } from "react";
import "bootstrap/dist/css/bootstrap.css"; // Polegamy mocno na gridach / kartkach bootstrapa stąd.

// Ważne: Baza musi wskazywać na folder publiczny dla ładowarki! Pliki muszą tam siedzieć (public/dzwieki/..._
const LISTA_UTWOROW = [
  { id: 1, tytuł: "Zimowy wiatr", wokalista: "Marek Pytlaś", src: "/dzwieki/wiatr.mp3" },
  { id: 2, tytuł: "Nocne gwieździste niebo", wokalista: "Dj GROM", src: "/dzwieki/niebo.mp3" },
  { id: 3, tytuł: "Energetyczny Pop 2026", wokalista: "Sygmond", src: "/dzwieki/pop.mp3" }
]

```

```

];

function OdtwarzaczZPlayLista() {
  // Trzymamy w całości obiekt w którym jest i autor i jego mp3!
  const [obecnyUtwor, setObecnyUtwor] = useState(LISTA_UTWOROW[0]);

  return (
    <div className="container mt-5">
      <div className="card shadow border-dark" style={{ maxWidth: "550px", margin: "auto" }}>

        {/* ===== Widok playera (górny ekran odtwarzania) ===== */}
        <div className="card-header bg-dark text-white text-center rounded-top">
          <h5 className="mb-0">🎵 Twoja Cyfrowa Playlista</h5>
        </div>
        <div className="card-body text-center bg-light">
          <h6 className="text-secondary text-uppercase ls-1">Aktualnie gra</h6>
          <h2 className="text-primary fw-bold">{obecnyUtwor.tytul}</h2>
          <p className="text-muted fs-5">Artysta: {obecnyUtwor.wokalista}</p>

          {/* Tag Audio ma atrybut kluczowy → key={}.
              Zmusza to silnik React'a do "zniszczenia i zrestartowania Playera" kiedy podepnimy mu nowy key'u
              w zapiętym źródle (mp3). Bez tego podmienimy src ale utwór nie zadziała.
          */}
          <audio controls autoPlay key={obecnyUtwor.src} className="w-100 mt-4 px-2">
            <source src={obecnyUtwor.src} type="audio/mpeg" />
          </audio>
        </div>

        {/* ===== Sekcja listy utworów do klikania ===== */}
        <ul className="list-group list-group-flush rounded-bottom">
          {LISTA_UTWOROW.map((utwor) => {
            // Wzmienie dynamiczne klasy jeżeli piosenka w pętli zgadza się ze stanem obecnym w pamięci
            const aktywnyClass = utwor.id === obecnyUtwor.id ? "active bg-primary border-primary fw-bold" : "";

            return (
              <li
                key={utwor.id}
                className={`list-group-item list-group-item-action ${aktywnyClass}`}
                onClick={() => setObecnyUtwor(utwor)}
                style={{ cursor: "pointer", transition: "0.2s" }}
              >
                <div className="d-flex justify-content-between align-items-center">
                  <span>{utwor.id}. {utwor.tytul}</span>
                  <span className="badge rounded-pill bg-dark">
                    {utwor.id === obecnyUtwor.id ? "Odtwarza się" : "Graj"}
                  </span>
                </div>
              </li>
            );
          })}
        </ul>

      </div>
    </div>
  );
}

```

```

    );
  }

  export default OdtwarzaczZPlayLista;

```

### 26.36. Akordeon FAQ z widocznością (Sekcje Rozwijane)

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Akordeon FAQ z widocznością (Sekcje Rozwijane)” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Akordeon FAQ z widocznością (Sekcje Rozwijane)” dodaj jeden konkretny wariant nieidealny: brak danych, błędną wartość, reset albo komunikat po wykonaniu akcji. Taki wariant pokazuje, czy stan komponentu jest zaprojektowany kompletnie.

Typowe zadanie architektoniczne - "Mam listę elementów, ale po kliknięciu konkretnego w dół rościąga mi się tekst/odpowiedź, gasząc resztę na biało!". Jest to idealne pole dla **Renderowania warunkowego**.

```

import React, { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

const BAZA_FAQ = [
  { id: 1, pyta: "Jak długo czekam na dostawę?", opowiada: "Zasze w 24 godziny od kupienia zlecenia po zaksięgowaniu P24 na naszym koncie firmowym." },
  { id: 2, pyta: "Czy dostawa jest darmowa dla paczek?", opowiada: "Dla zamówień przekraczających trefny pułap 199.99 PLN wysyłamy na nasz pełny, gigantyczny paczkowy koszt całkowicie za free." },
  { id: 3, pyta: "Co z procedurą zgłaszanych zwrotów 12 dniowych", opowiada: "Towar ze względów higienicznych i certyfikatów z rąk Reacta nie może zostać o dziwo wysłany w drogę powrotną a zaledwie w proces reklamacji wewnątrz chatu." }
];

function ModulPytanUzytkownikaFAQ() {
  // Stan "otwarteId" trzyma ID rozwiniętego segmentu. Wartość startowa to puste → 'null', czyli wszystkie ukryte.
  const [otwarteId, setOtwarteId] = useState(null);

  // Funkcja odbierająca kliknięcie:
  const nacisnietyPrzycisk = (idMiejsca) => {
    // Jeżeli kliknięto id tego samego, co jest już otwarte... ZAMKNIJ wszystkich (przypisać tu null).
    if (otwarteId === idMiejsca) {
      setOtwarteId(null);
    } else {
      setOtwarteId(idMiejsca); // Jak wciśnie inny guzik - po prostu OTWÓRZ go! A przy okazji reszta zgasi flagi.
    }
  };

  return (
    <div className="container mt-5">
      <h2 className="text-center mb-5 fw-bold text-dark">Baza Częstych Przyszłych Pytań</h2>

      <div className="list-group shadow-lg overflow-hidden border-0 rounded" style={{ maxWidth: "600px", margin: "auto" }}>
        {BAZA_FAQ.map((rekord) => {

          // Boolean (flaga bool) decydująca wewnątrz metody - jeżeli w koscie się zgadza to True
          const panelWidoczny = otwarteId === rekord.id;

```

```

return (
  <div key={rekord.id}>

    {/* Sekcja Clickable (Nagłówek Modułu z Zapytaniem) */}
    <button
      onClick={() => nacisnietyPrzycisk(rekord.id)}
      className={`list-group-item list-group-item-action d-flex justify-content-between align-items-center py-3 border-0
border-bottom
      ${panelWidoczny ? "bg-primary text-white" : ""}`}
      style={{ fontSize: "1.1rem" }}
    >
      <div className="fw-bold">
        <span className="me-3 fs-4 text-warning"?></span>
        Question. {rekord.pyta}
      </div>
      {/* Genialny feedback do strzałeczek kierunkowych! */}
      <span className="fs-5">{panelWidoczny ? "⏏ Zwiń" : "🔍 Rozwiń"}</span>
    </button>

    {/* Renderowanie warunkowe (wyświetl / zmiel). Gdyby zmienna pod bool'a u góry wynosiła fałsz, po prostu przeskoczy
tę sekcję. */}
    {panelWidoczny && (
      <div className="p-4 bg-light text-muted border-start border-5 border-warning opacity-75">
        <strong className="text-dark">Odpowiedź Eksperta:</strong> <br/>
        <p className="mt-2 mb-0 lh-lg">{rekord.opowiada}</p>
      </div>
    )}

  </div>
)
  )}
</div>
</div>
);
}

export default ModulPytanUzytkownikaFAQ;

```

## 26.37. CSS Gradient Generator

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „CSS Gradient Generator” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „CSS Gradient Generator” dodaj historię rund, licznik punktów i reset całej rozgrywki. Dane konfiguracyjne, takie jak lista pytań, możliwe ruchy albo ściany kostki, trzymaj poza komponentem.

Narzędzie do wizualnego generowania tła z kodem do skopiowania, operujące na stylach inline.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

```

```

function App() {
  const [kolor1, setKolor1] = useState("#ff0000");
  const [kolor2, setKolor2] = useState("#0000ff");
  const [kat, setKat] = useState(90);

  const gradientCSS = `linear-gradient(${kat}deg, ${kolor1}, ${kolor2})`;

  function kopiujKod() {
    navigator.clipboard.writeText(`background: ${gradientCSS};`);
    alert("Skopiowano do schowka!");
  }

  return (
    <main className="container mt-4" style={{ maxWidth: "600px" }}>
      <h1>Generator Gradientów CSS</h1>

      {/* Obszar podglądu */}
      <div
        className="w-100 rounded border border-2 border-dark shadow-sm my-4 d-flex align-items-center justify-content-center"
        style={{ height: "200px", background: gradientCSS }}
      >
        <span className="bg-light px-3 py-1 rounded shadow-sm fw-bold">Podgląd na żywo</span>
      </div>

      <div className="row mb-3">
        <div className="col">
          <label className="form-label">Kolor 1:</label>
          <input type="color" className="form-control form-control-color w-100" value={kolor1} onChange={e =>
setKolor1(e.target.value)} />
        </div>
        <div className="col">
          <label className="form-label">Kolor 2:</label>
          <input type="color" className="form-control form-control-color w-100" value={kolor2} onChange={e =>
setKolor2(e.target.value)} />
        </div>
      </div>

      <div className="mb-4">
        <label className="form-label">Kąt nachylenia: {kat}°</label>
        <input type="range" className="form-range" min="0" max="360" value={kat} onChange={e => setKat(Number(e.target.value))} />
      </div>

      <div className="input-group">
        <span className="input-group-text">CSS:</span>
        <input type="text" className="form-control" readOnly value={`background: ${gradientCSS};`} />
        <button className="btn btn-outline-secondary" onClick={kopiujKod}>Kopiuj</button>
      </div>
    </main>
  );
}

export default App;

```



## 26.38. Licznik słów, znaków i czasu czytania

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Licznik słów, znaków i czasu czytania” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Licznik słów, znaków i czasu czytania” dodaj obsługę pustego pola, wartości ujemnych i zaokrąglenia wyniku. Funkcję obliczającą trzymaj poza JSX, np. `obliczWynik(dane)`, a w komponencie zostaw tylko pobieranie danych i prezentację rezultatu.

Zaawansowane analizowanie zawartości pola tekstowego za pomocą metod na ciągach znaków (`split`, `trim`).

```
// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [tekst, setTekst] = useState("");

  const znakiZespacjami = tekst.length;
  const znakiBezSpacji = tekst.replace(/\s+/g, "").length;

  // Zliczanie słów (podział po spacji i enterach)
  const slowa = tekst.trim() ? tekst.trim().split(/\s+/).length : 0;

  // Czas czytania przy założeniu 200 słów na minutę
  const czasCzytania = (slowa / 200).toFixed(1);

  return (
    <main className="container mt-4" style={{ maxWidth: "600px" }}>
      <h1>Analizator Tekstu</h1>

      <textarea
        className="form-control mb-4"
        rows="8"
        placeholder="Wklej tutaj swój tekst do analizy..."
        value={tekst}
        onChange={(e) => setTekst(e.target.value)}
      />

      <div className="row text-center">
        <div className="col-6 col-md-3 mb-3">
          <div className="card p-3 shadow-sm bg-light">
            <h5 className="text-muted mb-1">Znaków</h5>
            <h3 className="mb-0">{znakiZespacjami}</h3>
          </div>
        </div>
        <div className="col-6 col-md-3 mb-3">
          <div className="card p-3 shadow-sm bg-light">
            <h5 className="text-muted mb-1">Znaków (bez spacji)</h5>
            <h3 className="mb-0">{znakiBezSpacji}</h3>
          </div>
        </div>
        <div className="col-6 col-md-3 mb-3">
          <div className="card p-3 shadow-sm bg-light">
            <h5 className="text-muted mb-1">Słów</h5>
```

```

        <h3 className="text-primary mb-0">{słowa}</h3>
      </div>
    </div>
    <div className="col-6 col-md-3 mb-3">
      <div className="card p-3 shadow-sm bg-light">
        <h5 className="text-muted mb-1">Czas czytania (min)</h5>
        <h3 className="text-success mb-0">{czasCzytania}</h3>
      </div>
    </div>
  </div>
</main>

  <button className="btn btn-danger mt-2" onClick={() => setTekst("")}>Wyczyść tekst</button>
</main>

);
}

export default App;

```

### 26.39. Minutnik Kuchenny (Odliczanie)

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Minutnik Kuchenny (Odliczanie)” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Minutnik Kuchenny (Odliczanie)” dodaj jeden konkretny wariant nieidealny: brak danych, błędną wartość, reset albo komunikat po wykonaniu akcji. Taki wariant pokazuje, czy stan komponentu jest zaprojektowany kompletnie.

Zastosowanie hooka `useEffect` i funkcji `setInterval` do manipulacji stanem odliczania czasu w dół.

```

// Plik: src/App.js
import { useState, useEffect } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [minuty, setMinuty] = useState("");
  const [sekundyLacznie, setSekundyLacznie] = useState(0);
  const [aktywny, setAktywny] = useState(false);

  useEffect(() => {
    let interval = null;
    if (aktywny && sekundyLacznie > 0) {
      interval = setInterval(() => {
        setSekundyLacznie((s) => s - 1);
      }, 1000);
    } else if (sekundyLacznie === 0 && aktywny) {
      setAktywny(false);
      alert("Czas minął!"); // Alert na koniec czasu
    }
    return () => clearInterval(interval);
  }, [aktywny, sekundyLacznie]);

  function start(e) {
    e.preventDefault();
    const min = Number(minuty);
  }
}

```



```
    );  
  }  
  
  export default App;
```

## 26.40. Kreator i podgląd menu restauracji (Karta dań)

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Kreator i podgląd menu restauracji (Karta dań)” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Kreator i podgląd menu restauracji (Karta dań)” dodaj wyszukiwarkę albo filtr kategorii i trzymaj wynik w zmiennej `widoczneElementy`. Dzięki temu logika wyboru danych jest oddzielona od JSX renderującego pojedynczą kartę lub wiersz.

Oddzielenie formularza tworzenia elementów od samej wizualizacji z podziałem na kategorie (np. wegetariańskie).

```
// Plik: src/App.js  
import { useState } from "react";  
import "bootstrap/dist/css/bootstrap.css";  
  
function App() {  
  const [menu, setMenu] = useState([  
    { id: 1, nazwa: "Burger Klasyczny", cena: 35, opis: "Wołowina 200g, ser, sałata", wege: false },  
    { id: 2, nazwa: "Sałatka Grecka", cena: 28, opis: "Feta, oliwki, pomidory, ogórek", wege: true }  
  ]);  
  
  const [nazwa, setNazwa] = useState("");  
  const [cena, setCena] = useState("");  
  const [opis, setOpis] = useState("");  
  const [wege, setWege] = useState(false);  
  
  function dodajDanie(e) {  
    e.preventDefault();  
    if (!nazwa || !cena) return;  
  
    setMenu([...menu, {  
      id: Date.now(),  
      nazwa,  
      cena: Number(cena),  
      opis,  
      wege  
    }]);  
  
    setNazwa(""); setCena(""); setOpis(""); setWege(false);  
  }  
  
  function usunDanie(id) {  
    setMenu(menu.filter(m => m.id !== id));  
  }  
  
  return (  
    <main className="container-fluid mt-4">  
      <div className="row">  
        {/* Panel Managera */}
```

```

<div className="col-md-4 bg-light p-4 border-end">
  <h3>Panel Dodawania</h3>
  <form onSubmit={dodajDanie}>
    <div className="mb-2">
      <input type="text" className="form-control" placeholder="Nazwa dania" value={nazwa} onChange={e =>
setNazwa(e.target.value)} required />
    </div>
    <div className="mb-2">
      <input type="number" step="0.1" className="form-control" placeholder="Cena (zł)" value={cena} onChange={e =>
setCena(e.target.value)} required />
    </div>
    <div className="mb-2">
      <textarea className="form-control" placeholder="Opis potrawy (składniki)" value={opis} onChange={e =>
setOpis(e.target.value)} />
    </div>
    <div className="form-check mb-3">
      <input type="checkbox" className="form-check-input" id="wege" checked={wege} onChange={e =>
setWege(e.target.checked)} />
      <label className="form-check-label text-success fw-bold" htmlFor="wege">Danie Wegetariańskie 🌱 </label>
    </div>
    <button type="submit" className="btn btn-primary w-100">Dodaj do Menu</button>
  </form>
</div>

{/* Widok Klienta */}
<div className="col-md-8 p-4">
  <h1 className="text-center mb-4 border-bottom pb-2">Nasz Jadłospis</h1>
  <div className="row">
    {menu.map(danie => (
      <div key={danie.id} className="col-md-6 mb-3">
        <div className={`card h-100 ${danie.wege ? 'border-success' : ''}`>
          <div className="card-body">
            <div className="d-flex justify-content-between align-items-start mb-2">
              <h4 className="card-title">
                {danie.nazwa} {danie.wege && <span className="text-success fs-5">🌱 </span>}
              </h4>
              <h4 className="text-primary fw-bold">{danie.cena.toFixed(2)} zł</h4>
            </div>
            <p className="card-text text-muted fst-italic">{danie.opis}</p>
          </div>
          <div className="card-footer bg-white border-top-0">
            <button className="btn btn-sm btn-outline-danger" onClick={() => usunDanie(danie.id)}>Usuń z oferty</button>
          </div>
        </div>
      </div>
    ))}
    {menu.length === 0 && <p className="text-center text-muted">Obecnie menu jest puste.</p>}
  </div>
</div>
</main>

);
}

```

```
export default App;
```

### 26.41. Interaktywny Quiz wiedzy (5 pytań)

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Interaktywny Quiz wiedzy (5 pytań)” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Interaktywny Quiz wiedzy (5 pytań)” dodaj historię rund, licznik punktów i reset całej rozgrywki. Dane konfiguracyjne, takie jak lista pytań, możliwe ruchy albo ściany kostki, trzymaj poza komponentem.

Wykorzystanie stanu do śledzenia obecnego indeksu elementu tablicy pytań i punktacji.

```
// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

const pytania = [
  { p: "Jak nazywa się stolica Francji?", odp: ["Londyn", "Paryż", "Berlin", "Madryt"], poprawna: 1 },
  { p: "Która planeta jest najbliżej Słońca?", odp: ["Ziemia", "Wenus", "Merkury", "Mars"], poprawna: 2 },
  { p: "Kto napisał 'Pana Tadeusza'?", odp: ["Sienkiewicz", "Prus", "Mickiewicz", "Słowacki"], poprawna: 2 },
  { p: "Ile to 2 + 2 * 2?", odp: ["8", "6", "4", "10"], poprawna: 1 },
  { p: "Jaki jest największy ocean?", odp: ["Spokojny", "Atlantycki", "Indyjski", "Arktyczny"], poprawna: 0 }
];

function App() {
  const [aktualne, setAktualne] = useState(0);
  const [punkty, setPunkty] = useState(0);
  const [koniec, setKoniec] = useState(false);

  function odpowiedz(indeksOdpowiedzi) {
    if (indeksOdpowiedzi === pytania[aktualne].poprawna) {
      setPunkty(p => p + 1);
    }

    const nastepne = aktualne + 1;
    if (nastepne < pytania.length) {
      setAktualne(nastepne);
    } else {
      setKoniec(true);
    }
  }

  function reset() {
    setAktualne(0);
    setPunkty(0);
    setKoniec(false);
  }

  return (
    <main className="container mt-5" style={{ maxWidth: "500px" }}>
      <h1 className="text-center mb-4">Quiz Wiedzy</h1>
    </main>
  );
}
```

```

    {koniec ? (
      <div className="card text-center p-5 shadow-sm">
        <h2>Koniec Gry!</h2>
        <p className="fs-4">Twój wynik: <strong className="text-primary">{punkty}</strong> na {pytania.length}</p>
        <button className="btn btn-success mt-3" onClick={reset}>Zagraj ponownie</button>
      </div>
    ) : (
      <div className="card shadow-sm">
        <div className="card-header d-flex justify-content-between">
          <span>Pytanie {aktualne + 1} z {pytania.length}</span>
          <span>Wynik: {punkty}</span>
        </div>
        <div className="card-body p-4">
          <h4 className="card-title mb-4">{pytania[aktualne].p}</h4>
          <div className="d-grid gap-2">
            {pytania[aktualne].odp.map((tekst, i) => (
              <button key={i} className="btn btn-outline-primary text-start fs-5" onClick={() => odpowiedz(i)}>
                {tekst}
              </button>
            ))}
          </div>
        </div>
      </div>
    )}
  </main>
);
}

export default App;

```

## 26.42. Tablica Kanban (Zadania w kolumnach)

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Tablica Kanban (Zadania w kolumnach)” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Tablica Kanban (Zadania w kolumnach)” dodaj wyszukiwarkę albo filtr kategorii i trzymaj wynik w zmiennej `widoczneElementy`. Dzięki temu logika wyboru danych jest oddzielona od JSX renderującego pojedynczą kartę lub wiersz.

Zmiana wartości określonego parametru w obiekcie zadania pozwala na wizualne "przeptywanie" zadań między kolumnami.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [zadania, setZadania] = useState([
    { id: 1, tekst: "Nauczyć się Reacta", status: "todo" },
    { id: 2, tekst: "Zrobić projekt zaliczeniowy", status: "in_progress" },
    { id: 3, tekst: "Odpocząć", status: "done" }
  ]);
  const [noweZadanie, setNoweZadanie] = useState("");

  function dodajZadanie(e) {

```

```

    e.preventDefault();
    if (!noweZadanie.trim()) return;
    setZadania([...zadania, { id: Date.now(), tekst: noweZadanie, status: "todo" }]);
    setNoweZadanie("");
  }

function zmienStatus(id, nowyStatus) {
  setZadania(zadania.map(z => z.id === id ? { ...z, status: nowyStatus } : z));
}

function usunZadanie(id) {
  setZadania(zadania.filter(z => z.id !== id));
}

const renderKolumny = (tytul, statusID, kolorH, lewoKrok, prawoKrok) => {
  const karty = zadania.filter(z => z.status === statusID);
  return (
    <div className="col-md-4">
      <div className="card bg-light h-100">
        <div className={`card-header text-white fw-bold bg-${kolorH}`}>{tytul} ({karty.length})</div>
        <div className="card-body p-2">
          {karty.map(z => (
            <div key={z.id} className="card mb-2 shadow-sm">
              <div className="card-body p-2 d-flex flex-column">
                <span className="mb-2">{z.tekst}</span>
                <div className="d-flex justify-content-between mt-auto">
                  <div>
                    {lewoKrok && <button className="btn btn-sm btn-outline-secondary me-1" onClick={() => zmienStatus(z.id,
lewoKrok)}></button>}
                    {prawoKrok && <button className="btn btn-sm btn-outline-secondary" onClick={() => zmienStatus(z.id,
prawoKrok)}></button>}
                  </div>
                  <button className="btn btn-sm btn-danger text-white" onClick={() => usunZadanie(z.id)}>x</button>
                </div>
              </div>
            </div>
          ))}
        </div>
      </div>
    </div>
  );
};

return (
  <main className="container mt-4">
    <h1 className="text-center mb-4">Mini Tablica Kanban</h1>

    <form onSubmit={dodajZadanie} className="mb-4 d-flex" style={{ maxWidth: "500px", margin: "0 auto" }}>
      <input type="text" className="form-control me-2" placeholder="Nowe zadanie ..." value={noweZadanie} onChange={e =>
setNoweZadanie(e.target.value)} />
      <button type="submit" className="btn btn-primary">Dodaj</button>
    </form>

    <div className="row g-3">

```



```

    {renderKolumny("Do zrobienia", "todo", "secondary", null, "in_progress")}
    {renderKolumny("W trakcie", "in_progress", "warning text-dark", "todo", "done")}
    {renderKolumny("Zrobione", "done", "success", "in_progress", null)}
  </div>
</main>
);
}

export default App;

```

### 26.43. System rezerwacji miejsc w kinie (Siatka miejsc)

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „System rezerwacji miejsc w kinie (Siatka miejsc)” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „System rezerwacji miejsc w kinie (Siatka miejsc)” dodaj co najmniej dwa realne błędy walidacji, np. zbyt krótki tekst i brak zaznaczenia zgody. Po poprawnym wysłaniu pokaż podsumowanie danych oraz wyczyść zarówno wartości pól, jak i obiekt błędów.

Zarządzanie wizualną siatką elementów - każde miejsce z reprezentacją "wolne", "wybrane", "zajęte".

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

// Generujemy początkowy stan sali: 5 rzędów po 6 miejsc (status 0: wolne, 1: wybrane, 2: zajęte)
const inicjalizujMiejsca = () => {
  const siatka = [];
  for (let i = 0; i < 5; i++) {
    for (let j = 0; j < 6; j++) {
      // Losowo symulujemy kilka zajętych miejsc na starcie
      const zajete = Math.random() < 0.2;
      siatka.push({ id: `${i}-${j}`, rzad: i + 1, miejsce: j + 1, status: zajete ? 2 : 0 });
    }
  }
  return siatka;
};

function App() {
  const [miejsc, setMiejsc] = useState(inicjalizujMiejsca());
  const cenaBiletu = 25;

  function kliknijMiejsce(id) {
    setMiejsc(miejsc.map(m => {
      if (m.id === id) {
        if (m.status === 0) return { ...m, status: 1 }; // Zaznacz
        if (m.status === 1) return { ...m, status: 0 }; // Odznacz
      }
      return m; // 2 (zajęte) nie reaguje na kliknięcie
    }));
  }

  function kupBilety() {
    setMiejsc(miejsc.map(m => m.status === 1 ? { ...m, status: 2 } : m));
  }
}

```



## 26.44. Akordeon FAQ z wyszukiwarką pytań

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Akordeon FAQ z wyszukiwarką pytań” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

W przykładzie „Akordeon FAQ z wyszukiwarką pytań” dodaj jeden konkretny wariant nieidealny: brak danych, błędną wartość, reset albo komunikat po wykonaniu akcji. Taki wariant pokazuje, czy stan komponentu jest zaprojektowany kompletnie.

Otwieranie jednego elementu na raz poprzez trzymanie jego ID w stanie głównym komponentu i wyszukiwanie w tablicy.

```
// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

const pytaniaBaza = [
  { id: 1, p: "Jak zresetować hasło?", o: "Wybierz opcję 'Zapomniałem hasła' na ekranie logowania i postępuj zgodnie z instrukcjami z maila." },
  { id: 2, p: "Czy aplikacja jest darmowa?", o: "Tak, podstawowa wersja jest darmowa. Oferujemy płatne pakiety premium z dodatkami." },
  { id: 3, p: "Jak skontaktować się z obsługą?", o: "Napisz na nasz adres mailowy pomoc@example.com lub zadzwoń pod numer z zakładki kontakt." },
  { id: 4, p: "Gdzie znajdę ustawienia konta?", o: "Ustawienia konta są widoczne w prawym górnym rogu ekranu po kliknięciu w awatar użytkownika." }
];

function App() {
  const [otwarteId, setOtwarteId] = useState(null);
  const [szukaj, setSzukaj] = useState("");

  const wyswietlane = pytaniaBaza.filter(faq =>
    faq.p.toLowerCase().includes(szukaj.toLowerCase())
  );

  function przełącz(id) {
    if (otwarteId === id) setOtwarteId(null); // Zamknij jeśli kliknięto ponownie to samo
    else setOtwarteId(id); // Otwórz wybrane (zamykając inne)
  }

  return (
    <main className="container mt-5" style={{ maxWidth: "600px" }}>
      <h1 className="mb-4">Najczęstsze pytania (FAQ)</h1>

      <div className="mb-4">
        <input
          type="text"
          className="form-control form-control-lg"
          placeholder="Wpisz słowo kluczowe z pytania..."
          value={szukaj}
          onChange={(e) => setSzukaj(e.target.value)}
        />
      </div>

      <div className="list-group shadow-sm">
        {wyswietlane.length === 0 && <div className="p-3 text-muted">Brak pytań spełniających kryteria.</div>}
      </div>
    </main>
  );
}
```

```

    {wyswietlane.map(faq => {
      const otwarte = otwarteId === faq.id;
      return (
        <div key={faq.id} className="list-group-item p-0 border-bottom">
          <button
            className={`w-100 text-start btn bg-transparent border-0 p-3 fs-5 d-flex justify-content-between align-items-center`}
            ${otwarte ? "fw-bold text-primary" : "text-dark"}`>
              <span>{faq.p}</span>
              <span className="text-muted">{otwarte ? "▲" : "▼"}</span>
            </button>

            {otwarte && (
              <div className="p-3 pt-0 text-muted lh-lg" style={{ backgroundColor: "#f8f9fa" }}>
                {faq.o}
              </div>
            )}
          </div>
        );
      })}
    </div>
  </main>
);
}

export default App;

```

## 26.45. Wyszukiwarka użytkowników z API

W tym wzorcu dopisz własny wariant danych, zamiast zmieniać tylko teksty w gotowym kodzie. Dla przykładu „Wyszukiwarka użytkowników z API” warto sprawdzić, które wartości są stanem, które są wyliczane, a które są tylko prezentacją.

Rozbuduj ten wzorec o przycisk ponownego pobrania danych, komunikat błędu i stan ładowania. Jeśli dane są filtrowane, wyszukiwarkę licz już z pobranej tablicy, a nie przez kolejne żądanie przy każdym znaku.

Przykład łączy kilka praktycznych elementów: pobranie danych z zewnętrznego API, obsługę ładowania i błędu, wyszukiwanie po stronie React oraz podgląd szczegółów wybranego użytkownika.

```

// Plik: src/App.js
import { useEffect, useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

function App() {
  const [uzytkownicy, setUzytkownicy] = useState([]);
  const [szukaj, setSzukaj] = useState("");
  const [wybranyId, setWybranyId] = useState(null);
  const [ladowanie, setLadowanie] = useState(true);
  const [blad, setBlad] = useState("");

  async function pobierzUzytkownikow() {
    try {

```

```
    setLadowanie(true);
    setBlad("");

    const response = await fetch("https://jsonplaceholder.typicode.com/users");

    if (!response.ok) {
      throw new Error(`Błąd HTTP: ${response.status}`);
    }

    const data = await response.json();
    setUzytkownicy(data);
    setWybranyId(data[0]?.id ?? null);
  } catch (error) {
    setBlad("Nie udało się pobrać użytkowników. Spróbuj ponownie.");
  } finally {
    setLadowanie(false);
  }
}

useEffect(() => {
  pobierzUzytkownikow();
}, []);

const wyniki = uzytkownicy.filter((user) => {
  const tekst = `${user.name} ${user.email} ${user.company.name}`.toLowerCase();
  return tekst.includes(szukaj.toLowerCase());
});

const wybrany = uzytkownicy.find((user) => user.id === wybranyId);

if (ladowanie) {
  return (
    <main className="container mt-5">
      <p className="alert alert-info">Ładowanie użytkowników... </p>
    </main>
  );
}

if (blad) {
  return (
    <main className="container mt-5">
      <p className="alert alert-danger">{blad}</p>
      <button className="btn btn-primary" onClick={pobierzUzytkownikow}>
        Spróbuj ponownie
      </button>
    </main>
  );
}

return (
  <main className="container mt-4">
    <div className="d-flex justify-content-between align-items-center mb-3">
      <h1>Użytkownicy z API</h1>
      <button className="btn btn-outline-primary" onClick={pobierzUzytkownikow}>
```

Odśwież

</button>

</div>

<input

type="text"

className="form-control mb-4"

placeholder="Szukaj po nazwie, e-mailu lub firmie..."

value={szukaj}

onChange={(e) => setSzukaj(e.target.value)}

/>

<div className="row g-3">

<section className="col-md-5">

<div className="list-group">

{wyniki.length === 0 && (

<div className="list-group-item text-muted">

Brak wyników dla podanej frazy.

</div>

))}

{wyniki.map((user) => (

<button

key={user.id}

type="button"

className={`list-group-item list-group-item-action \${wybranyId === user.id ? "active" : ""}`}

onClick={() => setWybranyId(user.id)}

>

<strong>{user.name}</strong>

<br />

<small>{user.email}</small>

</button>

))}

</div>

</section>

<section className="col-md-7">

{wybrany ? (

<div className="border rounded p-3 h-100">

<h2 className="h4">{wybrany.name}</h2>

<p className="text-muted">@{wybrany.username}</p>

<hr />

<p><strong>E-mail:</strong> {wybrany.email}</p>

<p><strong>Telefon:</strong> {wybrany.phone}</p>

<p><strong>Strona:</strong> {wybrany.website}</p>

<p><strong>Firma:</strong> {wybrany.company.name}</p>

<p>

<strong>Miasto:</strong>{" "}

{wybrany.address?.city ?? "Brak danych"}

</p>

</div>

) : (

```
        <div className="border rounded p-3 text-muted">
            Wybierz użytkownika z listy.
        </div>
    )}
</section>
</div>
</main>

};
}

export default App;
```

## 27. Najczęstsze błędy (Wyjątki i błędy konsolowe)

W procesie tworzenia aplikacji we frameworku React i języku JavaScript najczęściej spotyka się błędy w samej konsoli przeglądarki (Developer Tools) lub w narzędziu bundlującym (Vite/Babel) ukazanym w terminalu. Zrozumienie powyższych czerwonych komunikatów pozwala sprawnie radzić sobie z awariami interfejsu.

### 27.1. `SyntaxError: Unexpected token`

Jeden z najczęstszych błędów początkujących w React. Oznacza niezgodność składni (zazwyczaj JSX) lub próbę napisania czystego HTML zamiast poprawnego JSX w nieodpowiednim pliku.

❌ Kod powodujący błąd:

```
const el = <div class="kontener">Witaj</div>;
```

Komunikat w konsoli:

```
SyntaxError: Unexpected token '<' (jeśli plik nie jest skompilowany jako JSX/TSX)
```

✅ Poprawny kod:

```
const el = <div className="kontener">Witaj</div>; // i upewnij się, że plik ma rozszerzenie .jsx/.tsx
```

Upewnij się, że używasz odpowiedniego rozszerzenia pliku dla Babel lub Vite (.jsx) i pamiętaj, że JSX wymaga używania `className` zamiast `class`.

### 27.2. `TypeError: Cannot read properties of undefined`

Występuje powszechnie przy pobieraniu danych ze stanu lub API, gdy próbujemy odwołać się do pola wewnątrz obiektu, który jeszcze nie został załadowany i ma wartość `undefined`.

❌ Kod powodujący błąd:

```
function Profil({ user }) {
    return <div>{user.name}</div>; // jeśli user jest undefined, to rzuci wyjątek
}
```

Komunikat w konsoli:

Uncaught TypeError: Cannot read properties of undefined (reading 'name')

✅ Poprawny kod:

```
function Profil({ user }) {  
  return <div>{user?.name || 'Brak danych'}</div>;  
}
```

Należy sprawdzać istnienie danych przed renderowaniem, na przykład używając operatora opcjonalnego wywołania (`?.`) lub warunkowego (`&&`).

### 27.3. ReferenceError: X is not defined

Pojawia się, gdy używamy w kodzie zmiennej, komponentu lub funkcji, której nie zaimportowaliśmy na początku pliku lub nie zadeklarowaliśmy.

❌ Kod powodujący błąd:

```
function App() {  
  return <Header />;  
}
```

Komunikat w konsoli:

Uncaught ReferenceError: Header is not defined

✅ Poprawny kod:

```
import Header from './Header';  
  
function App() {  
  return <Header />;  
}
```

Skopiuj nazwę błędu i sprawdź, czy dany element został poprawnie zaincludowany poleceniem `import`.

### 27.4. Warning: Each child in a list should have a unique "key" prop

Ostrzeżenie w konsoli od samego Reacta (na żółto). Zgłaszane podczas renderowania listy elementów z tablicy w przypadku braku unikalnego klucza dla każdego z nich.

❌ Kod powodujący błąd:

```
{zadania.map(zadanie => (<li>{zadanie.nazwa}</li>))}
```

Komunikat w konsoli:

Warning: Each child in a list should have a unique "key" prop.



✅ Poprawny kod:

```
{zadania.map(zadanie => (<li key={zadanie.id}>{zadanie.nazwa}</li>))}
```

Dodaj własność `key` do najwyższego elementu zwracanego z funkcji `map()`. Unikaj używania indeksu tablicy jako klucza, jeśli elementy mogą zmieniać swoją kolejność.

### 27.5. Error: Rendered fewer hooks than expected

Zgłaszany przez React, gdy hooki (np. `useState`, `useEffect`) zostaną wywołane warunkowo lub wewnątrz pętli. Psuje to mechanizm wewnętrzny śledzenia stanu Reacta.

❌ Kod powodujący błąd:

```
if (czyZalogowany) {  
  const [imie, setImie] = useState('');  
}
```

Komunikat w konsoli:

```
Uncaught Error: Rendered fewer hooks than expected. This may be caused by an accidental early return statement.
```

✅ Poprawny kod:

```
const [imie, setImie] = useState('');  
if (czyZalogowany) { /* logika */ }
```

Hooki muszą zawsze być deklarowane na samym początku (top-level) komponentu funkcyjnego, a nie w blokach `if` czy `for`.

### 27.6. Error: Invalid hook call

Pojawia się, gdy wywołasz hook Reacta poza ciałem komponentu funkcyjnego, lub wewnątrz zwykłej funkcji JavaScript, która nie jest customowym hookiem.

❌ Kod powodujący błąd:

```
function pomocnicza() {  
  useEffect(() => { ... });  
}
```

Komunikat w konsoli:

```
Uncaught Error: Invalid hook call. Hooks can only be called inside of the body of a function component.
```

✅ Poprawny kod:

```
function usePomocnicza() { // Custom Hook
  useEffect(() => { ... });
}
```

Upewnij się, że nazwa funkcji wywołującej hooka zaczyna się od 'use' (np. `useWindowSize`), i że masz tylko jedną kopię Reacta w `node_modules`.

### 27.7. Error: Too many re-renders

Krytyczny błąd blokujący aplikację. Oznacza nieskończoną pętlę renderowania: komponent aktualizuje stan, co wymusza renderowanie, które znów aktualizuje stan.

❌ Kod powodujący błąd:

```
function Przycisk() {
  const [licznik, setLicznik] = useState(0);
  return <button onClick={setLicznik(licznik + 1)}>Klik</button>; // od razu się wywołuje
}
```

Komunikat w konsoli:

```
Uncaught Error: Too many re-renders. React limits the number of renders to prevent an infinite loop.
```

✅ Poprawny kod:

```
function Przycisk() {
  const [licznik, setLicznik] = useState(0);
  return <button onClick={() => setLicznik(licznik + 1)}>Klik</button>;
}
```

Upewnij się, że do handlerów zdarzeń (jak `onClick`) przekazujesz funkcję (np. arrow function `() =>`), a nie natychmiastowe jej wywołanie.

### 27.8. Error: Objects are not valid as a React child

Występuje, gdy spróbujesz wyrenderować cały obiekt (np. z API) bezpośrednio w drzewie JSX, zamiast jego konkretnych tekstowych właściwości.

❌ Kod powodujący błąd:

```
const dane = { tytuł: 'Test' };
return <div>{dane}</div>;
```

Komunikat w konsoli:

```
Uncaught Error: Objects are not valid as a React child (found: object with keys {tytuł}). If you meant to render a collection of children, use an array instead.
```

✅ Poprawny kod:

```
const dane = { tytuł: 'Test' };  
return <div>{dane.tytuł}</div>;
```

Wybierz konkretne pole typu prostego (string/number), np. `.nazwa` w obiekcie.

### 27.9. Warning: A component is changing an uncontrolled input to be controlled

Zgłaszany przez formularze, gdy wartość początkowa (np. `value={stan}`) wynosi na początku `undefined` (input niekontrolowany), a następnie zmienia się na string (input kontrolowany).

#### ✗ Kod powodujący błąd:

```
const [tekst, setTekst] = useState();  
return <input value={tekst} onChange={e => setTekst(e.target.value)} />;
```

#### Komunikat w konsoli:

```
Warning: A component is changing an uncontrolled input to be controlled. This is likely caused by the value changing from undefined to a defined value.
```

#### ✓ Poprawny kod:

```
const [tekst, setTekst] = useState(''); // pusty string zamiast undefined  
return <input value={tekst} onChange={e => setTekst(e.target.value)} />;
```

Zawsze podawaj wartość domyślną podczas inicjalizacji `useState`, np. puste ciągi znaków `useState('')`.

### 27.10. Module not found: Can't resolve

Błąd paczki (Vite/Webpack), mówiący o tym, że podany w imporcie plik ścieżki nie istnieje, ma złą wielkość znaków lub nie zainstalowano pakietu NPM.

#### ✗ Kod powodujący błąd:

```
import Header from './header';
```

#### Komunikat w konsoli:

```
Module not found: Error: Can't resolve './header' in '/src/components'
```

#### ✓ Poprawny kod:

```
import Header from './Header'; // duża litera
```

Sprawdź dokładnie nazwę pliku, zwracając uwagę na wielkość liter (Linux/Mac rozróżnia wielkość znaków), lub wykonaj `npm install [pakiet]`.

### 27.11. TypeError: Failed to fetch

Oznacza, że żądanie sieciowe z API w ogóle nie opuściło przeglądarki lub zostało całkowicie zablokowane ze względu na awarię sieci, brak internetu, błędy CORS lub po prostu serwer jest wyłączony.

#### ✗ Kod powodujący błąd:

```
fetch('http://nieistniejacy-serwer.lokalny')
  .then(res => res.json())
```

#### Komunikat w konsoli:

```
TypeError: Failed to fetch
```

#### ✓ Poprawny kod:

```
fetch('http://localhost:3000/api')
  .catch(err => console.error("Serwer wyłączony lub awaria sieci:", err));
```

Upewnij się, że lokalny serwer działa poprawnie i adres do API jest bezbłędny. Obsłuż zawsze stan błędu w sekcji `.catch()` bloków Promise.

### 27.12. CORS error: Access-Control-Allow-Origin

Błąd zgłaszany w konsoli przez przeglądarkę dla celów bezpieczeństwa, gdy próbujemy uderzać (Fetch/Axios) do zewnętrznego serwera, który zabrania żądań z innej domeny.

#### ✗ Kod powodujący błąd:

```
// Kod wysyłający strzał do API z innej domeny bez odpowiednich nagłówków na serwerze
```

#### Komunikat w konsoli:

```
Access to fetch at 'https://api.strona.com' from origin 'http://localhost:3000' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.
```

#### ✓ Poprawny kod:

```
// Poprawka następuje najczęściej w kodzie Backend-u, np. dodanie paczki cors w Express.js
```

Należy skonfigurować Backend aby wysyłał nagłówki CORS, włączenie proxy serwera w Vite lub użycie wtyczki CORS (podczas developingu).

### 27.13. TypeError: X is not a function

Pojawia się podczas próby wywołania jako funkcja czegoś, co nią nie jest - na przykład obiektu. Bardzo częste podczas pomyłek w destrukuryzacji ze zmienną `useState`.

#### ✗ Kod powodujący błąd:

```
const { licznik, setLicznik } = useState(0); // użyto klamerek dla tablicy
```

**Komunikat w konsoli:**

```
TypeError: setLicznik is not a function
```

✅ **Poprawny kod:**

```
const [licznik, setLicznik] = useState(0);
```

Hooki w React, takie jak `useState`, zwracają tablicę (Array), a nie obiekt. Zwróć uwagę na nawiasy kwadratowe.

#### 27.14. Error: Element type is invalid

Pojawia się, gdy poprosisz Reacta o wyrenderowanie komponentu, ale dostarczysz mu coś nieprawidłowego, np. `undefined` z powodu błędu z importem lub nieużywania default exportów.

❌ **Kod powodujący błąd:**

```
import Header from './Header'; // Gdy w Header.jsx brakuje export default Header
```

**Komunikat w konsoli:**

```
Uncaught Error: Element type is invalid: expected a string (for built-in components) or a class/function (for composite components) but got: undefined.
```

✅ **Poprawny kod:**

```
import { Header } from './Header'; // (Jeśli eksport był nazwany 'export const Header')
```

Upewnij się, że w pliku zawierającym komponent na końcu jest wpisane na przykład `export default NazwaKomponentu;`.

#### 27.15. Warning: React does not recognize the prop on a DOM element

Pojawia się, gdy podasz niestandardowe propsy (camelCase) bezpośrednio do elementów czystego HTML (`div`, `span`), zamiast do komponentu Reactowego.

❌ **Kod powodujący błąd:**

```
<div isDarkTheme={true}>Tło</div>
```

**Komunikat w konsoli:**

```
Warning: React does not recognize the `isDarkTheme` prop on a DOM element. If you intentionally want it to appear in the DOM as a custom attribute, spell it as lowercase `isdarktheme` instead.
```

✅ **Poprawny kod:**

```
const className = true ? 'dark' : 'light';  
<div className={className}>Tło</div>
```

Propsy we wbudowanych elementach HTML muszą być nazwane tak samo jak istniejące, prawdziwe atrybuty standardu HTML (np. `className`, `id`).

### 27.16. TypeError: Assignment to constant variable

Podstawowy błąd JavaScript polegający na próbie modyfikacji wartości w zmiennej przypisanej do stałej (`const`).

❌ Kod powodujący błąd:

```
const wiek = 10;  
wiek = 15;
```

Komunikat w konsoli:

```
TypeError: Assignment to constant variable.
```

✅ Poprawny kod:

```
let wiek = 10;  
wiek = 15;
```

Jeśli masz zamiar modyfikować zmienną, zadeklaruj ją za pomocą `let`. Jeśli jest to stan Reacta, zmieniaj go tylko przez funkcję settera (np. `setWiek(15)`).

### 27.17. Unhandled Promise Rejection

Ten błąd konsolowy oznacza, że funkcja asynchroniczna rzuciła wyjątkiem (na przykład padło zapytanie do API), a aplikacja nigdzie tego nie wyłapała w odpowiednim bloku `catch`.

❌ Kod powodujący błąd:

```
async function load() {  
  const res = await fetch('/api');  
  const data = await res.json();  
}
```

Komunikat w konsoli:

```
Uncaught (in promise) SyntaxError: Unexpected end of JSON input
```

✅ Poprawny kod:

```
async function load() {  
  try {  
    const res = await fetch('/api');  
    const data = await res.json();  
  }  
}
```

```
} catch (error) {  
  console.error("Złapano błąd:", error);  
}  
}
```

Zawsze dodawaj bloki `try { ... } catch (error) { ... }` używając operacji `await`.

### 27.18. SyntaxError: Cannot use import statement outside a module

Zgłaszany przez NodeJS (często przy uruchamianiu skryptów lub tworzeniu testów po stronie serwera), oznaczający próbę korzystania z nowoczesnych modułów ES (`import`) w starszym ekosystemie CommonJS.

#### ❌ Kod powodujący błąd:

```
// Próba wykonania czystego skryptu za pomocą node plik.js
```

#### Komunikat w konsoli:

```
SyntaxError: Cannot use import statement outside a module
```

#### ✅ Poprawny kod:

```
// Upewnij się, że to Vite buduje aplikację lub w package.json podaj: "type": "module"
```

Upewnij się, że wykonujesz środowisko za pomocą np. Vite. Czysty node wymaga rozszerzenia `.mjs` lub deklaracji w package.json.

### 27.19. JSON.parse error: Unexpected token

Próba przetworzenia odpowiedzi jako JSON z serwera, na przykład w wyniku `res.json()`, podczas gdy w rzeczywistości serwer zwrócił HTML (np. stronę błędu 404).

#### ❌ Kod powodujący błąd:

```
fetch('/nie-istnieje').then(res => res.json())
```

#### Komunikat w konsoli:

```
SyntaxError: Unexpected token '<', "<!DOCTYPE "... is not valid JSON
```

#### ✅ Poprawny kod:

```
fetch('/api/dane').then(res => {  
  if(!res.ok) throw new Error("Błąd pobierania danych");  
  return res.json();  
})
```

Sprawdź wartość `res.ok` (statusu HTTP 200) przed użyciem konwersji na `.json()`. Najwyraźniej strona odpowiedziata kodem tekstowym, nie danymi API.

## 27.20. Error: Minified React error

Pojawia się na produkcji po zbudowaniu aplikacji. Wersja zminifikowana zastępuje opis błędu linkiem.

### ✗ Kod powodujący błąd:

```
// Aplikacja produkcyjna ulega krytycznej usterce.
```

### Komunikat w konsoli:

```
Uncaught Error: Minified React error #185; visit https://reactjs.org/docs/error-decoder.html?invariant=185...
```

### ✓ Poprawny kod:

```
// Przejdź do wskazanego pod adresem URL błędu
```

Musisz odwiedzić podany link w przeglądarce, aby odkodować zminifikowany błąd i przeczytać jego oryginalną, deweloperską treść.